

Прикладные методы оптимизации

Содержание

Прикладные методы оптимизации	1
1. Постановка одномерной задачи оптимизации. Методы решения одномерных задач оптимизации	1
2. Поиск экстремума функции нескольких переменных. Матрица Гессе и критерий Сильвестра	3
3. Задачи динамического программирования и методология их решения. Принцип Беллмана. Алгоритмические методы и метод функциональных уравнений	5
4. Методы решения многокритериальных задач оптимизации	7
5. Целочисленное линейное программирование. Метод ветвей и границ на примере задачи коммивояжера	9
6. Принятие решений в условиях неопределенности: игры с природой. Основные критерии принятия решения	11
7. Принятие решений в условиях неопределенности. Идеальный эксперимент и целесообразность его проведения	13
8. Метод анализа иерархий и его использование для принятия решений в условиях определенности	15
9. Антагонистические матричные игры; методы решения игр фиксированного размера	17
10. Антагонистические матричные игры произвольной размерности и методы их решения	19
11. Позиционные игры: разновидности и методы решения	22
12. Кооперативные игры. Принципы рациональности и дележа	24

Прикладные методы оптимизации

1. Постановка одномерной задачи оптимизации. Методы решения одномерных задач оптимизации

Одномерная задача оптимизации состоит в выборе такого значения одной действительной переменной, при котором целевая функция достигает минимального или максимального значения на допустимом множестве. В задаче минимизации требуется найти точку x^* , для которой выполняется:

$$f(x^*) \leq f(x), \quad x \in X \subseteq \mathbb{R}.$$

Задача максимизации сводится к минимизации заменой функции: максимизация $f(x)$ эквивалентна минимизации $-f(x)$. Допустимое множество X обычно задаётся отрезком $[a, b]$, полуинтервалом, всей числовой прямой или системой ограничений. Если минимум достигается на всём допустимом множестве, он называется глобальным. Если неравенство выполняется только в некоторой окрестности точки, то минимум локальный.

Для письменного ответа важно различать точку экстремума x^* и значение экстремума $f(x^*)$. Схема постановки одномерной задачи:

Дано: $f(x)$, область X , тип экстремума, точность ε

↓

Найти: x^* , при котором $f(x^*)$ минимально или максимально

↓

Проверить: внутренние точки, границы, точки недифференцируемости

↓

Выдать: x^* , $f(x^*)$ и оценку точности

Если функция дифференцируема внутри отрезка, необходимое условие внутреннего локального экстремума имеет вид:

$$f'(x^*) = 0.$$

Точка, в которой производная равна нулю или не существует, называется критической. Стационарная точка, где $f'(x^*) = 0$, не обязательно является экстремумом: она может быть точкой перегиба. Достаточные условия через вторую производную:

$$f'(x^*) = 0, \quad f''(x^*) > 0 \Rightarrow x^* \text{ — локальный минимум};$$

$$f'(x^*) = 0, \quad f''(x^*) < 0 \Rightarrow x^* \text{ — локальный максимум}.$$

Для глобальной оптимизации на отрезке нужно проверить все критические точки внутри отрезка и значения функции на границах a и b . Если функция унимодальна на отрезке, то она имеет единственный минимум или максимум в смысле монотонного убывания до экстремума и монотонного возрастания после него. Для таких функций эффективны методы последовательного сужения интервала неопределённости. Основные методы одномерной оптимизации делят на аналитические и численные. Аналитический метод основан на решении уравнения $f'(x) = 0$ и проверке достаточных условий. Численные методы применяются, когда производная сложна, неизвестна или функция задана вычислительной процедурой. Метод дихотомии делит интервал двумя близкими точками около середины:

$$x_1 = \frac{a+b}{2} - \delta, \quad x_2 = \frac{a+b}{2} + \delta.$$

Если $f(x_1) \leq f(x_2)$, то минимум лежит в $[a, x_2]$; иначе в $[x_1, b]$. Метод золотого сечения использует постоянное отношение длин интервалов, благодаря чему на каждой итерации требуется только одно новое вычисление функции:

$$\tau = \frac{\sqrt{5} - 1}{2} \approx 0.618.$$

Точки выбираются так:

$$x_1 = b - \tau(b - a), \quad x_2 = a + \tau(b - a).$$

Метод Ньютона использует производные и строит приближение к стационарной точке:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Он быстро сходится около минимума, но требует существования второй производной и удачного начального приближения. Если $f''(x_k)$ близка к нулю, метод может работать нестабильно. **Пример 1. Аналитическое решение.** Пусть нужно минимизировать функцию

$$f(x) = x^2 - 6x + 10, \quad x \in [0, 5].$$

Находим производную:

$$f'(x) = 2x - 6.$$

Стационарная точка определяется из условия $2x-6=0$, то есть $x^*=3$. Вторая производная:

$$f''(x) = 2 > 0.$$

Следовательно, $x=3$ является точкой локального минимума. Проверим значения на отрезке:

$$f(0) = 10, \quad f(3) = 1, \quad f(5) = 5.$$

Глобальный минимум на $[0, 5]$ достигается в точке $x^*=3$, минимальное значение равно 1. **Пример 2. Один шаг метода золотого сечения.** Нужно минимизировать унимодальную функцию на $[0, 4]$ с использованием золотого сечения. Пусть

$$f(x) = (x - 1)^2 + 2.$$

Берём $a=0$, $b=4$, $\tau \approx 0.618$. Тогда:

$$x_1 = 4 - 0.618 \cdot 4 = 1.528,$$

$$x_2 = 0 + 0.618 \cdot 4 = 2.472.$$

Вычислим:

$$f(1.528) \approx (0.528)^2 + 2 = 2.279,$$

$$f(2.472) \approx (1.472)^2 + 2 = 4.167.$$

Так как $f(x_1) < f(x_2)$, минимум находится левее x_2 , новый интервал неопределённости равен $[0, 2.472]$. Далее процедура повторяется до выполнения условия $b-a \leq \epsilon$.

2. Поиск экстремума функции нескольких переменных. Матрица Гессе и критерий Сильвестра

Задача оптимизации функции нескольких переменных состоит в нахождении вектора переменных, при котором функция достигает экстремального значения. В общем виде задача безусловной минимизации записывается так:

$$f(x_1, x_2, \dots, x_n) \rightarrow \min, \quad x = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n.$$

Вектор x^* называется точкой глобального минимума, если:

$$f(x^*) \leq f(x), \quad \forall x \in \mathbb{R}^n.$$

Если это неравенство выполняется только в некоторой окрестности точки x^* , то минимум является локальным. Для максимума знак неравенства меняется на противоположный. Основным объект первого порядка — градиент функции:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^T.$$

Необходимое условие внутреннего локального экстремума дифференцируемой функции:

$$\nabla f(x^*) = 0.$$

То есть все частные производные первого порядка в точке экстремума должны быть равны нулю. Однако это условие не является достаточным. Для классификации стационарной точки используется матрица Гессе. Матрица Гессе — это квадратная матрица вторых частных производных:

$$H(x) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j} \right)_{i,j=1}^n.$$

Для функции двух переменных она имеет вид:

$$H(x, y) = \begin{pmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{pmatrix}.$$

Если вторые производные непрерывны, то $f_{xy}=f_{yx}$, и матрица Гессе симметрична. Смысл матрицы Гессе состоит в том, что она описывает кривизну функции около стационарной точки. Формула Тейлора второго порядка показывает это явно:

$$f(x^* + h) = f(x^*) + \nabla f(x^*)^T h + \frac{1}{2} h^T H(x^*) h + o(\|h\|^2).$$

В стационарной точке первый линейный член исчезает, и характер точки определяется квадратичной формой $h^T H h$. Если матрица Гессе положительно определена, стационарная точка является локальным минимумом. Если отрицательно определена — локальным максимумом. Если матрица неопределённая, то точка седловая. Если матрица полуопределённая, требуется дополнительный анализ. Критерий Сильвестра позволяет проверять знакоопределённость симметричной матрицы через главные угловые миноры. Матрица положительно определена тогда и только тогда, когда все её главные угловые миноры положительны:

$$\Delta_1 > 0, \quad \Delta_2 > 0, \quad \dots, \quad \Delta_n > 0.$$

Матрица отрицательно определена тогда и только тогда, когда знаки главных угловых миноров чередуются:

$$\Delta_1 < 0, \quad \Delta_2 > 0, \quad \Delta_3 < 0, \quad \dots,$$

то есть выполняется условие $(-1)^k \Delta_k > 0$ для каждого k . Схема исследования экстремума:

Задана $f(x_1, \dots, x_n)$

↓

Вычислить градиент ∇f

↓

Решить систему $\nabla f = 0$

↓

Найти стационарные точки

↓

Вычислить H в каждой точке

↓

Применить критерий Сильвестра

↓

Минимум / максимум / седло / требуется доп. анализ

Пример 1. Минимум функции двух переменных. Рассмотрим функцию:

$$f(x, y) = x^2 + 2y^2 - 4x - 8y + 10.$$

Градиент:

$$f_x = 2x - 4, \quad f_y = 4y - 8.$$

Стационарная точка находится из системы:

$$2x - 4 = 0, \quad 4y - 8 = 0.$$

Отсюда $x=2$, $y=2$. Матрица Гессе:

$$H = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}.$$

Главные угловые миноры:

$$\Delta_1 = 2 > 0, \quad \Delta_2 = 2 \cdot 4 - 0 = 8 > 0.$$

По критерию Сильвестра матрица положительно определена, значит точка $(2, 2)$ — локальный минимум. Так как функция квадратичная с положительно определённой матрицей, этот минимум является глобальным. **Пример 2. Седловая точка.** Пусть

$$f(x, y) = x^2 - y^2 + 2x + 4y.$$

Частные производные:

$$f_x = 2x + 2, \quad f_y = -2y + 4.$$

Стационарная точка:

$$\begin{aligned} 2x + 2 = 0 &\Rightarrow x = -1, \\ -2y + 4 = 0 &\Rightarrow y = 2. \end{aligned}$$

Матрица Гессе:

$$H = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}.$$

Главные миноры:

$$\Delta_1 = 2 > 0, \quad \Delta_2 = 2 \cdot (-2) - 0 = -4 < 0.$$

Матрица неопределённая, поэтому точка $(-1, 2)$ является седловой: в одних направлениях функция возрастает, в других убывает.

3. Задачи динамического программирования и методология их решения. Принцип Беллмана. Алгоритмические методы и метод функциональных уравнений

Динамическое программирование — это метод решения многошаговых задач оптимизации, в которых общее решение строится как последовательность частных решений. Главная идея состоит в разбиении исходной задачи на этапы и использовании оптимальности уже решённых подзадач. Метод применим, когда задача обладает двумя свойствами: оптимальной подструктурой и перекрывающимися подзадачами. Оптимальная подструктура означает, что оптимальное решение всей задачи содержит оптимальные решения её частей. Перекрывающиеся подзадачи означают, что одни и те же состояния возникают многократно, поэтому выгодно хранить уже найденные значения. Основные элементы задачи динамического программирования: этап, состояние, управление, функция выигрыша или затрат, рекуррентное соотношение и граничное условие. Этап — номер шага процесса. Состояние описывает всю информацию, необходимую для принятия дальнейших решений. Управление — решение, выбираемое на данном этапе. Функция перехода показывает, в какое состояние система перейдёт после выбора управления. Целевая функция обычно является суммой выигрышей или затрат по этапам. Схема динамического программирования:

Исходная многошаговая задача

↓

Разбиение на этапы

↓

Описание состояния s_k

↓

Выбор управления u_k

↓

Переход $s_{k+1}=T(s_k, u_k)$

↓

Рекуррентное соотношение Беллмана

↓

Обратный или прямой расчёт оптимального решения

Принцип оптимальности Беллмана формулируется так: каково бы ни было начальное состояние и начальное решение, последующие решения должны образовывать оптимальную стратегию относительно состояния, полученного после первого решения. Иначе говоря, “хвост” оптимальной стратегии сам должен быть оптимальным для соответствующей подзадачи. Типовое рекуррентное соотношение для задачи минимизации имеет вид:

$$F_k(s) = \min_{u \in U_k(s)} \{c_k(s, u) + F_{k+1}(T(s, u))\},$$

где $F_k(s)$ — оптимальное значение целевой функции от этапа k до конца при состоянии s , $U_k(s)$ — множество допустимых управлений, $c_k(s, u)$ — текущие затраты, $T(s, u)$ — функция перехода. Для конечной задачи задаётся граничное условие:

$$F_{N+1}(s) = 0.$$

Алгоритмические методы динамического программирования обычно реализуются через таблицы. Они удобны для дискретных задач: кратчайший путь, задача о рюкзаке, распределение ресурсов, замена оборудования. Метод функциональных уравнений используется в более общей форме: сначала записывают уравнение Беллмана для функции оптимального результата, затем решают его аналитически или численно. В непрерывных задачах функциональное уравнение может быть сложным и заменяется приближённой сеточной схемой. Алгоритм решения задачи методом динамического программирования:

1. Выделить этапы процесса.
2. Определить состояние на каждом этапе.
3. Записать допустимые управления.
4. Задать функцию перехода.
5. Записать текущий выигрыш или затраты.
6. Сформулировать рекуррентное уравнение Беллмана.
7. Задать граничные условия.
8. Заполнить таблицу значений.
9. Восстановить оптимальную стратегию.

Пример 1. Кратчайший путь в ациклической сети. Пусть из пункта S нужно попасть в T . Возможны переходы:

$S \rightarrow A$: 2	$A \rightarrow T$: 7
$S \rightarrow B$: 5	$B \rightarrow T$: 3
$S \rightarrow C$: 4	$C \rightarrow T$: 4

Состояние — текущая вершина. Функция $F(v)$ — минимальная длина пути из вершины v в T . Граничное условие:

$$F(T) = 0.$$

Для конечных вершин:

$$F(A) = 7,$$

$$F(B) = 3,$$

$$F(C) = 4.$$

Для начальной вершины:

$$F(S) = \min\{2 + F(A), 5 + F(B), 4 + F(C)\}.$$

Подставим значения:

$$F(S) = \min\{2 + 7, 5 + 3, 4 + 4\} = \min\{9, 8, 8\} = 8.$$

Оптимальные маршруты: S-B-T и S-C-T, оба имеют длину 8. **Пример 2. Задача о рюкзаке.** Есть рюкзак вместимости $W=5$ и три предмета:

Предмет 1: вес 2, ценность 6

Предмет 2: вес 3, ценность 10

Предмет 3: вес 4, ценность 12

Пусть $F(k, w)$ — максимальная ценность, которую можно получить, рассматривая первые k предметов при вместимости w . Рекуррентная формула:

$$F(k, w) = \max\{F(k-1, w), v_k + F(k-1, w - a_k)\},$$

если $a_k \leq w$; иначе

$$F(k, w) = F(k-1, w).$$

Для вместимости 5: можно взять предметы 1 и 2 с общим весом $2+3=5$ и ценностью $6+10=16$; предмет 3 даёт ценность 12; предметы 1 и 3 вместе не помещаются. Следовательно, оптимум равен 16, оптимальный набор — предметы 1 и 2.

4. Методы решения многокритериальных задач оптимизации

Многокритериальная задача оптимизации возникает тогда, когда качество решения оценивается не одним, а несколькими критериями. В общем виде задача может быть записана так:

$$F(x) = (f_1(x), f_2(x), \dots, f_m(x)) \rightarrow \text{opt}, \quad x \in X.$$

Здесь x — допустимое решение, X — множество допустимых решений, $f_i(x)$ — отдельные критерии. Критерии могут иметь разный смысл: прибыль максимизируется, затраты минимизируются, риск минимизируется, надёжность максимизируется. Главная трудность состоит в том, что критерии часто конфликтуют. Улучшение одного критерия может ухудшать другой. Решение x_1 называется доминирующим над x_2 , если оно не хуже по всем критериям и строго лучше хотя бы по одному. Для задачи максимизации это записывается так:

$$f_i(x_1) \geq f_i(x_2) \quad \forall i,$$

и существует хотя бы один критерий j , для которого:

$$f_j(x_1) > f_j(x_2).$$

Решение называется Парето-оптимальным, если не существует другого допустимого решения, которое доминирует его. Множество Парето содержит компромиссные варианты, среди которых лицо, принимающее решение, выбирает окончательный вариант. Схема многокритериальной оптимизации:

Допустимые решения X
 $\downarrow$
 Вычисление критериев $f_1, \dots, f_m$
 $\downarrow$
 Нормализация критериев
 $\downarrow$
 Удаление доминируемых решений
 $\downarrow$
 Построение множества Парето
 $\downarrow$
 Выбор компромисса по предпочтениям ЛПР

Если критерии имеют разные единицы измерения, их нужно нормализовать. Для критерия, который максимизируется, можно использовать нормировку:

$$\tilde{f}_i(x) = \frac{f_i(x) - f_i^{\min}}{f_i^{\max} - f_i^{\min}}.$$

Для критерия, который минимизируется:

$$\tilde{f}_i(x) = \frac{f_i^{\max} - f_i(x)}{f_i^{\max} - f_i^{\min}}.$$

После нормировки все критерии обычно приводятся к шкале от 0 до 1, где большее значение означает лучший результат. Один из основных методов — метод взвешенной суммы. После нормировки критериев строится свёртка:

$$\Phi(x) = \sum_{i=1}^m w_i \tilde{f}_i(x),$$

где веса удовлетворяют условиям:

$$w_i \geq 0, \quad \sum_{i=1}^m w_i = 1.$$

Метод прост, но результат сильно зависит от выбора весов. Если веса заданы неверно, можно получить решение, которое формально оптимально по свёртке, но практически не соответствует предпочтениям. Метод главного критерия выбирает один критерий как основной, а остальные переводит в ограничения. Например:

$$f_1(x) \rightarrow \max,$$

при условиях:

$$f_2(x) \geq b_2, \quad f_3(x) \leq b_3.$$

Метод последовательных уступок сначала оптимизирует главный критерий, затем допускает ухудшение этого критерия на величину уступки и оптимизирует следующий критерий. Лексикографический метод упорядочивает критерии по важности и сравнивает решения сначала по самому важному критерию, затем при равенстве по следующему. Метод идеальной точки сравнивает решения с воображаемым лучшим вектором критериев:

$$F^* = (f_1^*, f_2^*, \dots, f_m^*).$$

Обычно выбирают решение с минимальным расстоянием до идеальной точки:

$$D(x) = \sqrt{\sum_{i=1}^m w_i (\tilde{f}_i(x) - \tilde{f}_i^*)^2}.$$

Пример 1. Выбор варианта по взвешенной сумме. Есть три проекта. Нужно максимизировать прибыль и минимизировать риск.

Проект А: прибыль 100, риск 8

Проект В: прибыль 80, риск 4

Проект С: прибыль 60, риск 2

Нормируем прибыль как максимизируемый критерий: минимум 60, максимум 100.

$$\tilde{P}_A = 1, \quad \tilde{P}_B = \frac{80 - 60}{100 - 60} = 0.5, \quad \tilde{P}_C = 0.$$

Риск минимизируется: минимум 2, максимум 8.

$$\tilde{R}_A = \frac{8 - 8}{8 - 2} = 0, \quad \tilde{R}_B = \frac{8 - 4}{8 - 2} = 0.667, \quad \tilde{R}_C = 1.$$

Пусть веса равны $w_P=0.6$, $w_R=0.4$. Тогда:

$$\Phi_A = 0.6 \cdot 1 + 0.4 \cdot 0 = 0.6,$$

$$\Phi_B = 0.6 \cdot 0.5 + 0.4 \cdot 0.667 = 0.567,$$

$$\Phi_C = 0.6 \cdot 0 + 0.4 \cdot 1 = 0.4.$$

По методу взвешенной суммы выбирается проект А, хотя он самый рискованный. Это показывает чувствительность метода к весам. **Пример 2. Метод главного критерия.** Предприятие выбирает технологию. Главный критерий — максимальная прибыль, но риск должен быть не выше 5.

А: прибыль 100, риск 8

В: прибыль 80, риск 4

С: прибыль 60, риск 2

Задача:

$$P(x) \rightarrow \max,$$

при ограничении:

$$R(x) \leq 5.$$

Проект А исключается, так как $8 > 5$. Остаются В и С. Из них максимальная прибыль у проекта В: $80 > 60$. Следовательно, по методу главного критерия выбирается проект В.

5. Целочисленное линейное программирование. Метод ветвей и границ на примере задачи коммивояжера

Целочисленное линейное программирование — это раздел оптимизации, в котором целевая функция и ограничения линейны, но часть или все переменные должны принимать целые значения. Общая постановка задачи минимизации:

$$c^T x \rightarrow \min,$$

при условиях:

$$Ax \leq b, \quad x \geq 0, \quad x_i \in \mathbb{Z}.$$

Если все переменные целочисленные, задача называется полностью целочисленной. Если только часть переменных целочисленна, задача смешанная. Если переменные принимают значения только 0 или 1, задача называется булевой или бинарной. Целочисленные задачи сложнее обычного линейного программирования, потому что допустимое

множество становится дискретным. Решение линейной релаксации, где условие целочисленности временно удаляется, может оказаться дробным и потому недопустимым для исходной задачи. Метод ветвей и границ систематически перебирает области решений, но отсекает заведомо бесперспективные ветви. Идея метода ветвей и границ:

Исходная целочисленная задача

↓

Решить релаксацию без целочисленности

↓

Если решение целое → кандидат на оптимум

↓

Если решение дробное → разделить задачу на ветви

↓

Для каждой ветви вычислить границу

↓

Отсечь ветви, которые не могут улучшить текущий рекорд

↓

Продолжать до доказательства оптимальности

Для задачи минимизации нижняя граница получается из релаксации: если даже без целочисленных ограничений значение не лучше текущего целого решения, ветвь отбрасывается. Если в релаксации переменная $x_j = 2.6$, создаются две ветви:

$$x_j \leq 2,$$

и

$$x_j \geq 3.$$

Задача коммивояжера состоит в нахождении кратчайшего замкнутого маршрута, который проходит через каждый город ровно один раз и возвращается в начальный город. Пусть c_{ij} — расстояние из города i в город j , а переменная x_{ij} равна 1, если дуга (i, j) входит в маршрут, и 0 иначе. Тогда целевая функция:

$$\sum_i \sum_j c_{ij} x_{ij} \rightarrow \min.$$

Ограничения входа и выхода:

$$\sum_j x_{ij} = 1 \quad \forall i,$$

$$\sum_i x_{ij} = 1 \quad \forall j.$$

Дополнительно нужны ограничения, запрещающие подциклы, иначе можно получить несколько независимых циклов вместо одного маршрута. В методе ветвей и границ для TSP часто строят нижние оценки через редукцию матрицы расстояний: из каждой строки и каждого столбца вычитают минимальные элементы, а сумма вычтенных величин даёт нижнюю границу маршрута. **Пример 1. Простая целочисленная задача.** Рассмотрим задачу:

$$3x + 2y \rightarrow \max,$$

при условиях:

$$2x + y \leq 5,$$

$$x + 2y \leq 5,$$

$$x, y \geq 0, \quad x, y \in \mathbb{Z}.$$

Переберём допустимые целые точки. Если $x=0$, то $y \leq 2$, максимум даёт $(0, 2)$ со значением 4. Если $x=1$, то $y \leq 2$, точка $(1, 2)$ даёт 7. Если $x=2$, то ограничения дают $y \leq 1$, точка $(2, 1)$ даёт 8. Если $x=3$, первое ограничение нарушается. Оптимальное целочисленное решение: $(2, 1)$, значение 8. В релаксации максимум может достигаться в дробной точке, но целочисленность требует выбора ближайшего допустимого дискретного решения, а не простого округления. **Пример 2. Задача коммивояжера для четырёх городов.** Пусть матрица расстояний симметрична:

	A	B	C	D
A	-	10	15	20
B	10	-	35	25
C	15	35	-	30
D	20	25	30	-

Возможные маршруты с началом в A:

A-B-C-D-A: $10+35+30+20 = 95$
 A-B-D-C-A: $10+25+30+15 = 80$
 A-C-B-D-A: $15+35+25+20 = 95$
 A-C-D-B-A: $15+30+25+10 = 80$
 A-D-B-C-A: $20+25+35+15 = 95$
 A-D-C-B-A: $20+30+35+10 = 95$

Оптимальная длина равна 80. В методе ветвей и границ полный перебор заменяется деревом решений: например, ветвь A-B включается или исключается, затем для каждой ветви считается нижняя граница. Если нижняя граница ветви уже не меньше 80, ветвь можно отсечь, потому что она не даст лучший маршрут.

6. Принятие решений в условиях неопределённости: игры с природой. Основные критерии принятия решения

Принятие решений в условиях неопределённости означает, что лицо, принимающее решение, выбирает одну из альтернатив, но не знает, какое состояние внешней среды наступит. В отличие от риска, вероятности состояний природы неизвестны или не считаются надёжными. Такая модель называется игрой с природой. Природа не является разумным противником и не стремится специально ухудшить результат, но её состояние влияет на выигрыш. Модель задаётся матрицей выигрышей:

$$A = (a_{ij}),$$

где i — номер решения или стратегии лица, принимающего решение, j — состояние природы, a_{ij} — результат при выборе решения i и наступлении состояния j . Схема игры с природой:

Решения ЛПР: $A_1, A_2, \dots, A_m$

↓

Состояния природы: $S_1, S_2, \dots, S_n$

↓

Матрица выигрышей a_{ij}

↓

Выбор критерия: Вальд, максимакс, Лаплас, Сэвидж, Гурвиц

↓

Выбор альтернативы

Критерий Вальда является критерием крайнего пессимизма. Для каждой альтернативы берётся худший результат, затем выбирается альтернатива с наилучшим из худших результатов:

$$W_i = \min_j a_{ij},$$

$$i^* = \arg \max_i W_i.$$

Этот критерий подходит, когда ошибка опасна, а лицо, принимающее решение, осторожно. Он гарантирует наилучший результат в самом неблагоприятном состоянии. Критерий максимакса является критерием крайнего оптимизма. Для каждой альтернативы берётся лучший возможный результат, затем выбирается максимальный:

$$M_i = \max_j a_{ij},$$

$$i^* = \arg \max_i M_i.$$

Он подходит для склонного к риску поведения, но может привести к плохому результату при неблагоприятной природе. Критерий Лапласа основан на принципе недостаточного основания: если вероятности состояний неизвестны, они считаются равными. Для каждой альтернативы вычисляется средний выигрыш:

$$L_i = \frac{1}{n} \sum_{j=1}^n a_{ij},$$

и выбирается альтернатива с максимальным L_i . Критерий Сэвиджа использует не матрицу выигрышей, а матрицу сожалений или рисков. Сожаление показывает, сколько потеряно по сравнению с лучшим решением для данного состояния природы:

$$r_{ij} = \max_i a_{ij} - a_{ij}.$$

Затем для каждой альтернативы берётся максимальное сожаление:

$$S_i = \max_j r_{ij},$$

а выбирается альтернатива с минимальным максимальным сожалением:

$$i^* = \arg \min_i S_i.$$

Критерий Гурвица является компромиссом между оптимизмом и пессимизмом. Для каждой альтернативы вычисляется:

$$H_i = \alpha \max_j a_{ij} + (1 - \alpha) \min_j a_{ij}, \quad 0 \leq \alpha \leq 1.$$

Если $\alpha=0$, критерий совпадает с пессимистическим подходом. Если $\alpha=1$, он совпадает с оптимистическим подходом. На практике α задаёт степень оптимизма лица, принимающего решение. **Пример 1. Выбор стратегии по разным критериям.** Пусть есть три решения и три состояния природы:

	S1	S2	S3
A1	20	10	30
A2	15	25	20
A3	5	40	10

Критерий Вальда:

$$W_1 = 10, \quad W_2 = 15, \quad W_3 = 5.$$

Выбирается A2, так как худший результат у неё максимален. Критерий максимакса:

$$M_1 = 30, \quad M_2 = 25, \quad M_3 = 40.$$

Выбирается А3. Критерий Лапласа:

$$L_1 = \frac{20 + 10 + 30}{3} = 20,$$

$$L_2 = \frac{15 + 25 + 20}{3} = 20,$$

$$L_3 = \frac{5 + 40 + 10}{3} \approx 18.33.$$

По Лапласу решения А1 и А2 равноценны. **Пример 2. Критерии Сэвиджа и Гурвица.** Для той же матрицы найдём матрицу сожалений. В состоянии S1 максимум равен 20, в S2 максимум 40, в S3 максимум 30. Тогда:

	S1	S2	S3
A1	0	30	0
A2	5	15	10
A3	15	0	20

Максимальные сожаления:

$$S_1 = 30, \quad S_2 = 15, \quad S_3 = 20.$$

По критерию Сэвиджа выбирается А2, потому что её максимальное сожаление минимально. Для критерия Гурвица возьмём $\alpha=0.6$:

$$H_1 = 0.6 \cdot 30 + 0.4 \cdot 10 = 22,$$

$$H_2 = 0.6 \cdot 25 + 0.4 \cdot 15 = 21,$$

$$H_3 = 0.6 \cdot 40 + 0.4 \cdot 5 = 26.$$

По Гурвицу выбирается А3. Разные критерии дают разные решения, потому что отражают разные отношения к неопределённости.

7. Принятие решений в условиях неопределенности. Идеальный эксперимент и целесообразность его проведения

В задачах принятия решений часто возникает возможность провести эксперимент, исследование, тестирование или сбор дополнительной информации перед выбором решения. Эксперимент нужен для уменьшения неопределённости относительно состояния природы. Идеальный эксперимент — это эксперимент, который безошибочно сообщает, какое состояние природы наступит. Такая информация называется полной и совершенной. В реальности идеальные эксперименты встречаются редко, но их анализ важен, потому что он даёт верхнюю оценку полезности информации. Пусть есть решения A_i , состояния природы S_j , выигрыши a_{ij} и вероятности состояний p_j . Если вероятности известны, то для каждого решения вычисляется математическое ожидание выигрыша:

$$E_i = \sum_{j=1}^n p_j a_{ij}.$$

Лучшее решение без эксперимента имеет ожидаемый выигрыш:

$$E_0 = \max_i \sum_{j=1}^n p_j a_{ij}.$$

Если провести идеальный эксперимент, состояние природы станет известно до принятия решения. Тогда для каждого состояния можно выбрать лучшее решение именно для него. Ожидаемый выигрыш при полной информации:

$$E_{PI} = \sum_{j=1}^n p_j \max_i a_{ij}.$$

Ценность идеальной информации или ожидаемая ценность совершенной информации равна:

$$EVPI = E_{PI} - E_0.$$

Эксперимент целесообразен, если его стоимость C не превышает ожидаемую ценность информации:

$$C \leq EVPI.$$

Если $C > EVPI$, то даже идеальный эксперимент экономически невыгоден. Следовательно, любой реальный, менее точный эксперимент тем более не должен стоить дороже этой величины. Схема оценки идеального эксперимента:

Матрица выигрышей + вероятности состояний

↓

Расчёт лучшего решения без эксперимента E_0

↓

Расчёт выигрыша при полной информации E_{PI}

↓

$EVPI = E_{PI} - E_0$

↓

Сравнение $EVPI$ со стоимостью эксперимента C

↓

Проводить / не проводить

Важно отличать идеальный эксперимент от несовершенного. Идеальный эксперимент точно указывает состояние природы. Несовершенный эксперимент даёт сигнал, который связан с состоянием вероятностно. Тогда применяется формула Байеса: априорные вероятности состояний уточняются после получения результата эксперимента, превращаясь в апостериорные вероятности. Но даже в этом случае ценность несовершенной информации не может превышать $EVPI$. **Пример 1. Расчёт ценности полной информации.** Предприятие выбирает один из двух проектов. Возможны два состояния рынка: высокий спрос $S1$ с вероятностью 0.4 и низкий спрос $S2$ с вероятностью 0.6 .

	$S1$	$S2$
$A1$	100	20
$A2$	60	50

Без эксперимента математические ожидания:

$$E_1 = 0.4 \cdot 100 + 0.6 \cdot 20 = 40 + 12 = 52,$$

$$E_2 = 0.4 \cdot 60 + 0.6 \cdot 50 = 24 + 30 = 54.$$

Без эксперимента выбирается $A2$, ожидаемый выигрыш:

$$E_0 = 54.$$

При идеальном эксперименте в состоянии $S1$ выбирается $A1$, потому что $100 > 60$; в состоянии $S2$ выбирается $A2$, потому что $50 > 20$. Ожидаемый выигрыш при полной информации:

$$E_{PI} = 0.4 \cdot 100 + 0.6 \cdot 50 = 40 + 30 = 70.$$

Ценность идеальной информации:

$$EVPI = 70 - 54 = 16.$$

Если эксперимент стоит не больше 16 условных единиц, он может быть целесообразен. Если стоит дороже, проводить его невыгодно. **Пример 2. Проверка целесообразности эксперимента.** Пусть есть три решения и три состояния природы с вероятностями:

$$p_1 = 0.2, \quad p_2 = 0.5, \quad p_3 = 0.3.$$

Матрица выигрышей:

	S1	S2	S3
A1	40	30	10
A2	20	35	25
A3	10	20	50

Ожидания без эксперимента:

$$E_1 = 0.2 \cdot 40 + 0.5 \cdot 30 + 0.3 \cdot 10 = 8 + 15 + 3 = 26,$$

$$E_2 = 0.2 \cdot 20 + 0.5 \cdot 35 + 0.3 \cdot 25 = 4 + 17.5 + 7.5 = 29,$$

$$E_3 = 0.2 \cdot 10 + 0.5 \cdot 20 + 0.3 \cdot 50 = 2 + 10 + 15 = 27.$$

Лучшее решение без эксперимента — A2, поэтому $E_0=29$. При полной информации выбираем максимум в каждом столбце: для S1 максимум 40, для S2 максимум 35, для S3 максимум 50. Тогда:

$$E_{PI} = 0.2 \cdot 40 + 0.5 \cdot 35 + 0.3 \cdot 50 = 8 + 17.5 + 15 = 40.5.$$

$$EVPI = 40.5 - 29 = 11.5.$$

Если предлагаемый идеальный эксперимент стоит 9, его проведение целесообразно. Если стоимость равна 15, он нецелесообразен, потому что максимальная возможная выгода от информации меньше затрат.

8. Метод анализа иерархий и его использование для принятия решений в условиях определенности

Метод анализа иерархий — это метод многокритериального выбора, основанный на декомпозиции сложной задачи на иерархию целей, критериев и альтернатив. Он применяется в условиях определённости, когда альтернативы и критерии известны, а задача состоит в формализации предпочтений лица, принимающего решение. Иерархия обычно имеет три уровня:

Цель выбора

↓

Критерии и подкритерии

↓

Альтернативы

Например, цель — выбрать проект. Критерии — стоимость, срок, качество, риск. Альтернативы — проекты А, В, С. Основная процедура метода состоит в попарном сравнении элементов одного уровня относительно элемента вышестоящего уровня. Сравнения задаются по шкале относительной важности. В классическом варианте используется шкала Саати: 1 означает равную важность, 3 — умеренное превосходство, 5 — сильное, 7 —

очень сильное, 9 — абсолютное. Чётные значения 2, 4, 6, 8 используются как промежуточные. Матрица парных сравнений имеет вид:

$$A = (a_{ij}),$$

где a_{ij} показывает, во сколько раз элемент i предпочтительнее элемента j . Матрица обратно симметрична:

$$a_{ji} = \frac{1}{a_{ij}}, \quad a_{ii} = 1.$$

Если сравнения полностью согласованы, то выполняется:

$$a_{ij} = \frac{w_i}{w_j},$$

где w_i — вес элемента. На практике оценки человека не идеально согласованы, поэтому веса находят приближённо, например через нормировку столбцов и усреднение строк или через собственный вектор матрицы:

$$Aw = \lambda_{\max} w.$$

После нахождения локальных весов критериев и альтернатив строится глобальный приоритет альтернативы:

$$P(A_k) = \sum_{i=1}^m w_i p_{ki},$$

где w_i — вес критерия, p_{ki} — локальный приоритет альтернативы A_k по критерию i . Для проверки согласованности используется индекс согласованности:

$$CI = \frac{\lambda_{\max} - n}{n - 1}.$$

Отношение согласованности:

$$CR = \frac{CI}{RI},$$

где RI — средний случайный индекс для матрицы порядка n . Если CR достаточно мал, обычно меньше 0.1, суждения считаются приемлемо согласованными. Если CR велик, матрицу сравнений нужно пересмотреть. Алгоритм метода анализа иерархий:

1. Сформулировать цель.
2. Выделить критерии и альтернативы.
3. Построить иерархию.
4. Составить матрицы парных сравнений.
5. Найти веса критериев.
6. Найти веса альтернатив по каждому критерию.
7. Проверить согласованность.
8. Рассчитать глобальные приоритеты.
9. Выбрать альтернативу с максимальным приоритетом.

Пример 1. Вес критериев. Пусть выбирается ноутбук по трём критериям: цена $C1$, производительность $C2$, автономность $C3$. Лицо, принимающее решение, считает производительность умеренно важнее цены, цену умеренно важнее автономности, а производительность сильно важнее автономности. Матрица:

	$C1$	$C2$	$C3$
$C1$	1	1/3	3
$C2$	3	1	5
$C3$	1/3	1/5	1

Приближённо найдём веса через геометрические средние строк:

$$g_1 = (1 \cdot \frac{1}{3} \cdot 3)^{1/3} = 1,$$

$$g_2 = (3 \cdot 1 \cdot 5)^{1/3} \approx 2.466,$$

$$g_3 = (\frac{1}{3} \cdot \frac{1}{5} \cdot 1)^{1/3} \approx 0.405.$$

Сумма $g \approx 3.871$. Нормированные веса:

$$w_1 \approx 0.258, \quad w_2 \approx 0.637, \quad w_3 \approx 0.105.$$

Самым важным критерием является производительность. **Пример 2. Расчёт глобальных приоритетов.** Пусть есть две альтернативы: А и В. Веса критериев:

$$w_1 = 0.3, \quad w_2 = 0.5, \quad w_3 = 0.2.$$

Локальные приоритеты альтернатив:

	c1	c2	c3
A	0.7	0.4	0.6
B	0.3	0.6	0.4

Глобальный приоритет А:

$$P(A) = 0.3 \cdot 0.7 + 0.5 \cdot 0.4 + 0.2 \cdot 0.6 = 0.21 + 0.20 + 0.12 = 0.53.$$

Глобальный приоритет В:

$$P(B) = 0.3 \cdot 0.3 + 0.5 \cdot 0.6 + 0.2 \cdot 0.4 = 0.09 + 0.30 + 0.08 = 0.47.$$

По методу анализа иерархий выбирается альтернатива А, так как её суммарный приоритет выше.

9. Антагонистические матричные игры; методы решения игр фиксированного размера

Антагонистическая матричная игра — это игра двух игроков с нулевой суммой, в которой выигрыш одного игрока равен проигрышу другого. Игра задаётся матрицей выигрышей первого игрока:

$$A = (a_{ij}),$$

где i — стратегия первого игрока, j — стратегия второго игрока, a_{ij} — выигрыш первого игрока при выборе пары стратегий (i, j) . Второй игрок стремится минимизировать этот выигрыш. Если первый игрок выбирает строку i , он гарантирует себе не меньше, чем минимум по этой строке:

$$\alpha_i = \min_j a_{ij}.$$

Нижняя цена игры:

$$\alpha = \max_i \min_j a_{ij}.$$

Если второй игрок выбирает столбец j , он гарантирует, что выигрыш первого игрока не превысит максимум по этому столбцу:

$$\beta_j = \max_i a_{ij}.$$

Верхняя цена игры:

$$\beta = \min_j \max_i a_{ij}.$$

Всегда выполняется:

$$\alpha \leq \beta.$$

Если $\alpha = \beta$, игра имеет седловую точку и решается в чистых стратегиях. Значение $v = \alpha = \beta$ называется ценой игры. Если седловой точки нет, используются смешанные стратегии. Смешанная стратегия первого игрока — вероятностное распределение по строкам:

$$p = (p_1, p_2, \dots, p_m), \quad p_i \geq 0, \quad \sum_i p_i = 1.$$

Смешанная стратегия второго игрока:

$$q = (q_1, q_2, \dots, q_n), \quad q_j \geq 0, \quad \sum_j q_j = 1.$$

Ожидаемый выигрыш первого игрока:

$$E(p, q) = p^T A q.$$

Для игры 2×2 с матрицей

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

вероятность выбора первой строки первым игроком находится из условия безразличия второго игрока между столбцами:

$$pa + (1 - p)c = pb + (1 - p)d.$$

Отсюда:

$$p = \frac{d - c}{a - b - c + d}.$$

Вероятность выбора первого столбца вторым игроком находится из условия безразличия первого игрока между строками:

$$qa + (1 - q)b = qc + (1 - q)d.$$

Отсюда:

$$q = \frac{d - b}{a - b - c + d}.$$

Для игр $2 \times n$ используется графический метод: ожидаемый выигрыш каждой стратегии второго игрока изображается как линейная функция вероятности p выбора первой строки. Первый игрок выбирает p , максимизируя нижнюю огибающую этих прямых. Для игр $m \times 2$ аналогично строятся функции для второго игрока. Для игр фиксированного размера также применяют удаление доминируемых стратегий: если одна строка даёт не меньшие выигрыши, чем другая, и хотя бы где-то больший выигрыш, худшая строка удаляется. **Пример 1. Игра с седловой точкой.** Пусть матрица выигрышей первого игрока:

	B1	B2	B3
A1	3	4	2
A2	1	5	0
A3	2	3	1

Минимумы по строкам:

$$\min A_1 = 2, \quad \min A_2 = 0, \quad \min A_3 = 1.$$

Нижняя цена:

$$\alpha = \max\{2, 0, 1\} = 2.$$

Максимумы по столбцам:

$$\max B_1 = 3, \quad \max B_2 = 5, \quad \max B_3 = 2.$$

Верхняя цена:

$$\beta = \min\{3, 5, 2\} = 2.$$

Так как $\alpha = \beta = 2$, игра имеет седловую точку. Она находится в позиции (A1, B3). Оптимальная стратегия первого игрока — A1, второго — B3, цена игры равна 2. **Пример 2. Игра 2×2 без седловой точки.** Пусть

$$A = \begin{pmatrix} 2 & 6 \\ 5 & 3 \end{pmatrix}.$$

Минимумы строк:

$$\min A_1 = 2, \quad \min A_2 = 3, \quad \alpha = 3.$$

Максимумы столбцов:

$$\max B_1 = 5, \quad \max B_2 = 6, \quad \beta = 5.$$

Так как $\alpha < \beta$, седловой точки нет. Найдём смешанные стратегии. Для первого игрока:

$$p = \frac{d - c}{a - b - c + d} = \frac{3 - 5}{2 - 6 - 5 + 3} = \frac{-2}{-6} = \frac{1}{3}.$$

Значит первая строка выбирается с вероятностью 1/3, вторая — с вероятностью 2/3. Для второго игрока:

$$q = \frac{d - b}{a - b - c + d} = \frac{3 - 6}{2 - 6 - 5 + 3} = \frac{-3}{-6} = \frac{1}{2}.$$

Цена игры:

$$v = qa + (1 - q)b = \frac{1}{2} \cdot 2 + \frac{1}{2} \cdot 6 = 4.$$

Проверка по второй строке:

$$v = \frac{1}{2} \cdot 5 + \frac{1}{2} \cdot 3 = 4.$$

Оптимальная смешанная стратегия первого игрока: (1/3, 2/3), второго: (1/2, 1/2), цена игры равна 4.

10. Антагонистические матричные игры произвольной размерности и методы их решения

Антагонистическая матричная игра произвольной размерности задаётся матрицей выигрышей первого игрока размера $m \times n$:

$$A = (a_{ij}), \quad i = 1, \dots, m, \quad j = 1, \dots, n.$$

Первый игрок выбирает строку и стремится максимизировать выигрыш, второй выбирает столбец и стремится минимизировать выигрыш первого. Если седловой точки нет,

решение ищется в смешанных стратегиях. Смешанная стратегия первого игрока — вектор вероятностей:

$$p_i \geq 0, \quad \sum_{i=1}^m p_i = 1.$$

Смешанная стратегия второго игрока:

$$q_j \geq 0, \quad \sum_{j=1}^n q_j = 1.$$

Оптимальные стратегии удовлетворяют минимаксному принципу:

$$\max_p \min_j \sum_{i=1}^m p_i a_{ij} = \min_q \max_i \sum_{j=1}^n a_{ij} q_j = v,$$

где v — цена игры. Для решения игр произвольной размерности применяют несколько методов: поиск седловой точки, исключение доминируемых стратегий, сведение к линейному программированию, итерационные методы и приближённые методы. Доминирование позволяет уменьшить размерность игры. Для первого игрока строка r доминирует строку s , если:

$$a_{rj} \geq a_{sj} \quad \forall j,$$

и хотя бы для одного j неравенство строгое. Тогда строку s можно удалить, потому что она никогда не выгоднее. Для второго игрока столбец r доминирует столбец s , если первый игрок получает в столбце r не больше, чем в s , по всем строкам:

$$a_{ir} \leq a_{is} \quad \forall i.$$

Тогда второй игрок предпочитает столбец r , а столбец s можно удалить. Если все элементы матрицы положительны, игра может быть сведена к задаче линейного программирования. Если в матрице есть неположительные элементы, ко всем элементам можно прибавить константу K , чтобы сделать их положительными. Это изменит цену игры на K , но не изменит оптимальные стратегии. Для первого игрока задача может быть записана так:

$$\sum_{i=1}^m p_i a_{ij} \geq v, \quad j = 1, \dots, n,$$

$$\sum_{i=1}^m p_i = 1,$$

$$p_i \geq 0,$$

$$v \rightarrow \max.$$

Для второго игрока:

$$\sum_{j=1}^n a_{ij} q_j \leq v, \quad i = 1, \dots, m,$$

$$\sum_{j=1}^n q_j = 1,$$

$$q_j \geq 0,$$

$$v \rightarrow \min .$$

Через замену переменных можно получить стандартную форму ЛП. Для первого игрока при положительной цене игры вводят:

$$y_i = \frac{p_i}{v}.$$

Тогда задача сводится к минимизации суммы y_i при ограничениях:

$$\sum_{i=1}^m a_{ij} y_i \geq 1,$$

$$y_i \geq 0.$$

После решения:

$$v = \frac{1}{\sum_i y_i},$$

$$p_i = v y_i.$$

Схема решения произвольной матричной игры:

Матрица выигрышей A

↓

Проверка седловой точки

↓

Удаление доминируемых строк и столбцов

↓

Если размер стал 2×2, 2×n или m×2 → специальный метод

↓

Иначе сведение к линейному программированию

↓

Нахождение p, q и цены игры v

Пример 1. Удаление доминируемых стратегий. Пусть матрица выигрышей первого игрока:

	B1	B2	B3
A1	3	4	5
A2	2	3	4
A3	1	6	2

Сравним строки A1 и A2:

$$3 \geq 2, \quad 4 \geq 3, \quad 5 \geq 4.$$

Во всех столбцах A1 не хуже A2, а фактически строго лучше. Значит строка A2 доминируется строкой A1 и может быть удалена. Получаем сокращённую игру:

	B1	B2	B3
A1	3	4	5
A3	1	6	2

Дальше можно проверять доминирование столбцов или решать игру 2×3 графическим методом. **Пример 2. Сведение к линейному программированию.** Пусть матрица игры:

$$A = \begin{pmatrix} 2 & 4 & 3 \\ 5 & 1 & 2 \end{pmatrix}.$$

Пусть первый игрок выбирает первую строку с вероятностью p , вторую с вероятностью $1-p$. Ожидаемые выигрыши против столбцов:

$$g_1(p) = 2p + 5(1-p) = 5 - 3p,$$

$$g_2(p) = 4p + 1(1-p) = 1 + 3p,$$

$$g_3(p) = 3p + 2(1-p) = 2 + p.$$

Первый игрок максимизирует минимум из этих трёх выражений:

$$\max_p \min\{5 - 3p, 1 + 3p, 2 + p\}.$$

Пересечение первых двух функций:

$$5 - 3p = 1 + 3p \Rightarrow p = \frac{2}{3}.$$

Значения:

$$g_1 = 3, \quad g_2 = 3, \quad g_3 = 2 + \frac{2}{3} = \frac{8}{3}.$$

Минимум равен $8/3$, поэтому это не лучшая точка, если третья функция ниже. Пересечение второй и третьей:

$$1 + 3p = 2 + p \Rightarrow 2p = 1 \Rightarrow p = \frac{1}{2}.$$

Тогда:

$$g_1 = 5 - 1.5 = 3.5,$$

$$g_2 = 2.5,$$

$$g_3 = 2.5.$$

Минимум равен 2.5 . Пересечение первой и третьей:

$$5 - 3p = 2 + p \Rightarrow p = \frac{3}{4}.$$

Тогда минимум равен $g_2=3.25$, а $g_1=g_3=2.75$? Проверим: $g_1=2.75$, $g_2=3.25$, $g_3=2.75$, минимум 2.75 . Значит оптимально $p=3/4$, цена игры $v=2.75$. Этот пример показывает геометрический вариант решения игры $2 \times n$ как частный случай общего подхода.

11. Позиционные игры: разновидности и методы решения

Позиционная игра — это игра, в которой решения принимаются последовательно, а не одновременно. Такая игра представляется деревом. Вершины дерева называются позициями. Рёбра соответствуют ходам. Терминальные вершины содержат выигрыши игроков. Игрок, которому принадлежит текущая вершина, выбирает один из доступных ходов. Стратегия игрока в позиционной игре — это полный план действий, указывающий, какой ход делать в каждой позиции, где игрок должен принимать решение. Основные элементы позиционной игры:

Начальная позиция
 ↓
 Ходы игроков
 ↓
 Промежуточные позиции
 ↓
 Терминальные позиции
 ↓
 Выигрыши игроков

Позиционные игры различают по числу игроков, сумме выигрышей, наличию случайных ходов и полноте информации. Если каждый игрок в момент хода знает всю предшествующую историю игры, игра называется игрой с совершенной информацией. Шахматы, шашки и многие логические игры относятся к такому типу. Если игрок не различает некоторые позиции, игра имеет несовершенную информацию. Такие неразличимые для игрока позиции объединяются в информационное множество. Если в игре есть случайные ходы природы, например бросок кубика или случайное событие, игра называется стохастической. Для конечных игр с совершенной информацией основной метод решения — обратная индукция. Сначала анализируются последние ходы перед терминальными вершинами. В каждой такой позиции игрок выбирает ход, дающий ему лучший результат. Затем выбранные значения поднимаются на уровень выше, и процедура повторяется до начальной вершины. Результат обратной индукции даёт равновесное решение для конечной игры с полной информацией. Схема обратной индукции:

Терминальные выигрыши
 ↑
 Оптимальный выбор на последнем ходе
 ↑
 Оптимальный выбор на предпоследнем ходе
 ↑
 ...
 ↑
 Оптимальный выбор в начальной позиции

Для антагонистических позиционных игр используется минимаксный принцип. Если ход делает первый игрок, значение позиции равно максимуму значений дочерних позиций. Если ход делает второй игрок, значение позиции равно минимуму значений дочерних позиций:

$$V(s) = \max_{s' \in \text{Succ}(s)} V(s'),$$

если ход первого игрока, и

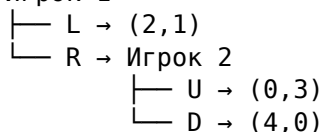
$$V(s) = \min_{s' \in \text{Succ}(s)} V(s'),$$

если ход второго игрока. Здесь $\text{Succ}(s)$ — множество позиций, достижимых из позиции s одним ходом. Для больших деревьев используется альфа-бета отсечение. Это улучшение минимаксного алгоритма, при котором не рассматриваются ветви, которые уже не могут повлиять на итоговый выбор. Значение α хранит лучшую уже найденную гарантию максимизирующего игрока, а β — лучшую гарантию минимизирующего игрока. Если становится ясно, что ветвь хуже уже найденного варианта, она отсекается. Позиционные игры бывают также кооперативными и некооперативными, с полной или неполной информацией, конечными и бесконечными, детерминированными и случайными. Метод решения зависит от структуры игры. Для конечных игр с совершенной информацией подходит обратная индукция. Для антагонистических игр — минимакс. Для игр с случайными ходами значения рассчитываются через математическое ожидание.

Пример 1. Обратная индукция в последовательной игре. Первый игрок выбирает

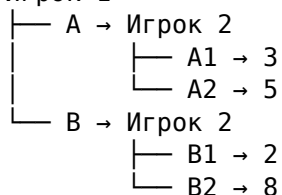
L или R. Если он выбрал L, игра заканчивается выигрышами $(2, 1)$. Если выбрал R, второй игрок выбирает U или D. При U выигрыши $(0, 3)$, при D — $(4, 0)$.

Игрок 1



Решаем с конца. Если второй игрок окажется в своей позиции, он сравнит свой выигрыш: при U он получает 3, при D — 0. Поэтому он выберет U. Первый игрок это предвидит. Если он выберет L, получит 2. Если выберет R, второй выберет U, и первый получит 0. Значит первый игрок выбирает L. Равновесный исход — $(2, 1)$. **Пример 2. Минимакс для антагонистической игры.** Пусть дерево игры имеет два хода. Первый игрок выбирает левую или правую ветвь. Затем второй игрок выбирает один из двух вариантов. Выигрыши первого игрока в терминальных вершинах:

Игрок 1



В позициях второго игрока берётся минимум, потому что второй игрок уменьшает выигрыш первого:

$$V(A) = \min\{3, 5\} = 3,$$

$$V(B) = \min\{2, 8\} = 2.$$

В начальной позиции первый игрок берёт максимум:

$$V = \max\{3, 2\} = 3.$$

Оптимальный ход первого игрока — A, цена игры в начальной позиции равна 3.

12. Кооперативные игры. Принципы рациональности и дележа

Кооперативная игра — это игра, в которой игроки могут образовывать коалиции, заключать соглашения и совместно распределять общий выигрыш. В отличие от некооперативной игры, основное внимание переносится с выбора индивидуальных стратегий на вопрос: какие коалиции образуются и как делится выигрыш между участниками. Кооперативная игра в характеристической форме задаётся множеством игроков:

$$N = \{1, 2, \dots, n\},$$

и характеристической функцией:

$$v(S), \quad S \subseteq N.$$

Значение $v(S)$ показывает, какой выигрыш может гарантировать коалиция S , действуя самостоятельно. Пустая коалиция обычно имеет нулевой выигрыш:

$$v(\emptyset) = 0.$$

Коалиция всех игроков называется большой коалицией N . Главная задача кооперативной игры — распределить выигрыш $v(N)$ между игроками. Вектор дележа:

$$x = (x_1, x_2, \dots, x_n),$$

где x_i — выплата игроку i . Делёж должен удовлетворять условию эффективности:

$$\sum_{i=1}^n x_i = v(N).$$

Индивидуальная рациональность означает, что каждый игрок должен получить не меньше, чем он может обеспечить себе один:

$$x_i \geq v(\{i\}).$$

Групповая рациональность означает, что любая коалиция должна получать не меньше, чем она может обеспечить себе самостоятельно:

$$\sum_{i \in S} x_i \geq v(S), \quad S \subseteq N.$$

Если делёж эффективен и индивидуально рационален, он называется иммутацией. Если дополнительно ни одна коалиция не может возразить против дележа, потому что получает не меньше своей самостоятельной ценности, такой делёж принадлежит ядру игры. Ядро — множество устойчивых дележей:

$$C(v) = \left\{ x : \sum_{i \in N} x_i = v(N), \sum_{i \in S} x_i \geq v(S) \forall S \subseteq N \right\}.$$

Ядро может быть пустым. Это означает, что невозможно распределить общий выигрыш так, чтобы ни одна коалиция не захотела отделиться. Ещё один важный принцип дележа — вектор Шепли. Он распределяет выигрыш по среднему предельному вкладу игрока во все возможные коалиции. Формула Шепли:

$$\varphi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{\|S\|!(n - \|S\| - 1)!}{n!} [v(S \cup \{i\}) - v(S)].$$

Здесь $v(S \cup \{i\}) - v(S)$ — вклад игрока i при присоединении к коалиции S . Коэффициент перед вкладом отражает вероятность того, что в случайном порядке формирования коалиции перед игроком i уже окажется именно множество S . Принципы рационального дележа:

Эффективность: весь выигрыш распределён

Индивидуальная рациональность: игроку выгодно участвовать

Групповая рациональность: коалициям невыгодно отделяться

Симметрия: равные игроки получают равные выплаты

Справедливость: больший вклад даёт большую долю

Устойчивость: нет коалиции, которая разрушает соглашение

Пример 1. Проверка дележа на рациональность. Пусть есть три игрока, большая коалиция получает:

$$v(N) = 100.$$

Одиночные выигрыши:

$$v(\{1\}) = 20, \quad v(\{2\}) = 15, \quad v(\{3\}) = 10.$$

Пары могут получить:

$$v(\{1, 2\}) = 50, \quad v(\{1, 3\}) = 60, \quad v(\{2, 3\}) = 40.$$

Рассмотрим делёж:

$$x = (40, 30, 30).$$

Эффективность:

$$40 + 30 + 30 = 100 = v(N).$$

Индивидуальная рациональность:

$$40 \geq 20, \quad 30 \geq 15, \quad 30 \geq 10.$$

Проверим пары:

$$x_1 + x_2 = 70 \geq 50,$$

$$x_1 + x_3 = 70 \geq 60,$$

$$x_2 + x_3 = 60 \geq 40.$$

Все условия выполнены, значит делёж принадлежит ядру и является устойчивым. **Пример 2. Расчёт вектора Шепли для трёх игроков.** Пусть:

$$v(\emptyset) = 0,$$

$$v(\{1\}) = 0, \quad v(\{2\}) = 0, \quad v(\{3\}) = 0,$$

$$v(\{1, 2\}) = 60, \quad v(\{1, 3\}) = 30, \quad v(\{2, 3\}) = 30,$$

$$v(\{1, 2, 3\}) = 90.$$

Для трёх игроков вклад одиночной пустой коалиции имеет вес $1/3$, вклад в коалицию из одного игрока — вес $1/6$ для каждой такой коалиции, вклад в коалицию из двух игроков — вес $1/3$. Для игрока 1:

$$\varphi_1 = \frac{1}{3}[v(\{1\}) - v(\emptyset)] + \frac{1}{6}[v(\{1, 2\}) - v(\{2\})] + \frac{1}{6}[v(\{1, 3\}) - v(\{3\})] + \frac{1}{3}[v(N) - v(\{2, 3\})].$$

$$\varphi_1 = \frac{1}{3} \cdot 0 + \frac{1}{6} \cdot 60 + \frac{1}{6} \cdot 30 + \frac{1}{3} \cdot (90 - 30) = 0 + 10 + 5 + 20 = 35.$$

Для игрока 2 аналогично:

$$\varphi_2 = \frac{1}{6} \cdot 60 + \frac{1}{6} \cdot 30 + \frac{1}{3} \cdot (90 - 30) = 10 + 5 + 20 = 35.$$

Для игрока 3:

$$\varphi_3 = \frac{1}{6} \cdot 30 + \frac{1}{6} \cdot 30 + \frac{1}{3} \cdot (90 - 60) = 5 + 5 + 20 = 30.$$

Вектор Шепли:

$$\varphi = (35, 35, 20).$$

Проверка эффективности:

$$35 + 35 + 20 = 90 = v(N).$$

Вектор Шепли показывает распределение выигрыша по среднему предельному вкладу игроков.

Contents

Базы данных	1
Использованные источники	1
1. Виды баз данных: реляционные, сетевые, иерархические. Примеры	2

Базы данных

Материал подготовлен по перечисленным учебным источникам, открытым конспектам ИТМО и официальной документации PostgreSQL, SQLite и Neo4j. Формат рассчитан на повторение перед письменным экзаменом: каждый билет дан связным ответом, с примерами, схемами, таблицами и SQL-фрагментами. В этой редакции акцент сделан на точных определениях, конкретных формулировках и проверяемых примерах.

> Как пользоваться этой версией. Текст ниже переработан под письменный экзамен: в каждом билете добавлен блок с точными определениями, формальными признаками и проверочными примерами. Если времени мало, сначала учи определения и примеры из блоков «Конкретика для ответа», затем дочитывай развернутое объяснение.

Использованные источники

1. Преснякова Г. В., Шахомиров А. В. *Проектирование реляционных баз данных*. СПб.: ГУАП, 2015.
2. Дейт К. *Введение в системы баз данных*. М.: Вильямс, 2018.
3. Советов Б. Я., Цехановский В. В., Чертовской В. Д. *Базы данных*. М.: Юрайт, 2015.
4. Лукин В. Н. *Введение в проектирование баз данных*. М.: Вузовская книга, 2015.
5. Аграновский А. В., Боженко В. В., Турнецкая Е. Л. *Разработка и администрирование баз данных с открытым исходным кодом*. СПб.: ГУАП, 2023.
6. Аграновский А. В., Боженко В. В., Турнецкая Е. Л. *Практическое применение реляционных баз данных с открытым исходным кодом*. СПб.: ГУАП, 2022.
7. База знаний ИТМО: https://neerc.ifmo.ru/wiki/index.php?title=Базы_данных
8. Григорьев Ю. А., Плутенко А. Д., Плужникова О. Ю. *Реляционные базы данных и системы NoSQL*. АмГУ, 2018.
9. PostgreSQL Documentation: <https://www.postgresql.org/docs/current/>
10. SQLite Documentation: <https://sqlite.org/docs.html>
11. Neo4j Documentation: <https://neo4j.com/docs/>

—

1. Виды баз данных: реляционные, сетевые, иерархические. Примеры

Конкретика для ответа

База данных — это именованная, структурированная и долговременно хранимая совокупность взаимосвязанных данных, предназначенная для многократного использования прикладными программами и пользователями. **Модель данных** — это способ описания структуры данных, допустимых связей между ними и операций над ними. Поэтому вопрос о видах БД надо раскрывать не как список названий, а как различие способов организации информации.

Иерархическая модель — модель, в которой записи организованы в дерево: у каждой дочерней записи существует не более одного непосредственного родителя. Формальный признак: связь между типами записей обычно имеет вид 1:N, а путь доступа к объекту задаётся маршрутом от корня дерева. Конкретный пример: база предприятия Компания → Отдел → Сотрудник. Запись сотрудника находится внутри отдела, отдел находится внутри компании. Если сотрудник одновременно участвует в нескольких проектах, иерархия начинает ломаться, потому что один объект приходится дублировать в разных ветвях.

Сетевая модель — модель, в которой запись может участвовать в нескольких связях и иметь несколько «родителей». Формальный признак: поддерживаются связи M:N без обязательного сведения всей структуры к дереву. Пример: Студент ↔ Дисциплина, где студент изучает много дисциплин, а дисциплина изучается многими студентами. В сетевой модели такая связь естественна, но программы часто зависят от конкретных путей перехода между записями.

Реляционная модель — модель, в которой данные представлены в виде отношений. В реализации СУБД отношение обычно выглядит как таблица: строки являются кортежами, столбцы — атрибутами, связи задаются первичными и внешними ключами. Пример: таблица `students(student\_id, full\_name, group\_id)` и таблица `groups(group\_id, group\_name)`. Значение `students.group\_id` ссылается на `groups.group\_id`, поэтому принадлежность студента группе задаётся не вложенностью, а совпадением значений ключей.

Экзамениционный пример можно оформить так:

```
|                               Предметная область |
↪ Иерархически |
↪ Сетево |                               Реляционно |
|:-----|:-----|:-----|:-----|:-----|:-----|
↪ -----|:-----|:-----|:-----|:-----|
↪ -----|:-----|:-----|:-----|:-----|
↪ -----|:-----|:-----|:-----|:-----|
|                               Учебный процесс |                               Выз →
↪ факультет → группа → студент |
↪ Студенты связаны с дисциплинами и преподавателями | `students`, `groups`,
↪ `disciplines`, `enrollments` |
|                               Каталог файлов |
↪ Диск → папка → файл |                               Файл
↪ может иметь ярлыки и связи с тегами | `files`, `folders`, `tags`,
↪ `file\_tags` |
|                               Продажи |
↪ Компания → склад → товар |
↪ Поставщик связан со многими товарами | `suppliers`,
↪ `products`, `orders` |
```

Мини-SQL для реляционного примера:

```
CREATE TABLE groups (  
    group_id integer PRIMARY KEY,  
    group_name text NOT NULL  
);  
  
CREATE TABLE students (  
    student_id integer PRIMARY KEY,  
    full_name text NOT NULL,  
    group_id integer NOT NULL REFERENCES groups(group_id)  
);
```

Главный вывод: иерархическая БД удобна для строгих деревьев, сетевая — для сложных навигационных связей, реляционная — для табличного представления данных с декларативными запросами и формальными ограничениями целостности. База данных представляет собой организованную совокупность данных, предназначенную для длительного хранения, поиска, изменения и совместного использования. Модель базы данных определяет, как именно представляются объекты предметной области и связи между ними. В классической теории обычно сравнивают иерархическую, сетевую и реляционную модели. Их удобно воспринимать как три уровня эволюции: дерево, сеть и таблицы. Дерево хорошо там, где структура подчинённая; сеть нужна, когда один объект участвует во множестве связей; реляционная модель отделяет логическое представление от физических путей доступа. Иерархическая база данных организует записи в виде дерева. Каждый дочерний объект имеет одного родителя, но родитель может иметь много потомков. Такой подход естественен для структур «один ко многим»: университет содержит факультеты, факультет содержит кафедры, кафедра содержит преподавателей. Достоинство иерархической модели — простая навигация по заранее известному пути и высокая скорость доступа при стабильной структуре. Недостаток — слабая поддержка связей «многие ко многим». Если студент посещает дисциплины разных кафедр, участвует в проектах и относится к нескольким классификациям, дерево начинает требовать дублирования или искусственных обходных ссылок.

```
graph TD  
    U[Университет] --> F1[Факультет ИТ]  
    F1 --> K1[Кафедра БД]  
    K1 --> T1[Преподаватель]  
    U --> F2[Экономический факультет]
```

Сетевая база данных расширяет иерархическую модель: запись может иметь несколько связей с другими записями. Она лучше подходит для отношений «многие ко многим»: студенты изучают много дисциплин, дисциплина изучается многими студентами; поставщики поставляют много товаров, один товар поставляется несколькими поставщиками. Сетевая модель выразительнее дерева, но часто требует навигационного доступа. Программа должна знать маршруты перехода между записями, поэтому логика приложения сильнее зависит от структуры хранения.

```
graph LR  
    S1[Студент Иванов] --> D1[Базы данных]  
    S1 --> D2[SQL]  
    S2[Студент Петров] --> D1  
    D1 --> P1[Преподаватель Сидоров]
```

Реляционная база данных представляет данные в виде отношений, которые в СУБД отображаются как таблицы. Строки соответствуют кортежам, столбцы — атрибутам, связи задаются значениями ключей. Например, таблица `students` хранит студентов, таблица `groups` хранит группы, а поле `students.group_id` ссылается на

groups.group_id. Пользователь описывает не путь обхода записей, а требуемый результат: SELECT full_name FROM students WHERE group_id = 101. СУБД сама выбирает план выполнения, индексы и методы доступа.

erDiagram

```
GROUPS ||--o{ STUDENTS : contains
GROUPS { int group_id PK string name }
STUDENTS { int student_id PK string full_name int group_id FK }
```

Примеры реляционных СУБД: PostgreSQL, MySQL, MariaDB, Oracle Database, Microsoft SQL Server, SQLite. SQLite при этом является встроенной SQL-СУБД: она хранит базу в файле и не требует отдельного серверного процесса. Реляционная модель стала основной благодаря строгой математической базе, нормализации, ключам, ограничениям целостности, транзакциям и SQL. На экзамене важно подчеркнуть: иерархическая модель удобна для дерева, сетевая — для сложной навигационной структуры, реляционная — для декларативной работы с таблицами и логическими связями.

— ## 2. Трехуровневая система организации базы данных ANSI. Логическая и физическая независимость данных

Конкретика для ответа

Трёхуровневая архитектура ANSI/SPARC — это архитектурная модель организации БД, в которой описание данных делится на три уровня: внешний, концептуальный и внутренний. Она нужна для обеспечения независимости данных, то есть возможности изменять один уровень без обязательного изменения других уровней.

Внешний уровень — совокупность пользовательских представлений данных. Он отвечает на вопрос: «Какие данные видит конкретный пользователь или приложение?». Например, преподаватель видит представление student_marks, бухгалтер — student_contracts, администратор — users_and_roles. Внешний уровень может скрывать конфиденциальные поля, объединять таблицы и показывать данные в удобном виде.

Концептуальный уровень — единая логическая схема всей базы данных. Он отвечает на вопрос: «Какие сущности, атрибуты, связи и ограничения существуют в предметной области?». Например: есть сущности Студент, Группа, Дисциплина, Оценка; студент принадлежит одной группе; оценка относится к одному студенту и одной дисциплине.

Внутренний уровень — физическое представление данных в памяти компьютера. Он отвечает на вопрос: «Как данные реально размещены и как к ним осуществляется доступ?». Здесь находятся файлы данных, страницы, индексы, методы хранения, кластеры, партиции, буферы.

graph TD

```
E1[Внешняя схема преподавателя: оценки] --> C[Концептуальная схема БД]
E2[Внешняя схема бухгалтерии: договоры] --> C
E3[Внешняя схема администратора: пользователи] --> C
C --> I[Внутренняя схема: файлы, страницы, индексы]
```

Логическая независимость данных — возможность изменить концептуальную схему без изменения внешних схем и прикладных программ, если их представление данных сохранено. Пример: таблицу students разделили на persons и students, но сохранили представление student_view. Приложение продолжает выполнять SELECT * FROM student_view, хотя внутренняя логическая структура изменилась.

Физическая независимость данных — возможность изменить внутреннее хранение без изменения концептуальной схемы и запросов пользователей. Пример: добавили индекс на students(group_id), перенесли таблицу в другой tablespace, изменили способ хранения или включили партиционирование. Запрос SELECT * FROM students WHERE group_id = 101 остался тем же.

SQL-пример внешнего представления:

```
CREATE VIEW student_progress AS
SELECT s.student_id, s.full_name, d.name AS discipline, m.mark
FROM students s
JOIN marks m ON m.student_id = s.student_id
JOIN disciplines d ON d.discipline_id = m.discipline_id;
```

Если позднее таблица marks будет физически индексована или логически разделена, но представление student_progress останется совместимым, прикладной код можно не менять. Это и есть практический смысл трёхуровневой архитектуры.

Трёхуровневая архитектура ANSI/SPARC описывает базу данных через внешний, концептуальный и внутренний уровни. Её задача — разделить пользовательские представления, общую логическую схему и физическое хранение. Такое разделение защищает приложения от изменений реализации и позволяет разным пользователям работать с одной базой через разные представления.

Внешний уровень — это уровень отдельных пользователей и приложений. Бухгалтер видит договоры и оплаты, преподаватель видит студентов и оценки, администратор видит пользователей и права. В SQL внешние представления часто реализуются через VIEW. Представление может объединять несколько таблиц и скрывать лишние поля. Например, student_progress может показывать ФИО студента, дисциплину и оценку, хотя физически данные лежат в таблицах студентов, дисциплин и ведомостей.

Концептуальный уровень описывает единую логическую схему всей базы: сущности, атрибуты, связи, ключи, ограничения целостности и правила предметной области. Для учебной базы это сущности «Студент», «Группа», «Дисциплина», «Преподаватель», «Оценка». На этом уровне важно, что студент принадлежит группе, оценка относится к студенту и дисциплине, группа имеет уникальный номер. Концептуальная схема не должна зависеть от того, на каком диске лежат файлы и какие индексы созданы.

Внутренний уровень описывает физическое хранение: файлы, страницы, индексы, методы доступа, журналы транзакций, буферизацию, партиционирование. Например, таблица заказов может храниться по разделам за разные годы, иметь B-tree индекс по дате и отдельный индекс по клиенту. Пользователь при этом продолжает работать с таблицей orders, не зная деталей размещения.

graph TD

```
E[Внешний уровень: представления пользователей] --> C[Концептуальный уровень:
  ↳ логическая схема]
C --> I[Внутренний уровень: файлы, страницы, индексы]
U1[Преподаватель] --> E
U2[Бухгалтер] --> E
U3[Администратор] --> E
```

Физическая независимость данных означает, что изменения внутреннего уровня не требуют изменения логической схемы и программ. Можно создать индекс, перенести таблицу на другой диск, изменить параметры хранения или выполнить кластеризацию. Запрос `SELECT * FROM students WHERE group\_id = 101` остаётся тем же, хотя СУБД после добавления индекса выполнит его быстрее.

Логическая независимость данных означает возможность изменять концептуальную схему без обязательного изменения внешних представлений. Например, таблицу students можно разделить на persons и student_profiles, а для старого приложения оставить представление students. Логическая независимость сложнее физической, потому что изменения логики затрагивают смысл данных. Тем не менее представления, совместимые миграции и API помогают сохранить старые интерфейсы.

Пример физической независимости: добавлен индекс `CREATE INDEX idx\_students\_group ON students(group\_id);`, но приложение не изменилось.

Пример логической независимости: адрес студента вынесен из таблицы студентов в `student_addresses`, а старый отчёт получает прежний вид через VIEW. Главная идея ANSI-архитектуры: пользователь работает с логической моделью, а детали хранения и часть изменений схемы скрываются за слоями абстракции.

— ## 3. Основные требования к системе управления базами данных

Конкретика для ответа

Система управления базами данных (СУБД) — это программная система, предназначенная для определения структуры базы данных, хранения, поиска, изменения, защиты данных и управления одновременным доступом пользователей. Требования к СУБД удобно раскрывать через конкретные функции, которые она обязана выполнять.

1. **Определение данных.** СУБД должна позволять создавать структуры хранения: таблицы, представления, индексы, ограничения, схемы. В SQL это реализуется командами DDL: `CREATE TABLE`, `ALTER TABLE`, `DROP INDEX`.

2. **Манипулирование данными.** СУБД должна обеспечивать добавление, изменение, удаление и выборку данных. В SQL это `INSERT`, `UPDATE`, `DELETE`, `SELECT`. Пример: `UPDATE accounts SET balance = balance - 100 WHERE account_id = 1`.

3. **Целостность данных.** СУБД должна не просто хранить значения, а не допускать противоречивых состояний. Для этого применяются `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL`, `CHECK`. Например, оценка студента не должна быть равна 99, если допустимы значения от 2 до 5.

4. **Транзакционность.** СУБД должна поддерживать ACID-свойства: атомарность, согласованность, изолированность и долговечность. Денежный перевод должен либо полностью списать и зачислить сумму, либо не выполнить ни одного действия.

5. **Многопользовательский доступ.** СУБД должна корректно работать, когда несколько пользователей одновременно читают и изменяют данные. Для этого используются блокировки, MVCC, уровни изоляции транзакций.

6. **Безопасность.** СУБД должна разграничивать права доступа: кто может читать, кто может изменять, кто может создавать таблицы, кто может выдавать права. Пример: аналитик имеет `SELECT`, но не имеет `DELETE`.

7. **Восстановление после сбоев.** СУБД должна сохранять данные при отказах питания, ошибках приложений и сбоях сервера. Для этого используются журнал транзакций, резервное копирование, контрольные точки, восстановление из WAL.

8. **Эффективность доступа.** СУБД должна обеспечивать приемлемое время ответа за счёт оптимизатора запросов, индексов, статистики, кэширования и выбора плана выполнения.

9. **Независимость данных.** Пользователь должен работать с логической схемой, а не с физическими файлами. Добавление индекса не должно требовать переписывания всех запросов.

Мини-пример требований в одной таблице:

	Требование	Механизм СУБД
↪	Конкретный пример	
	:-----	:-----
↪	-----	-----
↪	-----	
	Целостность	`CHECK`, `FOREIGN KEY`
↪	нельзя записать оценку 10 по пятибалльной шкале	
	Безопасность	роли и привилегии
↪	`GRANT SELECT ON students TO analyst`	

	Надёжность	транзакции и журнал
↪	после сбоя перевод не остаётся «наполовину»	
	Производительность	индексы
↪	быстрый поиск студентов по группе	

На экзамене важно не перечислять требования сухим списком, а показать, что СУБД является посредником между пользователем и физическими данными: она управляет структурой, доступом, согласованностью, безопасностью и восстановлением.

СУБД — программный комплекс, который создаёт, хранит, изменяет, защищает и предоставляет данные пользователям и приложениям. Основные требования к СУБД вытекают из того, что база данных обычно является общим, долговременным и критически важным ресурсом. СУБД должна не просто записывать строки, а обеспечивать корректность, независимость, безопасность, многопользовательскую работу и восстановление после сбоев.

Первое требование — поддержка модели данных и средств описания структуры. Реляционная СУБД должна позволять создавать таблицы, атрибуты, типы, ключи, связи, индексы, представления и ограничения. Для этого используются команды DDL: CREATE, ALTER, DROP. Структура должна храниться в системном каталоге, чтобы СУБД могла проверять операции и чтобы пользователи могли получать метаданные.

Второе требование — обеспечение целостности. СУБД должна контролировать первичные ключи, внешние ключи, уникальность, обязательность значений, доменные ограничения и бизнес-правила. Если задан внешний ключ, нельзя вставить студента в несуществующую группу. Если задано CHECK (grade BETWEEN 2 AND 5), нельзя поставить оценку 9. Ограничения должны работать независимо от приложения, потому что к базе могут подключаться разные клиенты.

Третье требование — независимость данных. Пользователь и приложение должны работать с логической схемой, а не с физическими адресами и файлами. Добавление индекса, изменение способа хранения или перенос таблицы не должны ломать запросы. Это снижает стоимость сопровождения и позволяет администратору оптимизировать систему без переписывания бизнес-логики.

Четвёртое требование — эффективная обработка запросов. СУБД должна иметь оптимизатор, статистику, индексы, буферизацию и методы доступа. Запросы соединения, группировки, сортировки и фильтрации должны выполняться с приемлемой скоростью. Индекс ускоряет поиск, но СУБД сама решает, когда его выгодно использовать.

Пятое требование — транзакционность и восстановление. Транзакции должны обладать свойствами ACID: атомарностью, согласованностью, изолированностью и долговечностью. Если перевод денег состоит из списания и зачисления, обе операции должны выполняться вместе или вместе отмениться. Журналы транзакций и контрольные точки позволяют восстановить базу после сбоя.

Шестое требование — многопользовательский доступ. СУБД должна корректно обслуживать параллельные транзакции, предотвращать потерянные обновления, грязные чтения и другие аномалии. Для этого используются блокировки, MVCC и уровни изоляции.

Седьмое требование — безопасность. СУБД должна поддерживать аутентификацию, роли, права, GRANT, REVOKE, аудит, шифрование соединений и резервных копий. Пользователь должен получать только необходимые права, а не полный доступ ко всем таблицам.

Восьмое требование — администрирование: резервное копирование, восстановление, мониторинг, анализ планов запросов, миграции схемы, репликация, журналирование. Хорошая СУБД должна обслуживать полный жизненный цикл данных, а не только

текущий запрос.

graph TD

```
DBMS[Требования к СУБД] --> M[Модель и язык данных]
DBMS --> I[Целостность]
DBMS --> P[Производительность]
DBMS --> T[Транзакции]
DBMS --> S[Безопасность]
DBMS --> A[Администрирование]
```

Итак, СУБД должна быть одновременно хранилищем, контролёром правил, диспетчером параллельной работы, механизмом восстановления и охранной системой. Без этих требований база быстро превращается в файл с таблицами, а не в управляемую информационную систему.

— ## 4. Терминология реляционных баз данных: отношение, сущность, атрибуты, домен, ключ

Конкретика для ответа

Отношение в реляционной модели — это множество кортежей одинаковой структуры. В практической SQL-СУБД отношение обычно соответствует таблице. Например, `students(student\_id, full\_name, birth\_date, group\_id)` — отношение, описывающее студентов. Строка таблицы — это **кортеж**, а столбец — **атрибут**.

Сущность — объект предметной области, о котором нужно хранить данные. Сущностью может быть студент, преподаватель, группа, заказ, товар, дисциплина. Важно различать сущность и отношение: сущность относится к предметной области, а отношение является способом её логического представления в БД.

Атрибут — именованное свойство сущности или отношения. Например, у сущности Студент атрибутами являются `student\_id`, `full\_name`, `birth\_date`, `group\_id`. Атрибут имеет имя, тип данных и множество допустимых значений.

Домен — множество допустимых значений атрибута. Например, домен атрибута `mark` может быть `{2, 3, 4, 5}`, домен `birth\_date` — множество корректных календарных дат, домен `email` — строки определённого формата. В SQL домен частично задаётся типом данных и ограничениями: `integer CHECK (mark BETWEEN 2 AND 5)`.

Ключ — атрибут или набор атрибутов, позволяющий однозначно идентифицировать кортеж отношения. **Суперключ** однозначно идентифицирует строку, но может содержать лишние атрибуты. **Потенциальный ключ** минимален: из него нельзя удалить атрибут без потери уникальности. **Первичный ключ** — выбранный потенциальный ключ. **Внешний ключ** — атрибут, значения которого ссылаются на первичный или уникальный ключ другого отношения.

Пример:

```
CREATE TABLE students (
  student_id integer PRIMARY KEY,
  full_name text NOT NULL,
  email text UNIQUE,
  group_id integer REFERENCES groups(group_id),
  mark integer CHECK (mark BETWEEN 2 AND 5)
);
```

В этом примере:

	Термин	Конкретный
↪ элемент		
:----- :-----		
↪ -----		
	Отношение	
↪ `students`		

	Кортеж	одна строка таблицы
↪ студентов		
	Атрибут	`full\_name`, `email`,
↪ `group\_id`		
	Домен	допустимые значения `mark`:
↪ от 2 до 5		
	Первичный ключ	
↪ `student\_id`		
	Внешний ключ	
↪ `group\_id`		
	Уникальный ключ	
↪ `email`		

Схема связи:

```
erDiagram
    GROUPS ||--o{ STUDENTS : includes
    GROUPS { int group_id PK string group_name }
    STUDENTS { int student_id PK string full_name string email UK int group_id FK }
```

Формулировка для экзамена: реляционная терминология связывает предметную область и формальную модель. Сущность описывает объект реального мира, отношение задаёт табличную структуру, атрибут описывает свойство, домен ограничивает допустимые значения, а ключи обеспечивают идентификацию строк и связи между таблицами.

Реляционная модель опирается на строгую терминологию. Отношение — это математическая структура, которая в практической СУБД обычно представляется таблицей. Отношение имеет заголовок, где перечислены атрибуты и их домены, и тело, состоящее из кортежей. В SQL кортеж соответствует строке, атрибут — столбцу, домен — допустимому множеству значений.

Сущность — объект предметной области, о котором требуется хранить данные: студент, группа, дисциплина, заказ, товар, клиент. Сущность имеет свойства, которые в реляционной схеме становятся атрибутами. Например, сущность «Студент» имеет `student\_id`, `full\_name`, `birth\_date`, `group\_id`.

Атрибут — именованная характеристика сущности или отношения. Домен задаёт допустимые значения атрибута. Для `birth\_date` доменом являются даты, для `grade` — числа от 2 до 5, для `email` — строки определённого формата. Домен шире, чем технический тип: тип `integer` допускает много чисел, но домен «оценка» ограничивает их до 2, 3, 4, 5.

Ключ — атрибут или набор атрибутов, идентифицирующий строки. Потенциальный ключ минимально уникален. Первичный ключ выбирается как основной идентификатор. Внешний ключ ссылается на ключ другой таблицы и задаёт связь между отношениями. Составной ключ состоит из нескольких атрибутов, например (`student\_id`, `discipline\_id`) в таблице результатов.

```
CREATE TABLE groups (
    group_id integer PRIMARY KEY,
    group_name text NOT NULL UNIQUE
);
CREATE TABLE students (
    student_id integer PRIMARY KEY,
    full_name text NOT NULL,
    group_id integer NOT NULL REFERENCES groups(group_id)
);
```

erDiagram

```
GROUPS ||--o{ STUDENTS : contains
GROUPS { int group_id PK string group_name }
STUDENTS { int student_id PK string full_name int group_id FK }
```

Пример отношения STUDENTS:

student_id	full_name	birth_date	group_id
1	Иванов И. И.	2004-03-10	101
2	Петров П. П.	2003-11-02	101
3	Сидорова А. В.	2004-07-22	102

Здесь STUDENTS — отношение, строки — кортежи, student_id, full_name, birth_date, group_id — атрибуты, student_id — первичный ключ, group_id — внешний ключ. На экзамене важно показать связь понятий: сущность предметной области описывается атрибутами; атрибуты имеют домены; множество однотипных сущностей хранится как отношение; ключи обеспечивают идентификацию и связи.

— ## 5. Правила реляционной модели: с нулевого по пятое правило

Конкретика для ответа

Правила Кодда формулируют требования к настоящей реляционной СУБД. Нулевое правило задаёт общий критерий, а правила 1–5 раскрывают базовые свойства реляционного представления, доступа и обработки неопределённых значений.

Правило 0. Основное правило. Система считается реляционной СУБД только тогда, когда она управляет базой данных исключительно с помощью реляционных возможностей. Иначе говоря, таблицы, ключи, ограничения и операции над отношениями должны быть не внешней оболочкой, а основой системы. Если продукт показывает данные в таблицах, но требует навигации по физическим указателям, это ещё не полноценная реляционная СУБД.

Правило 1. Информационное правило. Вся информация в реляционной БД должна быть представлена только в виде значений в таблицах. Это относится и к пользовательским данным, и к метаданным. Пример: имя студента, номер группы, оценка, название таблицы и описание столбца должны быть представлены как значения, доступные через табличные структуры.

Правило 2. Гарантированный доступ. Каждое атомарное значение должно быть доступно по имени таблицы, значению первичного ключа строки и имени столбца. Например, оценку студента Иванова по дисциплине можно получить, зная таблицу, идентификатор строки и атрибут mark. Это исключает зависимость от физического адреса записи в файле.

Правило 3. Систематическая обработка NULL. СУБД должна поддерживать специальное значение NULL для отсутствующих или неизвестных данных. NULL не равен нулю и не равен пустой строке. Пример: если дата отчисления студента неизвестна или студент не отчислен, поле expelled_at может быть NULL. В SQL выражение expelled_at = NULL неверно, надо писать expelled_at IS NULL.

Правило 4. Активный реляционный каталог. Описание структуры базы данных должно храниться в самой базе в виде таблиц системного каталога и быть доступно реляционными средствами. Например, в PostgreSQL сведения о таблицах и столбцах можно получать через information_schema.tables и information_schema.columns.

```
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public';
```

Правило 5. Исчерпывающий подязык данных. СУБД должна иметь язык, который поддерживает определение данных, выборку, изменение, ограничения

целостности, представления, транзакции и права доступа. В современных СУБД эту роль выполняет SQL: CREATE TABLE, SELECT, INSERT, UPDATE, DELETE, COMMIT, GRANT.

Краткая таблица для запоминания:

Правило	Пример
Смысл	
0	Система управляет данными не через физические
1	Все данные представлены таблицами пользовательские данные и метаданные
2	Доступ по таблице, ключу и `students` + `student\_id` + `full\_name`
3	`NULL` обрабатывается `IS NULL`, трёхзначная логика
4	Каталог тоже реляционный `information\_schema`
5	Есть полноценный язык SQL как DDL/DML/DCL/TCL

На экзамене важно не просто назвать правила, а показать, что они защищают пользователя от физической зависимости и делают базу данных логической табличной системой.

Правила Кодда описывают идеальные требования к реляционной СУБД. Нулевое правило утверждает: система, называющая себя реляционной, должна управлять базой данных именно через реляционные возможности. Нельзя быть реляционной только на вывеске, а внутри требовать навигации по физическим указателям.

Первое правило — правило информации. Вся информация в БД должна быть представлена на логическом уровне одним способом: значениями в таблицах. Это относится не только к пользовательским данным, но и к метаданным. В современных СУБД сведения о таблицах, столбцах, индексах и ограничениях доступны через системные каталоги и `information\_schema`.

Второе правило — гарантированный доступ. Каждое атомарное значение должно быть доступно через имя таблицы, значение первичного ключа строки и имя столбца. Пользователь обращается к данным логически, а не через адрес страницы на диске.

Третье правило — систематическая обработка неопределённых значений. СУБД должна поддерживать NULL как специальное значение для неизвестной, отсутствующей или неприменимой информации. NULL не равен нулю или пустой строке. В SQL для него используются IS NULL и трёхзначная логика.

Четвёртое правило — активный динамический каталог на основе реляционной модели. Описание базы должно храниться как данные и быть доступно тем же языком, что и обычные таблицы. Это делает структуру БД управляемой и проверяемой.

Пятое правило — полный подязык данных. Реляционная СУБД должна иметь язык, который поддерживает определение данных, представления, манипулирование данными, ограничения целостности, авторизацию и транзакции. В современных системах эту роль выполняет SQL.

graph TD

```
R0[0. Реляционное управление] --> R1[1. Информация в таблицах]
R1 --> R2[2. Гарантированный доступ]
```

R1 --> R4[4. Реляционный каталог]
R2 --> R3[3. NULL]
R4 --> R5[5. Полный язык данных]

Смысл первых правил — сформировать фундамент: данные и метаданные должны быть представлены реляционно, доступ к значениям должен быть логическим, отсутствующие значения должны обрабатываться единообразно, а язык должен покрывать полный цикл работы с БД.

— ## 6. Правила реляционной модели: с шестого по двенадцатое правило

Конкретика для ответа

Правила Кодда 6–12 продолжают идею: реляционная СУБД должна обеспечивать работу с представлениями, множественными операциями, независимостью данных и ограничениями целостности.

Правило 6. Обновляемость представлений. Если представление теоретически может быть обновлено без неоднозначности, СУБД должна позволять его обновлять. Например, простое представление `active\_students`, построенное из одной таблицы `students`, может быть обновляемым. Но представление с группировкой `COUNT(\*)` обычно нельзя обновить напрямую, потому что непонятно, какие исходные строки надо изменить.

```
CREATE VIEW active_students AS  
SELECT student_id, full_name, group_id  
FROM students  
WHERE expelled_at IS NULL;
```

Правило 7. Высокоуровневые операции вставки, обновления и удаления. СУБД должна поддерживать операции над множествами строк, а не только построчную навигацию. Пример: `UPDATE students SET status = 'archived' WHERE graduated\_at < DATE '2025-01-01'`; изменяет сразу множество записей.

Правило 8. Физическая независимость данных. Изменение физического хранения не должно ломать приложения. Например, добавление индекса, изменение файловой организации или перенос таблицы в другой раздел диска не должны менять SQL-запросы.

Правило 9. Логическая независимость данных. Изменение логической схемы не должно ломать приложения, если их внешнее представление можно сохранить. Например, таблицу `students` разделили на `persons` и `student\_profiles`, но создали представление `students\_old\_api`, совместимое со старым приложением.

Правило 10. Независимость ограничений целостности. Ограничения должны задаваться на уровне БД и храниться в каталоге, а не быть зашитыми только в код приложения. Пример: `CHECK (mark BETWEEN 2 AND 5)` гарантирует корректность оценки независимо от того, из какого приложения пришла запись.

Правило 11. Независимость распределения. Пользователь не должен зависеть от того, где физически расположены данные. Если часть таблиц находится на другом сервере или в другой партиции, логический запрос должен оставаться тем же, насколько это поддерживается СУБД.

Правило 12. Правило неподрыва. Низкоуровневые средства доступа не должны позволять обходить ограничения реляционной модели. Если у СУБД есть специальные интерфейсы, они не должны давать возможность нарушить ключи, ограничения и целостность.

Сводная таблица:

	Правило	
↪	Проверочный вопрос	
↪	Пример	

:----- :-----	
↪ ----- :-----	
↪ -----	
6	Можно ли
↪ обновлять корректные представления?	
↪ `UPDATE active\_students ...`	
7	Можно ли менять
↪ множество строк одной командой?	
↪ `UPDATE ... WHERE ...`	
8	Не
↪ ломается ли код при добавлении индекса?	
↪ добавили B-tree индекс	
9	Не ломается ли
↪ внешний интерфейс при изменении схемы?	
↪ сохранили совместимое `VIEW`	
10	
↪ Хранятся ли ограничения в БД?	
↪ `CHECK`, `FK`, `UNIQUE`	
11	
↪ Скрыто ли распределение данных?	удалённая
↪ таблица как логический объект	
12	Нельзя ли
↪ обойти целостность низкоуровнево?	
↪ прямой API не должен нарушать `FK`	

Для письменного ответа можно сделать вывод: правила 6–12 описывают зрелость СУБД. Система должна не только хранить таблицы, но и поддерживать представления, массовые операции, независимость от физической организации, централизованную целостность и защиту от обхода реляционных механизмов.

Шестое правило Кодда — обновляемость представлений. Все представления, которые теоретически можно обновлять без неоднозначности, должны быть обновляемыми. Например, представление активных студентов на основе одной таблицы может поддерживать UPDATE, но агрегированное представление с GROUP BY обычно неоднозначно.

Седьмое правило — высокоуровневые операции вставки, обновления и удаления. Реляционная система должна работать с множествами строк, а не заставлять программу изменять записи по одной. Команда UPDATE orders SET status='archived' WHERE created_at < '2024-01-01' изменяет множество строк одной операцией.

Восьмое правило — физическая независимость данных. Изменение индексов, файлов, страниц, размещения таблиц или параметров хранения не должно менять приложения.

Девятое правило — логическая независимость данных. Изменения логической схемы, не разрушающие внешние представления, не должны ломать программы. Например, таблицу можно разделить на две, а старое приложение оставить работать через VIEW.

Десятое правило — независимость целостности. Ограничения должны храниться в самой базе и задаваться языком данных, а не только кодом приложения. Одиннадцатое правило — независимость распределения: пользователь должен видеть единую логическую БД независимо от того, распределены данные физически или нет.

Двенадцатое правило — запрет обхода: низкоуровневый интерфейс не должен позволять нарушать реляционные ограничения и права.

graph TD

R6[6. Обновляемые VIEW] --> R7[7. Операции над множествами]

R8[8. Физическая независимость] --> N[Независимость приложений]

R9[9. Логическая независимость] --> N
 R10[10. Ограничения в БД] --> C[Целостность]
 R11[11. Распределённость скрыта] --> D[Единая логическая БД]
 R12[12. Нельзя обойти правила] --> S[Защита модели]

Эти правила защищают реляционную модель от возврата к построчной навигации, зависимости программ от хранения и размывания целостности. Они показывают, почему ограничения, представления, права и транзакции должны быть частью самой СУБД.

— ## 7. Операции реляционной алгебры: объединение, пересечение и разность

Конкретика для ответа

Операции **объединения**, **пересечения** и **разности** относятся к теоретико-множественным операциям реляционной алгебры. Они применимы только к **совместимым по объединению отношениям**. Это значит, что у отношений одинаковое число атрибутов, а соответствующие атрибуты имеют совместимые домены. Например, можно объединять таблицы студентов дневной и заочной формы, если обе имеют столбцы (student_id, full_name). Нельзя напрямую объединять (student_id, full_name) и (discipline_id, hours, semester).

Пусть есть отношения:

A = students_db

student_id	full_name
1	Иванов
2	Петров
3	Сидоров

B = students_python

student_id	full_name
2	Петров
3	Сидоров
4	Смирнова

Объединение A $\cup$ B возвращает все кортежи, которые входят хотя бы в одно из отношений. Дубликаты в классической реляционной алгебре устраняются.

student_id	full_name
1	Иванов
2	Петров
3	Сидоров
4	Смирнова

SQL-аналог:

```
SELECT student_id, full_name FROM students_db
UNION
SELECT student_id, full_name FROM students_python;
```

Пересечение A $\cap$ B возвращает кортежи, которые входят одновременно в оба отношения.

student_id	full_name
2	Петров
3	Сидоров

SQL-аналог:


```
SELECT student_id, full_name FROM students_db
INTERSECT
SELECT student_id, full_name FROM students_python;
```

Разность $A - B$ возвращает кортежи, которые входят в A , но не входят в B .

student_id	full_name
1	Иванов

SQL-аналог:

```
SELECT student_id, full_name FROM students_db
EXCEPT
SELECT student_id, full_name FROM students_python;
```

Диаграммы Венна можно описать так:

```
flowchart LR
    U[Объединение  $A \cup B$ : вся область  $A$  и  $B$ ]
    I[Пересечение  $A \cap B$ : только общая часть]
    D[Разность  $A - B$ : часть  $A$  вне  $B$ ]
```

Важная экзаменационная деталь: $A - B$ и $B - A$ обычно дают разные результаты, то есть разность не коммутативна. Объединение и пересечение коммутативны: $A \cup B = B \cup A$, $A \cap B = B \cap A$. Практический смысл операций: объединение собирает данные из нескольких источников, пересечение ищет общие элементы, разность выявляет отсутствующие или несовпадающие записи.

Реляционная алгебра — формальный язык операций над отношениями. Объединение, пересечение и разность относятся к операциям над множествами. Они применимы только к совместимым отношениям: у них должно быть одинаковое количество атрибутов и совместимые домены соответствующих столбцов.

Объединение $R \cup S$ возвращает все кортежи, которые есть хотя бы в одном отношении. В классической алгебре дубликаты не сохраняются. В SQL аналог — UNION, а UNION ALL сохраняет повторы.

$R \cup S = \{ t \mid t \in R \text{ или } t \in S \}$

Пересечение $R \cap S$ возвращает только кортежи, присутствующие одновременно в R и S .

$R \cap S = \{ t \mid t \in R \text{ и } t \in S \}$

Разность $R - S$ возвращает кортежи, которые есть в R , но отсутствуют в S . Порядок важен: $R - S$ не равно $S - R$.

$R - S = \{ t \mid t \in R \text{ и } t \notin S \}$

Пусть R — студенты курса «Базы данных», S — студенты курса «SQL».

R :

id	name
1	Иванов
2	Петров
3	Сидорова

S :

id	name
2	Петров
3	Сидорова
4	Кузнецова

$R \cup S$ даёт Иванова, Петрова, Сидорову, Кузнецову. $R \cap S$ даёт Петрова и Сидорову. $R - S$ даёт Иванова. $S - R$ даёт Кузнецову.

```
SELECT id, name FROM database_course
UNION
SELECT id, name FROM sql_course;
```

```
SELECT id, name FROM database_course
INTERSECT
SELECT id, name FROM sql_course;
```

```
SELECT id, name FROM database_course
EXCEPT
SELECT id, name FROM sql_course;
```

Диаграммы Венна: объединение закрашивает оба круга, пересечение — общую часть, разность — часть первого круга вне второго.

Объединение: $R + S$ целиком

Пересечение: только $R \cap S$

Разность: только R без области S

Практически эти операции применяются для объединения списков клиентов, поиска общих участников, поиска отсутствующих элементов, сверки данных между системами. Важно проверять не только техническую совместимость столбцов, но и смысловую: объединять `student\_id`, `name` со `teacher\_id`, `name` можно технически, но результат уже будет множеством персон, а не студентов.

— ## 8. Классификация СУБД по архитектуре: встроенные, файл-серверные, клиент-серверные

Конкретика для ответа

Классификация СУБД по архитектуре показывает, где выполняется обработка данных и как приложения получают доступ к хранилищу. Основные варианты: встроенная СУБД, файл-серверная архитектура и клиент-серверная архитектура.

Встроенная СУБД — СУБД, работающая внутри процесса приложения и не требующая отдельного серверного процесса. Пример: SQLite. База обычно хранится в одном файле, а библиотека SQLite подключается к приложению. Приложение вызывает функции библиотеки, а не отправляет запросы отдельному серверу.

flowchart LR

A[Приложение] --> L[Библиотека SQLite]

L --> F[(Файл базы данных .sqlite)]

Плюсы: простота установки, отсутствие администрирования сервера, удобство для мобильных приложений, локальных программ и небольших систем. Минусы: ограниченная масштабируемость для большого числа сетевых пользователей, сложнее централизовать доступ и разграничение прав.

Файл-серверная архитектура — архитектура, при которой файлы БД лежат на файловом сервере, а значительная часть обработки выполняется на клиентских машинах. Клиент получает доступ к файлам по сети. Примерная логика: приложение открывает файл базы, читает нужные блоки и само выполняет часть операций.

flowchart LR

C1[Клиент 1] --> FS[(Файловый сервер: файлы БД)]

C2[Клиент 2] --> FS

C3[Клиент 3] --> FS

Плюсы: простая организация для малых локальных сетей. Минусы: большой сетевой трафик, повышенный риск повреждения файлов при сбоях, слабая централизованная обработка запросов, проблемы при одновременной работе большого

числа пользователей.

Клиент-серверная СУБД — архитектура, при которой приложение-клиент отправляет запросы серверу БД, а сервер выполняет запрос, управляет транзакциями, индексами, блокировками, журналом и правами доступа. Примеры: PostgreSQL, MySQL, MariaDB, Oracle Database, Microsoft SQL Server.

flowchart LR

```
C1[Клиентское приложение] --> DB[Сервер СУБД]
C2[Веб-приложение] --> DB
C3[Админ-клиент] --> DB
DB --> S[(Файлы данных, WAL, индексы)]
```

Конкретное сравнение:

Архитектура	Где выполняется
SQL/обработка	Примеры
Когда использовать	
Встроенная	SQLite
Файл-сервер	частично на клиенте,
Клиент-сервер	на

Экзамнационный вывод: чем больше пользователей, транзакций и требований к безопасности, тем важнее клиент-серверная архитектура. Встроенные СУБД удобны для локального хранения, а файл-серверная архитектура исторически проста, но хуже подходит для современных нагруженных систем.

По архитектуре СУБД делятся на встроенные, файл-серверные и клиент-серверные. Архитектура определяет, где находится ядро СУБД, кто выполняет запросы и как приложение взаимодействует с данными.

Встроенная СУБД включается в приложение как библиотека. Нет отдельного серверного процесса: приложение само открывает файл базы и вызывает функции движка. Пример — SQLite. Такая архитектура удобна для мобильных приложений, локальных кэшей, настольных программ, тестов. Достоинства: простота, один файл, малые накладные расходы. Недостатки: ограниченная многопользовательская работа и слабая пригодность для больших серверных систем.

graph LR

```
A[Приложение] --> B[Библиотека СУБД]
B --> C[(Файл БД)]
```

Файл-серверная архитектура хранит файлы базы на сетевом файловом сервере, а обработка во многом выполняется на клиентах. Это просто, но плохо масштабируется: по сети могут передаваться большие объёмы данных, сложнее контролировать блокировки и безопасность, выше риск повреждения общего файла.

graph TD

```
FS[(Файловый сервер)]
C1[Клиент 1] --> FS
```

```
C2[Клиент 2] --> FS
C3[Клиент 3] --> FS
```

Клиент-серверная СУБД имеет отдельный сервер, который выполняет SQL-запросы, управляет файлами, индексами, правами, транзакциями и восстановлением. Клиенты отправляют запросы и получают результаты, не читая файлы напрямую. Примеры: PostgreSQL, MySQL, Oracle, SQL Server. Такая архитектура подходит для многопользовательских приложений, потому что контроль сосредоточен на сервере.

graph LR

```
C1[Клиент] --> S[Сервер СУБД]
C2[Клиент] --> S
S --> D[(Данные, индексы, журнал)]
```

	Архитектура	Примеры	
↪ Плюсы			Минусы
↪	Встроенная	SQLite	ограничена для
↪	просто, локально, быстро		серверных нагрузок
↪	Файл-серверная	shared-file БД	сеть,
↪	простое хранение общего файла		блокировки, безопасность
↪	Клиент-серверная	PostgreSQL, MySQL	требуется
↪	транзакции, права, оптимизация		администрирование сервера

Если нужно хранить заметки в мобильном приложении, подойдёт SQLite. Если нужно обслуживать интернет-магазин или учебную систему, нужна клиент-серверная СУБД. Файл-серверная модель исторически важна, но для современных многопользовательских систем обычно проигрывает.

— ## 9. Тонкий и толстый клиент. Многоуровневые решения. Распределенная обработка

Конкретика для ответа

Толстый клиент — клиентское приложение, в котором сосредоточена значительная часть бизнес-логики, обработки данных и интерфейса. Сервер в такой архитектуре часто используется в основном как хранилище данных. Пример: настольная программа, которая сама проверяет правила, формирует отчёты и напрямую обращается к БД.

Тонкий клиент — клиент, который выполняет в основном функции интерфейса, а бизнес-логика и обработка данных находятся на сервере приложений или сервере БД. Пример: браузер, который отправляет HTTP-запросы веб-приложению, а само приложение уже обращается к PostgreSQL.

Простая двухуровневая архитектура:

flowchart LR

```
Client[Клиентское приложение: интерфейс + часть логики] --> DB[Сервер БД]
```

Здесь клиент напрямую соединяется с БД. Это проще, но опаснее: приходится раздавать доступ к базе множеству рабочих мест, сложнее обновлять бизнес-логику и контролировать безопасность.

Трёхуровневая архитектура:

flowchart LR

```
UI[Тонкий клиент / браузер] --> APP[Сервер приложений]
APP --> DB[Сервер БД]
```

Здесь клиент отвечает за ввод и отображение, сервер приложений содержит бизнес-правила, а сервер БД отвечает за хранение, транзакции и запросы. Например, интернет-магазин: браузер показывает корзину, сервер приложения проверяет наличие товара и считает скидки, PostgreSQL хранит товары, заказы и платежи.

Многоуровневое решение — архитектура, где система делится более чем на три логических слоя: клиентский интерфейс, API gateway, сервер приложений, сервис авторизации, сервис отчётов, СУБД, кэш, очередь сообщений. Такое деление повышает масштабируемость и сопровождаемость, но усложняет администрирование.

Распределённая обработка означает, что разные части вычислений выполняются на разных узлах системы. Например, клиент формирует запрос, сервер приложений проверяет права, сервер БД выполняет SQL, кэш Redis хранит часто используемые результаты, очередь сообщений передаёт задачи на фоновую обработку.

```
flowchart LR
    Browser[Браузер] --> API[API / сервер приложений]
    API --> Cache[(Кэш)]
    API --> DB[(База данных)]
    API --> MQ[Очередь сообщений]
    MQ --> Worker[Фоновый обработчик]
```

Сравнение:

	Признак	Толстый
↪ клиент	Тонкий клиент	
:----- :-----		
↪ ----- :-----		
↪ станции	Где логика	на рабочей
	на сервере	
↪ клиентов	Обновление	надо обновлять
	обновляется сервер	
↪ выше	Требования к клиенту	ниже
↪ контролировать	Безопасность	сложнее
	проще централизовать	
↪ CRM	Пример	настольная
	веб-приложение	

На экзамене полезно подчеркнуть: «тонкий» и «толстый» клиент не означают размер программы в мегабайтах. Это характеристика распределения ответственности между клиентом, сервером приложений и сервером БД.

Тонкий клиент выполняет минимум логики: отображает интерфейс, принимает ввод и отправляет запросы на сервер. Пример — браузер, работающий с веб-приложением. Основная бизнес-логика находится на сервере, а данные — в СУБД. Плюсы тонкого клиента: централизованное обновление, контроль безопасности, меньшие требования к устройству. Минусы: зависимость от сети и сервера.

Толстый клиент содержит значительную часть логики на стороне пользователя. Он может кэшировать данные, проверять правила, выполнять вычисления, иногда напрямую обращаться к БД. Пример — настольная бухгалтерская система. Плюсы: богатый интерфейс, часть работы offline, разгрузка сервера. Минусы: сложнее обновлять, выше риск рассинхронизации и утечки логики.

```
graph TD
    Thin[Тонкий клиент: UI] --> App[Сервер приложения]
    App --> DB[(БД)]
```

graph TD

```
Thick[Толстый клиент: UI + логика + кэш] --> DB[(Сервер БД)]
```

Многоуровневая архитектура разделяет систему на уровень представления, уровень бизнес-логики и уровень данных. Уровень представления отвечает за интерфейс. Уровень бизнес-логики применяет правила: оформление заказа, расчёт цены, проверку доступа. Уровень данных хранит информацию в СУБД, кэше, файловом хранилище или очереди.

```
graph LR
    U[Пользователь] --> UI[Представление]
    UI --> BL[Бизнес-логика]
    BL --> DB[(СУБД)]
    BL --> Cache[(Кэш)]
    BL --> MQ[(Очередь)]
```

Распределённая обработка означает, что части задачи выполняются на разных узлах. Например, API Gateway принимает запрос, сервис каталога отвечает за товары, сервис заказов — за заказы, сервис оплаты — за платежи, а сервис уведомлений получает события из очереди. Это повышает масштабируемость и отказоустойчивость, но усложняет согласованность, мониторинг и транзакции.

```
graph TD
    Client[Клиент] --> GW[API Gateway]
    GW --> Catalog[Сервис каталога]
    GW --> Orders[Сервис заказов]
    GW --> Pay[Сервис оплаты]
    Orders --> ODB[(БД заказов)]
    Catalog --> CDB[(БД каталога)]
    Orders --> MQ[Очередь]
    MQ --> Notify[Уведомления]
```

На экзамене важно не абсолютизировать понятия: современные веб-приложения часто занимают середину. Браузер может выполнять много JavaScript-логики, мобильное приложение может иметь локальную SQLite-БД и синхронизацию, но критические операции всё равно выполняются на сервере. Главное — объяснить распределение ответственности между клиентом, сервером приложений и уровнем данных.

— ## 10. Нормальные формы и процесс нормализации

Конкретика для ответа

Нормализация — это процесс преобразования схемы реляционной БД с целью уменьшения избыточности данных и устранения аномалий вставки, обновления и удаления. Основа нормализации — анализ зависимостей между атрибутами.

Функциональная зависимость $X \rightarrow Y$ означает: если две строки имеют одинаковое значение набора атрибутов X , то они обязаны иметь одинаковое значение набора атрибутов Y . Например, $student_id \rightarrow full_name$, потому что идентификатор студента однозначно определяет его ФИО.

Три главные аномалии:

	Аномалия
↪ Суть	
↪ Пример	
↪	
↪	
↪	

	Обновления		одно и то же
↪	значение надо менять в нескольких строках		
↪	название группы повторяется у каждого студента		
	Вставки		
↪	нельзя добавить факт без другого факта		
↪	нельзя добавить новую дисциплину без студента		
	Удаления		удаление
↪	одной строки уничтожает другой важный факт		
↪	удалили последнего студента группы и потеряли сведения о группе		

1НФ: все значения атрибутов атомарны, нет повторяющихся групп и списков в одной ячейке. Нельзя хранить `phones = '111,222,333'`; надо вынести телефоны в отдельные строки.

2НФ: отношение находится в 1НФ, и каждый неключевой атрибут полностью зависит от всего составного ключа, а не от его части. Нарушение возникает в таблице `enrollments(student\_id, discipline\_id, student\_name, discipline\_name, mark)`, где ключ `(student\_id, discipline\_id)`, но `student\_name` зависит только от `student\_id`.

3НФ: отношение находится во 2НФ, и неключевые атрибуты не зависят транзитивно от ключа через другие неключевые атрибуты. Пример нарушения: `student\_id → group\_id, group\_id → curator\_name`, значит `student\_id → curator\_name` транзитивно.

НФБК (BCNF): для каждой нетривиальной функциональной зависимости $X \rightarrow Y$ левая часть X должна быть суперключом. BCNF строже 3НФ и устраняет случаи, когда детерминант не является ключом.

4НФ устраняет нетривиальные многозначные зависимости. Пример: у преподавателя независимо несколько языков и несколько дисциплин. Хранить всё в одной таблице нельзя, иначе появится декартово произведение языков и дисциплин.

5НФ устраняет зависимости соединения, когда отношение без потерь раскладывается на три и более проекции, и эти проекции лучше отражают независимые факты.

Короткая формула ответа: нормализация переводит таблицы от «всё в одной большой таблице» к набору отношений, где каждый факт хранится один раз, зависит от ключа, всего ключа и ничего кроме ключа.

Нормализация — процесс преобразования реляционной схемы для уменьшения избыточности и устранения аномалий вставки, обновления и удаления. Аномалия обновления возникает, когда одно и то же значение нужно менять во многих строках. Аномалия вставки — когда нельзя добавить факт без другого факта. Аномалия удаления — когда удаление одной строки случайно уничтожает важную информацию.

Первая нормальная форма требует атомарности значений: в ячейке не должно быть списков. Поле `products = 'мышь, клавиатура'` нарушает 1НФ. Нужно вынести позиции заказа в отдельные строки или таблицу `order\_items`.

Вторая нормальная форма требует 1НФ и отсутствия частичных зависимостей от части составного ключа. Если в `order\_items(order\_id, product\_id, product\_name, quantity)` ключ `(order\_id, product\_id)`, то `product\_name` зависит только от `product\_id`, значит его надо вынести в `products`.

Третья нормальная форма требует отсутствия транзитивных зависимостей неключевых атрибутов от ключа. В `students(student\_id, group\_id, group\_name)` есть зависимость `student\_id → group\_id → group\_name`; значит `group\_name` нужно хранить в `groups`.

НФБК усиливает 3НФ: для каждой нетривиальной функциональной зависимости $X \rightarrow Y$ детерминант X должен быть суперключом. Это устраняет сложные случаи с пересекающимися потенциальными ключами.

4НФ работает с многозначными зависимостями. Если преподаватель имеет несколько телефонов и несколько дисциплин, их нельзя хранить в одной таблице с перемножением

строк; нужны teacher_phones и teacher_disciplines. 5НФ работает с зависимостями соединения и применяется к сложным многоместным фактам.

graph TD

```
U[Ненормализованная таблица] --> NF1[1НФ: атомарность]
NF1 --> NF2[2НФ: нет частичных зависимостей]
NF2 --> NF3[3НФ: нет транзитивных зависимостей]
NF3 --> BCNF[НФБК: каждый детерминант - суперключ]
BCNF --> NF4[4НФ: многозначные зависимости]
NF4 --> NF5[5НФ: зависимости соединения]
```

Нормализация не является самоцелью. Обычно для прикладных OLTP-систем хорошей целью является 3НФ или НФБК. Денормализация допустима ради производительности, но только осознанно: например, хранение агрегатов или копий названий должно контролироваться правилами обновления.

— ## 11. Процесс нормализации с первой нормальной формы до нормальной формы Бойса-Кодда

Конкретика для ответа

Процесс нормализации удобно показать на одной исходной таблице. Пусть есть ненормализованная ведомость:

student_id	student_name	group_name	disciplines
1	Иванов	4311	БД, SQL
Петров,	Сидоров	5, 4	

Здесь нарушена **1НФ**, потому что в полях disciplines, teachers, marks хранятся списки, а не атомарные значения. Для перехода к 1НФ надо сделать так, чтобы каждая ячейка содержала одно значение:

student_id	student_name	group_name	discipline	teacher	mark
1	Иванов	4311	БД	Петров	5
1	Иванов	4311	SQL	Сидоров	4

Теперь таблица в 1НФ, но нарушает **2НФ**. Если ключ считать составным (student_id, discipline), то student_name зависит только от student_id, а не от всего ключа. teacher может зависеть от discipline, а не от пары целиком. Нужно разделить данные:

```
students(student_id PK, student_name, group_id FK)
groups(group_id PK, group_name)
disciplines(discipline_id PK, discipline_name)
marks(student_id FK, discipline_id FK, mark,
        PRIMARY KEY(student_id, discipline_id))
```

После этого каждое отношение описывает один вид фактов: студент, группа, дисциплина, оценка.

Для перехода к **3НФ** надо устранить транзитивные зависимости. Если в таблице students хранить group_name и curator_name, получится зависимость student_id → group_id, group_id → group_name, curator_name. Значит, сведения о группе надо хранить отдельно:

```
CREATE TABLE groups (
    group_id integer PRIMARY KEY,
    group_name text NOT NULL UNIQUE,
```



```

    curator_name text
);

CREATE TABLE students (
    student_id integer PRIMARY KEY,
    student_name text NOT NULL,
    group_id integer NOT NULL REFERENCES groups(group_id)
);

```

Для перехода к **BCNF** проверяют каждую функциональную зависимость. Если есть зависимость $teacher \rightarrow discipline$, но $teacher$ не является ключом исходной таблицы $schedule(group_id, discipline, teacher)$, то таблица может нарушать BCNF. Её можно разложить:

```

teacher_disciplines(teacher, discipline)
group_teachers(group_id, teacher)

```

При нормализации важно требовать **соединения без потерь**: после разложения исходные данные должны восстанавливаться соединением таблиц. Также желательно сохранять зависимости, чтобы ограничения можно было проверять без сложных соединений. Поэтому нормализация — не механическое дробление таблиц, а управляемое преобразование схемы по зависимостям.

Рассмотрим таблицу результатов экзаменов: $exam(student_id, student_name, group_id, group_name, discipline_id, discipline_name, teacher_id, teacher_name, grade)$. Если все значения атомарны, таблица находится в 1НФ. Но в ней много избыточности: имя студента повторяется во всех его оценках, название группы повторяется у всех студентов, название дисциплины и имя преподавателя повторяются для каждой сдачи.

Если ключ результата — $(student_id, discipline_id)$, то $grade$ зависит от всего ключа. Но $student_name$ и $group_id$ зависят только от $student_id$; $discipline_name$ и $teacher_id$ зависят только от $discipline_id$. Это частичные зависимости, нарушающие 2НФ. Для перехода ко 2НФ данные разделяют на $students$, $disciplines$, $exam_results$.

После этого остаются транзитивные зависимости. В $students$: $student_id \rightarrow group_id$, а $group_id \rightarrow group_name$, значит $group_name$ транзитивно зависит от студента. В $disciplines$: $discipline_id \rightarrow teacher_id$, а $teacher_id \rightarrow teacher_name$. Для перехода к 3НФ создают отдельные таблицы $groups$ и $teachers$.

```

erDiagram
    GROUPS ||--o{ STUDENTS : contains
    TEACHERS ||--o{ DISCIPLINES : teaches
    STUDENTS ||--o{ EXAM_RESULTS : receives
    DISCIPLINES ||--o{ EXAM_RESULTS : has
    GROUPS { int group_id PK string group_name }
    STUDENTS { int student_id PK string name int group_id FK }
    TEACHERS { int teacher_id PK string name }
    DISCIPLINES { int discipline_id PK string name int teacher_id FK }
    EXAM_RESULTS { int student_id PK,FK int discipline_id PK,FK int grade }

```

НФБК требует проверить все функциональные зависимости. Для каждой зависимости $X \rightarrow Y$ множество X должно быть суперключом. Пример нарушения: $consultations(student_id, discipline, teacher)$, где $(student_id, discipline) \rightarrow teacher$ и $teacher \rightarrow discipline$, потому что каждый преподаватель ведёт одну дисциплину. Детерминант $teacher$ не является суперключом всей таблицы, значит НФБК нарушена. Разложение: $teacher_disciplines(teacher, discipline)$ и $student_teachers(student_id, teacher)$.

Главная логика нормализации: 1НФ убирает списки в ячейках, 2НФ убирает

зависимости от части составного ключа, 3НФ убирает транзитивные зависимости, НФБК требует, чтобы любой детерминант был ключом. При разложении важно сохранять соединение без потерь, иначе схема будет формально красивой, но данные не восстановятся корректно.

— ## 12. Четвертая и пятая нормальные формы. Примеры нормализации

Конкретика для ответа

Четвёртая нормальная форма (4НФ) связана не с функциональными, а с многозначными зависимостями. **Многозначная зависимость** $X \twoheadrightarrow Y$ означает: одному значению X соответствует множество значений Y , причём это множество независимо от других атрибутов отношения. Отношение находится в 4НФ, если для каждой нетривиальной многозначной зависимости $X \twoheadrightarrow Y$ левая часть X является суперключом. Классический пример нарушения 4НФ:

teacher	language	discipline
Иванов	Английский	Базы данных
Иванов	Английский	SQL
Иванов	Немецкий	Базы данных
Иванов	Немецкий	SQL

Смысл: преподаватель Иванов знает два языка и ведёт две дисциплины. Но знание языка не связано с дисциплиной. Таблица создаёт искусственное декартово произведение. Если добавить третий язык, придётся дублировать его для всех дисциплин. Здесь есть независимые многозначные зависимости: $teacher \twoheadrightarrow language$ и $teacher \twoheadrightarrow discipline$.

Правильное разложение для 4НФ:

```
teacher_languages(teacher, language)
teacher_disciplines(teacher, discipline)
```

Пятая нормальная форма (5НФ) связана с зависимостями соединения. Отношение находится в 5НФ, если каждая нетривиальная зависимость соединения следует из потенциальных ключей. Проще: таблицу нельзя без потерь разложить на несколько меньших таблиц так, чтобы это разложение отражало независимые факты лучше исходной таблицы.

Пример для 5НФ: поставщик S, деталь P, проект J.

supplier	part	project
S1	P1	J1
S1	P2	J1
S1	P1	J2

Иногда факт «поставщик поставляет деталь проекту» определяется тремя независимыми бинарными фактами: поставщик работает с деталью, поставщик работает с проектом, деталь используется в проекте. Тогда отношение можно разложить:

```
SP(supplier, part)
SJ(supplier, project)
PJ(part, project)
```

Но это разложение допустимо только если соединение этих трёх таблиц не создаёт ложных троек. Поэтому 5НФ применяется осторожно и встречается реже, чем 1НФ-3НФ и BCNF.

Краткое различие:

Норма	Что устраняет
↪ Тип зависимости	Пример проблемы
↪ ----- :----- :----- :-----	
↪ ----- :----- :----- :-----	
↪ ----- :----- :----- :-----	
4НФ	независимые множества значений
↪ многозначная зависимость	языки преподавателя ×
↪ дисциплины	
5НФ	сложные зависимости соединения
↪ зависимость соединения	
↪ поставщик–деталь–проект	

Экзамениционный вывод: 4НФ нужна, когда одна сущность имеет несколько независимых наборов характеристик, а 5НФ нужна, когда многоместная связь корректно распадается на несколько меньших связей без потери смысла и без появления ложных комбинаций.

Четвёртая нормальная форма связана с многозначными зависимостями. Многозначная зависимость $A \twoheadrightarrow B$ означает, что одному значению A соответствует множество значений B , независимое от других атрибутов. Проблема возникает, когда в одной таблице хранятся два независимых множества. Например, преподаватель имеет несколько телефонов и ведёт несколько дисциплин.

Ошибочная таблица:

teacher	phone	discipline
Сидоров	111	Базы данных
Сидоров	111	SQL
Сидоров	222	Базы данных
Сидоров	222	SQL

Здесь телефоны и дисциплины перемножаются. Чтобы добавить третий телефон, придётся добавить строки для всех дисциплин. Для 4НФ отношение разбивают:

teacher	phone
Сидоров	111
Сидоров	222
teacher	discipline
Сидоров	Базы данных
Сидоров	SQL

graph TD

```
Bad[teacher, phone, discipline] --> P[teacher_phones]
Bad --> D[teacher_disciplines]
```

Пятая нормальная форма связана с зависимостями соединения. Она требует, чтобы отношение нельзя было без потерь разложить на более мелкие отношения иначе как по зависимостям, вытекающим из ключей. Классический пример — поставщики, детали и проекты: SPJ(supplier, part, project). Иногда тройной факт можно восстановить из бинарных отношений SP, SJ, PJ. Но это допустимо только если бизнес-правила действительно говорят, что такие независимые связи порождают допустимую тройку. 5НФ применяется реже, потому что требует глубокого анализа предметной области. Если разложить отношение неправильно, соединение проекций создаст ложные строки. Поэтому 4НФ обычно распознаётся по независимым спискам, а 5НФ — по

сложным многоместным фактам, восстанавливаемым соединением. На экзамене нужно подчеркнуть: 4НФ устраняет избыточность от многозначных зависимостей, 5НФ — от нетривиальных зависимостей соединения.

— ## 13. Проектирование БД: инфологическое, даталогическое, физическое

Конкретика для ответа

Проектирование БД обычно делят на три уровня: **инфологическое, даталогическое и физическое**. Эти этапы соответствуют движению от предметной области к конкретной СУБД.

Инфологическое проектирование — построение модели предметной области, независимой от конкретной СУБД. На этом этапе выясняют, какие объекты существуют, какими свойствами обладают и как связаны между собой. Результатом является концептуальная ER-модель. Например, для учебной БД выделяются сущности Студент, Группа, Дисциплина, Преподаватель, Оценка; задаются связи: студент принадлежит группе, студент получает оценки, дисциплина читается преподавателем.

Даталогическое проектирование — преобразование инфологической модели в логическую схему выбранной модели данных, обычно реляционной. На этом этапе сущности превращаются в таблицы, атрибуты — в столбцы, связи — во внешние ключи или промежуточные таблицы. Здесь выбираются первичные ключи, внешние ключи, нормальные формы и ограничения.

Пример перехода:

Инфологическая сущность: Студент

Атрибуты: ФИО, дата рождения, группа

Даталогическая таблица:

students(student_id PK, full_name, birth_date, group_id FK)

Физическое проектирование — настройка схемы под конкретную СУБД и условия эксплуатации. Здесь выбирают типы данных, индексы, партиционирование, tablespace, параметры хранения, резервное копирование, права доступа, стратегию выполнения запросов. Например, для таблицы marks можно создать индекс по (student_id, discipline_id), потому что часто ищут оценки конкретного студента.

```
CREATE INDEX idx_marks_student_discipline
ON marks(student_id, discipline_id);
```

Схема этапов:

```
flowchart TD
    A[Анализ предметной области] --> B[Инфологическая модель: сущности, связи, атрибуты]
    B --> C[Даталогическая модель: таблицы, ключи, нормализация]
    C --> D[Физическая модель: типы, индексы, хранение, права]
    D --> E[Реализация в СУБД]
```

Сравнительная таблица:

	Этап	Главный
↪ вопрос		Результат
↪ Пример решения		
:----- :-----		
↪ ----- :-----		
↪ :-----		
	Инфологический	что есть в предметной
↪ области?		ER-модель
↪ студент связан с группой		

	Даталогический	как это представить
↪	логически?	схема таблиц
↪	`students`, `groups`, FK	
	Физический	как это эффективно
↪	хранить?	SQL DDL, индексы, настройки
↪	индекс, тип `integer`	B-tree

Экзаменационный вывод: инфологическое проектирование описывает смысл данных, даталогическое — их логическую структуру в выбранной модели, физическое — способ реализации в конкретной СУБД. Ошибка на раннем этапе обычно дороже, чем ошибка физической настройки, потому что неправильные сущности и связи потом трудно исправить.

Проектирование БД переводит описание предметной области в работающую структуру хранения. Обычно выделяют инфологическое, даталогическое и физическое проектирование. Эти этапы идут от смысла к логической модели и затем к реализации в конкретной СУБД.

Инфологическое проектирование не зависит от конкретной СУБД. На этом этапе выявляют сущности, атрибуты, связи и бизнес-правила. Например, для учебной системы: студент, группа, дисциплина, преподаватель, оценка. Определяется, что группа содержит студентов, преподаватель ведёт дисциплины, студент получает оценки. Результат — ER-модель или описание предметной области.

Даталогическое проектирование переводит инфологическую модель в логическую схему выбранной модели данных. Для реляционной БД сущности становятся таблицами, атрибуты — столбцами, связи 1:N — внешними ключами, связи M:N — промежуточными таблицами. На этом этапе определяются ключи, домены, нормальные формы и ограничения.

Физическое проектирование адаптирует логическую схему к конкретной СУБД и нагрузке. Выбираются типы данных, индексы, партиционирование, схемы, права, параметры хранения, резервное копирование. Например, в PostgreSQL можно создать индекс по `group_id`, если часто выбираются студенты группы.

graph TD

```
I[Инфологическое: сущности и связи] --> D[Даталогическое: таблицы и ключи]
D --> P[Физическое: индексы, типы, хранение]
```

```
CREATE TABLE exam_results (
    student_id integer REFERENCES students(student_id),
    discipline_id integer REFERENCES disciplines(discipline_id),
    grade integer CHECK (grade BETWEEN 2 AND 5),
    PRIMARY KEY (student_id, discipline_id)
);
CREATE INDEX idx_exam_discipline ON exam_results(discipline_id);
```

Если пропустить инфологический этап, можно быстро написать таблицы, но неверно понять предметную область. Если пропустить даталогический этап, ER-диаграмма будет плохо отображена в SQL. Если пропустить физический этап, логически правильная база может работать медленно. Хороший проект итерационен: новые требования могут возвращать проектировщика к более ранним этапам.

— ## 14. Проектирование в нотации П. Чена. Типы связей. Атрибуты. Пример

Конкретика для ответа

Нотация Питера Чена — классическая ER-нотация для инфологического проектирования БД. В ней предметная область описывается через **сущности, атрибуты и связи**.

Сущность — множество однотипных объектов предметной области. В нотации Чена сущность изображается прямоугольником. Примеры: Студент, Группа, Дисциплина,

Преподаватель.

Атрибут — свойство сущности или связи. В классической нотации Чена атрибут изображается овалом. Примеры атрибутов студента: student_id, ФИО, дата_рождения. Ключевой атрибут подчёркивается. Составной атрибут может раскладываться на части: ФИО на фамилия, имя, отчество. Многозначный атрибут изображается двойным овалом, например несколько телефонов студента.

Связь — ассоциация между сущностями. В нотации Чена связь изображается ромбом. Примеры: студент «принадлежит» группе, преподаватель «читает» дисциплину, студент «сдаёт» дисциплину.

Типы связей:

	Тип	Обозначение
↪ Пример		
:----- :----- :-----		
↪ -----		
	один-к-одному	1:1
↪ человек – паспорт		
	один-ко-многим	1:N
↪ группа – студенты		
	многие-ко-многим	M:N
↪ студенты – дисциплины		

Пример проектирования учебной части:

erDiagram

```
GROUP ||--o{ STUDENT : contains
STUDENT }o--o{ DISCIPLINE : studies
TEACHER ||--o{ DISCIPLINE : teaches
STUDENT { int student_id PK string full_name date birth_date }
GROUP { int group_id PK string group_name }
DISCIPLINE { int discipline_id PK string name int hours }
TEACHER { int teacher_id PK string full_name }
```

В терминах Чена этот пример читается так: сущность Группа связана со Студентом отношением 1:N, потому что одна группа содержит много студентов, а студент принадлежит одной группе. Сущность Студент связана с Дисциплиной отношением M:N, потому что один студент изучает много дисциплин, а одна дисциплина изучается многими студентами. При переходе к реляционной схеме связь M:N превращается в отдельную таблицу:

```
CREATE TABLE student_disciplines (
    student_id integer REFERENCES students(student_id),
    discipline_id integer REFERENCES disciplines(discipline_id),
    semester integer NOT NULL,
    PRIMARY KEY (student_id, discipline_id)
);
```

Если связь имеет собственные атрибуты, они относятся не к одной сущности, а к факту связи. Например, mark и exam_date логично хранить в связи «студент сдаёт дисциплину», а не в таблице студента и не в таблице дисциплины.

Экзаменационный вывод: нотация Чена нужна для концептуального описания предметной области до выбора конкретной СУБД. Она помогает увидеть сущности, их атрибуты, типы связей и затем корректно перейти к реляционным таблицам.

Нотация Питера Чена — классическая ER-нотация для инфологического проектирования. Сущности изображаются прямоугольниками, связи — ромбами, атрибуты — овалами. Её смысл — отделить объект предметной области, его свойства и факт связи с другими объектами.

Сущность — класс объектов: студент, группа, дисциплина, преподаватель. Экземпляр сущности — конкретный студент Иванов или дисциплина «Базы данных». Сильная сущность имеет собственный ключ. Слабая сущность зависит от другой, например позиция заказа зависит от заказа.

Атрибут — характеристика сущности или связи. Атрибуты бывают простыми, составными, однозначными, многозначными, производными и ключевыми. Например, ФИО можно считать составным атрибутом, телефон — многозначным, возраст — производным от даты рождения.

Связи бывают 1:1, 1:N, M:N. Связь 1:1: человек и паспорт в упрощённой модели. Связь 1:N: группа содержит много студентов, студент относится к одной группе. Связь M:N: студенты изучают дисциплины, дисциплины изучаются многими студентами. Связь может иметь собственные атрибуты: оценка относится к факту сдачи дисциплины студентом.

```
graph TD
    Student[СТУДЕНТ] --- SId((student_id))
    Student --- SName((ФИО))
    Group[ГРУППА] --- GId((group_id))
    Group --- GName((название))
    Discipline[ДИСЦИПЛИНА] --- DId((discipline_id))
    Teacher[ПРЕПОДАВАТЕЛЬ] --- TId((teacher_id))
    Belongs{ПРИНАДЛЕЖИТ} --- Student
    Belongs --- Group
    Studies{ИЗУЧАЕТ} --- Student
    Studies --- Discipline
    Studies --- Grade((оценка))
    Teaches{ВЕДЁТ} --- Teacher
    Teaches --- Discipline
```

Переход к реляционной схеме: сильные сущности становятся таблицами, атрибуты — столбцами, ключевые атрибуты — первичными ключами. Связь 1:N реализуется внешним ключом на стороне N. Связь M:N превращается в отдельную таблицу, содержащую внешние ключи и атрибуты связи.

```
CREATE TABLE student_disciplines (
    student_id integer REFERENCES students(student_id),
    discipline_id integer REFERENCES disciplines(discipline_id),
    grade integer CHECK (grade BETWEEN 2 AND 5),
    PRIMARY KEY (student_id, discipline_id)
);
```

Нотация Чена полезна тем, что заставляет правильно определить, что является сущностью, что атрибутом, а что связью. Например, оценка — не свойство только студента и не свойство только дисциплины, а атрибут связи между студентом и дисциплиной.

— ## 15. Индексирование: В-дерево, битовые карты

Конкретика для ответа

Индекс в БД — это дополнительная структура данных, ускоряющая поиск строк по значениям одного или нескольких столбцов. Индекс похож на предметный указатель в книге: вместо просмотра всей таблицы СУБД быстро находит место, где находятся нужные строки. Индекс ускоряет чтение, но замедляет вставку, удаление и обновление, потому что его тоже надо поддерживать.

В-дерево — сбалансированная древовидная структура, в которой данные отсортированы по ключу, а путь от корня до любого листа имеет примерно одинаковую длину. В СУБД часто используется разновидность B+tree: внутренние узлы направляют поиск, а листья содержат ссылки на строки или сами значения индекса.

```

flowchart TD
    R[Корень: 40] --> L[Лист: 1..39]
    R --> M[Лист: 40..79]
    R --> H[Лист: 80..120]

```

B-tree эффективен для операций:

```

WHERE student_id = 10
WHERE birth_date BETWEEN DATE '2000-01-01' AND DATE '2002-12-31'
ORDER BY full_name

```

Пример создания индекса:

```

CREATE INDEX idx_students_group_id
ON students(group_id);

```

Если часто выполняется запрос `SELECT * FROM students WHERE group_id = 101`, индекс по `group_id` может существенно ускорить поиск.

Битовая карта (bitmap index) — индекс, в котором для каждого значения атрибута хранится битовый вектор: 1 означает, что строка имеет это значение, 0 — что не имеет. Такой индекс особенно эффективен для столбцов с малым числом различных значений: пол, статус, категория, флаг активности.

Пример для атрибута `status`:

row_id	status
1	active
2	archived
3	active
4	blocked

Битовые векторы:

Значение	Битовая карта
active	1010
archived	0100
blocked	0001

Преимущество bitmap-индексов — быстрые логические операции AND, OR, NOT. Например, условие `status = 'active' AND gender = 'F'` можно вычислить как побитовое AND двух карт. Недостаток — плохая пригодность для часто изменяемых столбцов с высокой кардинальностью, например `email` или `passport_number`.

Сравнение:

Признак	B-tree	Bitmap
Лучше для		
высокоселективных значений, диапазонов		малой
кардинальности		
Примеры полей		
`id`, `date`, `email`		`status`, `gender`, `is_active`
Обновления		
обычно приемлемы		могут быть дорогими
Диапазонный поиск		
хорош		не основная задача

Экзаменационный вывод: B-tree является универсальным индексом для равенства, диапазонов и сортировки, а bitmap-индекс полезен для аналитических запросов по признакам с небольшим числом значений.

Индекс — вспомогательная структура, ускоряющая поиск строк. Без индекса СУБД часто просматривает таблицу последовательно. Индекс работает как предметный указатель в книге: он занимает место и требует обновления при изменениях, но ускоряет чтение.

B-tree — самый распространённый универсальный индекс. Он хранит ключи в отсортированном сбалансированном дереве. Подходит для равенства, диапазонов, сортировки: =, <, >, BETWEEN, ORDER BY, LIKE 'abc%'. В PostgreSQL B-tree создаётся по умолчанию командой CREATE INDEX.

graph TD

```
R[50] --> A[10|30]
R --> B[70|90]
A --> A1[1,5,9]
A --> A2[12,20,29]
B --> B1[51,60]
B --> B2[91,99]
```

```
CREATE INDEX idx_students_last_name ON students(last_name);
```

Битовые карты представляют множество строк через биты. Для каждого значения признака создаётся битовая последовательность: 1 означает, что строка имеет значение, 0 — не имеет. Они особенно полезны для признаков низкой кардинальности: статус, пол, категория, регион.

row	status
1	paid
2	new
3	paid
4	cancelled
5	new

status	bitmap
paid	10100
new	01001
cancelled	00010

Для условия status='paid' OR status='new' можно выполнить битовое OR: 10100 OR 01001 = 11101. Битовые индексы хороши для аналитики, но хуже для часто обновляемых OLTP-таблиц. Также важно различать постоянный bitmap index и bitmap scan, когда СУБД строит битовую карту во время выполнения запроса по обычным индексам.

	Критерий	Bitmap
↪ B-tree		
↪		
↪		
↪	Лучше для	равенство, диапазоны,
↪ сортировка		фильтры по малому числу значений
↪	Кардинальность	средняя и
↪ высокая		низкая
↪	Обновления	переносит
↪ хорошо		может быть дорогим

	Пример	email,
↪ id, date		status, category

Индексирование — компромисс: чтение быстрее, запись медленнее, база больше. Индексы выбирают по реальным запросам, а не по желанию украсить каждый столбец. — ## 16. Индексирование: первичный, кластерный и некластерный индексы

Конкретика для ответа

Первичный индекс обычно связан с первичным ключом таблицы. Когда объявляется PRIMARY KEY, СУБД создаёт механизм уникального поиска по ключу. В PostgreSQL первичный ключ автоматически создаёт уникальный B-tree индекс. Важно различать логическое ограничение PRIMARY KEY и физическую структуру индекса: первичный ключ гарантирует уникальность и отсутствие NULL, а индекс является средством эффективной проверки и поиска.

```
CREATE TABLE students (
    student_id integer PRIMARY KEY,
    full_name text NOT NULL
);
```

Кластерный индекс — индекс, порядок которого определяет физический или близкий к физическому порядок хранения строк таблицы. Идея: строки на диске расположены примерно в том же порядке, что и ключ индекса. Это полезно для диапазонных запросов. Например, если таблица заказов физически упорядочена по order_date, запросы за период читают меньше разрозненных страниц.

Пример концепции:

Кластеризация по order_date:
[2026-01-01][2026-01-02][2026-01-03] ...

В разных СУБД термин отличается. В SQL Server clustered index действительно задаёт физический порядок таблицы. В PostgreSQL команда CLUSTER физически переупорядочивает таблицу по индексу, но этот порядок не поддерживается автоматически после всех изменений.

Некластерный индекс — отдельная структура, которая хранит ключи индекса и ссылки на строки таблицы, но не определяет физический порядок самих строк. У таблицы может быть несколько некластерных индексов: по email, по дате рождения, по группе.

```
CREATE INDEX idx_students_email ON students(email);
CREATE INDEX idx_students_group ON students(group_id);
```

Пример различия:

	Вид индекса	
↪	Что делает	Сколько может быть
↪	Пример	
	-----	-----
↪	-----	-----
↪	-----	-----
	Первичный	обеспечивает
↪	быстрый доступ по РК и уникальность	
↪	обычно один РК `student\_id`	
	Кластерный	задаёт
↪	порядок хранения/кластеризации строк	обычно один
↪	фактический порядок `order\_date`	
	Некластерный	
↪	отдельный указатель на строки	
↪	много `email`, `group\_id`	

Проверочный пример: таблица orders содержит 10 млн строк. Если часто ищут заказ по order_id, нужен первичный индекс. Если часто строят отчёты по периоду, полезна кластеризация или индекс по order_date. Если часто ищут заказы пользователя, нужен некластерный индекс по customer_id.

Экзамениционный вывод: первичный индекс связан с идентификацией строк, кластерный — с порядком размещения данных, некластерный — с дополнительными путями доступа. Индекс нельзя считать бесплатным: он ускоряет чтение, но требует места и увеличивает стоимость изменений.

Первичный индекс связан с первичным ключом. Первичный ключ уникально идентифицирует строку и не допускает NULL. Во многих СУБД при объявлении PRIMARY KEY автоматически создаётся уникальный индекс, чтобы быстро искать строки и проверять уникальность.

```
CREATE TABLE students (  
    student_id integer PRIMARY KEY,  
    full_name text NOT NULL  
);
```

Кластерный индекс определяет или приближает физический порядок хранения строк. Если данные упорядочены по дате, диапазонные запросы по дате читают меньше разрозненных страниц. В некоторых СУБД таблица может иметь только один кластерный индекс, потому что физический порядок один. В PostgreSQL команда CLUSTER может переупорядочить таблицу по индексу, но порядок не поддерживается автоматически навсегда.

Некластерный индекс хранится отдельно от таблицы и содержит ключи плюс ссылки на строки. Таблица может иметь много некластерных индексов: по email, внешнему ключу, дате, статусу. Они ускоряют поиск, но требуют обслуживания при INSERT, UPDATE, DELETE.

```
graph TD  
    T["Таблица"]  
    PK["Первичный индекс по id"] --> T  
    N1["Некластерный индекс по email"] --> T  
    N2["Некластерный индекс по group_id"] --> T  
    C["Кластеризация по дате"] -. "влияет на порядок" .-> T
```

```
CREATE INDEX idx_students_email ON students(email);  
CREATE INDEX idx_students_group ON students(group_id);
```

Первичный ключ — логическое ограничение, индекс — физическая структура доступа. Уникальный индекс может поддерживать альтернативную уникальность, например email. Кластерный индекс важен для диапазонных чтений. Некластерные индексы гибки, но избыточное количество индексов замедляет записи и увеличивает размер базы. Поэтому индексы проектируют по профилю запросов.

— ## 17. Понятие и свойства транзакций, изоляции. Примеры

Конкретика для ответа

Транзакция — это логически неделимая последовательность операций с БД, переводящая базу из одного согласованного состояния в другое. Транзакция либо выполняется полностью, либо полностью отменяется. Пример: перевод 100 рублей со счёта А на счёт В состоит минимум из двух операций: списания и зачисления. Если списание произошло, а зачисление нет, база становится противоречивой.

ACID-свойства:

	Свойство
↪ Определение	
↪ Пример	

	:	-----		:	-----
↪		-----			-----
↪		-----		:	-----
↪		-----			-----
		Atomicity, атомарность			
↪		все операции транзакции выполняются целиком или не выполняются вовсе			
↪		перевод не может быть наполовину выполнен			
		Consistency, согласованность			
↪		транзакция сохраняет ограничения целостности			
↪		баланс не уходит ниже разрешённого минимума			
		Isolation, изолированность			
↪		параллельные транзакции не должны некорректно влиять друг на друга			
↪		два кассира не списывают один остаток дважды			
		Durability, долговечность			
↪		после `COMMIT` результат сохраняется при сбое			
↪		подтверждённый платёж не пропадает			

SQL-пример:

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;
```

Если возникла ошибка:

```
ROLLBACK;
```

Изоляция транзакций определяет, какие изменения одной транзакции видны другим параллельным транзакциям. Основные проблемы изоляции:

1. **Dirty read, грязное чтение** — транзакция читает данные, которые другая транзакция ещё не зафиксировала.
2. **Non-repeatable read, неповторяемое чтение** — повторное чтение той же строки в одной транзакции даёт другой результат из-за фиксации чужого изменения.
3. **Phantom read, фантомное чтение** — повторный запрос по условию возвращает новый набор строк, потому что другая транзакция вставила или удалила подходящие строки.

Уровни изоляции SQL:

		Уровень	
↪		Смысл	
	:	-----	
↪		-----	
		Read Uncommitted	
↪		допускает чтение незафиксированных данных в некоторых СУБД	
		Read Committed	
↪		видны только зафиксированные данные	
		Repeatable Read	
↪		повторное чтение уже прочитанных строк стабильно	
		Serializable	
↪		результат эквивалентен последовательному выполнению транзакций	

Пример конфликта: два пользователя одновременно покупают последний билет. Без транзакций оба могут увидеть `available = 1` и оба оформить покупку. Правильное решение — выполнять проверку и изменение в одной транзакции с подходящими блокировками или уровнем изоляции.

```
BEGIN;
SELECT available FROM tickets WHERE ticket_id = 10 FOR UPDATE;
UPDATE tickets SET available = available - 1 WHERE ticket_id = 10 AND available > 0;
COMMIT;
```

Экзаменационный вывод: транзакции нужны для надёжного выполнения связанных операций, а изоляция — для корректной параллельной работы пользователей.

Транзакция — логическая единица работы с БД, состоящая из одной или нескольких операций. Она завершается COMMIT или отменяется ROLLBACK. Транзакции нужны, когда несколько операций образуют один смысловой факт: перевод денег, оформление заказа, запись результата экзамена.

Свойства ACID: атомарность, согласованность, изолированность, долговечность. Атомарность означает «всё или ничего». Согласованность — переход из одного корректного состояния в другое. Изолированность — защита параллельных транзакций от неконтролируемого влияния. Долговечность — сохранение зафиксированных изменений после сбоя.

```
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 2;
COMMIT;
```

Если вторая операция не выполнится, нужен ROLLBACK, иначе деньги исчезнут из одного счёта и не появятся на другом.

Аномалии изоляции: грязное чтение — чтение незакоммиченных данных; неповторяющееся чтение — повторное чтение строки даёт другое значение; фантомное чтение — повторный запрос по условию видит новые строки; потерянное обновление — результат одной транзакции перезаписывается другой.

SQL-стандарт выделяет уровни изоляции: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE. Чем выше уровень, тем меньше аномалий, но выше стоимость конкурентного выполнения. PostgreSQL по умолчанию обычно использует READ COMMITTED, а SERIALIZABLE стремится обеспечить результат, эквивалентный последовательному выполнению транзакций.

Пример потери обновления: два пользователя читают остаток товара 10. Первый продаёт 2 и пишет 8, второй продаёт 3 и пишет 7. Правильный остаток 5, но одно обновление потерялось. Безопаснее выполнять атомарное обновление:

```
BEGIN;
UPDATE products
SET stock = stock - 3
WHERE product_id = 10 AND stock >= 3;
COMMIT;
```

```
sequenceDiagram
    participant T1 as Транзакция 1
    participant DB as БД
    participant T2 as Транзакция 2
    T1->>DB: BEGIN, UPDATE
    T2->>DB: SELECT/UPDATE
    DB-->>T2: видимость зависит от уровня изоляции
    T1->>DB: COMMIT
```

Транзакции также поддерживают SAVEPOINT, позволяя откатывать часть большой операции. На экзамене важно показать: транзакция сохраняет смысл операции, ACID описывает требования корректности, уровни изоляции регулируют параллельную видимость изменений.

— ## 18. Основные категории команд языка SQL. Примеры

Конкретика для ответа

SQL-команды принято делить на категории по назначению. Это важно, потому что SQL — не только язык выборки, но и язык определения структуры, изменения данных, управления правами и транзакциями.

DDL (Data Definition Language) — язык определения данных. Команды DDL создают, изменяют и удаляют объекты БД: таблицы, схемы, индексы, представления.

```
CREATE TABLE students (
    student_id integer PRIMARY KEY,
    full_name text NOT NULL,
    group_id integer
);
```

```
ALTER TABLE students ADD COLUMN email text;
DROP TABLE old_students;
```

DML (Data Manipulation Language) — язык манипулирования данными. Команды DML изменяют содержимое таблиц.

```
INSERT INTO students(student_id, full_name, group_id)
VALUES (1, 'Иванов Иван', 101);
```

```
UPDATE students SET group_id = 102 WHERE student_id = 1;
DELETE FROM students WHERE student_id = 1;
```

DQL (Data Query Language) — язык запросов к данным. Часто его рассматривают как часть DML, но на экзамене удобно выделить SELECT отдельно.

```
SELECT s.full_name, g.group_name
FROM students s
JOIN groups g ON g.group_id = s.group_id
WHERE g.group_name = '4311';
```

DCL (Data Control Language) — язык управления доступом. Он назначает и отзывает права.

```
GRANT SELECT ON students TO analyst;
REVOKE DELETE ON students FROM analyst;
```

TCL (Transaction Control Language) — язык управления транзакциями.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
COMMIT;
-- или ROLLBACK при ошибке
```

DQL/SELECT дополнительно включает фильтрацию, группировку, сортировку, соединения и подзапросы:

```
SELECT group_id, COUNT(*) AS student_count
FROM students
GROUP BY group_id
HAVING COUNT(*) > 20
ORDER BY student_count DESC;
```

Таблица для запоминания:

Категория	Назначение
Команды	
DDL	структура БД
DML	изменение данных
DCL	управление доступом
TCL	управление транзакциями
DQL/SELECT	запросы к данным

	DQL	выборка данных
↪	`SELECT`	
	DCL	права доступа
↪	`GRANT`, `REVOKE`	
	TCL	транзакции `BEGIN`, `COMMIT`,
↪	`ROLLBACK`, `SAVEPOINT`	

Экзаменационный вывод: SQL является комплексным языком управления БД. Через DDL описывается структура, через DML и DQL выполняется работа с данными, через DCL защищается доступ, через TCL обеспечивается транзакционная согласованность. SQL делится на категории команд. DDL определяет структуру данных: CREATE, ALTER, DROP, CREATE INDEX, CREATE VIEW. DML изменяет строки: INSERT, UPDATE, DELETE, иногда MERGE. DQL отвечает за выборку, прежде всего SELECT. DCL управляет правами: GRANT, REVOKE. TCL управляет транзакциями: BEGIN, COMMIT, ROLLBACK, SAVEPOINT.

```
CREATE TABLE products (
    product_id integer PRIMARY KEY,
    name text NOT NULL,
    price numeric(10,2) CHECK (price >= 0)
);
```

```
INSERT INTO products(product_id, name, price)
VALUES (1, 'Клавиатура', 2500.00);
UPDATE products SET price = price * 1.1 WHERE product_id = 1;
DELETE FROM products WHERE product_id = 1;
```

```
SELECT name, price
FROM products
WHERE price > 1000
ORDER BY price DESC;
```

```
GRANT SELECT ON products TO analyst;
REVOKE UPDATE ON products FROM analyst;
```

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;
COMMIT;
```

```
graph TD
    SQL[SQL] --> DDL[DDL: структура]
    SQL --> DQL[DQL: выборка]
    SQL --> DML[DML: изменение данных]
    SQL --> DCL[DCL: права]
    SQL --> TCL[TCL: транзакции]
```

Логический порядок обработки SELECT: FROM, JOIN, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT. Синтаксически SELECT стоит первым, но логически сначала определяется источник строк.

Комплексный пример:

```
SELECT d.name, AVG(er.grade) AS avg_grade
FROM exam_results er
JOIN disciplines d ON d.discipline_id = er.discipline_id
WHERE er.grade IS NOT NULL
GROUP BY d.name
HAVING AVG(er.grade) >= 4
ORDER BY avg_grade DESC;
```

SQL является декларативным языком: пользователь описывает результат, а СУБД

выбирает план выполнения. Это отличает SQL от навигационных подходов, где программа сама задаёт маршрут обхода записей.

— ## 19. Методы обеспечения безопасности данных в БД

Конкретика для ответа

Безопасность данных в БД — это совокупность мер, защищающих данные от несанкционированного доступа, изменения, уничтожения и раскрытия. Её нельзя сводить только к паролю пользователя: защита строится на нескольких уровнях.

1. **Идентификация и аутентификация.** СУБД должна знать, кто подключается. Пользователь указывает логин, пароль, сертификат, Kerberos-токен или иной механизм. Пример: пользователь `app\_user` подключается к PostgreSQL с паролем или через сертификат.

2. **Авторизация и разграничение прав.** После входа СУБД проверяет, какие действия разрешены. Пользователь может иметь право читать таблицу, но не изменять её.

```
CREATE ROLE analyst LOGIN PASSWORD 'strong_password';
GRANT SELECT ON students TO analyst;
REVOKE INSERT, UPDATE, DELETE ON students FROM analyst;
```

1. **Принцип минимальных привилегий.** Каждому пользователю и приложению выдаются только те права, которые необходимы для работы. Веб-приложению не следует работать под суперпользователем БД.
2. **Ролевое управление доступом.** Права назначают не каждому пользователю отдельно, а ролям: `analyst`, `teacher`, `admin`, `app\_service`. Это упрощает сопровождение.
3. **Шифрование при передаче.** Соединение между клиентом и сервером должно быть защищено TLS/SSL, чтобы пароль и данные не передавались открытым текстом по сети.
4. **Шифрование при хранении.** Данные могут шифроваться на уровне диска, файловой системы, `tablespace`, отдельных колонок или приложения. Особенно это важно для персональных данных, платёжной информации и документов.
5. **Аудит.** Система должна фиксировать важные действия: входы, неудачные попытки доступа, изменение прав, удаление записей, изменение критических таблиц. Аудит нужен для расследования инцидентов.
6. **Резервное копирование и восстановление.** Безопасность включает доступность. Если данные уничтожены ошибкой или атакой, должна быть возможность восстановить их из резервной копии.
7. **Защита от SQL-инъекций.** Приложение должно использовать параметризованные запросы, а не склеивать SQL строками.

Опасный пример:

```
-- плохо: строка запроса склеивается с вводом пользователя
SELECT * FROM users WHERE login = '...';
```

Правильная идея: параметры передаются отдельно от текста запроса.

Сводная таблица:

	Угроза
Мера защиты	


```

|:-----|:-----
↪ -----|
|               чужой пользователь читает таблицу |
↪ роли, `GRANT`, `REVOKE` |
|               перехват данных в сети |
↪ TLS/SSL |
|               кража диска |
↪ шифрование хранения |
|               ошибка администратора |
↪ резервные копии |
|               вредоносный SQL через форму |
↪ параметризованные запросы |
|               незаметное удаление данных |
↪ аудит |

```

Экзаменационный вывод: безопасность БД — это сочетание технических, организационных и программных мер. Надёжная система одновременно проверяет пользователя, ограничивает права, шифрует данные, пишет журнал действий и позволяет восстановиться после сбоя.

Безопасность БД включает защиту от несанкционированного доступа, утечки, повреждения, уничтожения и злоупотребления правами. Она строится слоями: аутентификация, авторизация, шифрование, аудит, резервное копирование, сетевая защита, маскирование и контроль целостности.

Аутентификация отвечает на вопрос «кто подключается». Используются пароли, сертификаты, Kerberos, LDAP и другие механизмы. Авторизация отвечает на вопрос «что разрешено». В SQL права выдаются ролям и пользователям через GRANT и отзываются через REVOKE.

```

CREATE ROLE analyst LOGIN PASSWORD 'strong_password';
GRANT SELECT ON students TO analyst;
REVOKE INSERT, UPDATE, DELETE ON students FROM analyst;

```

Принцип минимальных привилегий требует выдавать только необходимые права. Приложение не должно подключаться под суперпользователем. Лучше иметь отдельные роли: чтение, запись, миграции, администрирование.

Шифрование защищает данные при передаче и хранении. TLS/SSL защищает соединение клиент-сервер. Шифрование диска, файлов, резервных копий или отдельных столбцов защищает данные при компрометации носителя. При этом нужно защищать ключи: шифрование бесполезно, если ключ лежит рядом с данными в открытом виде.

Аудит фиксирует важные действия: входы, ошибки доступа, изменение прав, чтение чувствительных данных, удаление и массовые обновления. Журналы помогают расследовать инциденты. Резервное копирование защищает от потери данных из-за ошибок, сбоев и атак. Ваксуп нужно не только создавать, но и регулярно проверять восстановление.

Маскирование и псевдонимизация нужны, когда разработчикам или аналитикам не требуются реальные персональные данные. Row-level security и column privileges позволяют ограничивать доступ к строкам и столбцам.

graph TD

```

S[Безопасность БД] --> A[Аутентификация]
S --> P[Права и роли]
S --> E[Шифрование]
S --> L[Аудит]
S --> B[Ваксуп]

```

```
S --> M[Маскирование]
S --> I[Ограничения целостности]
```

Ограничения целостности тоже являются защитой: первичные ключи, внешние ключи, CHECK, NOT NULL предотвращают логическую порчу данных. Надёжная безопасность — это не один замок, а коридор шлюзов: пользователь проверяется, права ограничиваются, соединение шифруется, действия пишутся в журнал, а данные можно восстановить.

— ## 20. Обеспечение целостности реляционных баз данных

Конкретика для ответа

Целостность реляционной базы данных — это свойство БД находиться в корректном, непротиворечивом состоянии относительно заданных правил предметной области и реляционной модели. Целостность обеспечивается не только приложением, но и самой СУБД через ограничения.

Основные механизмы целостности:

1. Первичный ключ (**PRIMARY KEY**). Гарантирует уникальность строк и запрещает NULL в ключевых атрибутах.

```
CREATE TABLE groups (
    group_id integer PRIMARY KEY,
    group_name text NOT NULL
);
```

Нельзя создать две группы с одинаковым group_id.

2. Внешний ключ (**FOREIGN KEY**). Обеспечивает ссылочную целостность: значение в дочерней таблице должно существовать в родительской.

```
CREATE TABLE students (
    student_id integer PRIMARY KEY,
    full_name text NOT NULL,
    group_id integer NOT NULL REFERENCES groups(group_id)
);
```

Нельзя записать студента в группу 999, если такой группы нет.

3. Уникальность (**UNIQUE**). Запрещает повторение значений в одном или нескольких столбцах.

```
ALTER TABLE students ADD CONSTRAINT uq_students_email UNIQUE(email);
```

4. Обязательность (**NOT NULL**). Запрещает отсутствие значения. Например, ФИО студента не должно быть пустым.

5. Проверочные ограничения (**CHECK**). Задают допустимый диапазон или условие.

```
CREATE TABLE marks (
    student_id integer REFERENCES students(student_id),
    discipline_id integer,
    mark integer CHECK (mark BETWEEN 2 AND 5)
);
```

6. Ограничения предметной области. Например, дата окончания обучения не может быть раньше даты поступления:

```
CHECK (graduated_at IS NULL OR graduated_at >= enrolled_at)
```

7. **Транзакции**. Поддерживают согласованность при выполнении нескольких связанных изменений. Например, при удалении группы надо либо перенести всех студентов, либо запретить удаление.

Варианты поведения внешнего ключа:

```
FOREIGN KEY (group_id) REFERENCES groups(group_id)
```

ON DELETE RESTRICT
ON UPDATE CASCADE

RESTRICT запрещает удалить группу, если на неё ссылаются студенты. CASCADE может автоматически распространить изменение ключа. Использовать каскадное удаление надо осторожно, чтобы случайно не удалить важные зависимые данные.

Экзаменационный пример нарушения целостности:

	Нарушение
↪ Почему плохо	Как запретить
:----- :-----	
↪ ----- :-----	
↪ ----- :-----	
	две строки с одним `student\_id`
↪ студент не идентифицируется однозначно	`PRIMARY KEY`
	студент ссылается на несуществующую группу
↪ ссылка «в никуда»	`FOREIGN KEY`
	оценка равна 8
↪ нарушена шкала оценивания	`CHECK`
	ФИО равно `NULL`
↪ неполная запись	`NOT NULL`

Вывод: целостность — это не косметическая проверка, а фундаментальное свойство реляционной БД. Если ограничения находятся в СУБД, они действуют для всех приложений одинаково и не зависят от добросовестности отдельного программиста.

Целостность реляционной БД означает корректность, непротиворечивость и соответствие данных правилам предметной области. Она должна обеспечиваться самой СУБД, а не только приложением, потому что к базе могут обращаться разные программы.

Структурная целостность требует соответствия таблиц, атрибутов, типов и ключей. Первичный ключ идентифицирует строку, NOT NULL запрещает отсутствие обязательного значения, типы данных не позволяют записать произвольную строку в дату.

Ссылочная целостность требует корректности связей между таблицами. Внешний ключ не позволяет создать студента в несуществующей группе.

```
CREATE TABLE groups (group_id integer PRIMARY KEY);
CREATE TABLE students (
    student_id integer PRIMARY KEY,
    group_id integer NOT NULL REFERENCES groups(group_id)
);
```

Доменная целостность ограничивает допустимые значения. Например, оценка от 2 до 5:

```
CREATE TABLE exam_results (
    student_id integer,
    discipline_id integer,
    grade integer CHECK (grade BETWEEN 2 AND 5),
    PRIMARY KEY (student_id, discipline_id)
);
```

Семантическая целостность выражает бизнес-правила: нельзя оформить заказ без остатка товара, нельзя поставить оценку по дисциплине вне учебного плана, нельзя назначить преподавателя в две аудитории одновременно. Простые правила задаются CHECK, сложные — триггерами, процедурами, транзакциями и логикой приложения.

Транзакционная целостность обеспечивает выполнение связанных операций как

единого целого. Если заказ создан, позиции добавлены, остатки списаны и оплата проведена, всё это должно быть зафиксировано вместе или отменено вместе.

```
graph TD
    I[Целостность] --> S[Структурная]
    I --> R[Ссылочная]
    I --> D[Доменная]
    I --> B[Семантическая]
    I --> T[Транзакционная]
```

При удалении связанных строк используются действия ON DELETE RESTRICT, CASCADE, SET NULL, SET DEFAULT. Выбор зависит от смысла. Для учебной системы удаление группы со студентами обычно запрещают, а не каскадно удаляют студентов.

Нормализация также поддерживает целостность, потому что уменьшает дублирование. Если название группы хранится один раз в groups, оно не расходится в разных строках студентов. Итак, целостность обеспечивается ключами, ограничениями, доменами, транзакциями, нормализацией и бизнес-правилами.

— ## 21. Структурная, ссылочная, языковая и семантическая целостность

Конкретика для ответа

В учебных пособиях по реляционным БД часто выделяют несколько видов целостности: структурную, ссылочную, языковую и семантическую. Их удобно различать по вопросу, на который они отвечают.

Структурная целостность отвечает на вопрос: «Правильно ли данные организованы как реляционные отношения?». Она требует, чтобы таблицы имели строки и столбцы, строки были однотипными, атрибуты имели допустимые типы, каждая строка могла быть идентифицирована ключом. Пример нарушения: таблица students не имеет первичного ключа, а в одном поле phones хранится список телефонов через запятую. Это нарушает структурную строгость и атомарность.

SQL-пример обеспечения структурной целостности:

```
CREATE TABLE students (
    student_id integer PRIMARY KEY,
    full_name text NOT NULL,
    birth_date date
);
```

Ссылочная целостность отвечает на вопрос: «Не ссылается ли запись на несуществующий объект?». Она обеспечивается внешними ключами. Пример: студент не может быть записан в группу, которой нет в таблице groups.

```
CREATE TABLE students (
    student_id integer PRIMARY KEY,
    full_name text NOT NULL,
    group_id integer NOT NULL REFERENCES groups(group_id)
);
```

Если выполнить INSERT INTO students VALUES (1, 'Иванов', 999), а группы 999 нет, СУБД должна отклонить операцию.

Языковая целостность отвечает на вопрос: «Можно ли корректно обращаться к данным средствами языка БД?». Она связана с тем, что операции должны выполняться в рамках формального языка запросов и не должны нарушать структуру отношений. Например, запрос должен ссылаться на существующие таблицы и атрибуты, типы в выражениях должны быть совместимы, результат запроса должен быть корректным отношением.

Пример языковой ошибки:

```
SELECT unknown_column FROM students;
```

СУБД отклонит запрос, потому что атрибут не определён в схеме.

Семантическая целостность отвечает на вопрос: «Соответствуют ли данные правилам предметной области?». Она задаётся ограничениями, триггерами, бизнес-правилами и иногда логикой приложения. Пример: оценка должна быть от 2 до 5, дата экзамена не может быть раньше даты начала семестра, сумма заказа не может быть отрицательной.

```
CREATE TABLE marks (
    mark integer NOT NULL CHECK (mark BETWEEN 2 AND 5),
    exam_date date NOT NULL
);
```

Сравнение:

Смысл	Вид целостности	Пример ограничения
↪	Структурная	корректная
↪ табличная организация		`PRIMARY
↪ KEY`, атомарные поля		
↪	Ссылочная	ссылки только на
↪ существующие строки		
↪ `FOREIGN KEY`		
↪	Языковая	корректность
↪ операций языка БД		существующие столбцы,
↪ совместимые типы		
↪	Семантическая	соответствие
↪ предметной области		`CHECK (mark
↪ BETWEEN 2 AND 5)`		

Экзаменационный вывод: структурная целостность описывает форму данных, ссылочная — связи между таблицами, языковая — корректность обращения к данным, семантическая — смысловые правила предметной области.

Структурная целостность означает, что данные соответствуют структуре реляционной модели: таблицы имеют атрибуты, атрибуты имеют домены, строки идентифицируются ключами, значения соответствуют типам. Пример: `student\_id` является первичным ключом, `full\_name` не может быть NULL, `birth\_date` имеет тип даты.

```
CREATE TABLE students (
    student_id integer PRIMARY KEY,
    full_name text NOT NULL,
    birth_date date
);
```

Ссылочная целостность означает корректность межтабличных ссылок. Если студент ссылается на группу, такая группа должна существовать. Это реализуется внешними ключами. При удалении родительской записи СУБД должна либо запретить удаление, либо выполнить заданное действие: `cascade`, `set null`, `restrict`.

Языковая целостность означает, что операции с данными выполняются через разрешённые языковые средства СУБД и подчиняются их правилам. Пользователь не должен обходить SQL и напрямую править файлы таблицы, нарушая ключи и ограничения. Также языковые средства должны позволять задавать права, ограничения и транзакции.

```
GRANT SELECT ON students TO analyst;
REVOKE DELETE ON students FROM analyst;
```

Семантическая целостность означает соответствие данных смыслу предметной области. Например, оценка должна быть от 2 до 5; дата окончания договора не раньше даты начала; студент не получает оценку по дисциплине, которую не изучал. Часть правил выражается CHECK, часть требует триггеров или процедур.

```
CREATE TABLE practice (  
    start_date date NOT NULL,  
    end_date date NOT NULL,  
    CHECK (end_date >= start_date)  
);  
  
graph TD  
    C[Целостность БД] --> A[Структурная: типы, ключи, домены]  
    C --> B[Ссылочная: внешние ключи]  
    C --> L[Языковая: операции через SQL и права]  
    C --> S[Семантическая: бизнес-правила]
```

База может быть структурно и ссылочно корректной, но семантически неверной. Например, все ключи существуют, но преподаватель одновременно назначен на две пары. Поэтому полноценная целостность требует сочетания всех видов: структура задаёт каркас, ссылки соединяют таблицы, язык контролирует операции, семантика делает данные правдивыми.

— ## 22. Содержание этапов проектирования базы данных

Конкретика для ответа

Этапы проектирования базы данных — это последовательность работ от анализа предметной области до реализации и сопровождения схемы в конкретной СУБД.

1. Анализ предметной области. Определяются цели системы, пользователи, документы, отчёты, операции и ограничения. Например, для учебной системы выясняют, что нужно хранить студентов, группы, дисциплины, преподавателей, оценки и ведомости.

2. Сбор требований. Фиксируются функциональные требования: какие данные вводятся, какие отчёты формируются, какие запросы выполняются чаще всего. Также определяются нефункциональные требования: объём данных, число пользователей, требования к безопасности, резервному копированию, скорости работы.

3. Инфологическое проектирование. Строится концептуальная модель: сущности, атрибуты, связи, кардинальности. Например, Группа 1:N Студент, Студент M:N Дисциплина.

4. Даталогическое проектирование. Концептуальная модель преобразуется в логическую схему таблиц. Выбираются первичные ключи, внешние ключи, типы связей, таблицы для связей многие-ко-многим.

5. Нормализация. Проверяются функциональные зависимости, устраняются избыточность и аномалии. Таблицы приводятся как минимум к 3НФ или BCNF, если это оправдано.

6. Физическое проектирование. Выбираются конкретные типы данных, индексы, ограничения, партиционирование, настройки хранения. Например, student_id integer, full_name varchar(200), индекс по group_id.

7. Реализация. Создаются SQL-скрипты: CREATE TABLE, CREATE INDEX, ALTER TABLE, ограничения, роли и представления.

8. Загрузка и проверка данных. Выполняется импорт начальных данных, проверяются ограничения, тестируются основные запросы и отчёты.

9. Эксплуатация и сопровождение. Проводятся резервное копирование, мониторинг производительности, обновление схемы, аудит доступа, оптимизация запросов.

Схема:

flowchart TD

```

A[Анализ предметной области] --> B[Требования]
B --> C[ER-модель]
C --> D[Логическая схема таблиц]
D --> E[Нормализация]
E --> F[Физическая схема]
F --> G[SQL-реализация]
G --> H[Тестирование и эксплуатация]

```

Конкретный пример результата этапов:

Этап	Результат для учебной БД
Анализ	студенты, группы, дисциплины, оценки
ER-модель	принадлежит группе, получает оценки
Логическая схема	`groups`, `disciplines`, `marks`
Нормализация	название группы вынесено в `groups`
Физическая схема	индексы по `group\_id`, `student\_id`
Реализация	SQL-скрипт создания БД

Вывод: проектирование БД — это не только рисование таблиц. Это управляемый процесс, который начинается с понимания предметной области и заканчивается рабочей, защищённой и производительной схемой в СУБД.

Проектирование БД начинается с анализа требований. Нужно выяснить, какие данные хранятся, кто ими пользуется, какие операции выполняются, какие отчёты нужны, какие есть ограничения безопасности, объёма и производительности. Для учебной системы выясняют правила групп, дисциплин, оценок, пересдач, ролей пользователей. Следующий этап — инфологическое проектирование. Строится концептуальная модель предметной области: сущности, атрибуты, связи, кардинальности, бизнес-правила. Результат — ER-диаграмма и текстовое описание.

Далее выполняется даталогическое проектирование. Концептуальная модель переводится в логическую реляционную схему: таблицы, столбцы, первичные и внешние ключи, связи М:Н через промежуточные таблицы. Затем схема нормализуется: устраняются повторяющиеся группы, частичные и транзитивные зависимости.

Физическое проектирование адаптирует схему к конкретной СУБД: выбираются типы данных, индексы, партиционирование, схемы, роли, права, стратегия backup. Здесь учитываются реальные запросы и нагрузка.

После этого выполняется реализация: SQL-скрипты, миграции, таблицы, ограничения, индексы, представления, функции, роли. Затем загрузка данных и проверка качества: дубли, отсутствующие справочники, неверные даты, нарушения внешних ключей.

Тестирование проверяет типовые сценарии, ограничения, права, транзакции, производительность, планы запросов. Эксплуатация включает мониторинг, резервное копирование, восстановление, миграции, оптимизацию и сопровождение.

graph TD

```

A[Анализ требований] --> B[Инфологическая модель]
B --> C[Даталогическая схема]

```

```

C --> D[Нормализация]
D --> E[Физическое проектирование]
E --> F[Реализация]
F --> G[Загрузка и проверка]
G --> H[Тестирование]
H --> I[Эксплуатация]
I -. новые требования .-> A

```

Проектирование итерационно: новые требования могут вернуть к концептуальной модели, проблемы производительности — к физической схеме, ошибки данных — к ограничениям. Хороший ответ на экзамене должен раскрывать содержание этапов, а не просто перечислять названия.

— ## 23. Основные категории инструкций языка запросов SQL

Конкретика для ответа

Основные категории инструкций SQL совпадают с функциональными частями языка. В ответе важно назвать категории, дать точное назначение и привести по 1–2 примера.

DDL — Data Definition Language. Инструкции определения данных. Используются для создания и изменения объектов БД.

```

CREATE SCHEMA education;
CREATE TABLE education.students (
    student_id integer PRIMARY KEY,
    full_name text NOT NULL
);
ALTER TABLE education.students ADD COLUMN email text;
DROP TABLE education.old_students;

```

DDL меняет структуру базы данных. После CREATE TABLE появляется новый объект схемы, после ALTER TABLE меняется структура таблицы.

DML — Data Manipulation Language. Инструкции изменения данных. Они работают со строками таблиц.

```

INSERT INTO education.students(student_id, full_name)
VALUES (1, 'Иванов Иван');

```

```

UPDATE education.students
SET email = 'ivanov@example.com'
WHERE student_id = 1;

```

```

DELETE FROM education.students
WHERE student_id = 1;

```

DQL — Data Query Language. Инструкции выборки. Главная команда — SELECT. Она может включать WHERE, JOIN, GROUP BY, HAVING, ORDER BY, подзапросы и агрегатные функции.

```

SELECT g.group_name, COUNT(*) AS cnt
FROM students s
JOIN groups g ON g.group_id = s.group_id
GROUP BY g.group_name
ORDER BY cnt DESC;

```

DCL — Data Control Language. Инструкции управления правами.

```

GRANT SELECT ON education.students TO analyst;
REVOKE SELECT ON education.students FROM analyst;

```

TCL — Transaction Control Language. Инструкции управления транзакциями.

```

BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;

```


COMMIT;

Если нужно отменить изменения:

ROLLBACK;

SAVEPOINT позволяет откатить не всю транзакцию, а её часть:

BEGIN;

INSERT INTO orders(order_id) VALUES (10);

SAVEPOINT before_items;

INSERT INTO order_items(order_id, product_id) VALUES (10, 99);

ROLLBACK TO before_items;

COMMIT;

Итоговая таблица:

Категория	Назначение	Основные команды
↪ Что меняет		
:----- :----- :-----		
↪ ----- :-----		
DDL	структура	`CREATE`, `ALTER`, `DROP`
↪ схему БД		
DML	данные	`INSERT`, `UPDATE`, `DELETE`
↪ строки таблиц		
DQL	выборка	`SELECT`
↪ возвращает результат		
DCL	права	`GRANT`, `REVOKE`
↪ доступ пользователей		
TCL	транзакции	`BEGIN`, `COMMIT`, `ROLLBACK`
↪ состояние транзакции		

Экзаменационный вывод: SQL является не одним оператором SELECT, а полным языком управления реляционной БД: он описывает структуру, изменяет данные, выполняет запросы, управляет доступом и обеспечивает транзакционность.

Категории SQL отражают полный жизненный цикл работы с базой. DDL создаёт и изменяет структуру: CREATE TABLE, ALTER TABLE, DROP TABLE, CREATE INDEX. DQL получает данные через SELECT. DML изменяет содержимое: INSERT, UPDATE, DELETE, MERGE. DCL управляет доступом: GRANT, REVOKE. TCL управляет транзакциями: BEGIN, COMMIT, ROLLBACK, SAVEPOINT.

```
CREATE TABLE products (  
    product_id integer PRIMARY KEY,  
    name text NOT NULL,  
    price numeric CHECK (price >= 0)  
);  
ALTER TABLE products ADD COLUMN is_active boolean DEFAULT true;
```

```
INSERT INTO products VALUES (1, 'Мышь', 1000, true);  
UPDATE products SET price = 1100 WHERE product_id = 1;  
DELETE FROM products WHERE is_active = false;
```

```
SELECT name, price FROM products  
WHERE price > 500  
ORDER BY price;
```

```
GRANT SELECT ON products TO analyst;  
REVOKE UPDATE ON products FROM analyst;
```

BEGIN;

```
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;
```

Категория	Назначение	Примеры
DDL	структура	CREATE, ALTER, DROP
DQL	выборка	SELECT
DML	изменение строк	INSERT, UPDATE, DELETE
DCL	права	GRANT, REVOKE
TCL	транзакции	BEGIN, COMMIT, ROLLBACK

graph LR

```
DDL[DDL: структура] --> DML[DML: данные]
DML --> DQL[DQL: запросы]
DCL[DCL: права] --> DQL
TCL[TCL: транзакции] --> DML
```

SQL декларативен: пользователь описывает требуемый результат или изменение, а СУБД выбирает физический план выполнения. Это поддерживает независимость данных и оптимизацию.

— ## 24. Нереляционные решения БД: key-value, документные, колоночные, графовые
Конкретика для ответа

Нереляционные БД (NoSQL) — это класс систем хранения данных, которые не используют реляционную таблицу как единственный основной способ представления данных. Они появились как ответ на задачи горизонтального масштабирования, хранения полуструктурированных данных, высоких нагрузок и гибких схем.

Key-value БД хранят данные как пары «ключ — значение». Ключ уникален, значение может быть строкой, JSON, бинарным объектом. Примеры: Redis, Riak, Amazon DynamoDB. Конкретный пример: ключ user:1001, значение {"name": "Ivan", "status": "active"}. Такие БД быстры для операций GET key и SET key value, но плохо подходят для сложных запросов по множеству полей.

```
user:1 -> {name: "Иван", city: "SPb"}
session:abc123 -> {user_id: 1, expires: "2026-06-03"}
```

Документоориентированные БД хранят документы, обычно в JSON/BSON-подобном формате. Пример: MongoDB, CouchDB. Документ может содержать вложенные структуры и массивы. Это удобно, когда объект естественно хранится целиком: профиль пользователя, заказ с позициями, карточка товара.

```
{
  "student_id": 1,
  "full_name": "Иванов Иван",
  "contacts": {
    "email": "ivanov@example.com",
    "phone": "+79990000000"
  },
  "skills": ["SQL", "Python"]
}
```

Колоночные БД в NoSQL-смысле хранят данные по семействам колонок и оптимизированы для больших распределённых наборов данных. Примеры: Apache Cassandra, HBase. Они подходят для событий, телеметрии, логов и временных рядов, где важны высокая скорость записи и горизонтальное масштабирование.

Графовые БД хранят данные как вершины и рёбра. Примеры: Neo4j, JanusGraph. Они удобны, когда главное — связи: социальные графы, рекомендации, маршруты,

зависимости, fraud detection.

Сравнение:

Важно сказать: NoSQL не означает «SQL запрещён» или «реляционные БД устарели». Это набор альтернативных моделей, которые выбирают под конкретные требования. Если нужны строгие транзакции, сложные JOIN и нормализованная структура, часто лучше реляционная СУБД. Если нужны гибкие документы, граф связей или масштабируемое key-value хранилище, уместны NoSQL-решения.

Key-value БД хранят пары ключ-значение. Примеры: Redis, DynamoDB, Riak. Они хороши для кэшей, сессий, токенов, счётчиков, быстрых lookup-операций. Недостаток — слабые запросы по внутренней структуре значения.

Документно-ориентированные БД хранят JSON/BSON-документы. Примеры: MongoDB, CouchDB. Они удобны для профилей, заказов, каталогов, где объект естественно хранится целиком и может иметь вложенные поля.

Колоночные wide-column БД хранят данные по семействам столбцов и проектируются под конкретные запросы. Примеры: Cassandra, HBase, ScyllaDB. Они подходят для телеметрии, журналов событий, временных рядов и распределённых нагрузок, но плохо заменяют произвольные SQL-соединения.

```
graph LR
    A((Person: Иван)) -- FRIEND_OF --> B((Person: Пётр))
    A -- BOUGHT --> C((Product: Ногтбук))
```

Тип	Модель	Примеры
↪ Лучшее всего для		
:----- :----- :-----		
↪ ----- :-----		
↪ ---		
Key-value	ключ → значение	Redis
↪ кэш, сессии		
Документная	JSON-документ	MongoDB
↪ гибкие объекты		
Колоночная	wide-column	Cassandra
↪ большие распределённые записи		
Графовая	узлы и связи	Neo4j
↪ связи и пути		

NoSQL не отменяет реляционные СУБД. В реальных системах их часто комбинируют: PostgreSQL для транзакционных данных, Redis для кэша, MongoDB для гибких документов, Neo4j для рекомендаций. Выбор зависит от модели данных, запросов, целостности, масштабирования и транзакций.

— ## 25. Принцип хранения данных в графовой базе Neo4j. Пример модели

Конкретика для ответа

Neo4j использует **property graph model**, то есть модель графа свойств. В ней данные представлены как **узлы, отношения, метки** и **свойства**.

Узел (node) — объект предметной области. Например, студент, дисциплина, преподаватель, группа. Узел может иметь одну или несколько меток. Метка показывает тип объекта: :Student, :Discipline, :Teacher.

Отношение (relationship) — направленная связь между двумя узлами. Отношение имеет тип: STUDIES, TEACHES, BELONGS_TO. В Neo4j отношение всегда имеет начало и конец, но при запросе направление можно учитывать или игнорировать.

Свойство (property) — пара «ключ — значение», хранящая данные узла или отношения. Например, у студента свойства student_id, full_name; у отношения STUDIES свойство semester или mark.

Путь (path) — последовательность узлов и отношений. Например: студент → изучает → дисциплину → читается → преподавателем.

Пример модели учебного процесса:

flowchart LR

S[Student: Иванов] -- STUDIES mark=5 --> D[Discipline: Базы данных]

T[Teacher: Петров] -- TEACHES --> D

S -- BELONGS_TO --> G[Group: 4311]

Сурpher-пример создания графа:

```
CREATE (s:Student {student_id: 1, full_name: 'Иванов Иван'})
CREATE (g:Group {group_id: 4311, name: '4311'})
CREATE (d:Discipline {discipline_id: 10, name: 'Базы данных'})
CREATE (t:Teacher {teacher_id: 5, full_name: 'Петров Петр'})
CREATE (s)-[:BELONGS_TO]->(g)
CREATE (s)-[:STUDIES {semester: 5, mark: 5}]->(d)
CREATE (t)-[:TEACHES]->(d);
```

Запрос: найти дисциплины студента Иванова:

```
MATCH (s:Student {full_name: 'Иванов Иван'})-[:STUDIES]->(d:Discipline)
RETURN d.name;
```

Запрос: найти преподавателей, которые связаны со студентом через изучаемые дисциплины:

```
MATCH (s:Student {full_name: 'Иванов
↪ Иван'})-[:STUDIES]->(d:Discipline)-[:TEACHES]-(t:Teacher)
RETURN t.full_name, d.name;
```

Сравнение с реляционной моделью:

	Реляционная БД	
↪ Neo4j		
:----- :-----		
↪ -----		
	таблица `students`	
↪ узлы `:Student`		
	таблица `disciplines`	
↪ узлы `:Discipline`		
	таблица связи `student\disciplines`	
↪ отношения `[:STUDIES]`		
	внешний ключ	
↪ явное ребро графа		
	SQL `JOIN`	обход
↪ графа по отношениям		

Преимущество Neo4j проявляется, когда запросы зависят от связей переменной длины: найти друзей друзей, кратчайший путь, рекомендации, цепочки зависимостей. В реляционной БД такие задачи требуют множества JOIN или рекурсивных запросов. В графовой БД связь является объектом первого класса, поэтому обход графа выражается естественно.

Экзаменационный вывод: Neo4j хранит данные не в виде набора нормализованных таблиц, а как граф объектов и связей. Узлы описывают сущности, отношения описывают связи, свойства хранят значения, а Cypher позволяет искать шаблоны в графе.

Neo4j использует модель property graph — граф свойств. Основные элементы: узлы, связи, свойства, метки, типы связей и пути. Узел представляет сущность: человека, студента, товар, город, организацию. Метка классифицирует узел: Student, Teacher, Discipline. Свойства узла — пары ключ-значение, например name, age, email.

Связь соединяет два узла, имеет направление и тип: FRIEND_OF, TEACHES, PASSED, WORKS_AT. Связь тоже может иметь свойства. Например, оценка является свойством связи PASSED, потому что относится не только к студенту и не только к дисциплине, а к факту сдачи дисциплины студентом.

Путь — последовательность узлов и связей. Работа с путями делает графовые БД удобными для рекомендаций, социальных графов, маршрутов и зависимостей.

graph LR

```
S((Student: Иванов)) -- BELONGS_TO --> G((Group: ИС-101))
S -- PASSED {grade:5} --> D((Discipline: Базы данных))
T((Teacher: Сидоров)) -- TEACHES --> D
```

Создание модели на Cypher:

```
CREATE (s:Student {student_id: 1, full_name: 'Иванов Иван'})
CREATE (g:Group {group_id: 101, name: 'ИС-101'})
CREATE (d:Discipline {discipline_id: 10, name: 'Базы данных'})
CREATE (t:Teacher {teacher_id: 5, full_name: 'Сидоров С. С.'})
CREATE (s)-[:BELONGS_TO]->(g)
CREATE (s)-[:PASSED {grade: 5}]->(d)
CREATE (t)-[:TEACHES]->(d);
```

Запрос дисциплин и преподавателей студента:

```
MATCH (s:Student {full_name: 'Иванов
↪ Иван'})-[p:PASSED]->(d:Discipline)-[:TEACHES]-(t:Teacher)
```

```
RETURN d.name AS discipline, p.grade AS grade, t.full_name AS teacher;
```

	Составляющая	Определение
↪	Пример	
	:-----	:-----
↪	-----	-----
↪	-----	
	Узел	объект предметной области
↪	`(:Student)`	
	Метка	класс узла
↪	`Student`, `Teacher`	
	Связь	направленное отношение
↪	`-[:TEACHES]->`	
	Тип связи	имя отношения
↪	`PASSED`	
	Свойство	значение узла или связи
↪	`grade: 5`	
	Путь	цепочка узлов и связей
↪	студент → дисциплина → преподаватель	

В реляционной БД связь обычно прячется во внешнем ключе и раскрывается через JOIN. В Neo4j связь является объектом первого класса: она явно хранится, имеет тип, направление и свойства. Поэтому Neo4j особенно полезна там, где главный вопрос звучит не «какие строки подходят», а «как объекты связаны между собой».

— ## Контрольный чек-лист конкретики перед экзаменом

Перед сдачей письменного ответа проверь каждый билет по четырём критериям:

1. **Есть точное определение.** Например, не «индекс ускоряет», а «индекс — дополнительная структура данных, хранящая значения ключа и ссылки на строки для ускорения поиска».
2. **Есть формальный признак.** Для 2НФ — полная функциональная зависимость от всего составного ключа; для BCNF — каждый детерминант является суперключом; для внешнего ключа — значение должно существовать в родительском отношении.
3. **Есть предметный пример.** Студенты, группы, дисциплины, оценки, заказы, товары, счета — конкретика лучше абстрактного дыма.
4. **Есть SQL, таблица или схема.** На письменном экзамене это выглядит убедительнее, чем сплошной текст.

Короткая формула качественного ответа: **определение → механизм → пример → ограничение/исключение → вывод**. Такой порядок даёт ответу структуру и не позволяет расползтись в общие слова.

Contents

Программная инженерия	1
1. Модель SOA для организации машинно-машинного взаимодействия. Схема и примеры реализации	1
2. Интеграция на основе SOAP-протокола. Схемы	3
3. Монолитная и микросервисная архитектура. Схема	5
4. Интеграция на основе REST и GraphQL. Схема и примеры реализации . . .	7
5. Интеграция на основе WebSocket и WebHook. Схема и примеры реализации	9
6. Протоколы взаимодействия между web-приложениями по сети Интернет .	13
7. Шаблоны интеграций информационных систем по структуре взаимодействия	14
8. Шаблоны интеграций информационных систем по типу обмена данными. Схемы	16
9. Модели разработки ПО: итеративная, инкрементная, водопадная, спиральная	19
10. Наборы практик Scrum и Kanban. Схемы и примеры использования . . .	22
11. Уровни и типы требований по К. Вигерсу. Схема и комментарии к схеме .	24
12. Свойства качественных требований. Пояснения с примерами	25
13. Методы анализа качества интерфейсов	26
14. Проектирование интерфейсов на основе UX/UI-методов	27
15. Протоколы взаимодействия между сервисами по сети Интернет	30
16. Типы пользовательских интерфейсов	31
17. Метод анализа пользовательского интерфейса по метрикам Констайна и Локвуда	32
18. Метод анализа пользовательского интерфейса по метрикам GOMS	33
19. Построение карты эмпатии	34
20. Модели качества программного обеспечения	35
21. Контроль и обеспечение качества ПО	37
22. Классификация тестирования по способу тестирования. Примеры	38
23. Классификация тестирования по целям тестирования. Примеры	39
24. Классификация тестирования по уровням тестирования. Примеры	40
25. Классификация дефектов и жизненный цикл дефекта. Схема	42
26. Организация и управление процессом тестирования. Схемы	43
27. Функциональное назначение Docker. Контейнеризация и виртуализация	44
28. Архитектура платформы контейнеризации Docker	45
29. Назначение Apache Airflow. DAG-файл, task, operators	46
30. Конвейер обработки данных. Примеры. ETL и ELT	47
31. Архитектура Airflow. Принципы построения пайплайнов	49

Программная инженерия

1. Модель SOA для организации машинно-машинного взаимодействия. Схема и примеры реализации

SOA, или сервис-ориентированная архитектура, представляет способ построения информационной системы как совокупности самостоятельных сервисов, каждый из которых выполняет отдельную бизнес-функцию и предоставляет её другим компонентам через формализованный интерфейс. Главная идея SOA состоит не в том,

чтобы просто «разбить программу на части», а в том, чтобы организовать машинно-машинное взаимодействие между независимыми программными компонентами. В такой архитектуре сервис становится единицей интеграции: он принимает запрос, выполняет операцию и возвращает результат в согласованном формате.

В контексте интеграции прикладных программных сред SOA удобна тем, что позволяет связывать разные информационные системы без прямого объединения их внутреннего кода. Например, система управления заказами может обращаться к сервису проверки остатков на складе, сервису расчёта доставки и сервису оплаты. Каждая из этих подсистем может быть разработана на разных языках, работать на разных серверах и использовать разные базы данных, но для внешнего потребителя это скрыто за интерфейсом сервиса. В учебном смысле SOA можно рассматривать как «контрактную» архитектуру: важнее не внутренняя реализация, а описание доступных операций, формата запроса, формата ответа, ограничений и ошибок.

Типовая схема SOA включает три роли: поставщик сервиса, потребитель сервиса и, в классическом варианте, реестр сервисов. Поставщик публикует сервис и описание его интерфейса. Потребитель находит сервис, формирует запрос и получает ответ. Реестр нужен для каталогизации сервисов, особенно в больших корпоративных средах, где количество сервисов может быть значительным. На практике реестр может быть заменён документацией API, API Gateway, сервис-дискавери или корпоративной шиной.



Figure 1: Схема 1

Пример реализации: интернет-магазин оформляет заказ. Клиентское приложение обращается к сервису заказов. Сервис заказов не хранит у себя всё, а вызывает сервис склада для резервирования товара, сервис платежей для списания средств и сервис уведомлений для отправки письма. Если сервис уведомлений временно меняет внутреннюю реализацию, остальная система не должна ломаться, пока внешний контракт остаётся прежним.

SOA полезна для организаций, где уже существуют разные прикладные системы и требуется объединить их в единый процесс. Её сильные стороны: повторное использование сервисов, независимость реализации, возможность поэтапной модернизации, формализация взаимодействия. Слабые стороны: усложнение инфраструктуры, необходимость управления контрактами, версиями, безопасностью и мониторингом. При плохом проектировании SOA превращается в хаотичный набор удалённых вызовов, где каждая операция зависит от множества других. Поэтому сервисы должны выделяться по устойчивым бизнес-функциям, а не по случайным техническим классам.

Сервис в SOA — автономная программная единица, предоставляющая внешним потребителям одну или несколько операций через явно описанный контракт.

Контракт сервиса — формальное описание имени операции, входных параметров, выходных данных, ошибок, ограничений безопасности и правил использования.

Слабая связанность означает, что потребитель сервиса зависит от контракта, а не от внутреннего кода, структуры базы данных или языка реализации поставщика.

Важно отделять SOA от обычного модульного программирования. Модуль в монолите

вызывается внутри одного процесса и обычно зависит от внутренней структуры приложения. Сервис в SOA вызывается по сетевому или межпроцессному интерфейсу и должен быть пригоден для независимого использования несколькими системами. Поэтому признаки SOA: наличие формального интерфейса, независимость реализации, повторное использование, сетевое взаимодействие, управление версиями и ориентация на бизнес-функции.

Классический жизненный цикл взаимодействия в SOA выглядит так: поставщик публикует описание сервиса; потребитель находит описание или получает его из документации; потребитель формирует запрос по контракту; сервис выполняет операцию; сервис возвращает ответ или формализованную ошибку. Например, операция `CheckPaymentStatus(paymentId)` должна иметь точный входной параметр `paymentId`, набор возможных статусов `PAID`, `PENDING`, `FAILED`, `CANCELLED` и описанные ошибки вроде `PAYMENT\_NOT\_FOUND`.

Итоговая логика вопроса состоит в следующем: SOA применяется не ради самого разбиения системы, а ради управляемой интеграции независимых программных систем. Её качество определяется не количеством сервисов, а устойчивостью контрактов, слабой связанностью и возможностью изменять реализацию сервиса без нарушения работы потребителей.

2. Интеграция на основе SOAP-протокола. Схемы

SOAP представляет протокол обмена структурированными сообщениями между приложениями. В интеграции информационных систем SOAP обычно используется тогда, когда требуется строго описанный контракт, формальная структура операций и повышенная предсказуемость обмена. В отличие от более лёгких HTTP API, SOAP исторически ориентирован на корпоративную интеграцию, где важны типизация, описание интерфейса, контроль ошибок и возможность использовать дополнительные стандарты безопасности и надёжной доставки.

SOAP-сообщение обычно передаётся поверх HTTP, хотя сам подход не сводится только к HTTP. Сообщение имеет XML-структуру. В нём выделяют оболочку, заголовок и тело. Заголовок может содержать служебные данные: параметры безопасности, маршрутизации, транзакционности. Тело содержит бизнес-запрос или бизнес-ответ. Интерфейс SOAP-сервиса часто описывается через WSDL. WSDL задаёт операции, входные и выходные сообщения, типы данных и адрес сервиса. За счёт этого клиент может заранее знать, какие методы доступны и какие данные нужно передать.

Упрощённая структура SOAP-сообщения:

```
<soap:Envelope>
  <soap:Header>
    <!-- служебные данные: безопасность, маршрутизация -->
  </soap:Header>
  <soap:Body>
    <GetOrderStatus>
      <orderId>12345</orderId>
    </GetOrderStatus>
  </soap:Body>
</soap:Envelope>
```

Пример реализации: система университета передаёт сведения о студенте во внешнюю государственную систему. Поскольку формат сведений должен быть



Figure 2: Схема 2

строго регламентирован, используется SOAP-сервис. Клиент формирует XML-запрос с идентификатором студента, сервис возвращает XML-ответ с данными или формализованной ошибкой. Такой подход особенно удобен, когда интеграция должна быть документирована и проверяема: структура запроса и ответа не «угадывается» из примеров, а задаётся контрактом.

Достоинства SOAP: строгая формализация контракта, поддержка сложных типов данных, развитая экосистема корпоративных стандартов, пригодность для межведомственной и банковской интеграции. Недостатки: многословность XML, более высокая сложность отладки по сравнению с REST, необходимость поддерживать WSDL и связанные схемы, большой объём сообщений. SOAP хорошо подходит для стабильных корпоративных интерфейсов, где важнее формальная надёжность, чем простота и лёгкость. Для быстрых web-интерфейсов и публичных API чаще выбирают REST или GraphQL, но SOAP остаётся значимым в системах, где контракт и регламент важнее гибкости.

SOAP — протокол обмена структурированными XML-сообщениями между программными системами, задающий стандартную оболочку сообщения, правила помещения бизнес-данных в тело и служебных данных в заголовок. **WSDL** — XML-описание web-сервиса, в котором фиксируются доступные операции, типы сообщений, формат входных и выходных данных, протокол привязки и адрес конечной точки. SOAP-сообщение состоит из Envelope, Header, Body и, при ошибке, Fault. Envelope определяет границы сообщения. Header содержит необязательную служебную информацию, например токены безопасности, идентификатор транзакции или параметры маршрутизации. Body содержит полезную нагрузку: запрос операции или ответ. Fault используется для стандартизированного описания ошибки, чтобы клиент получил не произвольный текст, а машинно-обрабатываемую структуру.

SOAP особенно уместен в системах, где важны строгая типизация, регламентированный обмен и долговременная стабильность интерфейса: банки, межведомственный документооборот, государственные информационные системы, интеграция с унаследованными корпоративными приложениями. В отличие от REST, где контракт часто описывается отдельно через документацию или OpenAPI, в SOAP контракт исторически является центральным элементом и тесно связан с WSDL и XML Schema.

SOAP — это не просто «XML по HTTP», а контрактный протокол обмена сообщениями, где структура запроса, ответа и ошибки заранее формализована. Его сильная сторона — предсказуемость и регламент, слабая — громоздкость и повышенная сложность разработки.

3. Монолитная и микросервисная архитектура. Схема

Монолитная архитектура предполагает, что основные функции приложения реализованы внутри единого программного блока. Пользовательский интерфейс, бизнес-логика, доступ к данным, обработка заказов, авторизация и другие части системы могут быть разделены логически, но развёртываются как одно приложение. В монолите изменения обычно проходят через единый цикл сборки, тестирования и поставки. Такой подход прост для небольших и средних систем, особенно на ранних этапах, когда команда мала, предметная область ещё уточняется, а инфраструктурная сложность нежелательна.

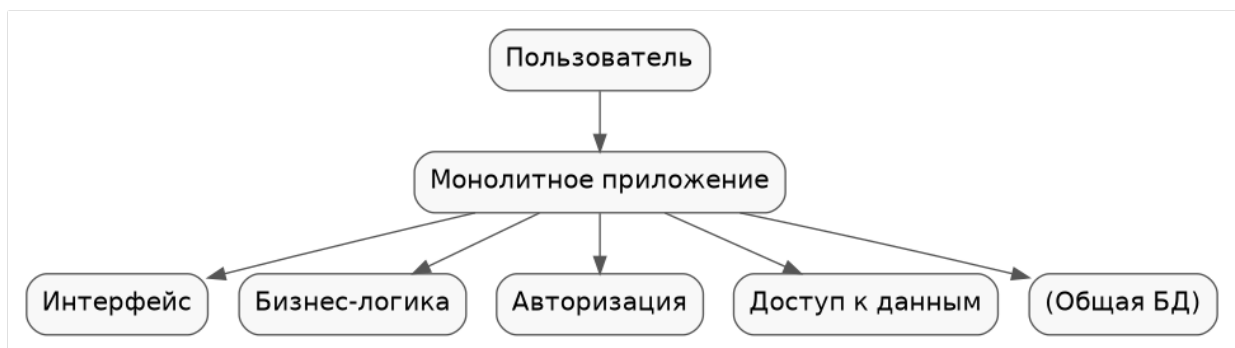


Figure 3: Схема 3

Преимущество монолита заключается в простоте разработки, локального запуска, отладки и развёртывания. Все части находятся в одном репозитории или одном проекте, вызовы между модулями обычно выполняются напрямую, без сетевых задержек. При этом монолит не обязательно является «плохой архитектурой». Качественный модульный монолит может долго оставаться удобным решением, если система не требует независимого масштабирования компонентов и частых автономных релизов.

Микросервисная архитектура строит приложение как набор небольших сервисов, каждый из которых отвечает за отдельную бизнес-возможность. Например, сервис пользователей, сервис заказов, сервис платежей, сервис уведомлений и сервис каталога товаров. Каждый микросервис может иметь собственную базу данных, собственный цикл развёртывания и собственный технологический стек.

Взаимодействие между сервисами выполняется по сетевым протоколам: HTTP/REST, SOAP, messaging, gRPC, WebSocket и т.д.

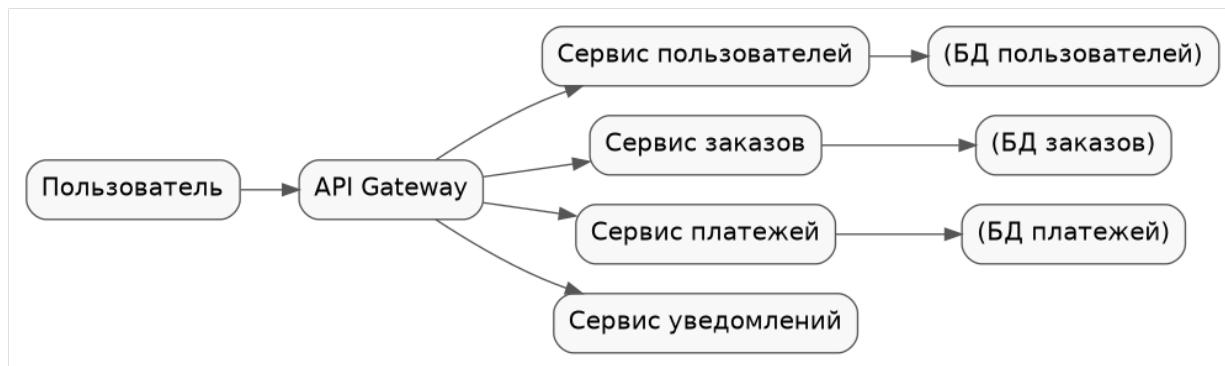


Figure 4: Схема 4

Микросервисный подход удобен, когда система большая, над ней работают несколько команд, разные части требуют разного масштабирования, а изменения нужно выпускать независимо. Например, сервис уведомлений можно масштабировать отдельно от сервиса каталога, а сервис платежей можно сделать более защищённым и изолированным. Но микросервисы приносят цену: сетевые сбои, распределённые транзакции, сложный мониторинг, управление версиями API, необходимость DevOps-инфраструктуры, логирование и трассировка запросов.

Главное различие между монолитом и микросервисами не только в количестве приложений, а в способе управления сложностью. Монолит концентрирует сложность внутри одного процесса. Микросервисы распределяют сложность между сервисами и инфраструктурой. Поэтому микросервисы не являются универсальным улучшением монолита. Если предметная область мала, команда одна, нагрузка умеренная, а релизы не требуют независимости, монолит может быть рациональнее. Если система разрастается и разные функции начинают жить разными темпами, микросервисная архитектура становится оправданной.

Монолит — архитектура, при которой приложение собирается, развёртывается и масштабируется как единая исполняемая единица. Внутри монолита могут быть модули и слои, но внешний жизненный цикл у них общий. **Микросервис** — самостоятельно развёртываемый сервис, реализующий отдельную бизнес-возможность и взаимодействующий с другими сервисами через сетевые интерфейсы. Для сравнения используются следующие критерии: единица развёртывания, единица масштабирования, границы данных, способ вызова, организационная независимость. В монолите единица развёртывания одна, база данных часто общая, вызовы локальные, а релиз затрагивает весь продукт. В микросервисной архитектуре каждый сервис имеет собственный релизный цикл, может масштабироваться отдельно, часто владеет собственной базой данных и общается с другими сервисами через API или сообщения.

Важно подчеркнуть, что микросервис не равен маленькому классу или модулю. Граница микросервиса должна проходить по бизнес-возможности: «управление заказами», «платежи», «уведомления», «каталог». Если разделить систему по техническим слоям, например отдельно «сервис контроллеров», «сервис репозитория» и «сервис сущностей», получится распределённый монолит: сетевых вызовов много, независимости мало.

Следовательно, монолит проще, дешевле в инфраструктуре и лучше подходит для старта продукта или небольшой команды. Микросервисы оправданы при большой системе, независимых командах, разных требованиях к масштабированию и частых автономных релизах. Главный риск микросервисов — перенос сложности из кода в сеть, инфраструктуру, наблюдаемость и согласованность данных.

4. Интеграция на основе REST и GraphQL. Схема и примеры реализации

REST — архитектурный стиль построения сетевого взаимодействия, чаще всего реализуемый поверх HTTP. В REST система рассматривается как набор ресурсов, к которым клиент обращается по URI. Операции над ресурсами выражаются стандартными HTTP-методами: GET для получения, POST для создания или запуска операции, PUT/PATCH для изменения, DELETE для удаления. Данные часто передаются в JSON, реже в XML или другом формате.



Figure 5: Схема 5

Пример REST-интеграции: мобильное приложение интернет-магазина отправляет GET /products для получения списка товаров, POST /orders для создания заказа и GET /orders/123 для проверки статуса. REST удобен благодаря простоте, понятной семантике HTTP и совместимости с браузерами, прокси, кэшем и инструментами тестирования API.

GraphQL решает другую проблему. В REST клиент часто получает либо слишком много данных, либо должен выполнять несколько запросов к разным эндпоинтам. GraphQL позволяет клиенту описать, какие именно поля нужны. Вместо набора URL для разных ресурсов используется единая точка входа, куда отправляется запрос

со структурой требуемых данных. Сервер возвращает ответ той формы, которую запросил клиент.

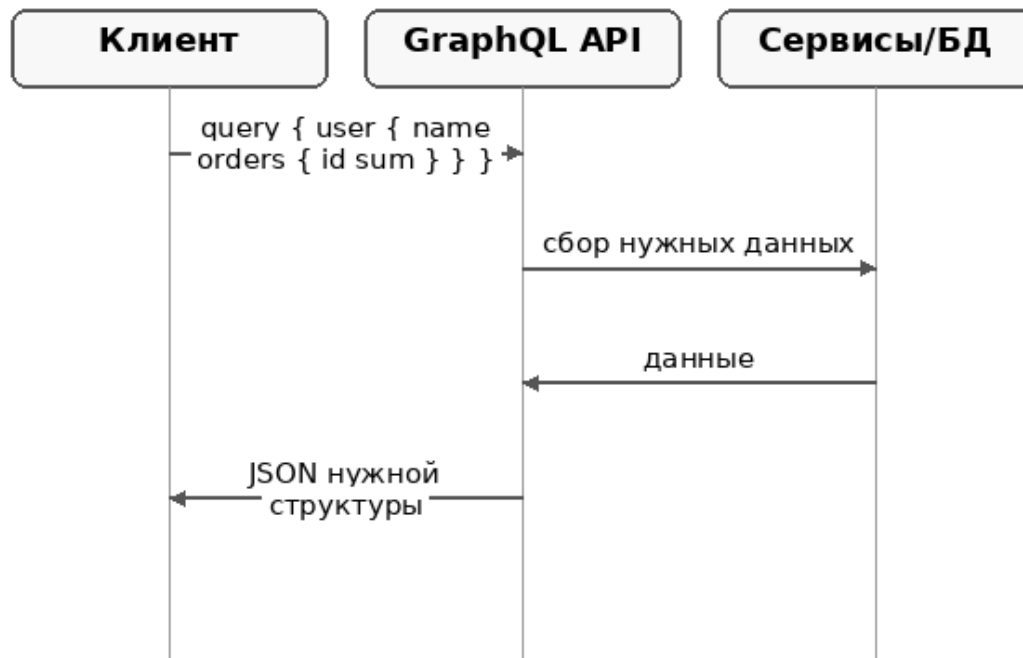


Figure 6: Схема 6

Пример GraphQL-интеграции: интерфейс личного кабинета должен показать имя пользователя, последние три заказа и количество бонусов. В REST могли бы потребоваться запросы к `/user`, `/orders`, `/bonus`. В GraphQL клиент формирует один запрос и получает ровно нужные поля. Это особенно полезно для сложных интерфейсов, мобильных приложений и агрегирующих API.

REST проще для классических CRUD-сценариев, где ресурсы стабильны и хорошо ложатся на HTTP. GraphQL удобнее, когда клиентам нужны разные представления одних и тех же данных. Но GraphQL сложнее в кэшировании, требует контроля сложности запросов и аккуратного проектирования схемы. REST легче документировать через набор эндпоинтов, GraphQL описывает API через типизированную схему. В интеграции ИС оба подхода могут сосуществовать: REST используется для базовых операций и внешних API, GraphQL — для агрегирующего слоя между фронтендом и несколькими внутренними сервисами.

REST — архитектурный стиль, в котором система представляется как набор ресурсов, идентифицируемых URI, а действия над ними выражаются стандартными методами HTTP. **Ресурс** — логически выделенный объект или коллекция предметной области: пользователь, заказ, платёж, список товаров. **Представление ресурса** — сериализованная форма его состояния, например JSON-документ.

Для REST важно назвать свойства: клиент-серверное разделение, отсутствие состояния сессии на сервере между запросами, единообразный интерфейс, кэшируемость ответов там, где это допустимо, многоуровневость системы. Пример строгой формулировки: `GET /orders/123` получает представление заказа, `POST /orders` создаёт новый заказ, `PATCH /orders/123` частично изменяет заказ, `DELETE /orders/123`

удаляет или отменяет заказ в соответствии с бизнес-правилами.

GraphQL — язык запросов и среда выполнения API, где клиент описывает структуру требуемых данных, а сервер возвращает ответ той же формы. Основные элементы GraphQL: схема типов, query для чтения, mutation для изменения, resolver для получения конкретного поля. GraphQL снижает overfetching, когда клиент получает лишние данные, и underfetching, когда для одного экрана требуется много запросов. Сравнение для экзамена: REST лучше подходит для простых ресурсных CRUD API, стандартного HTTP-кэширования и публичных интеграций. GraphQL удобен для сложных клиентских экранов, где нужно собрать разные данные за один запрос. Риск REST — рост числа эндпоинтов и избыточные ответы. Риск GraphQL — сложность контроля тяжёлых запросов, кэширования, авторизации на уровне полей и диагностики производительности.

5. Интеграция на основе WebSocket и WebHook. Схема и примеры реализации

WebSocket и WebHook применяются для сценариев, где простой запрос-ответ не всегда удобен. REST и SOAP в основном строятся вокруг инициативы клиента: клиент отправляет запрос, сервер отвечает. Но в реальных системах часто нужно передавать события: изменение статуса заказа, новое сообщение, завершение платежа, обновление курса, появление уведомления. Для таких задач используют событийные и двунаправленные механизмы.

WebSocket обеспечивает постоянное двунаправленное соединение между клиентом и сервером. После установления соединения обе стороны могут отправлять сообщения без повторного открытия HTTP-запроса. Это удобно для чатов, онлайн-игр, торговых терминалов, систем мониторинга, панелей с живыми статусами. В отличие от периодического опроса, WebSocket уменьшает задержку и избавляет от постоянных запросов «есть ли что-нибудь новое?».

Пример реализации WebSocket: система технической поддержки показывает оператору новые обращения в реальном времени. Сервер отправляет событие клиенту сразу после создания обращения. Оператору не нужно обновлять страницу, интерфейс получает событие и добавляет строку в список.

WebHook работает иначе. Это не постоянное соединение, а обратный HTTP-вызов по заранее указанному URL. Одна система регистрирует адрес, на который хочет получать события. Когда событие происходит, другая система отправляет HTTP-запрос на этот адрес. WebHook особенно часто используется для интеграции внешних сервисов: платёжная система уведомляет магазин об успешной оплате, Git-сервис уведомляет CI/CD о новом коммите, CRM уведомляет внешний сервис о новом лиде.

Различие между WebSocket и WebHook принципиально. WebSocket поддерживает живой канал и подходит для постоянного обмена. WebHook передаёт событие отдельным HTTP-запросом и подходит для асинхронных уведомлений между системами. WebSocket требует поддержки соединений и контроля их жизненного цикла. WebHook требует проверки подписи, повторной доставки при сбоях и идемпотентной обработки, потому что одно событие может прийти повторно. В интеграционной архитектуре WebSocket чаще связывает клиентский интерфейс с сервером, а WebHook чаще связывает серверные системы между собой.

WebSocket — протокол постоянного двунаправленного обмена сообщениями поверх одного установленного соединения. После начального HTTP Upgrade клиент и сервер

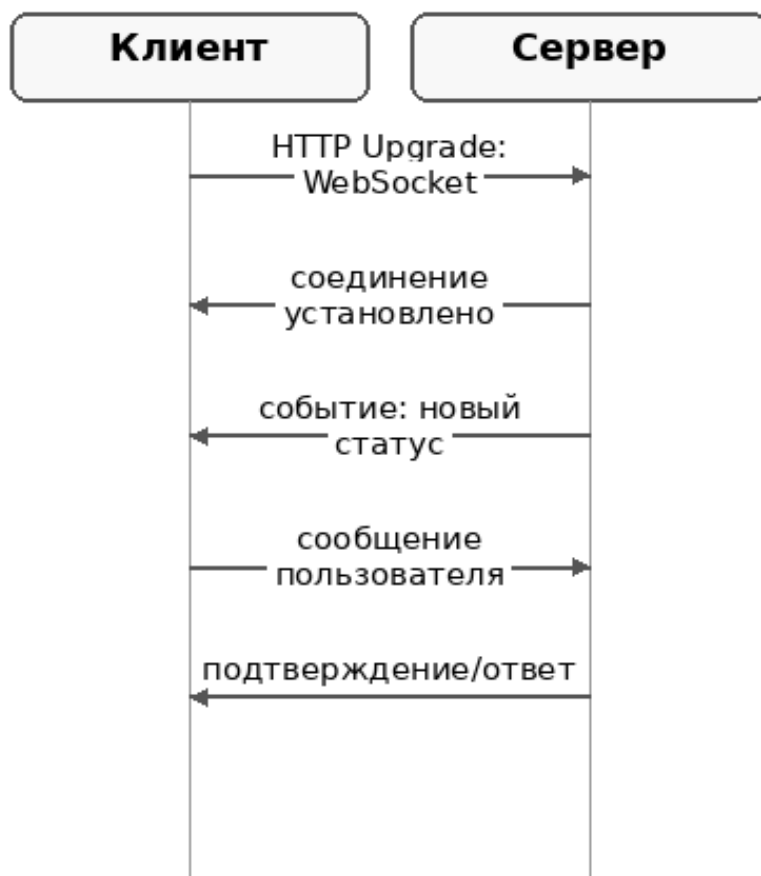


Figure 7: Схема 7

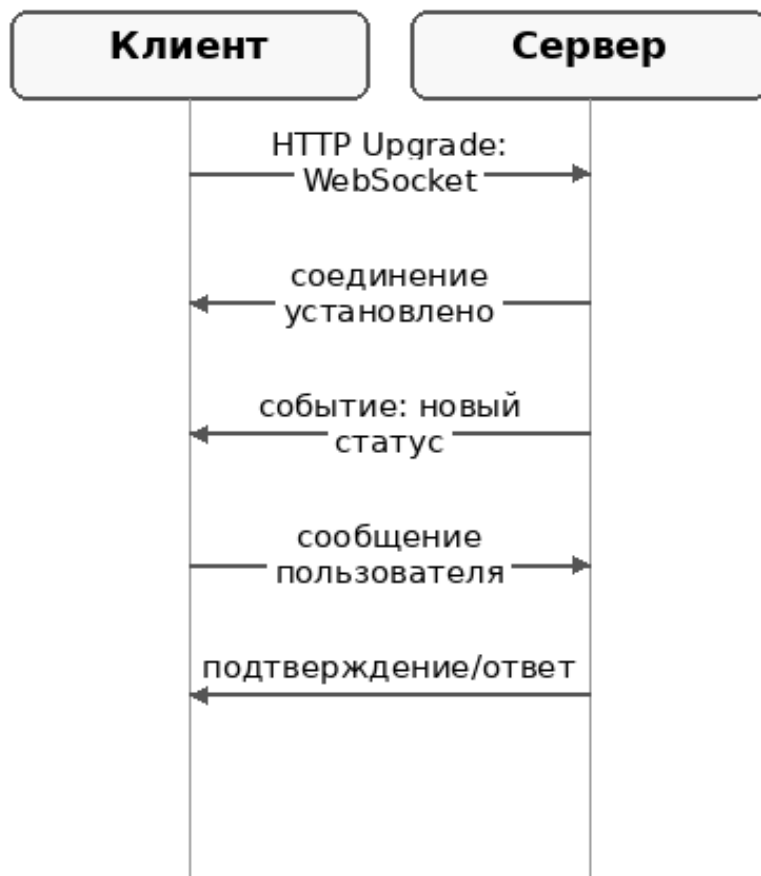


Figure 8: Схема 8



Figure 9: Схема 9

могут отправлять сообщения независимо друг от друга. **WebHook** — механизм событийной интеграции, при котором одна система отправляет HTTP-запрос на заранее зарегистрированный callback URL другой системы при наступлении события. Ключевое различие: WebSocket поддерживает живой канал связи, а WebHook выполняет отдельные обратные вызовы. WebSocket выбирают для интерактивного реального времени: чат, онлайн-игра, биржевой терминал, live-дашборд. WebHook выбирают для серверных уведомлений: платёж завершён, коммит отправлен, лид создан, документ подписан.

Для WebSocket существенны управление соединениями, heartbeat/ping-pong, переподключение, авторизация при установке соединения, ограничение числа активных клиентов. Для WebHook существенны подпись запроса, проверка источника, повторная доставка, обработка дублей и идиempотентность. Например, платёжный webhook может прийти дважды, поэтому обработчик не должен два раза начислять оплату по одному paymentId.

Экзаменационная фраза: WebSocket решает задачу низкой задержки и двунаправленного обмена, а WebHook решает задачу асинхронного уведомления между системами без постоянного соединения.

6. Протоколы взаимодействия между web-приложениями по сети Интернет

Взаимодействие web-приложений по сети Интернет строится на совокупности протоколов разных уровней. На базовом уровне обычно используется TCP/IP, который обеспечивает передачу данных между узлами сети. Поверх него располагаются прикладные протоколы, наиболее важным среди которых для web-среды является HTTP. HTTP задаёт модель запроса и ответа: клиент отправляет запрос с методом, адресом, заголовками и телом, сервер возвращает статус, заголовки и тело ответа.

HTTPS является защищённым вариантом HTTP, где обмен шифруется с использованием TLS. Для интеграции информационных систем HTTPS практически обязателен, поскольку по сети могут передаваться персональные данные, токены авторизации, платёжная информация и служебные ключи. Без защищённого канала запросы и ответы могут быть перехвачены или изменены.

На уровне форматов данных часто применяются JSON и XML. JSON легче и компактнее, поэтому широко используется в REST и GraphQL API. XML более формализован и часто связан с SOAP-интеграциями. Помимо формата сообщения важна схема взаимодействия: синхронная или асинхронная, однонаправленная или двунаправленная, запрос-ответ или событийная доставка.

REST использует возможности HTTP и представляет данные как ресурсы. SOAP передаёт XML-сообщения и опирается на строгий контракт. WebSocket используется для постоянного двунаправленного соединения. WebHook организует доставку событий через обратный HTTP-запрос. GraphQL предоставляет типизированный способ запроса данных, где клиент сам описывает нужную структуру ответа.

При выборе протокола учитывают требования к скорости, формализации, безопасности, совместимости и характеру обмена. Если нужен простой публичный API, подходит REST. Если нужен строгий корпоративный контракт, возможен SOAP. Если нужно обновлять интерфейс в реальном времени, применяют WebSocket. Если внешняя система должна уведомлять о событии, используют WebHook. Если интерфейсу нужно получать сложные агрегированные данные из нескольких

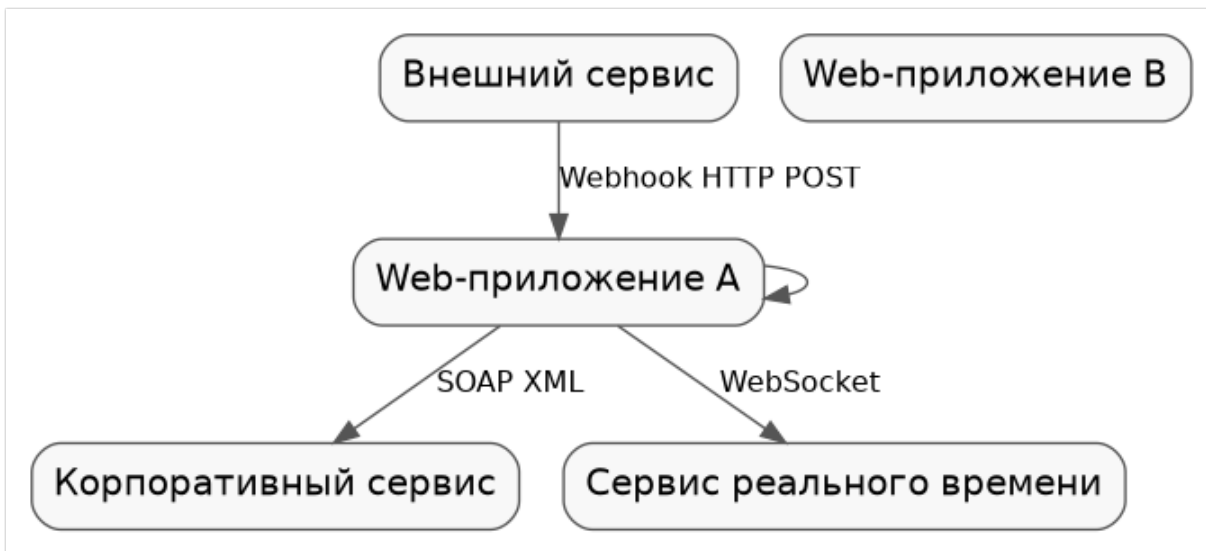


Figure 10: Схема 10

источников, удобен GraphQL.

Для точного ответа полезно разделить транспорт, прикладной протокол, формат данных и стиль API. **TCP** обеспечивает надёжную доставку байтового потока между узлами. **IP** отвечает за адресацию и маршрутизацию пакетов. **HTTP** задаёт прикладную модель запроса и ответа. **TLS** обеспечивает шифрование, целостность и проверку подлинности сервера, а иногда и клиента. **HTTPS** — это HTTP, работающий поверх TLS.

Формат данных не является протоколом взаимодействия сам по себе. JSON и XML задают представление данных, но не определяют правила маршрутизации, статусы ответа, методы или безопасность. Поэтому корректно писать: REST API обычно использует HTTP/HTTPS и JSON; SOAP обычно использует XML-сообщения и может передаваться поверх HTTP; WebSocket после HTTP Upgrade использует постоянное соединение для обмена фреймами.

При сравнении протоколов важно фиксировать следующее: HTTP/HTTPS подходит для запрос-ответ взаимодействия; REST — для ресурсных API; SOAP — для строгих корпоративных контрактов; GraphQL — для гибкого получения связанных данных; WebSocket — для постоянного real-time канала; WebHook — для событийных уведомлений отдельными HTTP-запросами.

Выбор протокола должен опираться на требования: нужен ли строгий контракт, нужно ли реальное время, допустима ли асинхронность, есть ли требования к безопасности, нужно ли кэширование, насколько важна совместимость с существующей инфраструктурой.

7. Шаблоны интеграций информационных систем по структуре взаимодействия

Шаблоны интеграции по структуре взаимодействия описывают, как именно связаны участники обмена. Самый простой вариант — точка-точка. В этом случае одна система напрямую обращается к другой. Например, сайт напрямую вызывает

API платёжного сервиса. Подход прост, но плохо масштабируется: если систем становится много, количество связей резко растёт, а изменение одного интерфейса может затронуть множество интеграций.

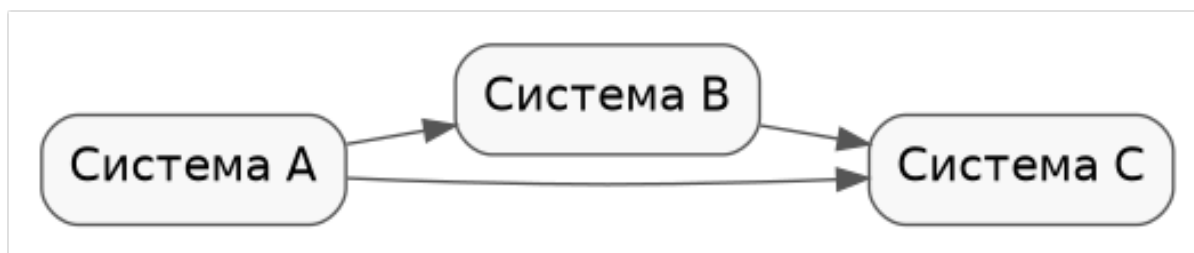


Figure 11: Схема 11

Второй распространённый шаблон — централизованная интеграция через посредника. Посредником может быть интеграционная шина, API Gateway, брокер сообщений или специализированная интеграционная платформа. Системы взаимодействуют не напрямую, а через центральный компонент. Это снижает связанность и позволяет централизованно управлять маршрутизацией, преобразованием форматов, безопасностью и мониторингом.

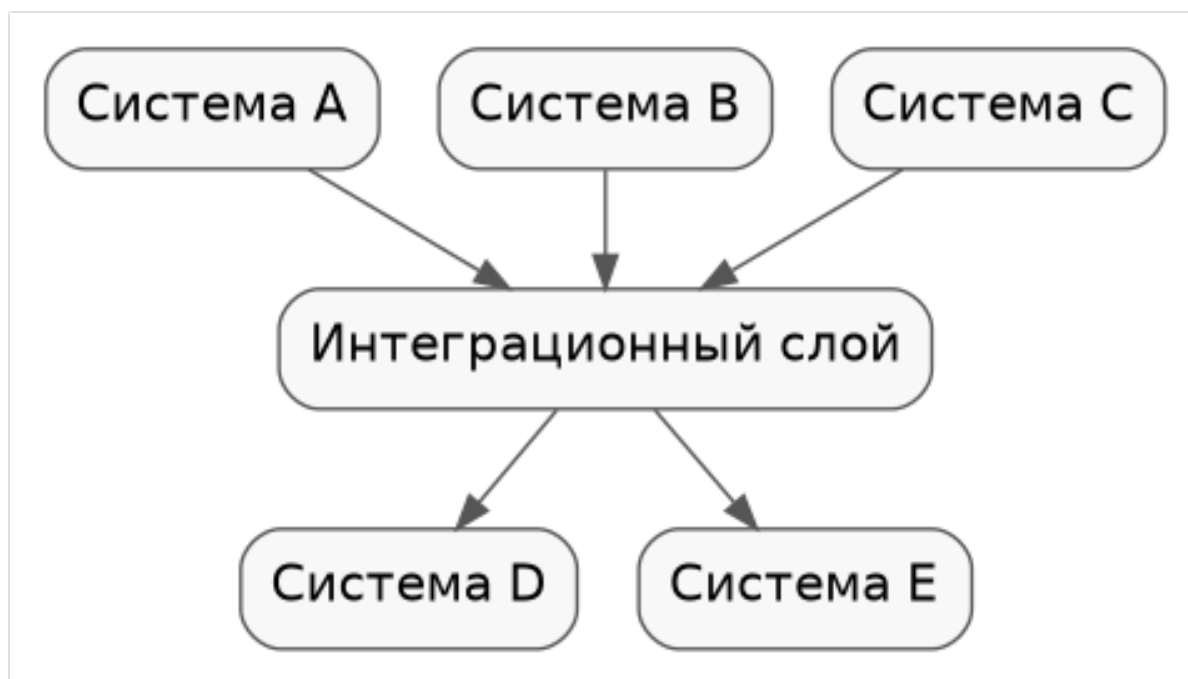


Figure 12: Схема 12

Третий шаблон — событийная интеграция. В этом случае источник публикует событие, а заинтересованные потребители получают его через брокер сообщений или механизм подписок. Источник не обязан знать всех получателей. Например, после создания заказа система заказов публикует событие `OrderCreated`, а сервис уведомлений, сервис аналитики и складская система обрабатывают его независимо.

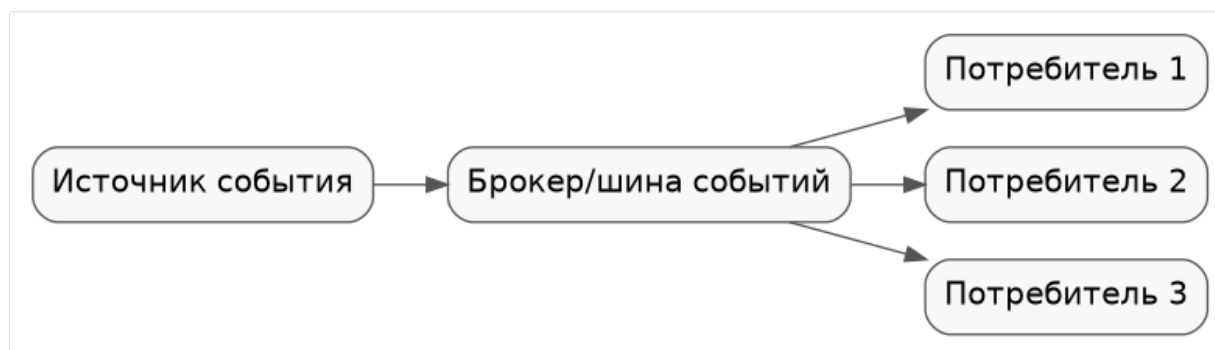


Figure 13: Схема 13

Выбор структуры зависит от масштаба и требований. Прямая интеграция хороша для простого обмена между двумя системами. Централизованный слой полезен в корпоративной среде с большим количеством приложений. Событийная архитектура удобна, когда важно слабое связывание и асинхронная реакция на изменения. Ошибка проектирования состоит в том, что один шаблон начинают применять ко всем задачам. Для простого синхронного запроса не всегда нужен брокер, а для множества независимых подписчиков прямые вызовы быстро превращаются в сеть из кабелей-спагетти.

Интеграция точка-точка — схема, при которой каждая система напрямую вызывает интерфейс другой системы. При n системах потенциальное число двусторонних связей растёт примерно как $n(n-1)/2$, поэтому при большом числе участников такая интеграция становится трудноуправляемой.

Централизованная интеграция использует общий посредник: интеграционную шину, API Gateway, ESB, брокер сообщений или iPaaS. Посредник выполняет маршрутизацию, преобразование форматов, контроль доступа, логирование и мониторинг. Преимущество — снижение числа прямых зависимостей. Недостаток — риск единой точки отказа и усложнение центрального компонента.

Событийная интеграция строится на публикации фактов о произошедших изменениях. Событие должно описывать то, что уже произошло, например `Order-Created`, а не команду «создай заказ». Источник события не обязан знать всех подписчиков, поэтому связанность снижается. Такая схема полезна для уведомлений, аналитики, аудита и независимых реакций нескольких систем.

Следовательно, структура интеграции выбирается по числу систем, требуемой связанности и характеру обмена. Для двух систем достаточно прямой интеграции, для корпоративной среды нужен интеграционный слой, для независимой реакции многих получателей удобна событийная модель.

8. Шаблоны интеграций информационных систем по типу обмена данными. Схемы

По типу обмена данных интеграции можно разделить на синхронные, асинхронные, пакетные и событийные. Синхронный обмен означает, что система-инициатор отправляет запрос и ожидает ответ до продолжения работы. Пример — проверка статуса платежа при оформлении заказа. Если ответ не получен, процесс не может

двигаться дальше.

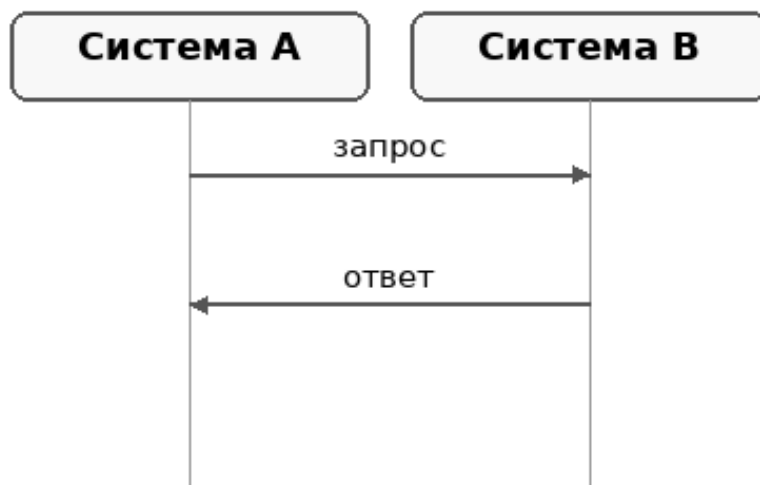


Figure 14: Схема 14

Асинхронный обмен означает, что отправитель передаёт сообщение и не блокируется до немедленного ответа. Сообщение может быть помещено в очередь, а получатель обработает его позже. Это полезно при длительных операциях, высокой нагрузке и необходимости повысить устойчивость. Например, сервис заказов кладёт сообщение в очередь на отправку email, а сервис уведомлений забирает его, когда готов.



Figure 15: Схема 15

Пакетный обмен предполагает передачу данных порциями по расписанию или по накоплению. Например, раз в сутки система продаж выгружает CSV/XML/JSON-файл с заказами в аналитическую систему. Такой подход проще для больших объёмов исторических данных, но не подходит для процессов, где требуется мгновенная реакция.

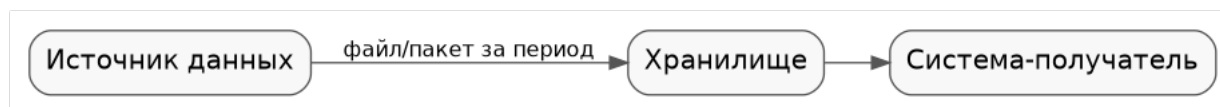


Figure 16: Схема 16

Событийный обмен строится вокруг факта изменения состояния. Система

публикует событие, например `UserRegistered`, `PaymentSucceeded`, `DocumentApproved`. Потребители подписываются на интересующие события и реагируют на них. Это близко к асинхронному обмену, но акцент сделан не просто на передаче команды, а на уведомлении о произошедшем факте.

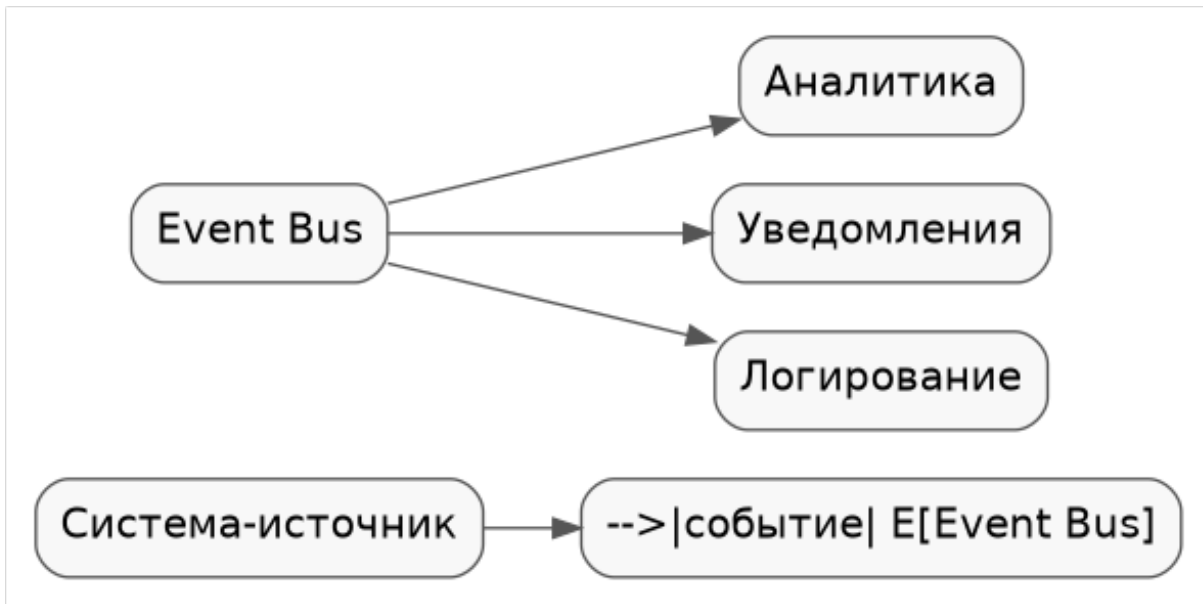


Figure 17: Схема 17

Также обмен можно классифицировать по направлению: однонаправленный, двунаправленный, широковещательный. По форме данных: передача файлов, обмен сообщениями, удалённый вызов процедур, доступ к общему хранилищу. По режиму согласования: жёстко контрактный обмен или более гибкий обмен документами. В практической интеграции часто комбинируют подходы: REST для оперативных запросов, очередь для фоновых операций, WebHook для уведомлений и пакетные выгрузки для отчётности.

Синхронный обмен — взаимодействие, при котором инициатор блокирует дальнейшее выполнение сценария до получения ответа или ошибки. Он прост для понимания, но чувствителен к задержкам и отказам удалённой системы. **Асинхронный обмен** — передача сообщения без ожидания немедленного результата обработки; часто реализуется через очередь или брокер сообщений.

Пакетный обмен применяется, когда данные передаются порциями за период: час, день, месяц. Он удобен для отчётности, миграций и аналитики, но даёт задержку актуальности. **Событийный обмен** фиксирует факт изменения состояния и доставляет его подписчикам. Событийность обычно асинхронна, но не всякая асинхронность является событийной: команда `SendEmail` — это поручение выполнить действие, а событие `OrderPaid` — сообщение о факте.

При описании обмена важно упомянуть гарантии доставки: `at most once` означает «не более одного раза, возможна потеря»; `at least once` означает «минимум один раз, возможны дубли»; `exactly once` означает «ровно один раз» и требует сложной инфраструктурной поддержки или идемпотентной логики. В прикладных системах часто проектируют обработчики под `at least once` и делают операции идемпотентными.

Пример строгой формулировки: если сервис платежей публикует событие PaymentSucceeded, сервис заказов должен проверять идентификатор платежа и не менять заказ повторно при дубле события. Это показывает не только знание видов обмена, но и понимание практической надёжности интеграций.

9. Модели разработки ПО: итеративная, инкрементная, водопадная, спиральная

Модель разработки ПО задаёт общий порядок выполнения работ: как собираются требования, проектируется система, пишется код, проводится тестирование и поставляется результат. В учебниках по программной инженерии подчёркивается, что модель жизненного цикла влияет не только на разработчиков, но и на тестировщиков, руководителей проекта и заказчика. Нет универсальной модели, подходящей всем проектам: выбор зависит от масштаба, рисков, изменчивости требований, компетенций команды и критичности продукта.

Водопадная модель предполагает последовательное прохождение фаз: требования, проектирование, реализация, тестирование, внедрение и сопровождение. Каждая следующая стадия начинается после завершения предыдущей. Схема проста и понятна, но плохо переносит изменения. Если ошибка в требованиях обнаружена на стадии тестирования, возврат назад дорог и организационно неприятен.

Инкрементная модель предполагает поставку продукта частями. Сначала реализуется минимальный набор функций, затем добавляются новые инкременты. Каждый инкремент увеличивает функциональность системы. Это удобно, когда заказчик хочет получать полезный результат постепенно.

Итеративная модель делает акцент на повторении цикла разработки. На каждой итерации команда уточняет требования, проектирует, реализует и проверяет часть решения. Результат каждой итерации помогает уточнить следующую. Итеративность полезна, когда требования не могут быть полностью определены заранее.

Спиральная модель сочетает итеративность с управлением рисками. Каждый виток спирали включает постановку целей, анализ рисков, разработку и оценку результата. Она подходит для крупных и рискованных проектов, где важно регулярно проверять технические, организационные и финансовые риски.

Пример использования: водопадная модель может применяться в небольшом проекте с жёсткими и стабильными требованиями. Инкрементная — при создании корпоративной системы, где сначала запускают учёт клиентов, затем отчёты, затем интеграцию. Итеративная — при разработке нового продукта, где пользовательские ожидания уточняются по прототипам. Спиральная — при создании сложной критичной системы, где высока цена архитектурной ошибки.

Модель жизненного цикла ПО — упорядоченная схема выполнения работ по созданию, поставке и сопровождению программного продукта. Она определяет, когда фиксируются требования, когда проектируется архитектура, как выполняется реализация, когда проводится тестирование и как принимаются изменения.

Водопадная модель предполагает последовательную фиксацию стадий и хорошо работает при стабильных требованиях, понятной предметной области и жёстком документообороте. Её слабость — дорогой возврат к ранним этапам. Итеративная модель предполагает повторение цикла с уточнением решения после каждой итерации. Инкрементная модель делает акцент на постепенной поставке готовых



Figure 18: Схема 18

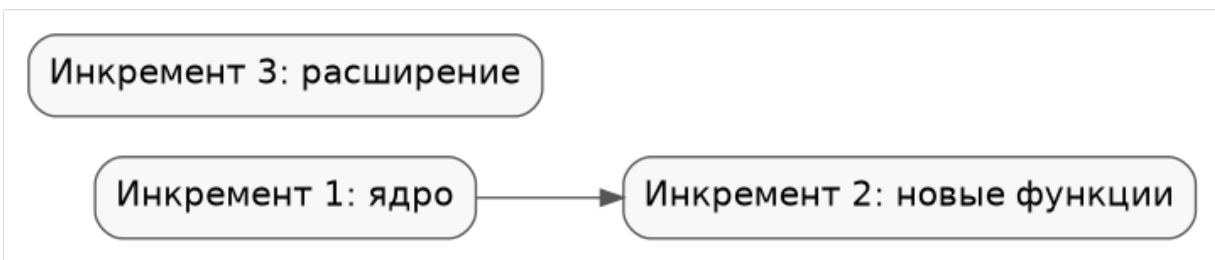


Figure 19: Схема 19



Figure 20: Схема 20

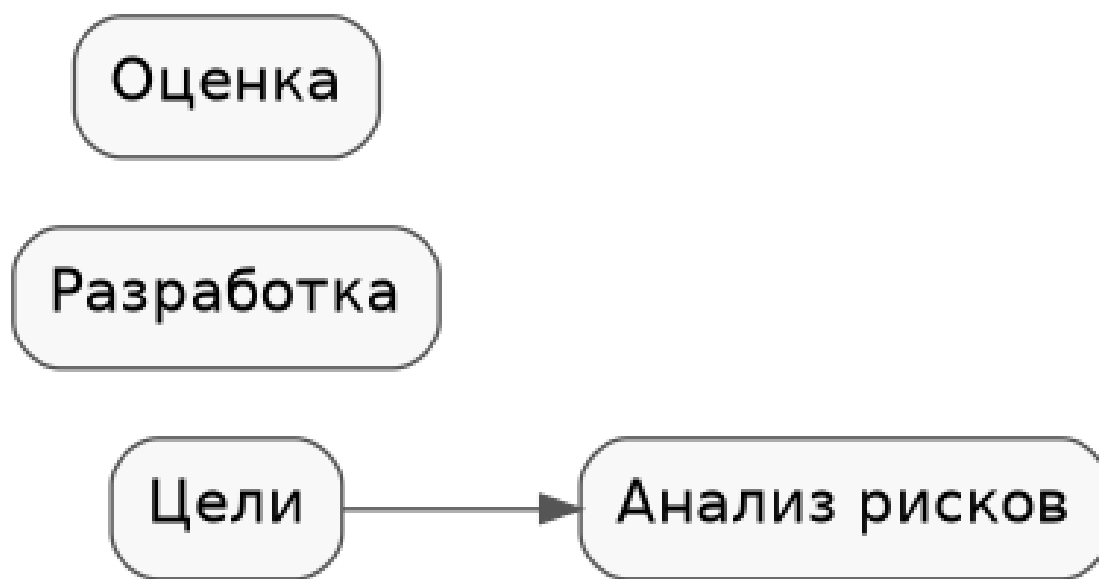


Figure 21: Схема 21

частей функциональности. Спиральная модель добавляет явный анализ рисков на каждом витке.

Важно различать итерацию и инкремент. Итерация — повтор процесса разработки и уточнения решения. Инкремент — поставляемое расширение функциональности продукта. Проект может быть одновременно итеративным и инкрементным: на каждой итерации создаётся новый полезный инкремент.

Выбор модели зависит от неопределённости требований, критичности продукта, стоимости ошибки, размера команды и участия заказчика. Для учебного ответа хорошая формула такая: чем выше неопределённость и потребность в обратной связи, тем полезнее итеративно-инкрементный подход; чем выше регламентированность и стабильность требований, тем допустимее водопадный подход; чем выше риски, тем уместнее спиральная модель.

10. Наборы практик Scrum и Kanban. Схемы и примеры использования

Scrum и Kanban относятся к гибким подходам организации работы. Они не являются просто «досками со стикерами», а задают разные способы управления потоком задач, обратной связью и изменениями. В гибкой разработке требования и решения развиваются через взаимодействие команды, заказчика и пользователей. Это особенно важно там, где заранее невозможно полностью зафиксировать все требования.

Scrum организует работу короткими итерациями — спринтами. Обычно спринт длится от одной до нескольких недель. Перед спринтом команда выбирает задачи из product backlog, формирует sprint backlog и берёт обязательство выполнить

определённый объём. В ходе спринта проводятся ежедневные встречи, а в конце — обзор результата и ретроспектива. Основные роли: Product Owner, Scrum Master и команда разработки.

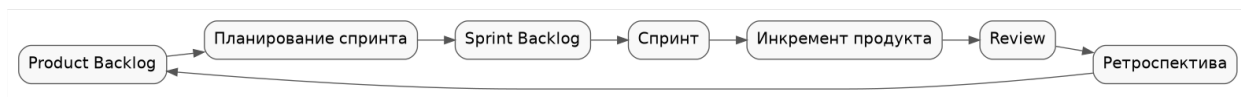


Figure 22: Схема 22

Пример Scrum: команда разрабатывает личный кабинет. В первый спринт реализуются авторизация и профиль, во второй — история заказов, в третий — уведомления. После каждого спринта заказчик видит работающий инкремент и может уточнить приоритеты.

Kanban управляет непрерывным потоком задач. Вместо фиксированных спринтов используется доска с колонками, например «Backlog», «В работе», «На проверке», «Готово». Важный элемент Kanban — ограничение незавершённой работы, WIP-limit. Оно не позволяет команде начинать слишком много задач одновременно и помогает быстрее доводить работу до завершения.

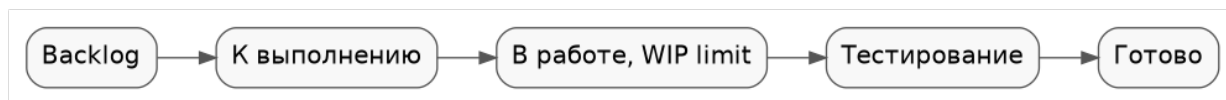


Figure 23: Схема 23

Пример Kanban: команда поддержки обрабатывает поток заявок. Новые задачи поступают постоянно, поэтому спринты неудобны. Заявки двигаются по доске, а ограничения WIP помогают не перегружать специалистов. Kanban хорошо подходит для сопровождения, DevOps, поддержки и задач с непрерывным поступлением.

Scrum даёт ритм и регулярную обратную связь. Kanban даёт прозрачность потока и контроль загрузки. Scrum лучше подходит для продуктовой разработки с планируемыми итерациями. Kanban удобен для операционной деятельности, поддержки и процессов, где задачи приходят непрерывно. В реальных командах часто используют гибрид: спринты Scrum плюс Kanban-доска для визуализации потока.

Scrum — фреймворк гибкой разработки, организующий работу короткими спринтами с фиксированной целью, регулярной обратной связью и поставкой потенциально готового инкремента. Основные роли: Product Owner управляет ценностью и приоритетами backlog; Scrum Master помогает соблюдать процесс и устранять препятствия; Developers создают инкремент продукта.

Основные артефакты Scrum: Product Backlog, Sprint Backlog и Increment. Product Backlog содержит упорядоченный список требований и задач продукта. Sprint Backlog содержит выбранные на спринт элементы и план их выполнения. Increment — результат спринта, соответствующий Definition of Done. События Scrum: планирование спринта, daily scrum, sprint review и retrospective.

Kanban — метод управления потоком работ, основанный на визуализации задач, ограничении незавершённой работы и непрерывном улучшении процесса. Его ключевые понятия: workflow, WIP-limit, lead time, cycle time, bottleneck. WIP-limit важен потому, что ограничивает распыление команды и выявляет узкие места.

Экзотическое различие: Scrum управляет работой через ритм спринтов и

цели итерации, Kanban управляет непрерывным потоком задач. Scrum удобен для продуктовой разработки с планируемыми релизами, Kanban — для поддержки, эксплуатации и потока заявок. Гибрид возможен, но важно не смешивать термины: наличие доски само по себе не делает процесс Scrum или Kanban.

11. Уровни и типы требований по К. Вигерсу. Схема и комментарии к схеме

В подходе К. Вигерса требования рассматриваются как многоуровневая система, связывающая бизнес-цели, потребности пользователей и конкретные характеристики программного продукта. Такое разделение нужно, чтобы команда не смешивала цели заказчика, сценарии пользователя и технические детали реализации. Ошибка на верхнем уровне требований особенно опасна: если неправильно понята бизнес-потребность, корректно реализованные функции всё равно могут оказаться бесполезными.



Figure 24: Схема 24

Бизнес-требования описывают цели организации: повысить скорость обработки заявок, снизить число ошибок, автоматизировать отчётность, обеспечить соответствие регламентам. Они отвечают на вопрос «зачем нужен проект». Например: «Сократить время обработки заявки с двух дней до четырёх часов». Это не функция интерфейса, а ожидаемый бизнес-результат.

Пользовательские требования описывают задачи пользователей и их ожидания. Например: «Оператор должен иметь возможность найти заявку по номеру телефона клиента». Такие требования часто выражаются через пользовательские сценарии, use case или user story.

Функциональные требования описывают поведение системы. Они отвечают на вопрос «что должна делать система». Например: «Система должна отображать список заявок, найденных по номеру телефона, с указанием статуса, даты создания и ответственного сотрудника». Функциональные требования должны быть проверяемыми, потому что на их основе строятся тесты.

Нефункциональные требования описывают качества системы: производительность, надёжность, безопасность, удобство использования, сопровождаемость, переносимость. Например: «Поиск заявки должен выполняться не более чем за две секунды при базе до одного миллиона записей». Такие требования нельзя заменять общими словами вроде «система должна быть быстрой», потому что это невозможно объективно проверить.

Бизнес-правила задают ограничения предметной области. Например: «Заявка со статусом “закрыта” не может быть переведена обратно в статус “новая” без прав администратора». Они могут влиять на разные функциональные требования. Схема Вигерса помогает видеть трассировку: от бизнес-цели к пользовательской задаче,

затем к функциям, качественным ограничениям и тестам.

В подходе К. Вигерса требования удобно рассматривать как иерархию от цели бизнеса к конкретной проверяемой спецификации. **Бизнес-требование** фиксирует ожидаемую ценность для организации. **Пользовательское требование** описывает задачу или сценарий пользователя. **Функциональное требование** задаёт наблюдаемое поведение системы. **Нефункциональное требование** задаёт качественную характеристику или ограничение работы системы.

Пример трассировки: бизнес-требование — «сократить время обработки заявок на 50%». Пользовательское требование — «оператор должен быстро находить заявку по телефону клиента». Функциональное требование — «система должна выполнять поиск заявки по номеру телефона и отображать номер, статус, дату создания и ответственного». Нefункциональное требование — «результаты поиска должны отображаться не более чем за 2 секунды при 100 одновременных пользователях».

Бизнес-правило отличается от требования тем, что описывает устойчивое правило предметной области, которое может существовать независимо от конкретной системы. Например, «закрытая заявка может быть возвращена в работу только руководителем» — бизнес-правило. Требование к системе формулирует, как именно программа должна обеспечить это правило.

Для экзамена важно подчеркнуть трассируемость: каждое низкоуровневое требование должно быть связано с пользовательской задачей и бизнес-целью. Это помогает проверять необходимость функций и оценивать влияние изменений.

12. Свойства качественных требований. Пояснения с примерами

Качественные требования — это требования, которые можно понять, реализовать, проверить и сопровождать. Требование должно быть корректным: оно должно отражать реальную потребность заказчика или пользователя. Если требование сформулировано красиво, но решает не ту задачу, оно некачественно. Например, требование «система должна отправлять отчёт всем сотрудникам» может быть некорректным, если отчёт содержит персональные данные и нужен только руководителям.

Требование должно быть однозначным. Формулировки вроде «быстро», «удобно», «надёжно», «при необходимости» опасны без численных критериев. Хороший вариант: «Страница списка заказов должна открываться не более чем за 2 секунды при 500 одновременных пользователях». Такое требование можно проверить тестом производительности.

Требование должно быть полным. Оно должно содержать все данные, нужные для реализации и проверки. Например, требование «пользователь может восстановить пароль» неполно, если не указано, через email или телефон выполняется восстановление, сколько действует ссылка, какие ограничения есть на новый пароль и что происходит при повторном запросе.

Требование должно быть непротиворечивым. Если в одном разделе написано, что пароль должен содержать минимум 8 символов, а в другом — минимум 12, команда не сможет корректно реализовать функцию. Противоречия часто возникают при изменении требований, когда один документ обновили, а другой забыли.

Требование должно быть проверяемым. Если нельзя построить тест, значит требование сформулировано слабо. «Интерфейс должен быть современным» — непроверяемо. «Основные действия пользователя должны выполняться не более чем за три клика из главного экрана» — уже проверяемо, хотя тоже требует уточнения

сценариев.

Требование должно быть реализуемым. Оно должно учитывать технические, организационные, правовые и ресурсные ограничения. Например, требование «система должна обрабатывать все запросы мгновенно» нереализуемо. Реалистичная версия: «95% запросов поиска должны обрабатываться менее чем за 1 секунду при нагрузке 100 запросов в секунду».

Требование должно быть трассируемым. Нужно понимать, откуда оно появилось, с какой бизнес-целью связано, в каком модуле реализуется и какими тестами проверяется. Трассируемость особенно важна в крупных проектах: она помогает оценивать влияние изменений.

Пример плохого требования: «Система должна удобно работать с документами». Пример улучшенного: «Пользователь с ролью “оператор” должен иметь возможность загрузить PDF-документ размером до 20 МБ, увидеть статус загрузки и получить сообщение об ошибке при недопустимом формате файла». Во втором варианте есть субъект, действие, объект, ограничение и проверяемый результат.

Качественное требование должно быть необходимым, корректным, однозначным, полным, непротиворечивым, реализуемым, проверяемым, трассируемым и атомарным. **Атомарность** означает, что одно требование описывает одно проверяемое условие, а не смесь нескольких функций. **Проверяемость** означает, что можно построить тест или критерий приёма, позволяющий установить выполнение требования.

Плохая формулировка: «система должна быть быстрой и удобной». Улучшенная формулировка: «страница списка заявок должна открываться не более чем за 2 секунды для 95% запросов при 200 одновременных пользователях и объёме базы до 1 000 000 заявок». Здесь есть объект, измеримый показатель, нагрузка и критерий.

Для функционального требования полезна структура: субъект, условие, действие системы, результат, исключения. Например: «Если пользователь с ролью “оператор” вводит корректный номер телефона в поле поиска и нажимает “Найти”, система должна отобразить список заявок, связанных с этим номером, либо сообщение “Заявки не найдены”». Такая формулировка сразу пригодна для тест-кейса.

Важно учитывать, что качество требований снижает стоимость разработки. Неоднозначное требование порождает разные трактовки у аналитика, разработчика, тестировщика и заказчика. Чем позже обнаружена неоднозначность, тем дороже исправление.

13. Методы анализа качества интерфейсов

Анализ качества интерфейса направлен на оценку того, насколько интерфейс помогает пользователю эффективно, результативно и с приемлемым уровнем удовлетворённости выполнять задачи. Интерфейс нельзя оценивать только по внешней красоте. Красивый экран может быть неудобным, если пользователь не понимает, что делать дальше, совершает много ошибок или тратит слишком много времени на простую операцию.

К основным методам анализа относятся экспертная оценка, эвристическая оценка, анализ пользовательских сценариев, тестирование удобства использования, сбор метрик, анкетирование и наблюдение. Экспертная оценка проводится специалистами, которые проверяют интерфейс по набору критериев: понятность навигации, согласованность элементов, видимость состояния системы, предотвращение ошибок, читаемость, доступность.

Пользовательское тестирование строится иначе. Реальным или типовым пользователям дают задачи и наблюдают, как они взаимодействуют с интерфейсом. Фиксируются время выполнения, количество ошибок, места затруднений, вопросы пользователя, лишние действия. Такой метод ценен тем, что показывает не то, как интерфейс задуман проектировщиком, а то, как он реально воспринимается.



Figure 25: Схема 25

Метрики качества интерфейса могут включать успешность выполнения задачи, время выполнения, количество ошибок, количество обращений к помощи, субъективную удовлетворённость, число шагов до результата. Например, если 80% пользователей не могут самостоятельно найти кнопку отправки формы, проблема не в пользователях, а в проектировании интерфейса.

Анализ интерфейса должен учитывать контекст использования. Один и тот же интерфейс может быть хорош для опытного оператора и плох для случайного пользователя. Для профессиональной системы важны скорость и плотность информации, для массового приложения — простота и очевидность. Поэтому оценка качества интерфейса всегда связана с целевой аудиторией, задачами и условиями применения.

Качество интерфейса оценивается по тому, насколько пользователь может эффективно, результативно и удовлетворительно выполнить свои задачи в заданном контексте. Результативность показывает, достиг ли пользователь цели. Эффективность показывает, сколько ресурсов затрачено: время, клики, усилия. Удовлетворённость показывает субъективное восприятие работы с системой.

Методы анализа делятся на экспертные и пользовательские. Экспертные методы включают эвристическую оценку, когнитивное прохождение, проверку по гайдлайнам, анализ доступности. Пользовательские методы включают usability-тестирование, наблюдение, интервью, анкетирование, A/B-тестирование и анализ продуктовой аналитики.

Метрики: task success rate, среднее время выполнения задачи, количество ошибок, количество шагов, доля пользователей, обратившихся к подсказке, SUS-оценка, conversion rate для целевых действий. Для профессиональных интерфейсов важны скорость и снижение ошибок; для массовых интерфейсов — понятность, обучаемость и доверие.

Экзампнационный пример: если 7 из 10 пользователей не находят кнопку отправки формы без подсказки, это интерфейсная проблема. Исправлением может быть изменение расположения кнопки, текста, визуального акцента или структуры сценария, а не только «обучение пользователей».

14. Проектирование интерфейсов на основе UX/UI-методов

Проектирование интерфейса на основе UX/UI-методов начинается не с выбора цветов, а с понимания пользователя, его задач, ограничений и контекста. UX отвечает за пользовательский опыт: насколько логично, удобно и предсказуемо человек достигает цели. UI отвечает за визуальное и интерактивное представление: элементы управления, расположение блоков, типографику, цвет, состояния кнопок, формы и

сообщения.

Первый этап — исследование пользователей. Команда определяет целевые группы, их потребности, уровень подготовки, частоту использования системы, типичные задачи и болевые точки. Для этого применяются интервью, наблюдение, анализ обращений в поддержку, изучение предметной области. Результатом могут стать персоны, карта эмпатии и список пользовательских сценариев.

Второй этап — проектирование структуры. Определяется информационная архитектура: какие разделы есть в системе, как пользователь переходит между ними, где находятся ключевые действия. Ошибка на этом уровне приводит к тому, что даже аккуратный визуальный дизайн не спасает продукт: пользователь просто не понимает, где искать нужную функцию.

Третий этап — создание прототипов. Низкодетализированные прототипы позволяют быстро проверить расположение блоков и сценарии. Высокдетализированные прототипы ближе к финальному интерфейсу и помогают согласовать визуальные решения. Прототипирование снижает риск дорогих переделок: исправлять схему экрана дешевле, чем переписывать реализованный модуль.

Четвёртый этап — проверка интерфейса. Пользователям дают типовые задачи и наблюдают, где они ошибаются. Если человек не может завершить действие без подсказок, интерфейс требует доработки. После проверки проектировщик корректирует сценарии, тексты, расположение элементов и визуальные акценты.

Пример: при проектировании интерфейса подачи заявки сначала описывают пользователей: студент, сотрудник деканата, администратор. Затем строят сценарии: создать заявку, прикрепить документ, проверить статус, вернуть на доработку. После этого создают прототип формы и тестируют его на пользователях. Такой подход лучше, чем сразу рисовать красивый экран, потому что интерфейс проектируется вокруг задач, а не вокруг декоративной оболочки.

UX-проектирование — процесс создания пользовательского опыта, в котором изучаются задачи, мотивация, ограничения и сценарии пользователя.

UI-проектирование — создание визуального и интерактивного слоя интерфейса: экранов, компонентов, состояний, типографики, цветов и поведения элементов.

Строгая последовательность проектирования: исследование пользователей; выделение ролей и сценариев; построение информационной архитектуры; создание user flow; подготовка wireframe; создание интерактивного прототипа; usability-тестирование; визуальный дизайн; передача в разработку; проверка реализации. Пропуск ранних этапов приводит к красивым, но функционально слабым интерфейсам.

Важные артефакты: персона описывает типового пользователя; customer journey map показывает путь пользователя через этапы взаимодействия; user flow описывает последовательность экранов и действий; wireframe задаёт структуру экрана без детального дизайна; design system обеспечивает согласованность компонентов.

Для экзамена хорошо привести пример: при проектировании формы заявки сначала определяют, кто её заполняет, какие данные у него есть, какие ошибки часты, какие поля обязательны. Затем строят сценарий заполнения, проверяют прототип на пользователях и только после этого финализируют визуальный стиль.

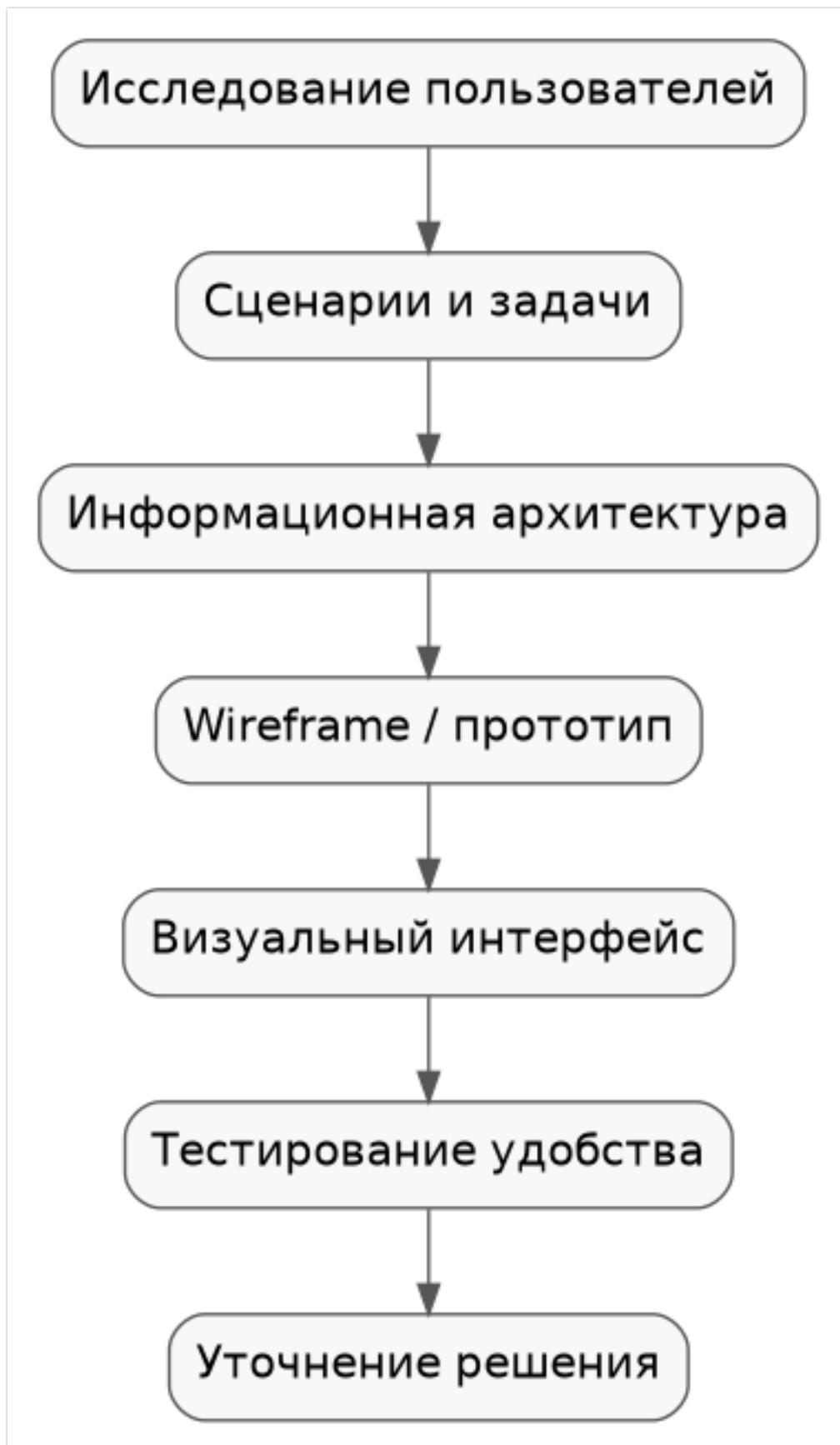


Figure 26: Схема 26
29

15. Протоколы взаимодействия между сервисами по сети Интернет

Сервисы в распределённой системе взаимодействуют через сетевые протоколы и форматы обмена. Наиболее распространённым основанием является HTTP/HTTPS. HTTP задаёт стандартную структуру запроса и ответа, а HTTPS добавляет защищённый транспортный слой. Поверх HTTP могут строиться REST API, SOAP-сервисы, GraphQL и WebHook-взаимодействие.

REST использует ресурсы и стандартные HTTP-методы. Он прост, хорошо поддерживается инструментами и подходит для большинства web-интеграций. SOAP использует XML-сообщения и формальный контракт, что полезно для корпоративных систем. GraphQL позволяет клиенту задавать структуру нужных данных. WebSocket создаёт постоянный двунаправленный канал. WebHook позволяет одной системе уведомлять другую о событии.

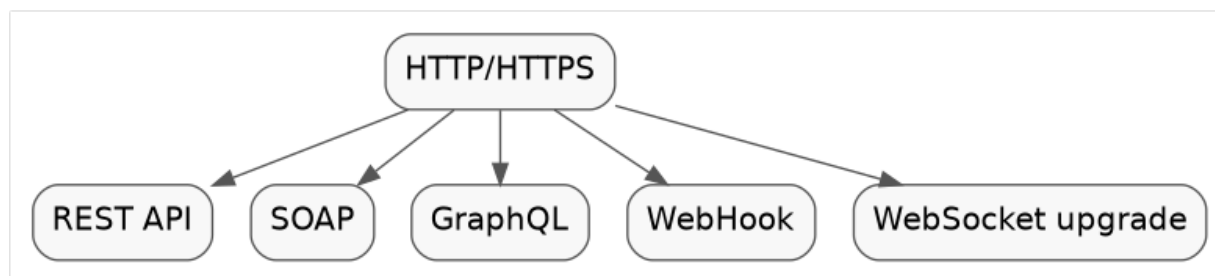


Figure 27: Схема 27

Выбор протокола зависит от характера взаимодействия. Если сервису нужно получить данные по запросу, подходит REST или GraphQL. Если нужно строгое описание операций и типов, подходит SOAP. Если нужно уведомить другую систему о событии, подходит WebHook. Если нужен постоянный обмен в реальном времени, подходит WebSocket.

Важны не только протоколы, но и сопутствующие аспекты: авторизация, формат ошибок, версионирование, идемпотентность, ограничения скорости, повторные попытки, мониторинг. Например, при оплате заказа повторный запрос может привести к двойному списанию, если операция не спроектирована идемпотентно. Поэтому протокол — только транспортная форма. Надёжная интеграция требует продуманного контракта и правил обработки сбоев.

При взаимодействии сервисов важно описывать не только протокол, но и контракт, безопасность и поведение при сбоях. **API-контракт** включает адрес, метод, формат данных, коды ошибок, правила авторизации, ограничения частоты запросов, версионирование и гарантии совместимости.

REST и GraphQL чаще используются для синхронного запроса данных. Messaging через брокеры, например RabbitMQ или Kafka, используется для асинхронного обмена. gRPC применяется для эффективного межсервисного взаимодействия с контрактом на основе Protocol Buffers. SOAP сохраняет значение в регламентированных корпоративных интеграциях. WebSocket нужен для постоянного канала, WebHook — для обратного уведомления о событии.

Надёжное межсервисное взаимодействие требует timeout, retry, circuit breaker, idempotency key, корреляционного идентификатора запроса и централизованного логирования. Без timeout зависший сервис может заблокировать цепочку вызовов. Без идемпотентности повтор запроса может создать дубликаты заказов или платежей.

Экзаменационная фраза: протокол определяет форму передачи данных, но качество интеграции определяется контрактом, обработкой ошибок, безопасностью, версионированием и наблюдаемостью.

16. Типы пользовательских интерфейсов

Пользовательский интерфейс — это способ взаимодействия человека с программной системой. Тип интерфейса зависит от устройства, задач, квалификации пользователя и контекста работы. Классический графический интерфейс использует окна, кнопки, меню, формы, таблицы и другие визуальные элементы. Он распространён в настольных, web- и мобильных приложениях.

Командный интерфейс основан на вводе команд. Он менее нагляден для новичка, но эффективен для опытного пользователя, администратора или разработчика. Командная строка позволяет автоматизировать действия, запускать сценарии и работать в средах, где графический интерфейс недоступен.

Web-интерфейс работает через браузер и не требует установки отдельного клиентского приложения. Он удобен для корпоративных систем и массовых сервисов. Мобильный интерфейс проектируется с учётом маленького экрана, сенсорного ввода, жестов, нестабильной сети и коротких пользовательских сессий.

Голосовой интерфейс позволяет взаимодействовать с системой через речь. Он полезен в условиях, где руки заняты или экран неудобен: автомобиль, умная колонка, доступная среда. Но голосовой интерфейс сложнее в обработке ошибок, конфиденциальности и точности распознавания.

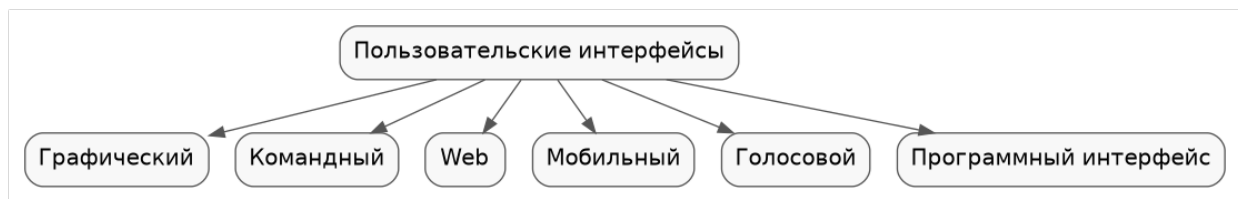


Figure 28: Схема 28

Отдельно можно выделить программный интерфейс — API. Его пользователем является не человек напрямую, а другая программа или разработчик. Но с точки зрения проектирования API тоже требует удобства: понятных названий, стабильных контрактов, документации и предсказуемых ошибок.

Выбор типа интерфейса должен соответствовать задаче. Для оператора call-центра важны скорость ввода и горячие клавиши. Для клиента интернет-магазина — простота и визуальная понятность. Для администратора — гибкость и диагностическая информация. Нельзя проектировать интерфейс абстрактно: его качество определяется тем, насколько он соответствует пользователю и задаче.

Пользовательский интерфейс — совокупность средств, с помощью которых пользователь управляет программной системой и получает от неё информацию. Интерфейс включает не только экранные элементы, но и правила взаимодействия, сообщения, обратную связь, обработку ошибок и навигацию.

GUI использует визуальные элементы и подходит для широкого круга пользователей. CLI использует текстовые команды и эффективен для администрирования, автоматизации и разработки. Web UI работает через браузер и упрощает

распространение системы. Mobile UI проектируется под сенсорный ввод, малый экран, жесты и нестабильную сеть. Voice UI использует речь и требует обработки естественного языка. API-интерфейс предназначен для программного взаимодействия и должен проектироваться с точки зрения разработчика-потребителя.

Также можно выделить TUI, то есть текстовый псевдографический интерфейс в терминале; NUI, естественные интерфейсы на жестах и касаниях; AR/VR-интерфейсы; conversational UI в чат-ботах. Каждый тип интерфейса имеет свои ограничения по вводу, обратной связи и ошибкам.

Выбор интерфейса определяется контекстом: оператору нужна скорость и горячие клавиши, мобильному пользователю — простота и крупные элементы, администратору — диагностическая информация и автоматизация. Хороший ответ должен показать связь типа интерфейса с задачей, а не просто перечислить виды.

17. Метод анализа пользовательского интерфейса по метрикам Констайна и Локвуда

Метрики Констайна и Локвуда связаны с оценкой удобства пользовательского интерфейса через результативность работы пользователя, затраты усилий и качество взаимодействия. В учебном контексте такой подход важен тем, что переводит разговор об интерфейсе из области вкуса в область наблюдаемых характеристик. Интерфейс оценивается не словами «красивый» или «не нравится», а через то, как пользователь решает задачу.

Основная логика анализа состоит в измерении того, насколько пользователь успешно достигает цели, сколько действий выполняет, сколько ошибок допускает и насколько путь через интерфейс соответствует естественной структуре задачи. Если интерфейс заставляет пользователя помнить лишнюю информацию, делать много переходов или исправлять непонятные ошибки, качество взаимодействия снижается.

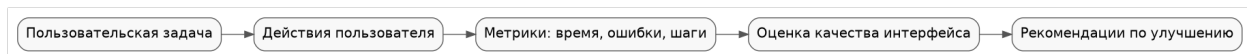


Figure 29: Схема 29

Пример анализа: пользователю нужно создать заявку. Наблюдатель фиксирует, сколько времени ушло на выполнение, нашёл ли пользователь нужную кнопку, сколько раз вернулся назад, какие поля вызвали затруднения, понял ли сообщение об ошибке. Если большинство пользователей ошибается на одном и том же поле, проблема связана не с обучением пользователей, а с интерфейсом.

Метрики могут включать: время выполнения задачи, количество шагов, число ошибок, количество обращений к справке, долю успешно завершённых сценариев, субъективную оценку понятности. Важна не отдельная цифра, а её связь с целью. Например, сокращение времени выполнения на 20% может быть значимым для оператора, который повторяет действие сотни раз в день.

Достоинство метода — возможность сравнивать варианты интерфейса. Можно протестировать два прототипа формы и выбрать тот, где меньше ошибок и быстрее выполнение. Ограничение — метрики нужно собирать на корректных сценариях и подходящей аудитории. Если тестировать профессиональную систему на случайных пользователях, выводы будут искажены.

В рамках подхода Константайна и Локвуда интерфейс анализируется через задачи пользователя, роли и качество взаимодействия. Центральная идея: интерфейс должен соответствовать структуре пользовательской деятельности, а не внутренней структуре программы или базы данных.

Для строгой оценки выделяют показатели: достижение цели, количество действий, количество ошибок, время выполнения, когнитивная нагрузка, необходимость запоминать промежуточные данные, соответствие экранов пользовательским ролям. Если пользователь вынужден думать о технических терминах системы вместо своей задачи, интерфейс спроектирован вокруг реализации, а не вокруг деятельности.

Пример: оператор оформляет заявку. Хороший интерфейс группирует поля по смыслу: данные клиента, описание проблемы, вложения, приоритет, отправка. Плохой интерфейс показывает поля в порядке таблиц базы данных или внутренних API. Метрики позволяют сравнить варианты: в варианте А заявка создаётся за 90 секунд и 2 ошибки на пользователя, в варианте В — за 55 секунд и 0,5 ошибки.

Следовательно, метод полезен тем, что переводит оценку интерфейса в измеримые показатели деятельности пользователя и помогает обосновать изменения не вкусом дизайнера, а снижением затрат и ошибок.

18. Метод анализа пользовательского интерфейса по метрикам GOMS

GOMS — метод моделирования пользовательских действий, название которого связано с четырьмя элементами: Goals, Operators, Methods, Selection rules. Цели описывают, чего хочет достичь пользователь. Операторы — элементарные действия: нажать кнопку, ввести текст, посмотреть на экран, принять решение. Методы — последовательности операторов для достижения цели. Правила выбора определяют, какой метод пользователь применяет в конкретной ситуации.

GOMS полезен для анализа интерфейсов, где важна эффективность повторяемых действий. Метод позволяет разложить задачу на элементарные шаги и оценить, сколько времени и усилий требуется пользователю. Например, создание заявки можно представить как цель, состоящую из выбора раздела, заполнения полей, прикрепления файла и отправки формы.

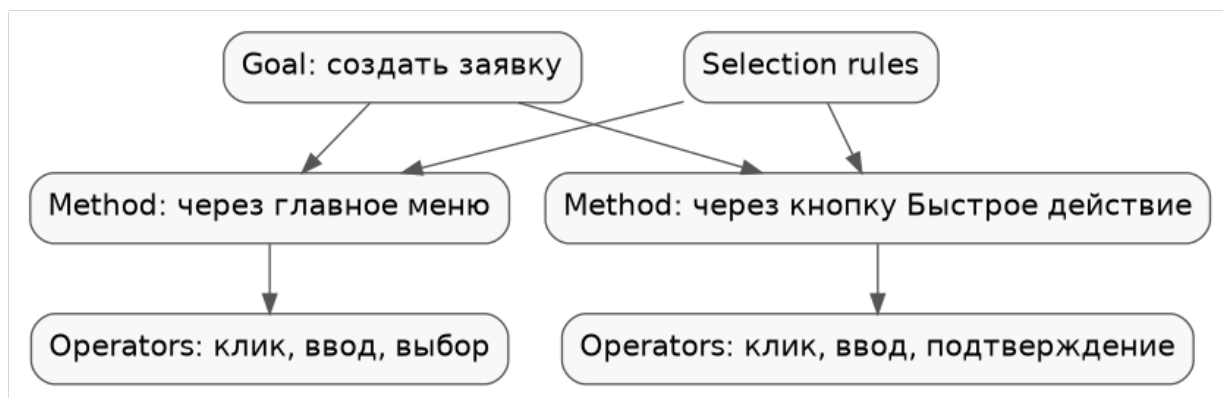


Figure 30: Схема 30

Пример: пользователь хочет найти заказ. Метод 1: открыть раздел «Заказы», нажать поле поиска, ввести номер, нажать Enter, открыть найденную строку. Метод

2: использовать глобальный поиск из верхней панели. Если второй метод требует меньше операторов и понятен пользователю, он может быть предпочтительным.

Метод GOMS помогает выявить лишние действия. Если для частой операции требуется десять переходов, интерфейс перегружает пользователя. Если одно действие можно выполнить несколькими способами, правила выбора должны быть очевидны: пользователь должен понимать, когда применять каждый путь.

Ограничение GOMS состоит в том, что он хорошо работает для рациональных, повторяемых, достаточно формализованных задач, но хуже учитывает эмоции, доверие, визуальную привлекательность и неопределённость поведения новичков. Поэтому GOMS желательно сочетать с пользовательским тестированием, интервью и анализом ошибок. Важно подчеркнуть: GOMS не заменяет UX-исследование, а даёт инструментальное описание действий пользователя.

GOMS расшифровывается как Goals, Operators, Methods, Selection rules. Goal — цель пользователя. Operator — элементарное действие или умственная операция. Method — последовательность операторов для достижения цели. Selection rules — правила выбора метода, если одну цель можно достичь разными путями.

Пример разложения цели «найти заказ по номеру»: Goal — открыть карточку заказа. Method 1 — перейти в раздел заказов, кликнуть поиск, ввести номер, нажать Enter, открыть результат. Operators — перемещение указателя, клик, ввод текста, чтение результата, принятие решения. Selection rule — если пользователь знает номер заказа, использовать поиск; если не знает, открыть список и фильтры.

GOMS особенно полезен для повторяемых профессиональных операций. Он позволяет сравнить два варианта интерфейса до полноценной разработки, оценив число действий и примерное время. Если одна операция выполняется сотни раз в день, сокращение даже на несколько секунд становится значимым.

Ограничение метода: GOMS плохо учитывает эмоциональную реакцию, доверие, визуальную привлекательность, обучение новичков и ситуации неопределённости. Поэтому его применяют вместе с usability-тестированием и анализом пользовательских ошибок.

19. Построение карты эмпатии

Карта эмпатии — UX-инструмент, который помогает описать пользователя не только как роль в системе, но и как человека с целями, ожиданиями, страхами, ограничениями и мотивацией. Она используется на ранних этапах проектирования интерфейса и требований, чтобы команда не строила продукт «для абстрактного пользователя». Карта эмпатии помогает понять, что пользователь видит, слышит, думает, чувствует, говорит и делает.

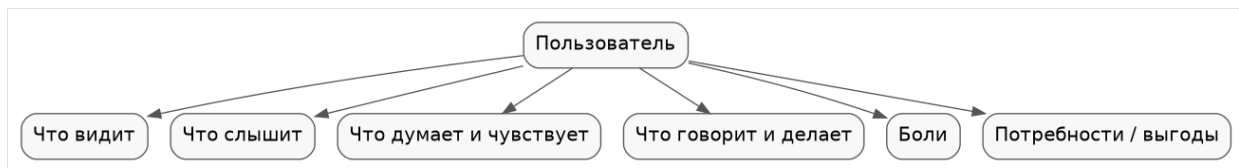


Figure 31: Схема 31

Построение карты начинается с выбора конкретной пользовательской группы. Нельзя строить одну карту «для всех». Например, в системе электронного

документооборота разные карты нужны для студента, сотрудника деканата, руководителя и администратора. У них разные задачи и разные боли.

Далее собирают данные: интервью, наблюдения, обращения в поддержку, результаты тестирования прототипа, анализ предметной области. Затем сведения распределяют по секторам карты. «Что видит» — рабочее окружение, интерфейсы, документы, ограничения. «Что слышит» — указания руководителя, требования преподавателя, советы коллег. «Что думает и чувствует» — опасения ошибиться, желание выполнить задачу быстро, недоверие к новой системе. «Что говорит и делает» — реальные действия и формулировки пользователя. «Боли» — трудности, риски, раздражители. «Выгоды» — желаемый результат.

Пример: студент подаёт заявление через портал. Он видит много полей и непонятные статусы. Слышит от одногруппников, что «заявки часто теряются». Думает, что может ошибиться с документом. Делает скриншоты после отправки. Его боль — неопределённость статуса. Его потребность — ясное подтверждение и понятные уведомления. Из такой карты рождается требование: после отправки заявления система должна показать номер заявки, статус, срок рассмотрения и отправить уведомление.

Карта эмпатии не является финальным требованием. Это промежуточный инструмент, который помогает формировать пользовательские сценарии, требования и решения интерфейса. Её ценность в том, что она сохраняет связь проектирования с реальным контекстом пользователя.

Карта эмпатии — визуальный UX-артефакт, описывающий выбранную группу пользователей через наблюдаемое поведение, мысли, чувства, боли и ожидаемые выгоды. Её цель — связать проектные решения с реальным контекстом пользователя. Строить карту нужно для конкретного сегмента, а не для «среднего пользователя». Например, «студент, подающий заявление впервые» и «сотрудник деканата, обрабатывающий 100 заявок в день» требуют разных карт. Источники данных: интервью, наблюдение, аналитика, обращения в поддержку, результаты тестирования прототипа. Выдуманная карта без данных полезна только как гипотеза. Типовая структура: что пользователь видит; что слышит; что говорит и делает; что думает и чувствует; боли; потребности и выгоды. Из карты должны вытекать требования. Например, боль «непонятно, дошло ли заявление» превращается в требование «после отправки система показывает номер заявки, дату, текущий статус и срок рассмотрения».

Экзамнационная формула: карта эмпатии не является спецификацией системы, но помогает получить более точные пользовательские требования и снизить риск проектирования интерфейса для абстрактного, несуществующего пользователя.

20. Модели качества программного обеспечения

Модель качества программного обеспечения описывает, по каким характеристикам можно оценивать продукт. Без модели качества команда рискует обсуждать качество слишком расплывчато: «хорошая система», «удобная», «надёжная». Модель превращает качество в набор характеристик и подхарактеристик, которые можно анализировать, проектировать и проверять.

К основным характеристикам качества обычно относят функциональную пригодность, производительность, совместимость, удобство использования, надёжность, безопасность, сопровождаемость и переносимость. Функциональная пригодность показывает, выполняет ли система нужные функции. Производительность описывает

время отклика, потребление ресурсов и пропускную способность. Совместимость показывает, как система взаимодействует с другими продуктами и средами. Удобство использования связано с понятностью, обучаемостью и комфортом пользователя.



Figure 32: Схема 32

Надёжность означает способность системы сохранять работоспособность при нормальных и неблагоприятных условиях. Безопасность связана с защитой данных, доступом, целостностью и подлинностью. Сопровождаемость показывает, насколько легко систему изменять, исправлять и развивать. Переносимость описывает способность работать в разных средах.

Модель качества нужна на всех этапах жизненного цикла. На этапе требований она помогает формулировать нефункциональные требования. На этапе проектирования — выбирать архитектурные решения. На этапе тестирования — строить проверки. Например, если важна безопасность, проектируются роли и права, а затем проводятся проверки авторизации. Если важна производительность, задаются количественные критерии и выполняются нагрузочные тесты.

Пример: для банковского приложения критичны безопасность, надёжность и функциональная корректность. Для мобильного приложения массового сервиса особенно важны удобство использования и производительность. Для библиотечного модуля, который будет встраиваться в разные продукты, важны сопровождаемость и переносимость. Качество всегда контекстно: невозможно одинаково максимизировать все характеристики без роста стоимости и сложности. Современная модель качества часто связывается с ISO/IEC 25010. В ней качество продукта описывается характеристиками: функциональная пригодность, производительность, совместимость, удобство использования, надёжность, безопасность, сопровождаемость и переносимость. Каждая характеристика раскрывается через подхарактеристики.

Функциональная пригодность показывает, насколько функции покрывают потребности пользователя и дают корректные результаты. Производительность включает время отклика, пропускную способность и потребление ресурсов. Совместимость включает сосуществование и взаимодействие с другими системами. Удобство использования включает узнаваемость, обучаемость, защищённость от пользовательских ошибок и доступность. Надёжность включает зрелость, доступность, отказоустойчивость и восстанавливаемость.

Безопасность включает конфиденциальность, целостность, неотказуемость, подотчётность и подлинность. Сопровождаемость включает модульность, повторное использование, анализируемость, изменяемость и тестируемость. Переносимость включает адаптируемость, устанавливаемость и заменяемость.

Для экзамена важно указать, что модель качества нужна для превращения общего слова «качество» в проверяемые требования. Например, «надёжность» превращается в требование доступности 99,9%, а «производительность» — в требование 95-го перцентиля времени ответа менее 500 мс при заданной нагрузке.

21. Контроль и обеспечение качества ПО

Контроль качества и обеспечение качества связаны, но не совпадают. Контроль качества направлен на проверку готового или промежуточного результата: соответствует ли продукт требованиям, есть ли дефекты, можно ли выпускать версию. Обеспечение качества шире: оно направлено на организацию процессов так, чтобы дефекты предотвращались, а не только обнаруживались в конце.



Figure 33: Схема 33

Контроль качества включает тестирование, ревью, статический анализ, проверку документации, контроль сборок, анализ дефектов. Его задача — дать информацию о состоянии продукта. Например, тестирующий проверяет, что функция восстановления пароля работает по требованиям, а отчёт о дефекте фиксирует найденное несоответствие.

Обеспечение качества включает настройку процесса разработки: правила описания требований, критерии готовности задач, code review, стандарты оформления документации, управление конфигурациями, планирование тестирования, анализ причин дефектов. Если контроль качества ловит ошибку, обеспечение качества задаёт вопрос: почему такая ошибка появилась и как не допустить её повторения.

Пример: если в релизе часто возникают ошибки из-за неоднозначных требований, одно только увеличение количества тестов не решит проблему. Нужно улучшить процесс анализа требований: добавить ревью требований, шаблоны описания, критерии проверяемости, трассировку требований к тестам. Это уже область обеспечения качества.

В зрелом процессе QA и QC работают вместе. QA создаёт условия для управляемой разработки, QC даёт обратную связь о фактическом состоянии продукта. Тестирование не должно восприниматься только как финальный барьер перед релизом. Чем раньше выявлена ошибка, тем дешевле её исправление и тем меньше вероятность цепочки последствий.

Quality Assurance, QA — обеспечение качества, то есть деятельность по построению процессов, предотвращающих появление дефектов. **Quality Control, QC** — контроль качества, то есть проверка конкретных результатов разработки на соответствие требованиям. **Тестирование** — один из методов контроля качества, связанный с выполнением проверок и сравнением фактического результата с ожидаемым.

Пример различия: введение шаблона требований и обязательного review требований — QA; проверка готовой функции по тест-кейсам — QC; запуск автотестов регистрации — тестирование. QA работает с процессом, QC — с продуктом и артефактами, тестирование — с поведением системы или её частей.

К обеспечению качества относятся стандарты кодирования, Definition of Done, code review, статический анализ, управление конфигурациями, план тестирования, анализ причин дефектов. К контролю качества относятся тестирование, инспекции, ревью документации, приёмочные проверки, проверка сборок.

Следовательно, зрелая разработка не может полагаться только на финальное тестирование. Дефекты дешевле предотвращать ранними практиками QA, чем обнаруживать перед релизом, когда уже затронуты код, тесты, документация и сроки поставки.

22. Классификация тестирования по способу тестирования. Примеры

По способу тестирования обычно выделяют статическое и динамическое тестирование, а также тестирование чёрного, белого и серого ящика. Статическое тестирование выполняется без запуска программы. Проверяются требования, проектная документация, код, схемы, тест-кейсы. Пример: ревью требований на полноту и однозначность или анализ кода статическим анализатором.

Динамическое тестирование выполняется с запуском программы. Тестировщик или автоматический тест подаёт входные данные, выполняет действия и сравнивает фактический результат с ожидаемым. Пример: открыть форму регистрации, ввести корректные данные, нажать кнопку и проверить создание аккаунта.

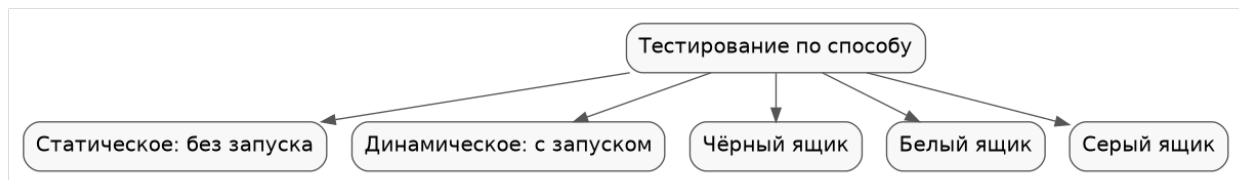


Figure 34: Схема 34

Тестирование чёрного ящика выполняется без знания внутренней структуры программы. Тестировщик ориентируется на требования, интерфейс и ожидаемое поведение. Например, проверяет, что поле возраста принимает значения от 18 до 99 и показывает ошибку при вводе 17.

Тестирование белого ящика учитывает внутреннюю структуру кода. Проверяются ветвления, циклы, условия, покрытие операторов и путей. Такой подход часто применяют разработчики при модульном тестировании.

Серый ящик сочетает оба подхода. Тестировщик знает общую архитектуру, структуру данных или особенности реализации, но проверяет систему через внешний интерфейс. Например, при проверке web-приложения тестировщик использует знания о кэше, очередях или базе данных, чтобы подобрать более точные сценарии.

Также по способу выполнения различают ручное и автоматизированное тестирование. Ручное полезно для исследовательских проверок, UX, новых функций. Автоматизированное эффективно для повторяемых регрессионных проверок, API-тестов и быстрой обратной связи. Нельзя считать автоматизацию полной заменой ручного тестирования: это инструмент, который должен применяться там, где окупается.

Статическое тестирование выполняется без запуска программы и проверяет артефакты: требования, проектные решения, код, тесты, документацию. Оно

позволяет находить дефекты до выполнения программы. **Динамическое тестирование** выполняется с запуском программы или её части и проверяет фактическое поведение на входных данных.

Чёрный ящик означает, что тестировщик не использует знания внутреннего кода и строит проверки по требованиям и интерфейсу. **Белый ящик** означает, что проверки строятся с учётом внутренней структуры: ветвлений, условий, циклов, путей выполнения. **Серый ящик** сочетает внешний способ проверки с частичным знанием реализации, например структуры базы данных или архитектуры кэша.

Также по способу выполнения выделяют ручное и автоматизированное тестирование. Ручное эффективно для исследовательских проверок, UX и новых нестабильных функций. Автоматизированное эффективно для повторяемых регрессионных сценариев, API, модульных проверок и CI/CD.

Короткий пример: проверка требования «поле возраста принимает 18–99» методом чёрного ящика использует значения 17, 18, 99, 100. Проверка белого ящика дополнительно смотрит условие в коде и покрывает обе ветви if. Серый ящик может учитывать, что возраст хранится в базе как integer и проверить недопустимые символы и переполнение.

23. Классификация тестирования по целям тестирования. Примеры

По целям тестирования проверки делят в зависимости от того, какую информацию команда хочет получить о продукте. Функциональное тестирование проверяет, выполняет ли система заявленные функции. Например, система должна позволять пользователю создать заказ, изменить адрес доставки и отменить заказ до оплаты.

Нефункциональное тестирование проверяет характеристики качества: производительность, безопасность, удобство использования, совместимость, надёжность. Например, нагрузочное тестирование выясняет, выдерживает ли система 1000 одновременных пользователей, а тестирование безопасности проверяет, может ли пользователь получить доступ к чужим данным.

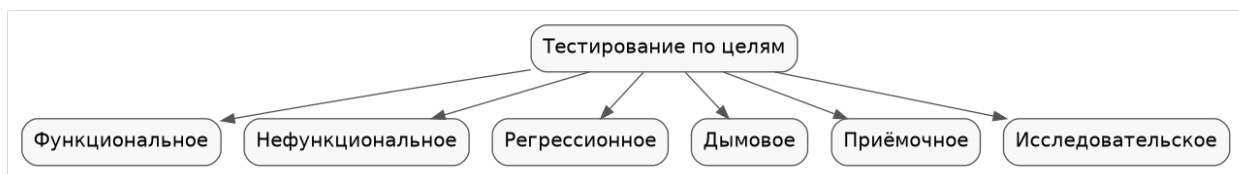


Figure 35: Схема 35

Регрессионное тестирование проверяет, не сломали ли новые изменения уже работавшую функциональность. Например, после изменения модуля оплаты нужно проверить, что оформление заказа, возврат и история платежей продолжают работать.

Дымовое тестирование — короткая проверка базовой работоспособности сборки. Его цель — понять, имеет ли смысл продолжать более глубокое тестирование. Если приложение не запускается или не работает авторизация, полное тестирование бессмысленно.

Приёмочное тестирование проверяет готовность продукта с точки зрения заказчика или пользователя. Оно отвечает на вопрос: можно ли принять результат работ. Например, заказчик проверяет, что система формирует отчёты в нужном формате и

поддерживает утверждённый процесс.

Исследовательское тестирование применяется, когда тестировщик одновременно изучает продукт, проектирует проверки и выполняет их. Оно полезно при недостаточной документации, новых функциях и поиске неожиданных дефектов. Цель классификации по задачам — правильно выбрать глубину, метод и критерии завершения тестирования.

Функциональное тестирование отвечает на вопрос, выполняет ли система требуемые функции. **Нефункциональное тестирование** отвечает на вопрос, с каким качеством система их выполняет: быстро ли, безопасно ли, удобно ли, устойчиво ли, совместимо ли.

Регрессионное тестирование проверяет, не нарушили ли изменения ранее работавшее поведение. Дымовое тестирование проверяет базовую пригодность сборки к дальнейшим проверкам. Sanity testing выполняет узкую проверку конкретного изменения или исправления. Приёмочное тестирование проверяет готовность продукта к принятию заказчиком или пользователем.

Нагрузочное тестирование проверяет поведение при ожидаемой нагрузке, стресс-тестирование — при нагрузке выше ожидаемой или в экстремальных условиях. Тестирование безопасности проверяет доступ, авторизацию, утечки данных, инъекции, небезопасные настройки. Usability-тестирование проверяет удобство выполнения пользовательских задач.

Для экзамена важно писать цель проверки: не просто «есть регресс», а «регрессионное тестирование необходимо после изменений, чтобы убедиться, что уже реализованные функции не сломаны». Это показывает понимание назначения каждого вида тестирования.

24. Классификация тестирования по уровням тестирования. Примеры

Уровни тестирования соответствуют разной степени детализации системы. Модульное тестирование проверяет отдельные функции, классы или компоненты. Обычно его выполняют разработчики. Пример: тест функции расчёта скидки проверяет, что при сумме заказа 10 000 рублей применяется скидка 5%.

Интеграционное тестирование проверяет взаимодействие компонентов между собой. Даже если каждый модуль работает отдельно, ошибка может возникнуть на стыке: неправильный формат данных, неверная обработка ошибок, различие в ожиданиях по протоколу. Пример: сервис заказов передаёт данные в сервис оплаты, и нужно проверить корректность обмена.

Системное тестирование проверяет всю систему целиком в условиях, близких к реальным. Здесь проверяются пользовательские сценарии, нефункциональные характеристики, конфигурация, совместимость. Например, полный сценарий интернет-магазина: регистрация, поиск товара, оформление заказа, оплата, уведомление.

Приёмочное тестирование проводится для подтверждения соответствия продукта ожиданиям заказчика и готовности к эксплуатации. Оно может выполняться заказчиком, пользователями или представителями бизнеса. Пример: университет принимает систему подачи заявлений, проверяя утверждённые сценарии и отчёты. Разделение на уровни помогает локализовать дефекты. Если ошибка найдена на модульном уровне, её обычно проще исправить. Если она дошла до системного или приёмочного уровня, стоимость исправления выше. Поэтому зрелый процесс

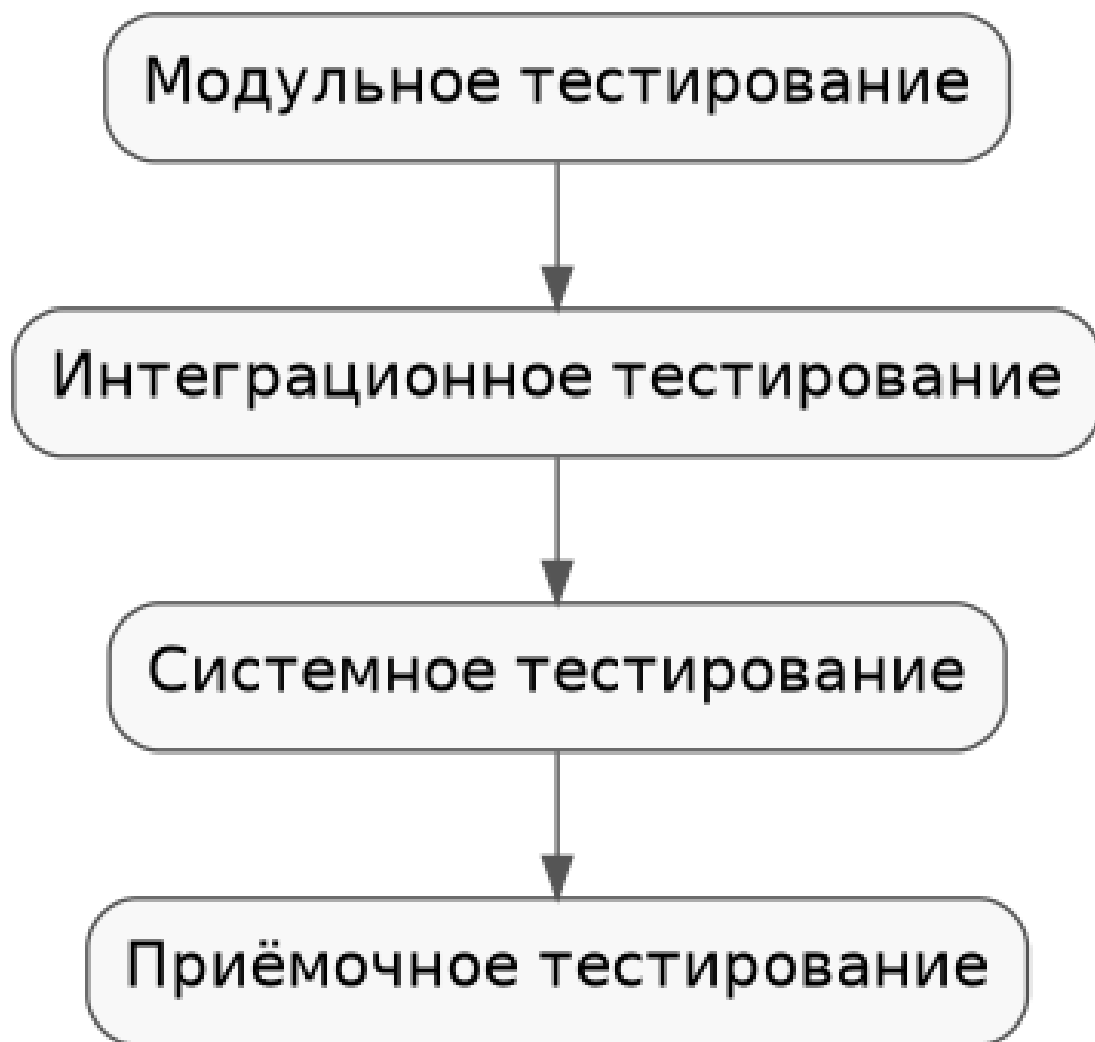


Figure 36: Схема 36

тестирования строится не только на финальных проверках, а на постепенном контроле качества на разных уровнях.

Модульное тестирование проверяет минимальные изолированные части программы: функцию, метод, класс, компонент. Цель — быстро обнаружить ошибку в локальной логике. **Интеграционное тестирование** проверяет взаимодействие частей: модулей, сервисов, базы данных, очередей, внешних API. Цель — найти ошибки на границах.

Системное тестирование проверяет всю систему целиком в окружении, близком к реальному. Оно включает сквозные пользовательские сценарии и нефункциональные проверки. **Приёмочное тестирование** подтверждает, что продукт соответствует ожиданиям заказчика, договорённостям и критериям приёмки.

Пример одной функции на разных уровнях: скидка рассчитывается модульным тестом функции `calculateDiscount`; интеграционно проверяется передача скидки из корзины в заказ; системно проверяется полный сценарий покупки; приёмочно заказчик проверяет, что бизнес-правила скидок соответствуют регламенту.

Следовательно, уровни тестирования отличаются объектом проверки и целью. Чем ниже уровень, тем дешевле и быстрее локализация дефекта. Чем выше уровень, тем ближе проверка к реальному пользовательскому поведению, но тем сложнее диагностика причины ошибки.

25. Классификация дефектов и жизненный цикл дефекта. Схема

Дефект — это несоответствие фактического результата ожидаемому. Ошибка может возникнуть в требованиях, проектировании, коде, данных, конфигурации или документации. Важно различать ошибку человека, дефект в артефакте, сбой при выполнении и отказ системы как наблюдаемое нарушение работы.

Дефекты классифицируют по серьёзности, приоритету, области возникновения, воспроизводимости, типу ошибки и стадии обнаружения. Серьёзность показывает влияние дефекта на систему: блокирующий, критический, значительный, незначительный. Приоритет показывает срочность исправления с точки зрения бизнеса. Например, опечатка на главной странице может иметь высокий приоритет из-за репутации, хотя техническая серьёзность низкая.

```
stateDiagram-v2
    [*] --> New: найден
    New --> Assigned: назначен
    Assigned --> InProgress: исправляется
    InProgress --> Fixed: исправлен
    Fixed --> Retest: передан на проверку
    Retest --> Closed: подтверждено исправление
    Retest --> Reopened: ошибка сохраняется
    Reopened --> Assigned
    New --> Rejected: не дефект/дубликат
    Rejected --> Closed
```

Жизненный цикл дефекта начинается с обнаружения. Тестировщик создаёт отчёт: описание, шаги воспроизведения, фактический и ожидаемый результат, окружение, вложения, серьёзность и приоритет. Затем дефект анализируется и назначается ответственному. Разработчик исправляет ошибку или отклоняет дефект с объяснением. После исправления тестировщик выполняет повторную проверку. Если дефект устранён, он закрывается. Если нет — переоткрывается.

Качественный отчёт о дефекте должен быть понятным, воспроизводимым и полезным для исправления. Плохой отчёт «ничего не работает» не помогает команде. Хороший отчёт указывает конкретный сценарий, данные, окружение и результат.

Жизненный цикл дефекта нужен не только для исправления конкретной ошибки, но и для управления качеством. Анализ дефектов показывает проблемные модули, слабые места требований, недостатки процесса разработки и тестирования. Если много дефектов связано с неоднозначными требованиями, нужно улучшать анализ требований, а не только исправлять код.

Ошибка человека, error — неправильное действие аналитика, разработчика, тестировщика или пользователя. **Дефект, defect/bug** — следствие ошибки, зафиксированное в артефакте: требованиях, коде, конфигурации, данных или документации. **Сбой/отказ, failure** — наблюдаемое неправильное поведение системы при выполнении.

Серьёзность дефекта показывает техническое влияние на систему: блокирующий, критический, major, minor, trivial. Приоритет показывает срочность исправления с точки зрения бизнеса. Эти параметры не равны. Например, редкая ошибка в админском отчёте может быть высокой серьёзности, но низкого приоритета; опечатка на главной странице может быть низкой серьёзности, но высоким приоритетом из-за репутации.

Качественный баг-репорт включает заголовок, окружение, предусловия, шаги воспроизведения, фактический результат, ожидаемый результат, вложения, логи, серьёзность и приоритет. Заголовок должен быть конкретным: не «форма сломана», а «при отправке формы заявки с PDF больше 20 МБ отображается 500 вместо сообщения о превышении размера».

Жизненный цикл дефекта нужен для управляемого исправления: New, Assigned, In Progress, Fixed, Retest, Closed или Reopened. Возможны статусы Duplicate, Won't Fix, Cannot Reproduce, Rejected. Анализ дефектов помогает найти системные причины: слабые требования, проблемные модули, нехватку тестов или ошибки процесса.

26. Организация и управление процессом тестирования. Схемы

Организация процесса тестирования начинается с планирования. На этом этапе определяются цели тестирования, объекты проверки, риски, ресурсы, сроки, тестовая среда, критерии начала и завершения. Тестирование не должно быть хаотичным набором проверок. Оно должно отвечать на вопрос: какую информацию о качестве продукта нужно получить и какими средствами.



Figure 37: Схема 37

После планирования выполняется анализ требований. Тестировщик проверяет требования на полноту, однозначность, непротиворечивость и проверяемость. Чем раньше найдена ошибка в требовании, тем дешевле её исправить. Далее проектируются тесты: чек-листы, тест-кейсы, тестовые наборы, тестовые данные. Затем готовится тестовая среда. Она должна быть достаточно близка к реальной эксплуатации, чтобы результаты были полезны. Для web-систем это может включать сервер приложения, базу данных, тестовые учётные записи, интеграции, версии браузеров и мобильных устройств. Если среда нестабильна, результаты тестирования

теряют доверие.

Во время выполнения тестов фиксируются результаты: пройдено, не пройдено, заблокировано, требуется уточнение. Найденные дефекты оформляются в системе отслеживания ошибок. После исправления выполняется повторная проверка и регрессионное тестирование.

Управление тестированием включает распределение задач, контроль прогресса, анализ рисков, приоритизацию проверок, отчётность и принятие решений о готовности продукта. В отчёте о тестировании могут отражаться количество выполненных проверок, найденные дефекты, открытые критические проблемы, покрытие требований, ограничения тестирования и рекомендации по релизу.

Процесс тестирования начинается не с выполнения тестов, а с анализа требований и рисков. **План тестирования** определяет объекты проверки, цели, подход, виды тестирования, ресурсы, окружение, критерии входа и выхода, риски и отчётность. **Тестовая стратегия** описывает общий подход к проверке качества продукта.

Критерии входа могут включать готовность требований, доступность сборки, тестовой среды и тестовых данных. Критерии выхода могут включать выполнение запланированных проверок, отсутствие открытых критических дефектов, достижение требуемого покрытия, принятие остаточных рисков.

Управление тестированием включает приоритизацию по рискам. Сначала проверяют наиболее критичные функции: платежи, авторизацию, сохранность данных, основные пользовательские сценарии. Менее рискованные проверки могут выполняться позже или выборочно, если сроки ограничены.

Отчёт о тестировании должен содержать не только число тестов, но и смысловую оценку качества: что проверено, что не проверено, какие дефекты открыты, какие риски остаются, можно ли выпускать версию. Хороший тест-менеджмент даёт команде информацию для решения, а не просто набор галочек.

27. Функциональное назначение Docker. Контейнеризация и виртуализация

Docker предназначен для упаковки, доставки и запуска приложений в контейнерах. Контейнер содержит приложение, зависимости, библиотеки и настройки окружения, необходимые для запуска. Это решает типичную проблему «у меня работает, а на сервере нет»: если приложение запускается в контейнере, его окружение становится более воспроизводимым.

Контейнеризация отличается от виртуализации. При виртуализации создаётся полноценная виртуальная машина с собственной гостевой операционной системой. Гипервизор распределяет ресурсы между несколькими ВМ. Каждая ВМ тяжелее, потому что содержит ОС целиком. При контейнеризации контейнеры используют ядро операционной системы хоста, но изолируются на уровне процессов, файловой системы, сети и ресурсов.

Контейнеры легче, быстрее запускаются и удобны для микросервисной архитектуры, CI/CD и локальной разработки. Виртуальные машины обеспечивают более сильную изоляцию и позволяют запускать разные ОС на одном хосте. На практике оба подхода могут сочетаться: контейнеры часто запускают внутри виртуальных машин в облачной инфраструктуре.

Пример использования Docker: backend-приложение зависит от PostgreSQL и Redis. Вместо ручной установки разработчик запускает `docker compose`, который поднимает



Figure 38: Схема 38

контейнеры приложения, базы данных и кэша. Все участники команды получают одинаковое окружение.

Docker — платформа контейнеризации, предназначенная для сборки, доставки и запуска приложений в изолированных контейнерах. **Контейнер** — изолированный процесс или группа процессов, использующие ядро операционной системы хоста, но имеющие собственное файловое окружение, сетевое пространство, переменные среды и ограничения ресурсов.

Контейнеризация отличается от виртуализации уровнем изоляции. Виртуальная машина содержит гостевую ОС и работает поверх гипервизора. Контейнер не содержит отдельного ядра ОС, поэтому запускается быстрее и потребляет меньше ресурсов. Однако изоляция контейнера обычно слабее изоляции полноценной виртуальной машины, потому что ядро разделяется с хостом.

Docker решает задачи воспроизводимости окружения, упрощения локальной разработки, доставки приложений, CI/CD и развёртывания микросервисов. Пример: приложение на Python требует конкретной версии Python и библиотек. Dockerfile фиксирует базовый образ, зависимости и команду запуска, поэтому приложение одинаково стартует у разработчика, на CI и на сервере.

Следовательно, Docker не является виртуальной машиной в строгом смысле. Он использует контейнеризацию для изоляции приложений и упаковки зависимостей, что делает окружение переносимым и воспроизводимым.

28. Архитектура платформы контейнеризации Docker

Архитектура Docker строится вокруг клиент-серверной модели. Пользователь работает с Docker Client, например через команду `docker`. Клиент отправляет команды Docker Daemon. Демон управляет образами, контейнерами, сетями и томами. Образы хранятся локально или загружаются из registry, например Docker Hub или корпоративного реестра.

Docker Image — шаблон для создания контейнера. Он содержит файловую систему, зависимости и инструкции запуска. Образ обычно создаётся по Dockerfile. Docker Container — запущенный экземпляр образа. Один образ может использоваться для создания множества контейнеров.

Dockerfile описывает сборку образа: базовый образ, копирование файлов, установку зависимостей, переменные окружения, команду запуска. Registry используется для распространения образов. Разработчик собирает образ, публикует его в registry, а сервер загружает и запускает.

Сети Docker позволяют контейнерам взаимодействовать друг с другом. Например, backend-контейнер обращается к контейнеру базы данных по имени сервиса. Volumes

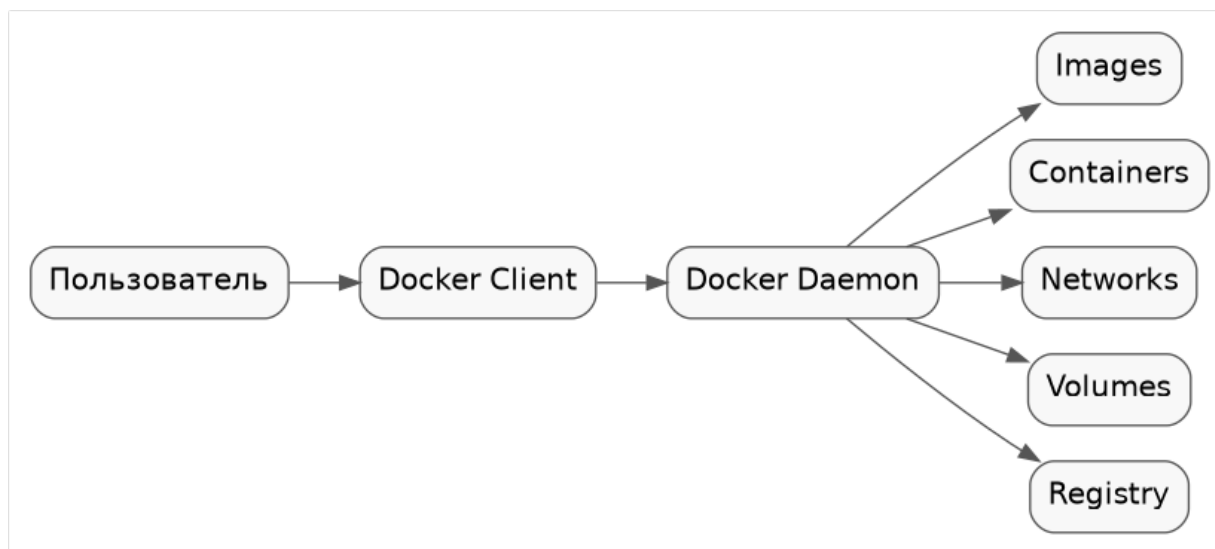


Figure 39: Схема 39

используются для хранения данных вне жизненного цикла контейнера. Это важно для баз данных: если удалить контейнер без volume, можно потерять данные.

Docker Compose добавляет уровень описания многоконтейнерного приложения. В одном YAML-файле можно описать приложение, базу данных, кэш, сети, переменные окружения и тома. Это удобно для локальной разработки и учебных стендов.

Архитектура Docker включает Docker Client, Docker Daemon, Docker Images, Docker Containers, Registry, Networks и Volumes. **Docker Client** принимает команды пользователя. **Docker Daemon** выполняет операции: сборку образов, запуск контейнеров, создание сетей и томов. **Registry** хранит и распространяет образы.

Docker Image — неизменяемый шаблон файловой системы и метаданных, из которого создаются контейнеры. Образы состоят из слоёв, что ускоряет сборку и распространение. **Docker Container** — запущенный экземпляр образа с собственным writable layer. Если контейнер удалить, данные в его временном слое исчезнут, поэтому постоянные данные выносят в volume.

Dockerfile описывает сборку образа инструкциями FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, CMD или ENTRYPOINT. **Docker Compose** описывает многоконтейнерное приложение в YAML: сервисы, сети, volumes, переменные окружения, зависимости запуска.

Практический пример: сервис app подключается к сервису postgres по имени контейнера внутри docker-сети, а данные базы хранятся в volume postgres_data. Это показывает понимание не только терминов, но и практической архитектуры Docker-приложения.

29. Назначение Apache Airflow. DAG-файл, task, operators

Apache Airflow предназначен для описания, планирования, запуска и мониторинга рабочих процессов, состоящих из зависимых задач. Он особенно часто применяется для конвейеров обработки данных: загрузки данных, преобразования, проверки качества, выгрузки в хранилище, запуска отчётов и моделей.

Ключевое понятие Airflow — DAG, Directed Acyclic Graph, направленный ациклический

граф. DAG описывает набор задач и зависимости между ними. Направленность означает, что зависимости имеют порядок: задача В запускается после задачи А. Ацикличность означает отсутствие циклов: процесс не должен бесконечно возвращаться к предыдущим задачам.



Figure 40: Схема 40

DAG-файл — Python-файл, в котором описывается граф: расписание запуска, задачи, зависимости, параметры повторных попыток, таймауты и другие настройки. Task — отдельная задача внутри DAG. Например, загрузить файл, выполнить SQL-запрос, вызвать API, запустить Python-функцию.

Operator — шаблон задачи. Он определяет, что именно будет выполнено. Например, PythonOperator запускает Python-функцию, BashOperator выполняет shell-команду, SQL-оператор выполняет запрос к базе данных, HTTP-оператор вызывает внешний API. Оператор — это тип действия, task — конкретный экземпляр действия в DAG.

Пример: ежедневный пайплайн продаж. Первая задача загружает данные из CRM, вторая проверяет структуру, третья очищает данные, четвёртая записывает их в хранилище, пятая строит отчёт. Airflow позволяет видеть статус каждой задачи, перезапускать упавшие шаги и контролировать расписание.

Apache Airflow — платформа оркестрации рабочих процессов, позволяющая описывать зависимости задач как DAG, запускать их по расписанию или событию, отслеживать статусы и перезапускать отдельные шаги. Airflow не является системой потоковой обработки данных; он управляет выполнением задач и зависимостями между ними.

DAG — направленный ациклический граф задач. Направленность задаёт порядок выполнения, ацикличность запрещает циклы зависимостей. **DAG run** — конкретный запуск DAG за определённый интервал или момент. **Task** — конкретная задача внутри DAG. **Task instance** — выполнение конкретной task в конкретном DAG run. **Operator** — класс-шаблон, определяющий тип работы: PythonOperator, BashOperator, EmptyOperator, SQL-операторы, HTTP-операторы и другие.

Важно различать operator и task: operator — тип действия, task — конкретный узел графа, созданный на основе operator. Например, PythonOperator(task_id='validate_data', python_callable=validate) создаёт задачу validate_data, которая запускает Python-функцию.

Экзаменационный пример: DAG ежедневной загрузки данных состоит из задач extract, validate, transform, load, notify. Если validate завершилась ошибкой, Airflow не запускает зависимые задачи, фиксирует статус, сохраняет логи и может выполнить retry по настройкам.

30. Конвейер обработки данных. Примеры. ETL и ELT

Конвейер обработки данных — организованная последовательность операций, через которую данные проходят от источника к целевому использованию. Источниками могут быть базы данных, API, файлы, логи, очереди сообщений. Целью может быть аналитическое хранилище, отчёт, витрина данных, ML-модель или другая

информационная система.



Figure 41: Схема 41

Пример конвейера: интернет-магазин ежедневно собирает заказы, платежи и данные доставки. Затем данные очищаются, приводятся к единому формату, связываются по идентификаторам и загружаются в хранилище. На их основе строятся отчёты по выручке, среднему чеку и просроченным доставкам.

ETL означает Extract, Transform, Load: извлечь данные, преобразовать их и затем загрузить в целевое хранилище. Такой подход удобен, когда преобразования нужно выполнить до попадания данных в хранилище, например для строгой очистки или унификации.

ELT означает Extract, Load, Transform: извлечь данные, загрузить их в хранилище и затем преобразовывать уже внутри него. Такой подход часто применяют в современных аналитических платформах, где хранилище обладает достаточной вычислительной мощностью.

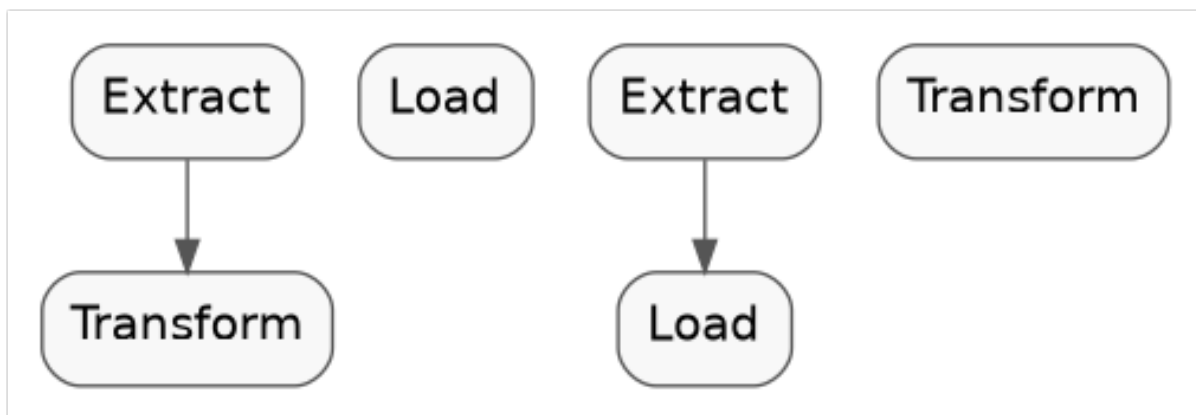


Figure 42: Схема 42

Различие между ETL и ELT связано с местом преобразования. В ETL данные очищаются до загрузки, в ELT сначала сохраняется более сырой слой данных, а затем строятся преобразованные витрины. ETL удобен для контролируемых корпоративных процессов. ELT удобен для гибкой аналитики и больших хранилищ, где нужно сохранять исходные данные и строить разные представления.

Конвейер обработки данных — управляемая последовательность операций, преобразующая данные от источника к целевому использованию. Он включает извлечение, проверку качества, очистку, преобразование, загрузку, контроль ошибок, логирование и мониторинг.

ETL означает Extract, Transform, Load. Данные сначала извлекаются из источников, затем преобразуются во внешнем обработчике и только потом загружаются в хранилище. Этот подход полезен, когда данные должны быть очищены до попадания в целевую систему или когда хранилище не предназначено для тяжёлых преобразований.

ELT означает Extract, Load, Transform. Данные сначала загружаются в хранилище в

более сыром виде, а преобразования выполняются уже внутри него. ELT характерен для современных аналитических хранилищ и lakehouse-подходов, где вычислительная мощность хранилища позволяет строить разные витрины из исходного слоя.

Критерии качества пайплайна: полнота загрузки, корректность преобразований, идемпотентность перезапуска, наблюдаемость, обработка ошибок, контроль схемы данных, проверка дублей, сохранение истории. Пример: если пайплайн за 2026-06-01 запущен повторно, он не должен удвоить продажи в витрине, а должен корректно перезаписать или обновить нужную партицию.

31. Архитектура Airflow. Принципы построения пайплайнов

Архитектура Airflow включает несколько основных компонентов. Scheduler анализирует DAG-файлы, определяет, какие задачи нужно запустить, и ставит их в очередь. Executor отвечает за способ выполнения задач: локально, параллельно, через Celery, Kubernetes и т.д. Workers выполняют задачи. Metadata Database хранит сведения о DAG, задачах, запусках, статусах и логах. Webserver предоставляет пользовательский интерфейс для мониторинга и управления.

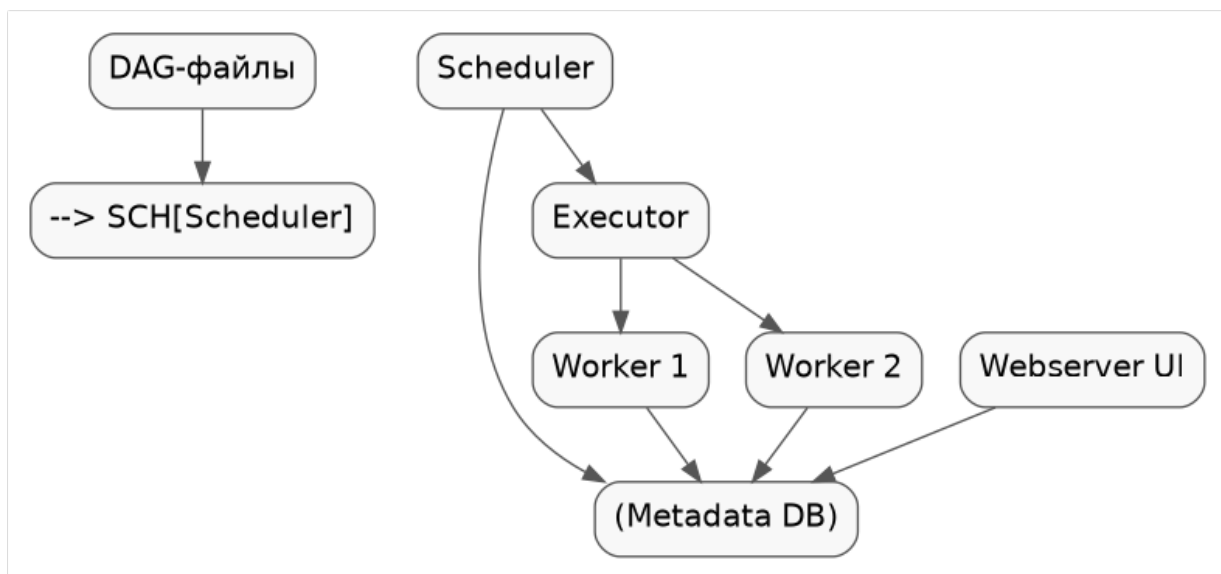


Figure 43: Схема 43

Принцип построения пайплайнов в Airflow состоит в явном описании зависимостей. Каждая задача должна быть достаточно автономной и иметь понятный результат. Нежелательно создавать огромную задачу, которая делает всё сразу: её трудно перезапускать и диагностировать. Лучше разбивать процесс на логические шаги: извлечение, проверка, преобразование, загрузка, отчёт.

Задачи должны быть идемпотентными, насколько это возможно. Повторный запуск не должен портить данные или создавать дубли. Например, загрузка данных за дату должна либо перезаписывать партицию, либо корректно проверять уже загруженные записи.

Пайплайн должен учитывать ошибки: повторные попытки, таймауты, уведомления, обработку отсутствующих данных. Также важно логировать действия и сохранять

промежуточные результаты там, где это необходимо для диагностики.

Хороший DAG читается как схема бизнес-процесса или обработки данных. Его зависимости понятны, задачи названы осмысленно, параметры вынесены в конфигурацию, а расписание соответствует потребности бизнеса. Плохой DAG превращается в невидимый клубок Python-кода, где непонятно, что запускается и почему упало.

Архитектура Airflow включает Scheduler, Executor, Workers, Metadata Database, Webserver и папку с DAG-файлами. **Scheduler** читает DAG, определяет готовые к запуску задачи и создаёт task instances. **Executor** определяет механизм выполнения задач: локальный, Celery, Kubernetes и другие. **Worker** фактически выполняет задачу. **Metadata Database** хранит состояния DAG run, task instance, расписания, подключения и другую служебную информацию. **Webserver** показывает UI для мониторинга.

Принципы хорошего пайплайна: явные зависимости, небольшие атомарные задачи, идемпотентность, понятные имена task_id, ограничение тяжёлой логики внутри DAG-файла, хранение бизнес-логики в отдельных модулях, настройка retries и timeout, логирование, уведомления об ошибках. DAG-файл должен быстро парситься scheduler-ом; тяжёлые запросы и вычисления на уровне импорта ухудшают работу Airflow.

Airflow хорошо подходит для batch-процессов и периодических workflow, но не предназначен для низколатентной потоковой обработки событий. Для streaming чаще применяют Kafka Streams, Flink, Spark Structured Streaming и похожие инструменты, а Airflow может запускать и координировать batch-задачи вокруг них. Следовательно, Airflow ценен не самим Python-файлом, а оркестрацией зависимостей, повторными запусками, журналированием, мониторингом и управлением состояниями задач. Хороший DAG должен читаться как схема процесса, а не как запутанный скрипт.

4 Мультимедиа технологии

4.1 Аудиосигналы, их свойства. Спектральные характеристики.

Короче это мегабилет где можно очень много чего рассказать, но не бойтесь, зато остальные билеты меньше, так как я там ссылаюсь сюда

Аудиосигналы представляют собой сумму нескольких гармонических колебаний (синусоид). Свойства: частота (ну и длина волны соответственно) и амплитуда

По Y всегда амплитуда, по X здесь время.

Громкость всегда измеряется в децибелах. В физике вообще децибел это даже тип абсолютная величина которой измеряют громкость исходя из P (ватт/см²). Короче это прям что-то из реального мира что можно померить и почувствовать.

$$P_{dB} = 10 \cdot \lg \left(\frac{P}{P_{weakest}} \right)$$

Интенсивность (Вт/м ²)	Уровень интенсивности (дБ)	Качественная субъективная оценка	Источник звука
10 ⁻¹¹	10	Едва слышно	Спокойное дыхание
10 ⁻¹⁰	20	Очень тихо	Шелест листьев
10 ⁻⁹	30	Тихо	Перелистывание книги
10 ⁻⁸	40	Умеренно	Тихая контора
10 ⁻⁷	50	Умеренно	Домашняя обстановка
10 ⁻⁶	60	Умеренно	Обычный разговор
10 ⁻⁵	70	Умеренно	Лектор
10 ⁻⁴	80	Шумно	Уличный транспорт
10 ⁻³	90	Очень шумно	Близко идущий поезд
10 ⁻²	100	Очень шумно	Пожарная сирена
10 ⁻¹	110	Невыносимо	Взлет авиалайнера
10	120	Болевое ощущение	Разрыв артиллерийского снаряда

НО. Децибелы бывают разные. В проге по другому!

Записанный звук => файл на компьютере => набор чисел. У набора чисел нет громкости. Громкость (которая реальная, из физики, которая в ватт/см²) зависит от динамиков.

В проге децибелы это относительная величина. А ещё есть формула через напряжение. Короче всякая электроника (типо микрофона) она работает с напряжением (которые вольты) и есть такая формула

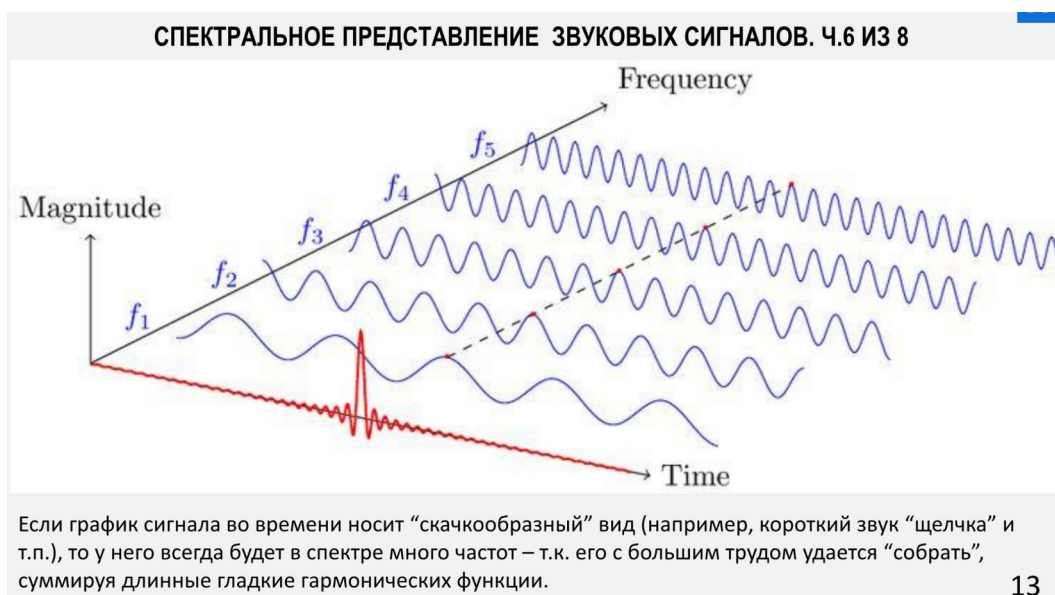
$$\text{дБ} = 20 \lg \frac{U_1}{U_0}$$

по напряжению

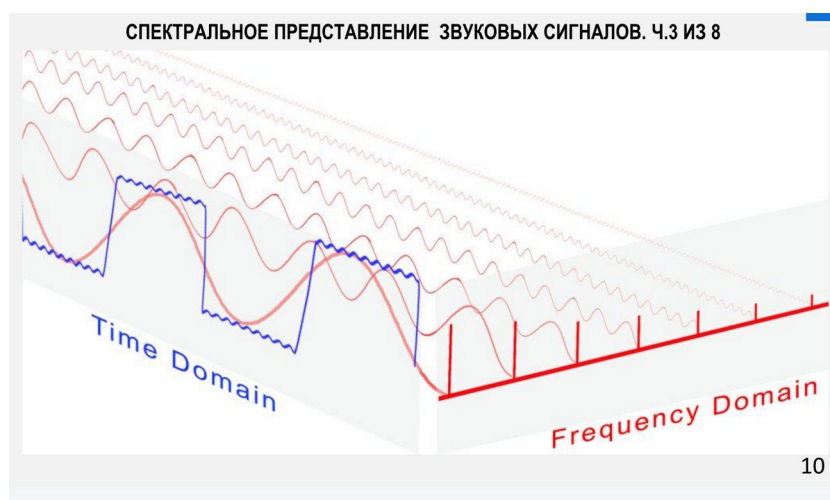
Что такое U_0 я не знаю. Может быть типичная чувствительность микрофона или ещё что-то. Но я думаю преподаватели сами не знают. Просто скажите что децибелы разные бывают разные. Это понт.

Теперь про спектральные характеристики

Короче любую функцию от t можно разложить на сумму гармоник (синусоид). Т.е. звук который меняется во времени это вот такая хитровыведенная сумма синусоид, каждая из которых постоянная во времени, но в сумме они все дают нужный звук. Внизу пример как собрать щелчок.



А теперь что такое спектр



Вот синяя это звук (функция амплитуды от времени). Звук это сумма синусоид. А красные палочки это размер этих самых синусоид. Вот эти палочки это и есть спектр. У спектра по оси **X частота!** по оси Y амплитуда. Чтобы получить спектр нужно

$$X(f) = \int_{-\infty}^{\infty} x(t) \cdot e^{-j2\pi ft} dt$$

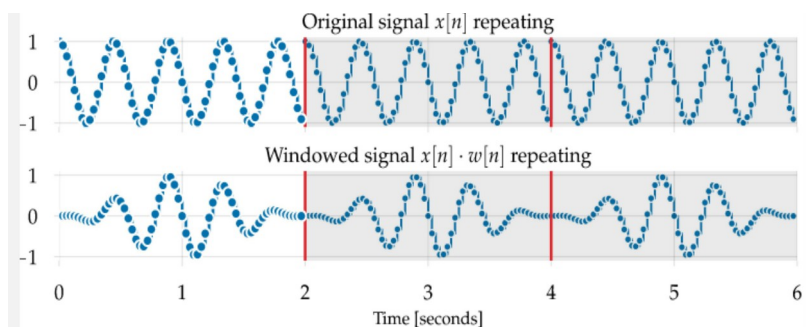
- в теории взять интеграл Фурье
- на практике выполнить дискретное преобразование Фурье (ДПФ) (ну заменяем интеграл суммой). ОДПФ - обратное ДПФ (это мы из спектра сигнал получаем)

$$X_k = \sum_{n=0}^{N-1} x_n e^{-j2\pi \frac{nk}{N}} \quad \text{— это ДПФ}$$

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{j2\pi \frac{nk}{N}} \quad \text{— это ОДПФ}$$

- ну и совсем на практике существуют другие алгоритмы которые вычисляют то же самое но быстрее (типо быстрое преобразование Фурье)

А есть ещё одна штука для умных. Преобразование Фурье круто работает если сигнал начинается и заканчивается одинаково. Чтобы работать с реальными звуками придумали оконирование. Короче делаем сигнал красивее чтобы спектр получился красивее.



4.2 Методологические подходы к фильтрации аудиосигналов.

Короче фильтры работают по разному, но цель у них одна — “убрать чистоты которые нас не устраивают и оставить те которых нас устраивают” (цитата Жаринова).

МЕТОДОЛОГИЧЕСКИЕ АСПЕКТЫ ФИЛЬТРАЦИИ СИГНАЛОВ. Ч.2 ИЗ 5

Закреть

Для изменения спектра звуковых сигналов путем регулирования АЧХ трактов используют различные устройства, создающие спады или подъемы АЧХ в области нижних и верхних частот или в ограниченных участках на средних частотах. Распространенные задачи изменения спектра аудиосигналов методом фильтрации в мультимедиа:

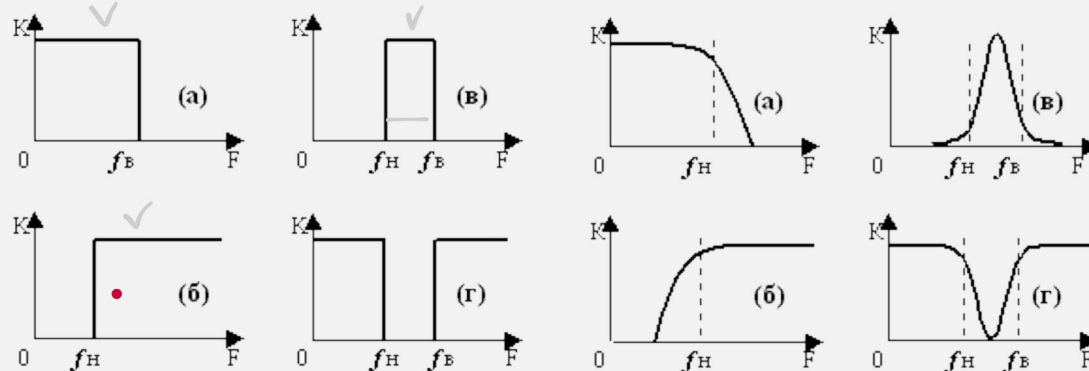
- придание большей выразительности звучанию певческих голосов и музыкальных инструментов;
- уменьшение заметности некоторых недостатков речи (неприятного тембра, шепелявости; посвистывания);
- создание различных звуковых эффектов, например имитации звучания речи по телефону, по радио, через рупор;
- имитация акустической обстановки передаваемых сцен;
- получение новых, необычных тембров;
- подчёркивание характерной особенности звучания инструментов и их выделение на фоне массивной оркестровки.
- ослабление влияния помех (шумов) при реставрации старых фонограмм, записанных механическим, оптическим или магнитным способом, и при записи в неудовлетворительных акустических условиях, с частотными искажениями.

Регуляторы АЧХ иногда называют **частотными корректорами**.

к.т.н., доц. Жарин О.О., ФГАОУ ВО "Санкт-Петербургский государственный университет аэрокосмического приборостроения" (ГУАП). Кафедра прикладной информатики (№41)
Дисциплина "Мультимедиа-технологии". Лекция 1. 2025 г.

30

Какие бывают фильтры. Если что это графики так называемой передаточной функции $W(f)$. f - частота, K это 0 если убираем, 1 оставляем, 1.5 усиливаем (например)

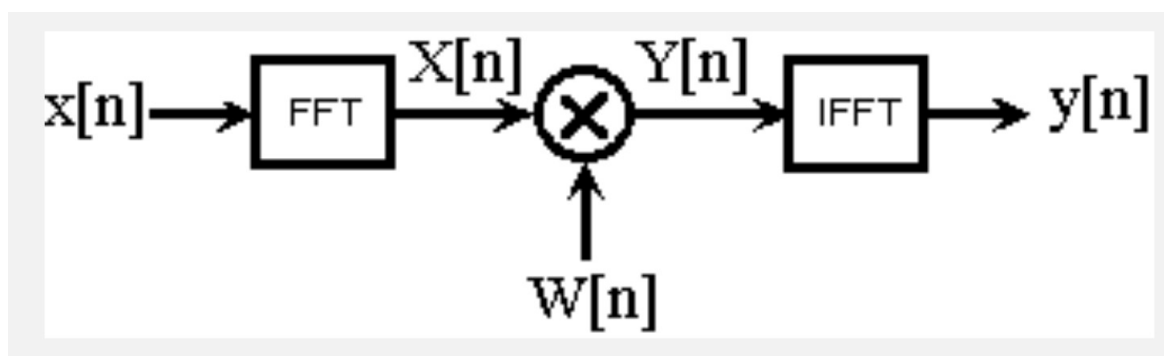


1. По частотной характеристике	
Тип фильтра	Что делает
ФНЧ (Фильтр низких частот, LPF)	Пропускает низкие частоты, ослабляет высокие
ФВЧ (Фильтр высоких частот, HPF)	Пропускает высокие, ослабляет низкие
ПФ (Полосовой фильтр, BPF)	Пропускает только определенный диапазон
Режекторный (Notch)	Подавляет узкую полосу частот
Полочный (Shelving)	Усиливает/ослабляет ВЧ или НЧ без резкого среза

Режекторный фильтр это если кто не понял “анти полосовой фильтр”. Врежекторный = 1 - Wполосовой

Ну и понятно фильтры бывают фурье, рекурсивные и нерекурсивные.

4.3 Фильтрация аудиосигналов с использованием обработки в спектральном пространстве.



FFT - преобразование Фурье (ДПФ)

IFFT - обратное преобразование Фурье (ОПФ)

1. Есть x - сигнал (массив чисел), значение - это амплитуда
2. С помощью ДПФ получаем X – спектр. Это тоже массив но там значения это амплитуда частоты. Короче у нас есть типо n частот (от 0 до частоты дискретизации 44100 обычно). И значение это размер [палочки](#)
3. Колдуем над массивом X и получаем Y . Это делается так $Y = X * W$
4. Получаем выходной сигнал y из Y с помощью ОДПФ.
5. Круто? Круто!

Есть 1 подводный камень

Массив дискретных значений передаточной функции фильтра можно формировать либо по взятой за основу передаточной функции аналогового прототипа $W(p)$, прибегая к замене переменной:

$$f_k = \frac{k}{N \cdot T_{\Delta}} \Rightarrow W[k], \forall k$$

либо даже предельно упрощенно (а-ля “идеальный фильтр”, например для РФ):

$$W[k] = \begin{cases} 1, & \text{для } k, \text{ которые соответствуют частотам, которые нужно пропускать} \\ 0, & \text{для } k, \text{ которые соответствуют частотам, которые нужно подавить} \end{cases}$$

Единственная тонкость при формировании массива дискретных значений частотной передаточной функции, если используется пара алгоритмов комплексных БПФ – это учесть требование комплексно-сопряженной симметрии для массива частотной передаточной функции:

$$W[k] = W^*[N - k], k = 0, 1, \dots, N - 1.$$

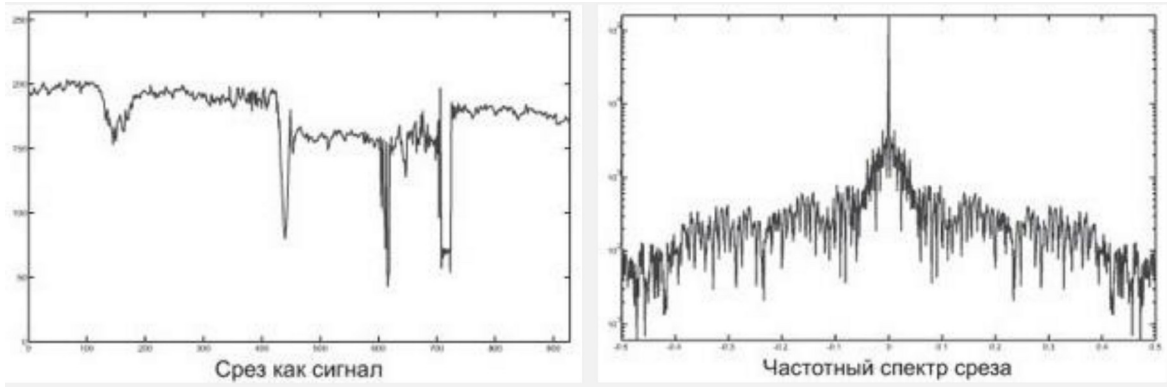
Заметим, что у $W[0]$ нет пары!, а $W[N/2]$ является сам себе комплексно-сопряженным.

Т.е. оба должны быть вещественными числами.

Поэтому нужно действовать так: получить $W[k], k = 1, 2, \dots, \frac{N}{2} - 1$ и тут же $W[k] = \text{Re}(W[k]) - j \text{Im}(W[k])$.

А потом уже вне цикла формировать $W[0]$ и $W[N/2]$

Если вы ничего не поняли. Простыми словами. Спектр содержит в 2 раза больше частот чем изначально (там есть ещё фантомные отрицательные частоты, не спрашивайте почему) и он симметричен.



Типо вот так. И он ВСЕГДА должен быть симметричным. Поэтому W тоже должна быть симметрична.

4.4 Фильтрация аудиосигналов во временной области. Нерекурсивные фильтры.

Требования к фильтру определяются необходимостью подавлять частотные компоненты в заданное количество раз. Фильтрация записанных реализаций аудиосигналов может осуществляться в частотной области, через модификацию спектра, вычисляемого с применением алгоритма БПФ, однако такой способ не подходит для сигналов, идущих в реальном масштабе времени (стриминговых сигналов). Для решения задачи фильтрации в этом случае разработано 2 типа фильтров - нерекурсивные и рекурсивные фильтры, каждый из которых имеет свои преимущества и недостатки. Достоинство нерекурсивных фильтров - они всегда сохраняют устойчивость, недостаток - при прочих равных условиях требуют большого объема вычислений. Цифровой фильтр представляет собой специализированное устройство, объединяющее цифровое вычислительное ядро и средства ввода-вывода сигналов. В рассматриваемой схеме входной аналоговый сигнал $x(t)$ поступает на аналого-цифровой преобразователь (АЦП), который осуществляет преобразование $x(t)$ в упорядоченную последовательность чисел $x[k]$, величина

каждого из которых равна значению входного сигнала в дискретные моменты времени:

$$x[k]=x(k\cdot T_{\Delta}),$$

где k – натуральные числа, T_{Δ} – период дискретизации, который задается таймером в составе вычислительного устройства. Вычислительный алгоритм тем или иным способом формирует последовательность выходных цифровых кодов $y[k]$, которые преобразуются в непрерывный выходной сигнал фильтра $y(t)$ с помощью цифроаналогового преобразователя (ЦАП):

$$y(t)=y[k], t\in[kT_{\Delta}, (k+1)T_{\Delta}).$$

Поскольку аппаратная часть ЦФ обычно реализуется по типовой структуре, под термином «цифровой фильтр» в литературе часто подразумевается только вычислительный алгоритм, описывающий преобразование последовательности дискретных отсчетов входного сигнала $\{x[k]\}$ в выходную последовательность отсчетов $\{y[k]\}$. Нерекурсивные цифровые фильтры для расчета отсчетов выходного сигнала используют только отсчеты входного сигнала. Отсутствие рекурсивных членов в разностном уравнении ЦФ гарантирует устойчивость фильтра при любом наборе коэффициентов $\{c_v\}$, а также отсутствие предельных циклов автоколебаний. Еще одним достоинством нерекурсивных ЦФ по сравнению с рекурсивными является возможность при проведении расчетов задавать функции для АЧХ и ФЧХ независимо друг от друга. Это, в частности, позволяет получать ЦФ со строго линейной ФЧХ, что минимизирует искажения сигнала в процессе фильтрации.

Недостатком нерекурсивных фильтров является только необходимость выполнения большого объема вычислений, поскольку опыт показывает, что при прочих равных условиях обеспечения необходимой частотной избирательности порядок нерекурсивного фильтра q оказывается во много раз больше, чем суммарный порядок $(q+p)$ рекурсивного фильтра. Последнее положение можно проиллюстрировать примером однородного нерекурсивного фильтра (фильтра скользящего среднего, представляющего собой простейший ФНЧ), разностное уравнение которого имеет вид:

$$y[k]=\frac{1}{q}\sum_{v=0}^{q-1}x[k-v].$$

Расчеты по данному уравнению могут быть организованы по рекурсивной схеме:

$$y[k]=y[k-1]+\frac{1}{q}(x[k]-x[k-q]).$$

Для расчета одного отсчета выходного сигнала рассматриваемого ЦФ в нерекурсивном разностном уравнении требуется q вычислительных операций, а в рекурсивном – всего 3 при любом значении q .

Для расчета нерекурсивных фильтров существует несколько методов:

1) метод разложения передаточной функции рекурсивного ЦФ в ряд Лорана;

2) аналитический метод расчета, основанный на соотношениях между частотной

передаточной функцией фильтра и его импульсной характеристикой;

3) методы, основанные на дискретизации частотной характеристики фильтра (метод

частотной выборки, оконный метод).

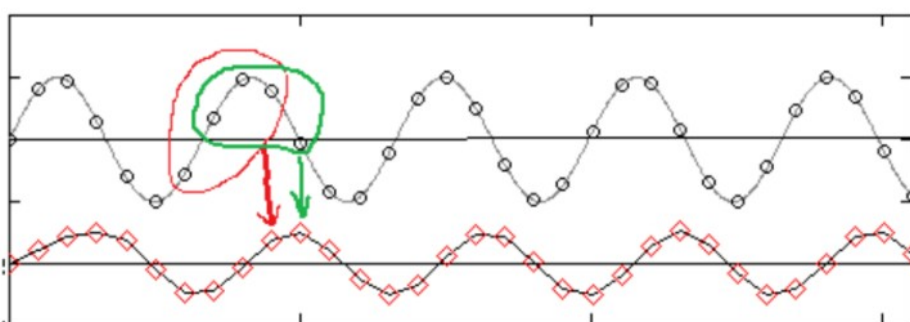
Пример одного из возможных фильтров:

ЦИФРОВАЯ ФИЛЬТРАЦИЯ АУДИОСИГНАЛОВ. НЕРЕКУРСИВНЫЕ ФИЛЬТРЫ. Ч.2 ИЗ 6

Пример - однородный нерекурсивный ЦФ

$$y[k] = \frac{1}{q} \sum_{v=0}^{q-1} x[k-v], \quad \text{Т.е. все коэффициенты одинаковы и равны величине } 1/q$$

Фильтр скользящего среднего (moving average)



Пример для $q=4$.

Однородный фильтр по сути работает как ФНЧ, сглаживает сигнал.

Пример расчета ФНЧ

ЦИФРОВАЯ ФИЛЬТРАЦИЯ АУДИОСИГНАЛОВ. НЕРЕКУРСИВНЫЕ ФИЛЬТРЫ. Ч.4 ИЗ 6

$$c_n = \frac{T_\Delta}{2\pi} \int_{-\frac{\pi}{T_\Delta}}^{\frac{\pi}{T_\Delta}} W(j\omega) e^{j\omega n T_\Delta} d\omega \quad \text{Рассчитаем идеальный ФНЧ}$$

$$c_n = \frac{T_\Delta}{2\pi} \int_{-\omega_{\text{гр}}}^{\omega_{\text{гр}}} e^{j\omega n T} e^{-j\lambda T \omega} d\omega = \frac{T e^{j\omega_{\text{гр}} T (n-\lambda)}}{2\pi j (n-\lambda)} \Big|_{-\omega_{\text{гр}}}^{\omega_{\text{гр}}} = \frac{(e^{j\omega_{\text{гр}} T (n-\lambda)} - e^{-j\omega_{\text{гр}} T (n-\lambda)})}{2\pi j (n-\lambda)} =$$

$$= \frac{2j \sin(\omega_{\text{гр}} T (n-\lambda))}{2\pi j (n-\lambda)} = \frac{\sin(\omega_{\text{гр}} T (n-\lambda))}{\pi (n-\lambda)}.$$

Проблема - необходимо использовать отрицательные индексы в c_n к тому же n должно меняться от $-\infty$ до ∞ . Другими словами, нужно $q \rightarrow \infty$

Это нереализуемо, а для конечных значений q последовательность c_n будет искусственно усечена. Что скажется на частотных характеристиках фильтра.

Реализация нерекурсивных фильтров в Matlab (6-9 строка)

4.5 Фильтрация аудиосигналов во временной области. Рекурсивные фильтры.

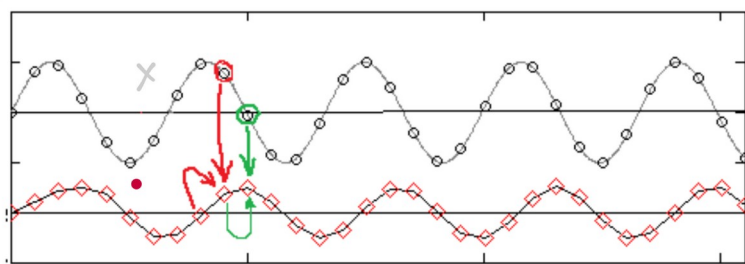
Было бы прикольно фильтровать сигнал без этого Фурье репа. Так вот так можно.

ЦИФРОВАЯ ФИЛЬТРАЦИЯ АУДИОСИГНАЛОВ. РЕКУРСИВНЫЕ ФИЛЬТРЫ. Ч.2 ИЗ 7

Заккрыть

Пример – простейший рекурсивный фильтр - фильтр 1-го порядка

$$y[k] = a_1 y[k-1] + b_0 x[k]$$



При $a_1 > 0$ этот фильтр работает как ФНЧ, при $a_1 < 0$ как усилитель-корректор, поднимающий ВЧ.

Но для данного фильтра всегда должно выполняться условие $|a_1| < 1$, иначе - беда.

Пусть $a_1 = -2$, $b_0 = 1$, $x[0] = 1$.

Получим: 1, -2, 4, -8, 16, -32, ...

$$y[k] = b_0 x[k] + b_1 x[k-1] + b_2 x[k-2] - a_1 y[k-1] - a_2 y[k-2]$$

Передаточная функция данного фильтра:
$$W(z) = \frac{b_0 z^2 + b_1 z + b_2}{z^2 + a_1 z + a_2}$$

ВСЕ корни полинома в знаменателе д.б. по модулю меньше 1

Короче тут всё просто: есть формула $y[k] = \dots$ что-то там. Если $k-1$ то мы берём 1 “кадр” из прошлого, если $k-2$ то 2 кадра. Вот эта глубина называется порядком. В общем на слайде всё есть, поясню только за передаточную функцию.

Вот у нас снова есть [W](#) но она какая-то другая. Это тоже передаточная функция но уже от z . Что такое z Жаринов не стал объяснять, в общем это что-то на очень умном. Тут у нас другая тема. Есть уже придуманные способы вычисления этих коэффициентов

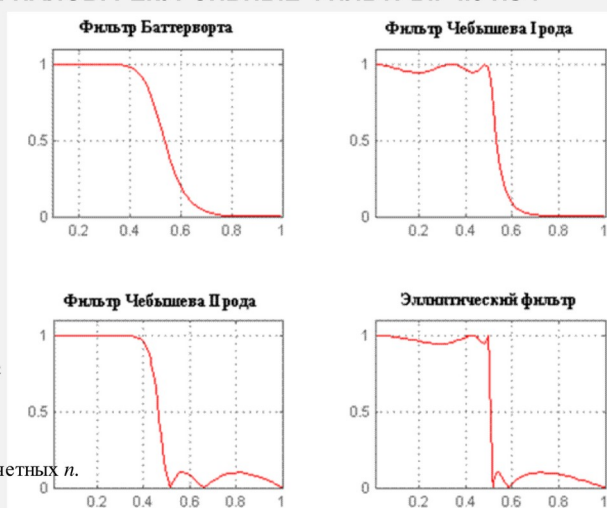
ЦИФРОВАЯ ФИЛЬТРАЦИЯ АУДИОСИГНАЛОВ. РЕКУРСИВНЫЕ ФИЛЬТРЫ. Ч.3 ИЗ 7

Заккрыть

Расчет коэффициентов рекурсивного фильтра производится исходя из свойств желаемого т.н. аналогового фильтра-прототипа. Существует несколько родов (или семейств) таких фильтров-прототипов. Для каждого из них известна формула для передаточной функции – она и является основой расчета цифровых фильтров с нужными свойствами.

Напр., для фильтра Баттерворта:

$$W_{Bt}(p) = \begin{cases} \frac{1}{\prod_{k=1}^{\frac{n}{2}} \left(p^2 - 2p \cos\left(\frac{2k+n-1}{2n}\pi\right) + 1 \right)}, & \text{для четных } n \\ \frac{1}{\prod_{k=1}^{\frac{n-1}{2}} \left(p^2 - 2p \cos\left(\frac{2k+n-1}{2n}\pi\right) + 1 \right) (p+1)}, & \text{для нечетных } n. \end{cases}$$



Сравнение характеристик фильтров разного рода

к.т.н., доц. Жаринов О.О., ФГАОУ ВО "Санкт-Петербургский государственный университет аэрокосмического приборостроения" (ГУАП). Кафедра прикладной информатики (№41)
Дисциплина "Мультимедиа-технологии". Лекция 2. 2025 г.

Вот посмотрите на разные фильтры низких частот.

4.6 Обработка изображений. Методы линейного контрастирования.

ПОЭЛЕМЕНТНАЯ ОБРАБОТКА. УЛУЧШЕНИЕ КАЧЕСТВА ИЗОБРАЖЕНИЙ. ЛИНЕЙНОЕ КОНТРАСТИРОВАНИЕ. Ч.1 ИЗ 3

Изображение – это матрица $\{x_{m,n}\}, m = 0, 1, \dots, M-1; n = 0, 1, \dots, N-1$.
Каждый элемент – целое число от 0 до 255

Если пиксель с максимальной яркостью “не дотянул” до 255, а пиксель с минимальной яркостью имеет величину больше нуля, то можно весь диапазон от минимума до максимума растянуть на диапазон от 0 до 255, выполнив общее преобразование для всех пикселей.

$$y = ax + b$$

$$\begin{cases} y_{\max} = ax_{\max} + b = 255 \\ y_{\min} = ax_{\min} + b = 0 \end{cases}$$

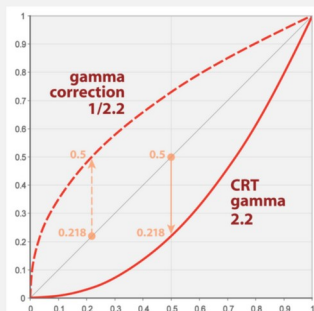
$$y = \frac{x - x_{\min}}{x_{\max} - x_{\min}} (y_{\max} - y_{\min}) + y_{\min}$$

к.т.н., доц. Жаринов О.О., ФГАОУВО “Санкт-Петербургский государственный университет аэрокосмического приборостроения” (ГУАП). Кафедра прикладной информатики (№41)
Дисциплина “Мультимедиа-технологии”. Лекция 3. 2025 г.

12

4.7 Обработка изображений. Гамма-коррекция.

ПОЭЛЕМЕНТНАЯ ОБРАБОТКА. УЛУЧШЕНИЕ КАЧЕСТВА ИЗОБРАЖЕНИЙ. ГАММА-КОРРЕКЦИЯ ИЗОБРАЖЕНИЙ



$$y = y_{\max} \left(\frac{x}{x_{\max}} \right)^{\gamma}$$

Смысл гамма-коррекции заключается в перераспределении относительной доли пикселей с разными яркостями в изображении: если в исходном изображении слишком много темных областей, их переводят в более светлые, и наоборот. При этом закон гамма-коррекции очень простой и является частным случаем более общего подхода, называемого градационными преобразованиями или эквализацией

к.т.н., доц. Жаринов О.О., ФГАОУВО “Санкт-Петербургский государственный университет аэрокосмического приборостроения” (ГУАП). Кафедра прикладной информатики (№41)
Дисциплина “Мультимедиа-технологии”. Лекция 3. 2025 г.

17

4.8 Обработка изображений. Эквализация гистограмм.

Эквализация гистограммы (histogram equalization) — это метод улучшения контраста изображения за счёт перераспределения яркости пикселей.

Представьте, что всё изображение состоит из тёмных, средних и светлых тонов. Если, например, тёмных пикселей слишком много (изображение "затенённое"), эквализация растягивает гистограмму так, чтобы яркости распределились равномерно.

Гистограмма — это график, показывающий, сколько пикселей имеют определённую яркость (от 0 до 255 для 8-битного изображения).

Алгоритм "растягивает" гистограмму так, чтобы все значения яркости использовались равномерно.

Формула (для дискретного случая): $s_k = T(r_k) = \sum_{j=0}^k (n_j / N)$, где r_k — исходная яркость пикселя, s_k — новая яркость, n_j — количество пикселей с яркостью j , N — общее число пикселей.

Результат: Темные участки становятся светлее. Светлые — темнее. (Столбики гистограмм уравниваются). Яркость как бы равномерно размазывается по изображению.

Применяется если: Детали изображения теряются в тенях или когда на изображении требуется максимальный контраст

Минусы: Если в тенях был скрытый шум, он станет заметнее. Искажает оттенки цветов, на портретах, например, может исказиться оттенок кожи.

4.9 Частотные методы в обработке изображений. Фурье-фильтрация.

Внизу пишу для понимания, если нужно что-то адекватное листай 2 страницы вниз

Короче тут можно что рассказать. Я не просто так написал, что ЛЮБУЮ функцию можно разложить на [спектр](#). Если в звуке у нас функция от амплитуды от t . То изображение мы можем представить как функцию яркости. Короче

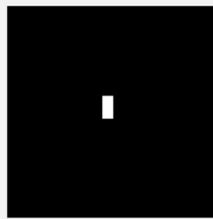
1. Главное в изображении это яркость пикселя, т.е. так альфа (0-255) составляющая. В общем вся эта математика это про работу с чб составляющей изображения.
2. Мы можем записать все альфы пикселей слева направо, сверху вниз в один линейный массив.
3. Т.е. у нас есть функция и мы можем засунуть её в ДПФ!!!
4. Ахуеть

Короче получается у нас функция (массив) яркостей. Там где яркости резко меняются — частота выше, где более плавно — частоты ниже. Отсюда идея что можно эти частоты менять а значит менять например резкость или ещё что-то.

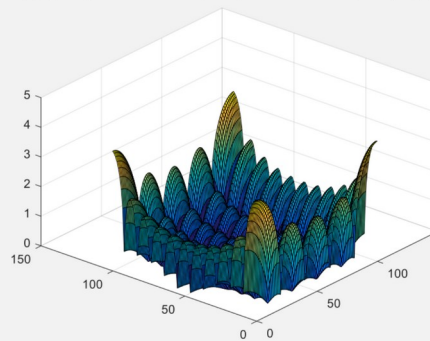
Если вы внимательно читали, то в спектре звука у нас были [фантомные части](#). Тут у нас как бы двумерный спектр и будет вот так

ЧАСТОТНЫЕ МЕТОДЫ ОБРАБОТКИ ИЗОБРАЖЕНИЙ. СВОЙСТВА 2D СПЕКТРА. Ч.2 ИЗ 4

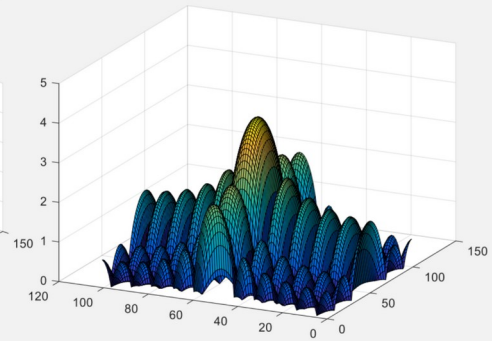
Для удобства работы с двумерным спектром, его часто центрируют.



Исходное изображение



Фурье спектр (нецентрированный)



Фурье спектр (центрированный)

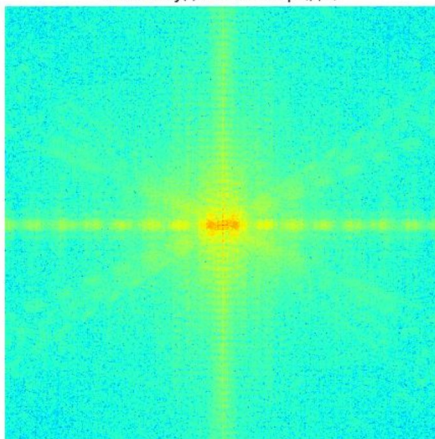
$X = \text{fft2}(x)$, $XS = \text{fftshift}(X)$ в MATLAB

Вместо центрирования спектра можно пиксели исходного изображения умножить на минус 1 в степени $(m+n)$, после чего обычный спектр автоматом будет центрированным

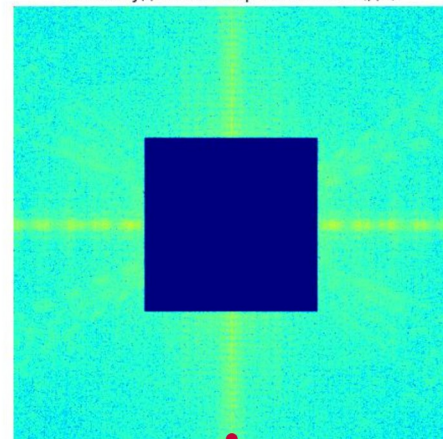
На левой картинке видно что у нас 4 одинаковых конца. Так вот их разворачивают к центру и получают вот такую башенку справа.

Спектрограммы выглядят вот так

Амплитудный спектр (дБ)



Амплитудный спектр после ФВЧ (дБ)



Как это понимать? Смотрим на левую картинку

Оно всегда симметрично относительно центра по кругу. Смотрим в центр. Там низкие частоты, ближе к краям квадрата там высокие частоты.

Цвета это как бы значение на сколько эти частоты влияют на картинку. Синее – 0, Красное ну много.

Справа Жаринов вырезал квадрат, т.е. занулил низкие частоты, т.е. сделал фильтр высоких частот. По идее должен был быть не квадрат а круг (это правильнее!!!! просто ему на лекции было лень делать круг). Если есть только высокие частоты, то в картинке остаются только переходы АКА границы

Исходное изображение

А Б В Г Д
Е Ж З И К
Л М Н О П
Р С Т У Ф
Х Ц Ч Ш

Фильтр высоких частот

А Б В Г Д
Е Ж З И К
Л М Н О П
Р С Т У Ф
Х Ц Ч Ш

ОПА

Ну и слайды

ФИЛЬТРАЦИЯ ИЗОБРАЖЕНИЙ. ОБЩИЕ ПОЛОЖЕНИЯ

Линейная фильтрация сигналов и изображений может осуществляться как в пространственной (т.е. непосредственно по пикселям), так и в частотной области. Полная аналогия с методами фильтрации аудиосигналов (во временной области и в спектральном пространстве).

- Низкочастотным составляющим соответствует основное содержание изображения – фон и крупноразмерные объекты,
- Высокочастотные составляющие – мелкоразмерные объекты, мелкие детали крупных форм и шумовые компоненты (зернистость).

Традиционно для перехода между пространственным и частотным пространствами используются методы, основанные на преобразовании Фурье. В последние годы все большее применение находят также методы, основанные на вейвлет-преобразовании.

При помощи фильтрации разными фильтрами решаются следующие задачи:

- **ФНЧ**: подавление случайных помех (“снега”), создание эффекта “размытия”,
- **ФВЧ**: выделение краев объектов в изображении, подъем ВЧ улучшает чёткость,
- **ПФ**: получение разных эффектов и создание шумоподобных фоновых изображений,
- **РФ**: удаление из изображений периодических помех (полосы, и т.п.),
- Более сложные фильтры используются восстановления изображений, искаженных в результате нарушения фокусировки и “размазывания”: при съемке с подвижного носителя.

к.т.н., доц. Жаринов О.О., ФГАОУВО “Санкт-Петербургский государственный университет аэрокосмического приборостроения” (ГУАП). Кафедра прикладной информатики (№41)
Дисциплина “Мультимедиа-технологии”. Лекция 3. 2025 г.

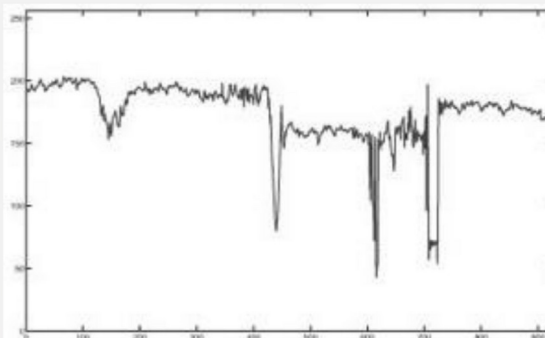
32

ЧАСТОТНЫЕ МЕТОДЫ ОБРАБОТКИ ИЗОБРАЖЕНИЙ. ОБЩАЯ КОНЦЕПЦИЯ. Ч.2 ИЗ 3

Собственно, когда изображение нужно передать на расстояние, оно и передается по строкам, вытягиваясь в длиннющий одномерный сигнал

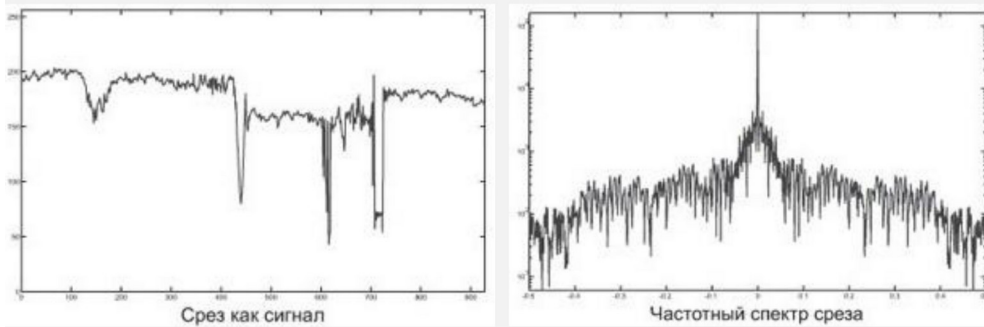


Изображение и срез



Срез как сигнал

ЧАСТОТНЫЕ МЕТОДЫ ОБРАБОТКИ ИЗОБРАЖЕНИЙ. ОБЩАЯ КОНЦЕПЦИЯ. Ч.3 ИЗ 3



Ну а раз есть сигнал, значит у него есть спектр. Времени, правда в таком сигнале нет – есть пространство. Вместо времени - 1 у.е., никаких Герц тоже нет, а есть простр. F с размерностью 1/1у.е.

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-j2\pi \frac{nk}{N}} \quad \text{– это ДПФ} \quad x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \cdot e^{j2\pi \frac{nk}{N}} \quad \text{– это ОДПФ}$$

35

Только на практике там немного подругому там формула которая ниже, а спектр который квадрат на прошлом листе

ЧАСТОТНЫЕ МЕТОДЫ ОБРАБОТКИ ИЗОБРАЖЕНИЙ. ДВУМЕРНОЕ ДПФ

$$X_{u,v} = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} x_{m,n} \cdot e^{-j2\pi \left(\frac{um}{M} + \frac{vn}{N} \right)} \quad \text{– это двумерное ДПФ}$$

$$x_{m,n} = \frac{1}{NM} \sum_{v=0}^{N-1} \sum_{u=0}^{M-1} X_{u,v} \cdot e^{j2\pi \left(\frac{um}{M} + \frac{vn}{N} \right)} \quad \text{– это двумерное ОДПФ}$$

4.10 Методы линейной пространственной фильтрации изображений. Улучшение четкости.

Определение: Это метод обработки изображений, где значение каждого пикселя вычисляется как линейная комбинация его соседей с помощью матрицы (ядра свертки).

Представьте, что вы размазываете детали изображения или наводите на них резкость, путём умножения пикселей на небольшую матрицу (например, 3×3) и суммируя результаты.

Основные шаги:

1. Выбор ядра (фильтра) — матрицы коэффициентов (например, для размытия или повышения резкости).
2. Свёртка — применение ядра к каждому пикселю изображения.
3. Нормализация (если нужно) — чтобы избежать переполнения яркости.

Общая формула свёртки для пикселя при линейной пространственной фильтрации:

$$I'(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k K(i, j) \cdot I(x + i, y + j)$$

K — ядро фильтра, k — его радиус (например, 1 для матрицы 3×3).

Методы:

1. Фильтр повышения резкости

Ядро (пример): $[[0, -1, 0], [-1, 5, -1], [0, -1, 0]]$

Повышает резкость центрального пикселя за счёт вычитания соседних. Края становятся чётче, но может появиться шум.

2. Лапласиан (усиление границ)

Ядро (пример): $[[0, 1, 0], [1, -4, 1], [0, 1, 0]]$

Выделяет границы (резкие перепады яркости)

Часто используется вместе с исходным изображением:

$I_{sharp} = I + c * Laplacian(I)$, I - Исходное изображение, c - коэффициент усиления (обычно 0.2–0.5)

4.11 Методы линейной пространственной фильтрации изображений. Сглаживание помех.

Сглаживание помех - процесс уменьшения шумов (случайных искажений) на изображении с сохранением полезных деталей.

Формула свёртки (4.10) тут такая же, т.к. это формула линейной пространственной фильтрации в общем.

$$I'(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k K(i, j) \cdot I(x + i, y + j)$$

Основной целью сглаживания является уменьшение шума, а также сглаживание резких переходов.

Фильтры для сглаживания:

1. Усредняющий фильтр (Размытие по Гауссу)

Ядро 3×3 : $(1/9) [[1, 1, 1], [1, 1, 1], [1, 1, 1]]$

Суть: Каждый пиксель заменяется средним значением соседей.

Коэффициент перед матрицей нормализует яркость: сумма весов матрицы равна девяти, и без нормализации яркость изображения будет завышена в 9 раз, поэтому на соответствующую дробь и умножаем

Гауссово размытие (более мягкое): $(1/16) [[1, 2, 1], [2, 4, 2], [1, 2, 1]]$ - это оптимальная целочисленная аппроксимация Гауссова размытия

При такой конфигурации весов, близкие пиксели сильнее влияют на наш целевой пиксель.

2. Фильтр Гаусса

Использует нормальное распределение для весов:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

3. Биномиальный фильтр

Аппроксимация Гаусса с целочисленными с целочисленными коэффициентами

Такой метод грубее фильтра Гаусса, однако он быстрее и во многих задачах его результат достаточен.

Фильтр	Эффект	Плюсы	Минусы
Усредняющий	Сильное размытие, подавление шума	Простота реализации	Потеря чёткости
Гауссов	Мягкое размытие	Сохраняет контуры лучше	Вычислительно сложнее
Биномиальный	Компромисс между скоростью и качеством	Быстрее сохраняет контуры	Менее точный, чем Гауссов

4.12 Методы нелинейной пространственной фильтрации изображений. Медианный фильтр.

Медианный фильтр - значения отсчетов внутри окна фильтра сортируются в порядке возрастания (убывания); и значение, находящееся в середине упорядоченного списка, поступает на выход фильтра. В случае чётного числа отсчетов в окне выходное значение фильтра равно среднему значению двух отсчетов в середине упорядоченного списка. Окно перемещается вдоль фильтруемого сигнала, и вычисления повторяются.

Медианная фильтрация — эффективная процедура обработки сигналов, подверженных воздействию импульсных помех.

Сущность работы медианного фильтра заключается в том, что каждый элемент выходного изображения равен медиане данных исходного изображения, находящихся в апертуре. Полезным свойством медианных фильтров является то, что они не искажают ступенчатые и линейно-

изменяющиеся перепады яркости, а также обеспечивают полное подавление точечных (импульсных) помех.

Отсчеты изображения, оказавшиеся в пределах окна, образуют рабочую выборку текущего шага. Обозначим рабочую выборку в виде одномерного массива $Y = \{y_1, y_2, \dots, y_n\}$; число его элементов равняется размеру окна, а их расположение произвольно. Обычно применяют окна с нечетным числом точек n (это автоматически обеспечивается при центральной симметрии апертury и при вхождении самой центральной точки в ее состав). Если упорядочить последовательность $\{y_i, i = \overline{1, n}\}$ по возрастанию, то ее *медианой* (средним значением) будет тот элемент выборки, который занимает центральное положение в этой упорядоченной последовательности. Полученное таким образом число и является продуктом фильтрации для текущей точки кадра. Понятно, что результат такой обработки в самом деле не зависит от того, в какой последовательности представлены элементы изображения в рабочей выборке. Формальное обозначение описанной процедуры выглядит следующим образом:

$$x^* = med(y_1, y_2, \dots, y_n)$$

Предположим, что апертюра фильтра находится вблизи границы, разделяющей светлый и темный участки изображения, при этом ее центр располагается в области темного участка. Тогда, вероятнее всего, рабочая выборка будет содержать большее количество элементов с малыми значениями яркости, и, следовательно, медиана будет находиться среди тех элементов рабочей выборки, которые соответствуют этой области изображения. Ситуация меняется на противоположную, если центр апертury смещен в область более высокой яркости.

Рассмотрим пример. Предположим, что выборка имеет вид: $\{136, 110, 99, 45, 250, 55, 158, 104, 75\}$, а элемент 250, расположенный в ее центре, соответствует текущей точке фильтрации. Упорядоченная по возрастанию выборка имеет при этом вид: $\{45, 55, 75, 99, 104, 110, 136, 158, 250\}$, следовательно, получаем . Как видим, влияние соседей на результат фильтрации в текущей точке привело к игнорированию импульсного выброса яркости, что следует рассматривать как эффект фильтрации. Если импульсная помеха не является точечной, а покрывает некоторую локальную область, то она также может быть подавлена. Это произойдет, если размер этой локальной области будет меньше, чем половина размера окна. Поэтому для подавления импульсных помех, поражающих локальные участки изображения, следует увеличивать размеры окна.

4.13 Методы нелинейной пространственной фильтрации изображений. Выделение контурных линий.

Процесс нелинейной фильтрации изображений во многом схож с линейной фильтрацией. Основное отличие заключается в том, что выход нелинейного фильтра формируется нелинейным преобразованием данных исходного изображения.

В нелинейных алгоритмах используют нелинейные операторы дискретного дифференцирования. В *фильтре Робертса* используется перемещающаяся по изображению маска 2×2 :

$$\begin{vmatrix} f_{i,j} & f_{i,j+1} \\ f_{i+1,j} & f_{i+1,j+1} \end{vmatrix}.$$

дифференцирования производится с помощью одного из выражений

$$g_{i,j} = \left((f_{i,j} - f_{i+1,j+1})^2 + (f_{i,j+1} - f_{i+1,j})^2 \right)^{1/2},$$

$$g_{i,j} = |f_{i,j} - f_{i+1,j+1}| + |f_{i,j+1} - f_{i+1,j}|,$$

В *фильтре Собела* используется окно 3×3 :

$$\begin{vmatrix} f_{i-1,j-1} & f_{i-1,j} & f_{i-1,j+1} \\ f_{i,j-1} & f_{i,j} & f_{i,j+1} \\ f_{i+1,j-1} & f_{i+1,j} & f_{i+1,j+1} \end{vmatrix}.$$

Центральному (i,j) -му пикселю вместо $f_{i,j}$ присваивается значение яркости

$$g_{i,j} = (X^2 + Y^2)^{1/2} \text{ либо } g_{i,j} = |X| + |Y|, \text{ где}$$

$$X = (f_{(i-1),(j-1)} + 2f_{(i-1),j} + f_{(i-1),j+1}) - (f_{i+1,(j-1)} + 2f_{i+1,j} + f_{i+1,j+1});$$

$$Y = (f_{(i-1),(j-1)} + 2f_{i,(j-1)} - f_{i+1,(j-1)}) - (f_{(i-1),j+1} + 2f_{i,j+1} + f_{i+1,j+1}).$$

Если выходу фильтра присваивать не значение медианы данных, а значение любой r -ой ($r = 1, \dots, R$, R - площадь окна фильтра в пикселах) порядковой статистики, то можно получить $R - 1$ так называемых процентильных фильтров. Одним из важных свойств процентильных фильтров является либо уменьшение размеров объектов, находящихся на изображении (это достигается при $r > \lfloor (R+1)/2 \rfloor$), либо их увеличение ($r < \lfloor (R+1)/2 \rfloor$).

Фильтр Собеля - дискретный дифференциальный оператор, вычисляющий приближённое значение градиента яркости изображения. Результатом применения оператора Собеля в каждой точке изображения является либо вектор градиента яркости в этой точке, либо его норма. Оператор вычисляет градиент яркости изображения в каждой точке. Так находится направление наибольшего увеличения яркости и величина её изменения в этом направлении. Результат показывает, насколько «резко» или «плавно» меняется яркость изображения в каждой точке, а значит, вероятность нахождения точки на грани, а также ориентацию границы.

Дифференциальные и масочные фильтры основаны на различных аппроксимациях величины градиента яркости в точках изображения и служат для выделения границ изображений объектов. В этой связи они близки к линейным высокочастотным фильтрам.

К использованию дифференциальных и высокочастотных фильтров следует подходить осторожно, т.к. они подчеркивают как полезные, так и шумовые перепады яркости. При очень слабом шуме в исходном изображении общее снижение отношения сигнал/шум в отфильтрованном изображении визуально замечается слабо. Однако, уже при средней зашумленности исходного кадра в результирующем изображении полезные перепады маскируются ложными, причем в такой степени, что даже такой мощный распознающий инструмент как человеческий глаз проще решает эту задачу по исходному изображению.

4.14 Восстановление смазанных изображений.

** (не нашла - поэтому подвязываю ЛР 7 от Жаринова где восстанавливали смазанные изображения)

Для того, чтобы снимок получился достаточно четким камера должна быть неподвижна. Если в момент снимка камера хотя бы немного меняет свое положение, световой поток, отраженный от объекта съемки смещается относительно оптической оси, соответственно смещается и проекция изображения на светочувствительную матрицу, и в результате изображение получается смазанным.

Для восстановления четкости фрагментов смазанных изображений можно применять в том числе методы линейной пространственной фильтрации.

Математически принцип линейной пространственной фильтрации изображения, представленного матрицей $\{I[m,n]\}$, описывается формулой:

$$J[m,n] = \sum_{p=-P}^P \sum_{q=-Q}^Q H[p,q] \cdot I[m+p,n+q],$$

где массив коэффициентов $\{H[p,q]\}_{p=-P,\dots,P; q=-Q,\dots,Q}$ называется маской или ядром (kernel) фильтра и полностью определяет его свойства.

Основная идея улучшения четкости изображения заключается в добавлении к исходному размытому изображению дополнительного изображения, в котором подчеркнуты компоненты границ объектов, с некоторым весовым коэффициентом. Процедура восстановления изображения из искаженного смазом называется деконволюцией (deconvolution).

Некоторые методы деконволюции требуют для своей работы задания искажающей маски гипотетического пространственного фильтра, исказившего исходное идеальное изображение, другие – не требуют (заложенные в них алгоритмы называются методами слепой деконволюции – blind deconvolution).

Иногда идентифицировать величину и направления смаза можно, рассматривая увеличенный фрагмент смазанного изображения. При знании величины и направления смаза в качестве методов деконволюции можно использовать метод *Винера*, метод *Люси-Ричардсона* и метод *Тихонова*.

Метод Винера рассматривает изображение и шум как случайные процессы и находит такую оценку $\hat{f}$ для неискаженного изображения f , чтобы среднеквадратическое отклонение этих величин было минимальным. Минимум этого отклонения достигается на функции в частотной области:

$$\hat{F}(u, v) = \left(\frac{1}{H(u, v)} \frac{|H(u, v)|^2}{|H(u, v)|^2 + S_\eta(u, v)/S_f(u, v)} \right) G(u, v)$$

Где $H(u, v)$, $F(u, v)$ – Фурье-образы соответствующих функций.

Метод Люси-Ричардсона является нелинейным и итерационным. Основная идея состоит в использовании метода максимального правдоподобия для которого предполагается, что изображение подчиняется распределению Пуассона. Формулы для вычисления в пространственной области:

$$\hat{f}_{k+1}(x, y) = \hat{f}_k(x, y) \left[h(-x, -y) * \frac{g(x, y)}{h(x, y) * \hat{f}_k(x, y)} \right]$$

Метод Тихонова. Его идея заключается в формулировке задачи в матричном виде с дальнейшим решением соответствующей задачи оптимизации. Это решение записывается в виде:

$$\hat{F}(u, v) = \left(\frac{H^*(u, v)}{|H(u, v)|^2 + \gamma |P(u, v)|^2} \right) G(u, v)$$

4.15 Методы линейной пространственной фильтрации изображений. Спецэффекты: тиснение, соляризация.

Линейная пространственная фильтрация не только устраняет шумы и размывает изображения, но и создаёт художественные эффекты, такие как тиснение и соляризация.

1. Тиснение

Создаёт иллюзию объёма, как будто изображение выдавлено на металлическом холсте (монета, к примеру) или бумаге.

Работает он так:

Подчёркивает перепады яркости, имитируя освещение под углом.

Использует разностные ядра, выделяющие границы.

Пример ядра с направлением света "сверху-слева":

$[-1, -1, 0], [-1, 0, 1], [0, 1, 1]$

Как применять:

Применяем свёртку с ядром.

Добавляем 127 (для сдвига значений в серую зону):

$$g(x, y) = \text{clip} \left(\sum w(i, j) \cdot f(x + i, y + j) + 127, 0, 255 \right)$$

Если обрабатывать каждый сигнал (RGB) отдельно, то можно получить эффект цветного тиснения! (Применяется в дизайне, играх и соц.сетях)

2. Соляризация — это нелинейный эффект, который инвертирует тона в определённом диапазоне яркости, создавая "фантастическое" или "сюрреалистическое" искажение.

Основная идея: инвертировать пиксели, превышающие порог яркости

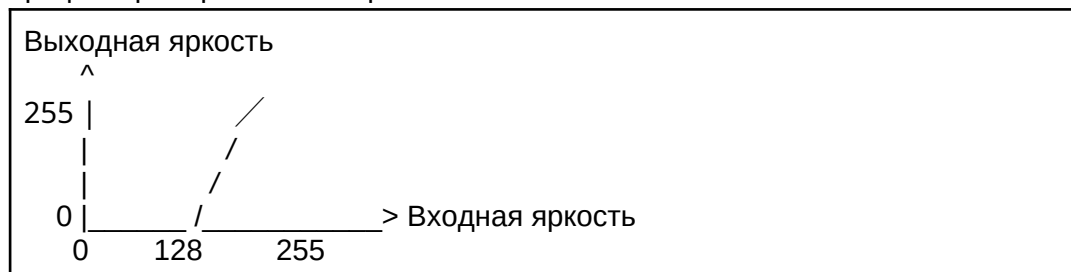
Светлые участки (близкие к белому) → становятся тёмными.

Тёмные участки (близкие к чёрному) → сохраняются.

Средние тона (серые) → превращаются в максимально белые.

Это соответствует классическому аналоговому эффекту, возникающему при частичном обращении серебра в фотоплёнке. Применяется для стилизации фотографий под "ретро"

График преобразования яркости



Участки с яркостью > 128 инвертируются.

Участки < 128 остаются без изменений.

Реализация соляризации:

1. Аналоговый метод (пороговый метод)

Формула:

$$g(x, y) = \begin{cases} 255 - f(x, y), & \text{если } f(x, y) > T \\ f(x, y), & \text{иначе} \end{cases}$$

где T - порог, обычно принимает значение 128

2. Плавная соляризация (синусоидальное преобразования)

Используется для более мягкого перехода:

$$g(x, y) = 255 \cdot \left| \sin \left(\frac{\pi \cdot f(x, y)}{255} \right) \right|$$

Линейная аппроксимация соляризации

1. Применение фильтра Лапласа (выделение границ + инверсия):
[[0, -1, 0], [-1, 4, -1], [0, -1, 0]]
2. Наложение на исходное изображение:

$$g(x, y) = \text{clip}(f(x, y) - k \cdot L(x, y), 0, 255),$$

где $L(x, y)$ - результат Лапласиана, k - коэффициент (например, 0.5)

Contents

1	Статистическая обработка информации	1
1.1	1. Разделы статистической обработки информации: теория оценок, теория проверки статистических гипотез	1

1 Статистическая обработка информации

1.1 1. Разделы статистической обработки информации: теория оценок, теория проверки статистических гипотез

Статистическая обработка информации начинается с формальной вероятностной модели наблюдений. Пусть задано вероятностное пространство $(\Omega, \mathcal{F}, P)$, а результаты эксперимента описываются случайными величинами $X_1, X_2, \dots, X_n$. Выборкой объёма n обычно называют набор независимых одинаково распределённых случайных величин $X_1, X_2, \dots, X_n$, имеющих ту же функцию распределения $F(x)$, что и изучаемая генеральная совокупность. Реализованная выборка $x_1, x_2, \dots, x_n$ — это уже конкретные числа, полученные в опыте. Статистикой называется любая измеримая функция от выборки $T = T(X_1, \dots, X_n)$, не содержащая неизвестных параметров. Например, выборочное среднее $\bar{X}$, выборочная дисперсия S^2 , максимум $X_{(n)}$, эмпирическая функция распределения $F_n(x)$ — всё это статистики.

Главные разделы статистической обработки информации — теория статистических оценок и теория проверки статистических гипотез. Теория оценок решает задачу восстановления неизвестных характеристик генеральной совокупности по выборке. Если распределение зависит от параметра θ , то оценкой параметра θ называется статистика $\hat{\theta} = \hat{\theta}(X_1, \dots, X_n)$, используемая вместо неизвестного θ . Оценивание бывает точечным и интервальным. При точечном оценивании результатом является одно число, например $\bar{X} = (1/n) \sum_{i=1}^n X_i$ как оценка математического ожидания μ . При интервальном оценивании строится случайный интервал, который с заданной доверительной вероятностью накрывает неизвестный параметр. Например, при нормальной модели с неизвестной дисперсией доверительный интервал для μ имеет вид

$$\bar{X} \pm t_{1-\alpha/2, n-1} \frac{S}{\sqrt{n}}$$

Теория оценок изучает не только способы построения оценок, но и их качество. Основные свойства оценок: несмещённость, состоятельность, эффективность, асимптотическая нормальность, устойчивость к выбросам. Несмещённость означает отсутствие систематической ошибки: $E\hat{\theta} = \theta$. Состоятельность означает сходимость оценки к истинному параметру при $n \rightarrow \infty$. Эффективность означает минимальный разброс или минимальный риск среди допустимых оценок. На практике также важны методы построения оценок: метод моментов, метод максимального правдоподобия, метод наименьших квадратов, робастные методы. Метод моментов приравнивает выборочные моменты к теоретическим, а метод максимального правдоподобия выбирает такие параметры, при которых наблюденная выборка имеет наибольшую вероятность или плотность.

Теория проверки статистических гипотез решает другую задачу: не просто оценить параметр, а принять решение о правдоподобности некоторого утверждения. Статистической гипотезой называется предположение о виде распределения или о значениях его параметров. Нулевая гипотеза H_0 — проверяемое основное

предположение, альтернативная гипотеза H_1 — конкурирующее предположение. Например, $H_0: \mu = 10$, $H_1: \mu \neq 10$. Критерий проверки гипотезы — правило, которое по выборке решает, отвергать H_0 или нет. Для критерия задают статистику критерия, критическую область и уровень значимости α . Ошибка первого рода — отвергнуть верную H_0 , её вероятность равна α . Ошибка второго рода — не отвергнуть ложную H_0 , её вероятность обозначают β . Мощность критерия равна $1 - \beta$.

Конкретные критерии зависят от задачи. t -критерий Стьюдента применяется для проверки гипотез о математическом ожидании нормальной совокупности при неизвестной дисперсии. χ^2 -критерий согласия используется для проверки того, согласуются ли наблюдаемые частоты с предполагаемым распределением. F -критерий Фишера применяется для сравнения дисперсий двух нормальных совокупностей. Критерий Колмогорова основан на максимальном отклонении эмпирической функции распределения от теоретической. В задачах однородности проверяют, можно ли считать несколько выборок полученными из одного распределения.

Интервальное оценивание тесно связано с проверкой гипотез. Доверительный интервал можно понимать как множество значений параметра, которые не отвергаются соответствующим двусторонним критерием на уровне значимости α . Например, если 95%-й доверительный интервал для μ равен $[8,4; 11,6]$, то гипотеза $\mu = 10$ не отвергается, а гипотеза $\mu = 12$ отвергается на уровне значимости около 0,05. Эта связь важна для письменного ответа: оценивание и проверка гипотез не изолированы, а являются двумя способами статистического вывода.

Числовой пример точечного оценивания. Пусть выборка времени обслуживания равна 8, 10, 12, 10 минут. Тогда $\bar{X} = (8 + 10 + 12 + 10)/4 = 10$. Это точечная оценка математического ожидания времени обслуживания. Если нужно оценить дисперсию, сначала считаем отклонения: $-2, 0, 2, 0$; квадраты: $4, 0, 4, 0$. Неисправленная дисперсия равна $S_n^2 = 8/4 = 2$, а исправленная $S^2 = 8/3 \approx 2,67$.

Числовой пример проверки гипотезы. Пусть проверяется $H_0: \mu = 10$ по выборке $n = 25$, $\bar{X} = 11$, $S = 2$. Статистика t равна $t = (\bar{X} - \mu_0)/(S/\sqrt{n}) = (11 - 10)/(2/5) = 2,5$. При уровне $\alpha = 0,05$ и 24 степенях свободы критическое значение около 2,064 для двустороннего критерия. Так как $2,5 > 2,064$, H_0 отвергается. Это не означает, что μ точно не равно 10, но означает, что наблюдаемое отклонение слишком велико для принятого уровня значимости.

Числовой пример связи доверительного интервала и гипотезы. Для той же выборки 95%-й интервал равен $11 \pm 2,064 \cdot 2/5 = 11 \pm 0,826$, то есть $[10,174; 11,826]$. Значение $\mu_0 = 10$ не входит в интервал, поэтому гипотеза $\mu = 10$ отвергается. Значение $\mu_0 = 11,5$ входит в интервал, поэтому такая гипотеза не отвергается. Так на числах видно, что доверительный интервал фактически перечисляет допустимые значения параметра.

2. Смещённость оценки; примеры смещённых и несмещённых оценок

Смещённость — свойство не отдельного вычисленного числа, а метода оценивания как случайной величины. Если в одной конкретной выборке получено $\theta = 4,8$ при истинном $\theta = 5$, то по одному опыту нельзя сказать, смещённа оценка или нет. Нужно рассматривать математическое ожидание оценки при многократном повторении эксперимента при одном и том же истинном параметре θ .

Пусть $\hat{\theta} = \hat{\theta}(X_1, \dots, X_n)$ — оценка параметра θ . Смещением оценки называется величина

$$b(\hat{\theta}) = E(\hat{\theta}) - \theta$$

Оценка называется несмещённой, если для всех допустимых θ выполняется

$$E(\hat{\theta}) = \theta$$

Если $E\theta \neq \theta$, оценка называется смещённой. Если $b(\theta) \rightarrow 0$ при $n \rightarrow \infty$, оценка называется асимптотически несмещённой. Несмещённость полезна тем, что исключает систематическую ошибку: при большом числе повторений эксперимента среднее значение оценок совпадает с истинным параметром. Но несмещённость сама по себе не гарантирует малой ошибки в каждой выборке, потому что оценка может иметь большую дисперсию.

Простейший пример несмещённой оценки — выборочное среднее. Пусть $E X_i = \mu$. Тогда

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

$$E(\bar{X}) = E\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n} \sum_{i=1}^n E(X_i) = \mu$$

Следовательно, $\bar{X}$ — несмещённая оценка μ . Для схемы Бернулли, где X_i принимает значения 1 и 0, а $P(X_i = 1) = p$, несмещённой оценкой вероятности p является относительная частота

$$\hat{p} = \frac{1}{n} \sum_{i=1}^n X_i$$

поскольку $E \hat{p} = p$.

Классический пример смещённой оценки — выборочная дисперсия с делением на n :

$$S_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$$

Если истинная дисперсия равна σ^2 , то

$$E(S_n^2) = \frac{n-1}{n} \sigma^2$$

Значит, S_n^2 систематически занижает σ^2 . Смещение равно

$$b(S_n^2) = E(S_n^2) - \sigma^2 = -\frac{\sigma^2}{n}$$

Но это смещение стремится к нулю, поэтому S_n^2 является смещённой, но асимптотически несмещённой оценкой. Исправленная выборочная дисперсия

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$$

является несмещённой: $E S^2 = \sigma^2$.

Числовой пример для дисперсии. Пусть выборка равна 2, 4, 6. Тогда $\bar{X} = 4$. Отклонения: -2, 0, 2. Квадраты: 4, 0, 4. Сумма квадратов равна 8. Неисправленная дисперсия $S_n^2 = 8/3 \approx 2,67$. Исправленная дисперсия $S^2 = 8/(3-1) = 4$. Если эта выборка взята из совокупности с истинной дисперсией около 4, то деление на n даёт систематически заниженный результат, а деление на $n-1$ устраняет смещение в среднем.

Числовой пример несмещённой оценки вероятности. Пусть в 10 испытаниях успех

произошёл 7 раз. Тогда $\hat{p} = 7/10 = 0,7$. В одной серии это может отличаться от истинного p , например $p = 0,6$, но метод $\hat{p} = m/n$ несмещён, потому что при многократном повторении серий средняя относительная частота будет равна p .

Числовой пример смещённой оценки математического ожидания. Пусть оценивать μ по формуле $\theta = \bar{X} + 1$. Если истинное $\mu = 5$, то $E\theta = E\bar{X} + 1 = 6$. Смещение равно 1. Даже при очень большой выборке эта оценка будет стремиться к 6, а не к 5, поэтому смещение не исчезает.

Несмещённость не всегда является единственным критерием качества. В некоторых задачах допустима небольшая смещённость, если она резко уменьшает дисперсию и среднеквадратическую ошибку. Для квадратичной функции потерь

$$MSE(\hat{\theta}) = E(\hat{\theta} - \theta)^2 = \text{Var}(\hat{\theta}) + b^2(\hat{\theta})$$

Эта формула показывает компромисс: можно иметь несмещённую оценку с большой дисперсией или смещённую оценку с малым общим риском. В регуляризованных методах статистики и машинного обучения преднамеренно вводят смещение ради устойчивости, особенно при плохой обусловленности данных или мультиколлинеарности. В классической экзаменационной постановке, однако, несмещённость ценится потому, что исключает систематическое завышение или занижение параметра.

Среди несмещённых оценок выделяют лучшую несмещённую оценку, то есть оценку с минимальной дисперсией в заданном классе. Здесь возникает связь со следующими свойствами: эффективность и граница Крамера-Рао. Если несмещённая оценка имеет минимально возможную дисперсию, она является не просто несмещённой, а оптимальной среди несмещённых оценок по разбросу.

3. Состоятельность оценки; примеры состоятельных и несостоятельных оценок
Состоятельность — асимптотическое свойство оценки, описывающее её поведение при увеличении объёма выборки. Пусть для каждого n построена оценка θ_n параметра θ . Оценка называется состоятельной, если θ_n сходится к θ по вероятности:

$$\hat{\theta}_n \xrightarrow{P} \theta, \quad n \rightarrow \infty$$

то есть для любого $\varepsilon > 0$ выполняется

$$P\{|\hat{\theta}_n - \theta| > \varepsilon\} \rightarrow 0, \quad n \rightarrow \infty$$

Это означает, что вероятность заметной ошибки стремится к нулю. Сходимость по вероятности слабее, чем сходимость почти наверное: если $\theta_n \rightarrow \theta$ почти наверное, то $\theta_n \rightarrow \theta$ по вероятности, но обратное вообще говоря неверно. Для экзамена важно подчеркнуть: состоятельность не требует точности при малом n , она требует правильного предельного поведения.

Основной источник состоятельности многих оценок — закон больших чисел. Если $X_1, \dots, X_n$ независимы и одинаково распределены, $E X_i = \mu$ и дисперсия конечна, то

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{P} \mu$$

Следовательно, выборочное среднее является состоятельной оценкой математического ожидания. Через неравенство Чебышёва это видно напрямую. Если $\text{Var}(X_i) = \sigma^2$, то $\text{Var}(\bar{X}) = \sigma^2/n$, и

$$P\{|\bar{X} - \mu| \geq \varepsilon\} \leq \frac{\sigma^2}{n\varepsilon^2}$$

Правая часть стремится к нулю, значит $\bar{X}$ состоятельно.

Относительная частота также состоятельна как оценка вероятности. Если событие A имеет вероятность p , а m — число появлений A в n независимых испытаниях, то

$$\hat{p}_n = \frac{m}{n} \xrightarrow{P} p$$

Это частный случай закона больших чисел для индикаторов события.

Смещённость не исключает состоятельности. Неисправленная дисперсия

$$S_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$$

смещена, поскольку $E S_n^2 = ((n - 1)/n)\sigma^2$, но смещение равно $-\sigma^2/n$ и стремится к нулю. При стандартных условиях $S_n^2 \rightarrow \sigma^2$ по вероятности, то есть оценка состоятельна. Исправленная дисперсия S^2 также состоятельна. Этот пример принципиален: оценка может быть смещённой при конечном n , но правильной в пределе.

Числовой пример через неравенство Чебышёва. Пусть $\sigma^2 = 16$, $n = 100$ и нужно оценить μ по $\bar{X}$. Вероятность ошибки не меньше 2 ограничивается сверху:

$$P\{|\bar{X} - \mu| \geq 2\} \leq \frac{16}{100 \cdot 4} = 0,04$$

Если $n = 400$, то

$$P\{|\bar{X} - \mu| \geq 2\} \leq \frac{16}{400 \cdot 4} = 0,01$$

Рост n уменьшает верхнюю границу вероятности большой ошибки. Это и есть числовая иллюстрация состоятельности.

Числовой пример смещённой, но состоятельной оценки. Пусть истинная дисперсия $\sigma^2 = 9$. Для $n = 10$ математическое ожидание S_n^2 равно $(9/10) \cdot 9 = 8,1$, смещение $-0,9$. Для $n = 100$ оно равно $(99/100) \cdot 9 = 8,91$, смещение $-0,09$. Для $n = 1000$ оно равно $8,991$, смещение $-0,009$. Смещение остаётся при конечном n , но исчезает при $n \rightarrow \infty$.

Пример несостоятельной оценки — оценка математического ожидания по одному первому наблюдению:

$$\hat{\theta}_n = X_1$$

Даже если $E X_1 = \mu$, распределение X_1 не сужается при росте n , потому что новые наблюдения вообще не используются. Например, если X_1 имеет нормальное распределение $N(\mu, 4)$, то $P(|X_1 - \mu| > 2) \approx 0,317$ независимо от n . Вероятность ошибки не стремится к нулю, значит оценка несостоятельна.

Другой пример несостоятельности — оценка с постоянным смещением:

$$\hat{\theta}_n = \bar{X} + 1$$

Так как $\bar{X} \rightarrow \mu$, то $\theta_n \rightarrow \mu + 1$, а не к μ . При любом объёме выборки предельное значение неверно. Численно: если $\mu = 10$ и n очень велико, $\bar{X}$ будет близко к 10, но $\bar{X} + 1$ будет

близко к 11.

Состоятельность может нарушаться, если нарушены условия закона больших чисел. Например, если данные не являются одинаково распределёнными из-за тренда, то среднее может не оценивать постоянный параметр. Пусть $X_i = \mu + 0,01i + \varepsilon_i$. Тогда $\bar{X}$ содержит средний тренд примерно $0,01(n + 1)/2$ и при росте n уходит от μ . Если данные сильно зависимы или нестационарны, механическое применение выборочного среднего может дать несостоятельную процедуру. Поэтому в строгом ответе нужно говорить не только “среднее состоятельно”, но и “при независимости, одинаковом распределении и существовании соответствующих моментов”.

4. Эффективность оценки; функции штрафа и риска

Эффективность оценки характеризует, насколько точно оценка использует информацию выборки по сравнению с другими оценками того же параметра. Для несмещённых оценок естественная мера точности — дисперсия. Если θ_1 и θ_2 несмещённо оценивают θ и $\text{Var}(\theta_1) < \text{Var}(\theta_2)$, то θ_1 эффективнее θ_2 : она имеет меньший разброс вокруг истинного значения.

В классической параметрической теории центральную роль играет информация Фишера. Пусть плотность или вероятность наблюдения зависит от параметра θ и равна $f(x; \theta)$. Для одного наблюдения информация Фишера задаётся формулой

$$I(\theta) = E \left[\left(\frac{\partial}{\partial \theta} \ln f(X; \theta) \right)^2 \right]$$

Для независимой выборки объёма n информация обычно равна $nI(\theta)$. При регулярных условиях для любой несмещённой оценки $\hat{\theta}$ выполняется неравенство Крамера-Рао:

$$\text{Var}(\hat{\theta}) \geq \frac{1}{I_n(\theta)}$$

где $I_n(\theta)$ — информация Фишера во всей выборке. Если оценка достигает этой нижней границы, она называется эффективной в строгом смысле. Поэтому нужно различать два смысла: эффективность как сравнительная характеристика, когда одна оценка имеет меньшую дисперсию или риск, и эффективная оценка как оценка, достигающая границы Крамера-Рао.

Пример для нормального распределения. Пусть $X_i \sim N(\mu, \sigma^2)$, где σ^2 известно, а μ неизвестно. Информация Фишера по μ для одного наблюдения равна $I(\mu) = 1/\sigma^2$, для n наблюдений $I_n(\mu) = n/\sigma^2$. Поэтому граница Крамера-Рао:

$$\text{Var}(\hat{\mu}) \geq \frac{\sigma^2}{n}$$

Выборочное среднее $\bar{X}$ имеет $\text{Var}(\bar{X}) = \sigma^2/n$ и является несмещённой оценкой μ . Значит, оно достигает границы Крамера-Рао и является эффективной оценкой μ в нормальной модели с известной дисперсией.

Числовой пример. Пусть $\sigma^2 = 9$, $n = 25$. Тогда граница Крамера-Рао для несмещённой оценки μ равна $9/25 = 0,36$. У $\bar{X}$ дисперсия также 0,36. Оценка по одному наблюдению X_1 имеет дисперсию 9, то есть в 25 раз больше. Обе оценки несмещённые, но $\bar{X}$ гораздо эффективнее.

Для сравнения оценок вводят относительную эффективность. Если две несмещённые оценки имеют дисперсии $\text{Var}(\theta_1)$ и $\text{Var}(\theta_2)$, то эффективность θ_1 относительно θ_2 можно оценивать отношением $\text{Var}(\theta_2)/\text{Var}(\theta_1)$. Например, если $\text{Var}(\theta_1) = 0,25$, а $\text{Var}(\theta_2) = 1$, то первая оценка в 4 раза эффективнее второй по дисперсии.

При наличии смещения сравнения только по дисперсии недостаточно. Вводится функция штрафа, или функция потерь $L(\theta, \hat{\theta})$, где θ — истинный параметр, а $\hat{\theta}$ — принятое решение или значение оценки. Квадратичная функция потерь:

$$L(\theta, \hat{\theta}) = (\hat{\theta} - \theta)^2$$

Модульная функция потерь:

$$L(\theta, \hat{\theta}) = |\hat{\theta} - \theta|$$

Асимметричная функция потерь может задаваться так:

$$L(\theta, \hat{\theta}) = c_1(\theta - \hat{\theta}), \quad \text{если } \hat{\theta} < \theta$$

$$L(\theta, \hat{\theta}) = c_2(\hat{\theta} - \theta), \quad \text{если } \hat{\theta} \geq \theta$$

где $c_1 \neq c_2$. Такая функция нужна, когда недооценка и переоценка имеют разную цену. Например, при оценке запаса прочности недооценить риск опаснее, чем переоценить, поэтому штраф за $\hat{\theta} < \theta$ может быть больше.

Риск оценки — это математическое ожидание функции потерь:

$$R(\theta, \hat{\theta}) = E_{\theta} L(\theta, \hat{\theta})$$

Для квадратичной функции потерь риск равен среднеквадратической ошибке:

$$R(\theta, \hat{\theta}) = E_{\theta}(\hat{\theta} - \theta)^2 = \text{Var}(\hat{\theta}) + [E(\hat{\theta}) - \theta]^2$$

Эта формула показывает, почему смещённая оценка иногда может быть предпочтительнее несмещённой: если небольшое смещение резко уменьшает дисперсию, общий риск может снизиться.

Числовой пример риска. Оценка А несмещённая и имеет $\text{Var}(A) = 4$. Тогда $\text{MSE}(A) = 4$. Оценка В смещена на 1, но имеет $\text{Var}(B) = 1$. Тогда $\text{MSE}(B) = 1 + 1^2 = 2$. По квадратичному риску В лучше, несмотря на смещение. Если же смещение В равно 3, то $\text{MSE}(B) = 1 + 9 = 10$, и В хуже.

Числовой пример асимметричной потери. Пусть истинный спрос $\theta = 100$. Если заказать $\theta = 90$, недопоставка 10 единиц штрафует по 5 за единицу: потеря 50. Если заказать $\theta = 110$, избыток 10 единиц штрафует по 1 за единицу: потеря 10. При такой функции потерь рациональная оценка будет склонна завышать спрос, потому что недооценка опаснее переоценки. Это показывает, что выбор функции штрафа влияет на выбор оценки.

Асимптотическая эффективность описывает поведение оценок при $n \rightarrow \infty$. Оценка может не достигать точной границы при конечном n , но её асимптотическая дисперсия может совпадать с нижней возможной границей. Многие оценки максимального правдоподобия при регулярных условиях являются состоятельными, асимптотически нормальными и асимптотически эффективными. Это значит, что при больших выборках они используют информацию почти оптимально.

5. Смещённость симметричного распределения: выборочное среднее, выборочная медиана, усечённое среднее

Распределение называется симметричным относительно точки a , если отклонения вправо и влево от a имеют одинаковый закон. Для непрерывного распределения с плотностью это можно записать так:

$$f(a+x) = f(a-x) \quad \text{для всех } x$$

Через функцию распределения симметрия записывается так:

$$F(a+x) = 1 - F(a-x)$$

если функция распределения непрерывна в соответствующих точках. При симметрии центр a одновременно является медианой. Если существует математическое ожидание, то $E X = a$. Поэтому для симметричных распределений естественная задача — оценивать центр a .

Выборочное среднее равно

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

Если $E X_i = a$, то $E \bar{X} = a$. Следовательно, выборочное среднее — несмещённая оценка центра симметрии при существовании математического ожидания. Его дисперсия равна

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n}$$

Для нормального распределения без выбросов $\bar{X}$ является наиболее эффективной оценкой центра среди стандартных оценок.

Выборочная медиана — центральная порядковая статистика. Для вариационного ряда $X_{(1)} \leq X_{(2)} \leq \dots \leq X_{(n)}$ при нечётном $n = 2k + 1$ медиана равна

$$Me = X_{(k+1)}$$

При чётном $n = 2k$ часто используют

$$Me = \frac{X_{(k)} + X_{(k+1)}}{2}$$

Для симметричного распределения выборочная медиана является естественной оценкой центра. Если распределение симметрично относительно a , то распределение выборочной медианы также симметрично относительно a . Поэтому при существовании математического ожидания медианы её ожидание равно a . Даже когда точная несмещённость зависит от определения медианы при чётном n , медиана является асимптотически несмещённой и состоятельной оценкой центра.

Усечённое среднее строится после удаления одинакового числа крайних наблюдений слева и справа. Если удаляется по k наблюдений, то

$$\bar{X}_{tr} = \frac{1}{n-2k} \sum_{i=k+1}^{n-k} X_{(i)}$$

Для симметричного распределения симметричное усечение не создаёт систематического сдвига: левые и правые хвосты удаляются одинаково. Поэтому усечённое среднее является естественной несмещённой или асимптотически несмещённой оценкой центра симметрии. Оно менее чувствительно к выбросам, чем обычное среднее, но использует больше информации, чем медиана.

Ключевое сравнение эффективности для нормального распределения. Если $X_i \sim N(a, \sigma^2)$, то

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n}$$

Асимптотическая дисперсия выборочной медианы равна

$$\text{Var}(Me) \approx \frac{\pi}{2} \cdot \frac{\sigma^2}{n}$$

Коэффициент $\pi/2 \approx 1,57$ означает, что при чистом нормальном распределении медиана примерно в 1,57 раза более разбросана, чем среднее. Иными словами, среднее эффективнее медианы, если выбросов нет и нормальная модель верна.

Числовой пример без выбросов. Пусть $\sigma^2 = 4$, $n = 100$. Тогда $\text{Var}(\bar{X}) = 4/100 = 0,04$. Для медианы при нормальной модели $\text{Var}(Me) \approx (\pi/2) \cdot 0,04 \approx 0,0628$. Стандартная ошибка среднего равна 0,2, а медианы около $\sqrt{0,0628} \approx 0,251$. Среднее точнее.

Числовой пример с выбросом. Выборка: 9, 10, 10, 11, 100. Среднее равно $\bar{X} = 140/5 = 28$. Медиана равна 10. Усечённое среднее при удалении одного минимального и одного максимального значения равно $(10 + 10 + 11)/3 \approx 10,33$. Если 100 — ошибка измерения или редкий выброс, то медиана и усечённое среднее намного лучше описывают центр основной массы данных.

Числовой пример, когда усечённое среднее является компромиссом. Выборка: 8, 9, 10, 10, 11, 12, 30. Среднее равно $90/7 \approx 12,86$, медиана равна 10, усечённое среднее с удалением 8 и 30 равно $(9 + 10 + 10 + 11 + 12)/5 = 10,4$. Если значение 30 является подозрительно большим, усечённое среднее сохраняет информацию о пяти центральных наблюдениях и меньше искажается выбросом, чем обычное среднее.

Таким образом, для симметричных распределений все три оценки направлены на оценивание одного центра a , но имеют разные свойства. Среднее несмещённо и эффективно при нормальности, но чувствительно к выбросам. Медиана устойчива, но менее эффективна при чистой нормальной модели. Усечённое среднее снижает влияние хвостов и часто даёт лучший практический баланс между эффективностью и устойчивостью.

6. Метод моментов: пример нахождения параметров равномерного распределения
Метод моментов — способ построения оценок, при котором теоретические моменты распределения приравниваются к выборочным моментам. Если распределение зависит от k неизвестных параметров $\theta_1, \dots, \theta_k$, то обычно составляют k уравнений по первым k моментам. Начальный момент порядка r равен

$$\alpha_r = E(X^r)$$

Выборочный начальный момент порядка r равен

$$\hat{\alpha}_r = \frac{1}{n} \sum_{i=1}^n X_i^r$$

По методу моментов решают систему

$$\alpha_r(\theta_1, \dots, \theta_k) = \hat{\alpha}_r, \quad r = 1, \dots, k$$

Метод моментов отличается от метода максимального правдоподобия. Метод моментов подгоняет моменты распределения: среднее, дисперсию и т.п. Метод максимального

правдоподобия выбирает параметры, при которых наблюденная выборка наиболее правдоподобна. Метод моментов обычно проще, но не всегда даёт наиболее эффективные оценки.

Пусть X имеет равномерное распределение на $[a, b]$, где a и b неизвестны. Плотность равна

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

$$E(X) = \frac{a+b}{2}$$

Теоретические характеристики:

$$\text{Var}(X) = \frac{(b-a)^2}{12}$$

Выборочные аналоги:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

$$S_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$$

Приравниваем моменты:

$$\frac{a+b}{2} = \bar{X}$$

$$\frac{(b-a)^2}{12} = S_n^2$$

Обозначим центр $m = (a+b)/2$ и длину $d = b-a$. Тогда

$$m = \bar{X}$$

$$d = \sqrt{12S_n^2}$$

Следовательно,

$$\hat{a}_{MM} = \bar{X} - \sqrt{3S_n^2}$$

$$\hat{b}_{MM} = \bar{X} + \sqrt{3S_n^2}$$

Числовой пример. Пусть выборка равна 2, 4, 5, 7, 8. Тогда $\bar{X} = 26/5 = 5,2$. Отклонения: $-3,2; -1,2; -0,2; 1,8; 2,8$. Квадраты: 10,24; 1,44; 0,04; 3,24; 7,84. Сумма квадратов равна 22,8. Тогда $S_n^2 = 22,8/5 = 4,56$. Получаем $\sqrt{(3S_n^2)} = \sqrt{13,68} \approx 3,70$. Поэтому

$$\hat{a}_{MM} \approx 5,2 - 3,70 = 1,50$$

$$\hat{b}_{MM} \approx 5,2 + 3,70 = 8,90$$

Значит, по методу моментов равномерный интервал оценивается примерно как [1,50; 8,90].

Можно вывести те же оценки через первые два начальных момента. Для равномерного распределения

$$E(X^2) = \frac{a^2 + ab + b^2}{3}$$

Если $\hat{\alpha}_1 = \bar{X}$ и $\hat{\alpha}_2 = (1/n)\sum X_i^2$, то система имеет вид

$$\frac{a + b}{2} = \hat{\alpha}_1$$

$$\frac{a^2 + ab + b^2}{3} = \hat{\alpha}_2$$

Так как $S_n^2 = \hat{\alpha}_2 - \bar{X}^2$, решение совпадает с формулами через среднее и дисперсию. Сравним метод моментов с методом максимального правдоподобия. Для равномерного распределения $[a, b]$ правдоподобие положительно только тогда, когда $a \leq X_{(1)}$ и $b \geq X_{(n)}$, где $X_{(1)}$ — минимум выборки, $X_{(n)}$ — максимум. При фиксированной выборке плотность равна $1/(b - a)^n$, поэтому она максимальна при минимальной длине интервала, покрывающего все наблюдения. Отсюда ММП-оценки:

$$\hat{a}_{MLE} = X_{(1)}$$

$$\hat{b}_{MLE} = X_{(n)}$$

Для выборки 2, 4, 5, 7, 8 получаем $\hat{a}_{MLE} = 2$, $\hat{b}_{MLE} = 8$. Метод моментов дал [1,50; 8,90], а ММП дал [2; 8]. Метод моментов использует среднее и дисперсию, а ММП использует крайние порядковые статистики.

У метода моментов есть важный недостаток: оценки границ могут не согласовываться с фактическими крайними наблюдениями. Например, выборка 0, 5, 5, 5, 5 имеет $\bar{X} = 4$, $S_n^2 = 4$. Тогда $\sqrt{(3S_n^2)} = \sqrt{12} \approx 3,46$, и $\hat{a}_{MM} \approx 0,54$. Но минимум выборки равен 0, то есть оценённая левая граница оказалась правее наблюдаемого значения: $\hat{a}_{MM} > X_{(1)}$. Для равномерного распределения это выглядит плохо, потому что все наблюдения должны лежать внутри $[a, b]$. ММП такой ошибки не допускает, поскольку всегда берёт границы по минимуму и максимуму.

При этом оценки метода моментов для равномерного распределения при стандартных условиях состоятельны: $\bar{X}$ сходится к $E X$, S_n^2 сходится к $\text{Var}(X)$, значит $\hat{a}_{MM}$ и $\hat{b}_{MM}$ сходятся к истинным a и b . Но они обычно не являются наиболее эффективными для оценки границ. Поэтому метод моментов удобен для быстрого получения формул и первичной оценки, но качество полученных оценок нужно анализировать отдельно.

7. Оценка закона распределения случайной величины: эмпирическая интегральная функция распределения

Если закон распределения случайной величины неизвестен, его можно оценивать без предположения о параметрическом семействе. Основной непараметрической оценкой функции распределения является эмпирическая интегральная функция распределения.

Пусть $X_1, \dots, X_n$ — выборка из распределения с функцией $F(x)$. Эмпирическая функция распределения определяется как

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n I\{X_i \leq x\}$$

где $I\{X_i \leq x\}$ — индикатор, равный 1, если $X_i \leq x$, и 0 иначе. Таким образом, $F_n(x)$ — доля наблюдений, не превосходящих x .

Если записать вариационный ряд

$$X_{(1)} \leq X_{(2)} \leq \dots \leq X_{(n)}$$

то $F_n(x)$ имеет ступенчатый вид:

$$F_n(x) = 0, \quad \text{если } x < X_{(1)}$$

$$F_n(x) = \frac{k}{n}, \quad \text{если } X_{(k)} \leq x < X_{(k+1)}$$

$$F_n(x) = 1, \quad \text{если } x \geq X_{(n)}$$

Эмпирическая функция распределения неубывает, принимает значения от 0 до 1, имеет скачки в точках наблюдений и является правосторонне непрерывной при стандартном определении через $\leq$. Если значения повторяются, скачок в соответствующей точке равен относительной частоте повторяющегося значения.

Для фиксированного x величина $F_n(x)$ является средним арифметическим индикаторов $I\{X_i \leq x\}$. Поскольку $E I\{X_i \leq x\} = P(X_i \leq x) = F(x)$, по закону больших чисел

$$F_n(x) \xrightarrow{P} F(x)$$

в каждой точке непрерывности F . Более сильный результат — теорема Гливенко-Кантелли:

$$\sup_x |F_n(x) - F(x)| \rightarrow 0 \quad \text{почти наверное}$$

Она означает равномерную сходимость эмпирической функции к истинной функции распределения по всем x одновременно. Это фундаментальное обоснование применения $F_n(x)$ как оценки закона распределения.

Числовой пример построения. Пусть выборка равна 3, 1, 4, 2, 2. Вариационный ряд: 1, 2, 2, 3, 4. Тогда $F_n(0,5) = 0$, $F_n(1) = 1/5 = 0,2$, $F_n(2) = 3/5 = 0,6$, $F_n(2,5) = 0,6$, $F_n(3) = 4/5 = 0,8$, $F_n(4) = 1$. В точке 2 скачок равен $2/5$, потому что значение 2 встретилось два раза. Числовой пример оценки вероятности. По той же выборке вероятность события $X \leq 2$ оценивается как $F_n(2) = 0,6$. Вероятность события $2 < X \leq 4$ оценивается как $F_n(4) - F_n(2) = 1 - 0,6 = 0,4$. Так эмпирическая функция позволяет оценивать вероятности интервалов без знания теоретического распределения.

Эмпирическая функция распределения связана с порядковыми статистиками. Квантили распределения оцениваются через элементы вариационного ряда. Например, выборочная медиана для выборки 1, 2, 2, 3, 4 равна 2, потому что на уровне 0,5 эмпирическая функция впервые достигает или превышает 0,5 именно в точке 2.

На основе эмпирической функции строятся критерии согласия. Например, статистика Колмогорова имеет вид

$$D_n = \sup_x |F_n(x) - F_0(x)|$$

где $F_0(x)$ — предполагаемая теоретическая функция распределения. Если D_n слишком велико, гипотеза о распределении F_0 отвергается. Также можно строить доверительные полосы для $F(x)$. Идея состоит в том, что с заданной вероятностью вся истинная функция $F(x)$ лежит внутри полосы вокруг $F_n(x)$:

$$F_n(x) - \frac{c}{\sqrt{n}} \leq F(x) \leq F_n(x) + \frac{c}{\sqrt{n}}$$

Константа c выбирается по распределению статистики Колмогорова или по соответствующим табличным значениям. Такая полоса показывает неопределённость оценки функции распределения сразу для всех x .

Числовой пример доверительной полосы в упрощённой форме. Пусть $n = 100$ и для уровня около 95% используется $c \approx 1,36$. Тогда ширина полосы равна $1,36/\sqrt{100} = 0,136$. Если $F_n(5) = 0,70$, то приближённо $F(5)$ находится в пределах $[0,564; 0,836]$ с учётом общей доверительной полосы. На концах интервал дополнительно ограничивается 0 и 1.

Эмпирическая функция распределения хороша тем, что не требует выбора интервалов, как гистограмма. Каждое наблюдение учитывается точно, без группировки. Недостаток состоит в ступенчатом виде, который не всегда удобен для визуального анализа плотности. Поэтому $F_n(x)$ часто используют вместе с гистограммой: первая описывает накопленные вероятности, вторая показывает распределение массы по интервалам.

8. Оценка закона распределения случайной величины: метод гистограмм

Гистограмма — непараметрический способ приближённой оценки распределения и плотности случайной величины по выборке. В отличие от эмпирической функции распределения, которая показывает накопленную долю наблюдений, гистограмма показывает, как наблюдения распределены по интервалам. Она особенно полезна для первичного анализа формы распределения: центра, разброса, асимметрии, выбросов, многомодальности.

Пусть дана выборка $X_1, \dots, X_n$. Сначала выбирают интервалы разбиения. Если интервалы одинаковой ширины h , то диапазон от $X_{\min}$ до $X_{\max}$ делится на m частей:

$$h = \frac{X_{\max} - X_{\min}}{m}$$

Пусть n_j — число наблюдений в j -м интервале. Относительная частота равна $p_j = n_j/n$. Если гистограмма строится как оценка плотности, высота j -го столбца равна

$$\hat{f}_j = \frac{n_j}{nh}$$

Тогда площадь столбца равна $h \cdot \hat{f}_j = n_j/n$, а сумма площадей всех столбцов равна 1. Для неравных интервалов с ширинами h_j высота задаётся формулой

$$\hat{f}_j = \frac{n_j}{nh_j}$$

Гистограмма содержит сглаживающий параметр — ширину интервала h или число интервалов m . Если m слишком мало, оценка имеет большое смещение: форма распределения чрезмерно сглаживается, детали исчезают. Если m слишком велико, возрастает дисперсия: столбцы становятся случайно рваными, отдельные наблюдения создают ложные пики. Поэтому выбор h — это дилемма смещение-дисперсия. Для выбора числа интервалов используют практические правила. Правило Стёрджеса:

$$m \approx 1 + \log_2 n$$

Оно удобно для умеренных выборок и даёт небольшое число интервалов. Правило Фридмана-Диакониса выбирает ширину интервала через межквартильный размах IQR:

$$h = 2 \cdot \frac{IQR}{n^{1/3}}$$

Это правило более устойчиво к выбросам, потому что IQR основан на квартилях, а не на полном размахе. После нахождения h число интервалов можно взять как $m \approx (X_{\max} - X_{\min})/h$.

Числовой пример по правилу Стёрджеса. Пусть $n = 100$. Тогда $m \approx 1 + \log_2 100 \approx 1 + 6,64 = 7,64$. Обычно берут 8 интервалов. Если $X_{\min} = 20$, $X_{\max} = 60$, то $h = (60 - 20)/8 = 5$. Интервалы можно взять $[20;25)$, $[25;30)$, ..., $[55;60]$.

Числовой пример нормированной высоты. Пусть $n = 100$, $h = 5$, а в интервал $[30;35)$ попало $n_j = 20$ наблюдений. Тогда относительная частота $p_j = 20/100 = 0,20$. Высота столбца как оценки плотности равна $\hat{f}_j = 20/(100 \cdot 5) = 0,04$. Площадь столбца равна $5 \cdot 0,04 = 0,20$, то есть совпадает с относительной частотой.

Числовой пример по правилу Фридмана-Диакониса. Пусть $n = 64$, $IQR = 12$, $X_{\min} = 10$, $X_{\max} = 58$. Тогда $n^{1/3} = 4$, $h = 2 \cdot 12/4 = 6$. Число интервалов $m \approx (58 - 10)/6 = 8$. Это даёт 8 интервалов шириной около 6. Если в данных есть один сильный выброс, например $X_{\max} = 120$, правило Стёрджеса через размах сильно растянет интервалы, а Фридман-Диаконис устойчивее, потому что h зависит от IQR.

Числовой пример влияния m . Пусть данные: 1, 2, 2, 3, 4, 10, 11, 12, 13, 14. Если взять $m = 2$, интервалы будут грубыми и можно не увидеть две группы около 2-4 и 10-14. Если взять $m = 10$, многие интервалы будут пустыми или с одним наблюдением. При $m = 4-5$ структура двух групп видна лучше. Это показывает, что гистограмма не является нейтральной картинкой: её вид зависит от выбранного сглаживания.

Гистограмма отличается от полигона частот. Гистограмма использует прямоугольники над интервалами, а полигон соединяет точки, соответствующие серединам интервалов и частотам. Для непрерывных данных гистограмма чаще используется как оценка плотности, а полигон удобен для сравнения нескольких распределений на одном графике.

Гистограмма помогает предварительно выбрать модель распределения. Симметричная колоколообразная гистограмма указывает на возможную нормальность. Длинный правый хвост может говорить о логнормальном, гамма- или экспоненциальном типе распределения. Две вершины могут указывать на смесь двух групп. Изолированные столбцы далеко от основной массы данных могут быть выбросами. Но гистограмма не является строгим критерием: выводы нужно подтверждать числовыми характеристиками и критериями согласия.

Таким образом, метод гистограмм оценивает закон распределения через интервальные частоты. Основные формулы — $p_j = n_j/n$ и $\hat{f}_j = n_j/(n \cdot h_j)$. Ключевой практический вопрос — выбор ширины интервала, потому что он управляет балансом между смещением и дисперсией оценки. Правила Стёрджеса и Фридмана-Диакониса дают конкретные способы выбора m и h , но окончательный анализ должен учитывать смысл данных и

устойчивость формы гистограммы к изменению разбиения.

9. Коэффициенты асимметрии и эксцесса; диаграммы Каллена-Фрея

Коэффициенты асимметрии и эксцесса описывают форму распределения. Математическое ожидание показывает центр, дисперсия — разброс, а асимметрия и эксцесс — перекося, тяжесть хвостов и остроту вершины. Эти характеристики особенно полезны при выборе вероятностной модели и проверке нормальности.

Теоретический коэффициент асимметрии определяется через третий центральный момент:

$$\gamma_1 = \frac{\mu_3}{\sigma^3}$$

где $\mu_3 = E(X - \mu)^3$, $\sigma^2 = \text{Var}(X)$. Если $\gamma_1 = 0$, распределение симметрично в моментном смысле. Если $\gamma_1 > 0$, правый хвост длиннее или тяжелее левого. Если $\gamma_1 < 0$, левый хвост длиннее или тяжелее правого.

Теоретический куртозис равен

$$\beta_2 = \frac{\mu_4}{\sigma^4}$$

где $\mu_4 = E(X - \mu)^4$. Эксцесс часто определяют как

$$\gamma_2 = \beta_2 - 3$$

Для нормального распределения $\beta_2 = 3$ и $\gamma_2 = 0$. Положительный эксцесс означает более тяжёлые хвосты или более острую вершину по сравнению с нормальным распределением. Отрицательный эксцесс означает более плоскую форму и более лёгкие хвосты.

Выборочные центральные моменты:

$$m_r = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^r$$

Простые выборочные коэффициенты:

$$g_1 = \frac{m_3}{m_2^{3/2}}$$

$$g_2 = \frac{m_4}{m_2^2} - 3$$

На практике часто используют поправки на смещение. Если s — исправленное стандартное отклонение, то скорректированный коэффициент асимметрии можно записать как

$$G_1 = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)$$

Для эксцесса одна из распространённых скорректированных форм имеет вид

$$G_2 = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}$$

Поправки особенно важны при малых n , потому что простые моментные коэффициенты могут иметь заметное смещение.

Числовой пример симметричной выборки. Пусть данные: 1, 2, 3, 4, 5. Среднее $\bar{x} = 3$. Отклонения: -2, -1, 0, 1, 2. Кубы отклонений: -8, -1, 0, 1, 8, сумма равна 0. Поэтому $m_3 = 0$ и $g_1 = 0$. Выборка симметрична относительно 3.

Числовой пример правой асимметрии. Данные: 1, 2, 2, 3, 12. Среднее $\bar{x} = 4$. Отклонения: -3, -2, -2, -1, 8. Кубы: -27, -8, -8, -1, 512. Сумма кубов равна 468, $m_3 = 93,6$. Третий момент положителен, значит асимметрия положительная: правый хвост вызван значением 12.

Числовой пример эксцесса. Сравним выборки А: 1, 2, 3, 4, 5 и В: 1, 2, 3, 4, 20. Во второй выборке значение 20 резко увеличивает четвёртые степени отклонений, поэтому m_4 и g_2 становятся намного больше. Это показывает чувствительность эксцесса к выбросам и тяжёлым хвостам. Именно поэтому эксцесс полезен для диагностики хвостов, но на малых выборках его нужно интерпретировать осторожно.

Для ориентировочной проверки значимости асимметрии и эксцесса используют стандартные ошибки. При достаточно больших n для нормальной совокупности приближённо

$$SE(g_1) \approx \sqrt{\frac{6}{n}}$$

$$SE(g_2) \approx \sqrt{\frac{24}{n}}$$

Если $|g_1| > 2 \cdot SE(g_1)$, асимметрию часто считают заметной. Если $|g_2| > 2 \cdot SE(g_2)$, эксцесс считают заметно отличающимся от нуля. Это эмпирическое правило, а не строгий универсальный критерий.

Числовой пример стандартных ошибок. Пусть $n = 100$. Тогда $SE(g_1) \approx \sqrt{(6/100)} = 0,245$, а $2 \cdot SE(g_1) \approx 0,49$. Если наблюдаем $g_1 = 0,8$, асимметрия заметна. Для эксцесса $SE(g_2) \approx \sqrt{(24/100)} = 0,49$, $2 \cdot SE(g_2) \approx 0,98$. Если $g_2 = 0,3$, серьёзного основания говорить о ненормальной тяжести хвостов только по эксцессу нет.

Диаграмма Каллена-Фрея используется для предварительного выбора семейства распределений по асимметрии и куртозису. На горизонтальной оси обычно откладывают квадрат асимметрии γ_1^2 , на вертикальной — куртозис β_2 . Нормальное распределение имеет точку

\$\$

$$\gamma_1^2 = 0, \quad \beta_2 = 3$$

\$\$ Равномерное распределение симметрично, поэтому $\gamma_1^2 = 0$, но его $\beta_2 = 1,8$, то есть оно лежит ниже нормального. Экспоненциальное распределение имеет $\gamma_1^2 = 2$, $\gamma_1^2 = 4$, $\beta_2 = 9$. Логнормальные и гамма-распределения занимают области с положительной асимметрией и повышенным куртозисом. Бета-распределения могут давать широкий набор форм, включая ограниченные и асимметричные распределения.

Числовой пример для диаграммы Каллена-Фрея. Если по выборке получено $g_1 = 0,1$ и $\beta_2 \approx 3,1$, точка близка к нормальному распределению. Если $g_1 = 2$ и $\beta_2 = 9$, точка близка к экспоненциальному типу. Если $g_1 = 0$ и $\beta_2 = 1,8$, это похоже на равномерное распределение. Если $g_1 = 1,5$ и $\beta_2 = 6$, стоит рассмотреть правосторонне асимметричные распределения: гамма, логнормальное, Вейбулла.

Диаграмма Каллена-Фрея не доказывает закон распределения, а только помогает выбрать кандидатов. Её используют вместе с гистограммой, эмпирической функцией распределения, критериями согласия и содержательным смыслом данных. На малых

выборках точка может сильно колебаться, поэтому иногда строят бутстреп-облако: многократно пересэмпблируют данные, считают асимметрию и куртозис и смотрят область возможных точек. Если облако широко, вывод о семействе распределений должен быть осторожным.

Итак, асимметрия отвечает за перекося, эксцесс — за хвосты и остроту, а диаграмма Каллена-Фрея превращает эти две характеристики в инструмент выбора распределения. В письменном ответе важно дать не только смысл, но и формулы γ_1 , β_2 , γ_2 , выборочные g_1 и g_2 , стандартные ошибки и примеры расположения нормального, равномерного и экспоненциального распределений на диаграмме.

Contents

Построение и анализ графовых моделей

1

Построение и анализ графовых моделей

Файл объединяет 12 билетов в связные ответы для письменного экзамена. Формулировки сокращены без изменения смысла: сохранены определения, основные свойства, критерии, формулы, алгоритмы и типовые примеры. # Билет 1. Графы: определения и основные операции над графами. Графовая модель описывает систему через объекты и связи между ними. Объекты называются вершинами, связи — рёбрами или дугами. Графы применяются для описания дорожных сетей, зависимостей задач, компьютерных сетей, социальных отношений, схем состояний, маршрутов и структур данных. В графе важна не природа объекта, а структура связей: какие вершины соединены, существуют ли пути, циклы, компоненты связности, какие вершины являются «важными» по степени или положению. Неориентированный граф задаётся парой

$$G = (V, E),$$

где V — множество вершин, E — множество рёбер. В простом неориентированном графе каждое ребро является неупорядоченной парой различных вершин:

$$e = \{u, v\}, \quad u, v \in V, \quad u \neq v.$$

Если $e = \{u, v\}$, то вершины u и v называются смежными, а ребро e инцидентно обоим вершинам. Ориентированный граф, или орграф, задаётся парой

$$D = (V, A),$$

где A — множество дуг. Дуга является упорядоченной парой (u, v) , где u — начало дуги, v — конец дуги. В орграфе наличие дуги (u, v) не означает наличие дуги (v, u) . Петля — ребро или дуга, соединяющая вершину с самой собой: $\{v, v\}$ или (v, v) . Кратные рёбра — несколько рёбер между одной и той же парой вершин. Граф с петлями и кратными рёбрами называют мультиграфом. Простой граф не содержит ни петель, ни кратных рёбер. Степень вершины v в неориентированном графе обозначается $\deg(v)$ и равна числу рёбер, инцидентных вершине. Петля обычно даёт вклад 2, так как оба её конца совпадают с одной вершиной. Для орграфа различают полустепень исхода и захода:

$$\deg^+(v) = \text{число дуг, выходящих из } v,$$

$$\deg^-(v) = \text{число дуг, входящих в } v,$$

$$\deg(v) = \deg^+(v) + \deg^-(v).$$

Для неориентированного графа выполняется лемма о рукопожатиях:

$$\sum_{v \in V} \deg(v) = 2\|E\|.$$

Каждое ребро имеет два конца, поэтому при суммировании степеней учитывается дважды. Следствие: число вершин нечётной степени в любом неориентированном графе чётно. Маршрут — последовательность вершин и рёбер

$$v_0, e_1, v_1, e_2, \dots, e_k, v_k,$$

где каждое ребро e_i соединяет v_{i-1} и v_i . В орграфе маршрут должен учитывать направления дуг. Цепь — маршрут без повторения рёбер. Путь — маршрут без повторения вершин. Замкнутый маршрут начинается и заканчивается в одной вершине. Цикл — замкнутый путь, в котором все вершины, кроме первой и последней, различны. Граф называется связным, если для любых двух его вершин существует путь между ними. Компонента связности — максимальный по включению связный подграф. Если граф имеет одну компоненту, он связан; если несколько — несвязен. В орграфах различают сильную связность, когда для любых u, v существует путь из u

в v и из v в u , и слабую связность, когда после удаления направлений дуг получается связный неориентированный граф. Подграф $H = (V_H, E_H)$ графа $G = (V, E)$ определяется условиями

$$V_H \subseteq V, \quad E_H \subseteq E.$$

Остовный подграф содержит все вершины исходного графа. Индуцированный подграф на множестве $U \subseteq V$ содержит все рёбра исходного графа, оба конца которых лежат в U . Основные операции над графами: удаление вершины, удаление ребра, добавление вершины, добавление ребра, объединение, пересечение, дополнение, стягивание ребра, взятие подграфа. При удалении вершины удаляются все инцидентные ей рёбра. При стягивании ребра uv вершины u и v заменяются одной вершиной, а инцидентные связи переносятся на неё. Дополнение простого графа $\bar{G}$ имеет то же множество вершин, а ребро uv присутствует в $\bar{G}$ тогда и только тогда, когда оно отсутствует в G :

$$E(\bar{G}) = \{\{u, v\} : u, v \in V, u \neq v, \{u, v\} \notin E(G)\}.$$

Изоморфизм графов — взаимно однозначное соответствие между вершинами двух графов, сохраняющее смежность. Графы $G_1 = (V_1, E_1)$ и $G_2 = (V_2, E_2)$ изоморфны, если существует биекция $f : V_1 \rightarrow V_2$, такая что

$$\{u, v\} \in E_1 \iff \{f(u), f(v)\} \in E_2.$$

При изоморфизме сохраняются число вершин, число рёбер, степени вершин, связность, число компонент, наличие циклов, эйлеровость, гамильтоновость, планарность и другие структурные свойства. # Билет 2. Формы задания графов и метрические характеристики графа Граф можно задать разными способами. Выбор формы зависит от задачи: требуется ли быстро проверять наличие ребра, перечислять соседей, хранить веса, запускать алгоритмы обхода или работать с плотным графом. Общая запись:

$$G = (V, E), \quad \|V\| = n, \quad \|E\| = m.$$

Если рёбра имеют направление, граф является ориентированным. Если каждому ребру сопоставлен вес $w(e)$, граф называется взвешенным. Вес может означать длину, стоимость, время, пропускную способность, риск или штраф. Список рёбер — перечисление всех рёбер графа. В неориентированном графе ребро записывается как $\{u, v\}$, в ориентированном — как (u, v) . Для взвешенного графа используют тройку (u, v, w) , где w — вес ребра. Эта форма компактна для разреженных графов и удобна для алгоритмов, которые перебирают все рёбра, например алгоритма Краскала. Память — $O(m)$. Недостаток: проверка наличия конкретного ребра обычно требует просмотра списка. Список смежности для каждой вершины хранит её соседей. Для неориентированного графа ребро uv записывается дважды: v в списке u и u в списке v . Для орграфа в списке вершины u обычно хранят вершины, в которые выходят дуги из u . Для взвешенного графа рядом с соседом указывают вес: (v, w) . Память — $O(n + m)$. Это основная форма для обходов BFS/DFS, поиска компонент связности и алгоритмов на разреженных графах. Список инцидентности для каждой вершины хранит рёбра, которым она инцидентна. Эта форма полезна, когда рёбра являются самостоятельными объектами с атрибутами: весом, номером, цветом, пропускной способностью. Она менее удобна для быстрой проверки смежности, но естественна в сетевых моделях. Матрица смежности — квадратная матрица $A = (a_{ij})$ размера $n \times n$, где строки и столбцы соответствуют вершинам. Для невзвешенного графа

$$a_{ij} = \begin{cases} 1, & \text{если } v_i \text{ и } v_j \text{ смежны,} \\ 0, & \text{иначе.} \end{cases}$$

Для ориентированного графа $a_{ij} = 1$, если есть дуга (v_i, v_j) . Для взвешенного графа вместо 1 можно хранить вес ребра, а отсутствие ребра обозначать 0, ∞ или специальным значением. В неориентированном графе матрица смежности симметрична:

$$a_{ij} = a_{ji}.$$

Память — $O(n^2)$. Главное достоинство — проверка наличия ребра за $O(1)$. Недостаток — неэкономичность для разреженных графов. Матрица инцидентности — матрица B размера $n \times m$, строки которой соответствуют вершинам, а столбцы — рёбрам. В неориентированном графе

$$b_{ij} = \begin{cases} 1, & \text{если вершина } v_i \text{ инцидентна ребру } e_j, \\ 0, & \text{иначе.} \end{cases}$$

В ориентированном графе часто используют знаки: -1 для начала дуги, $+1$ для конца, 0 при отсутствии инцидентности. Матрица инцидентности полезна в алгебраическом анализе графов, потоках и сетевых задачах. Основные метрические характеристики графа строятся через расстояния. Расстояние $d(u, v)$ между вершинами u и v — длина кратчайшего пути между ними. В невзвешенном графе длина пути равна числу рёбер, во взвешенном — сумме весов:

$$\text{len}(P) = \sum_{e \in P} w(e).$$

Если пути между вершинами нет, расстояние считают бесконечным или неопределённым. В связном неориентированном графе расстояние является метрикой: $d(u, v) \geq 0$, $d(u, v) = 0$ только при $u = v$, $d(u, v) = d(v, u)$, выполняется неравенство треугольника:

$$d(u, w) \leq d(u, v) + d(v, w).$$

Эксцентриситет вершины v — максимальное расстояние от неё до других вершин:

$$e(v) = \max_{u \in V} d(v, u).$$

Радиус графа — минимальный эксцентриситет:

$$r(G) = \min_{v \in V} e(v).$$

Диаметр графа — максимальный эксцентриситет:

$$\text{diam}(G) = \max_{v \in V} e(v) = \max_{u, v \in V} d(u, v).$$

Центр графа — множество вершин с минимальным эксцентриситетом:

$$C(G) = \{v \in V : e(v) = r(G)\}.$$

Периферийные вершины имеют эксцентриситет, равный диаметру. Метрические характеристики показывают, насколько «растянут» граф, какие вершины центральны, какова максимальная удалённость узлов и где выгоднее размещать ресурс или центр управления. # Билет 3. Специальные графы, дополнение и изоморфизм В простом неориентированном графе $G = (V, E)$ ребро является неупорядоченной парой различных вершин, петли и кратные рёбра отсутствуют. Обозначения: $\|V\| = n$, $\|E\| = m$, $\deg(v)$ — степень вершины. Всегда выполняется

$$\sum_{v \in V} \deg(v) = 2m.$$

Полный граф K_n — простой граф на n вершинах, в котором каждая пара различных вершин соединена ребром. Его множество рёбер:

$$E(K_n) = \{\{u, v\} : u, v \in V, u \neq v\}.$$

Каждая вершина имеет степень $n - 1$, поэтому K_n является $(n - 1)$ -регулярным. Число рёбер:

$$\|E(K_n)\| = C_n^2 = \frac{n(n-1)}{2}.$$

Это максимальное число рёбер в простом графе на n вершинах. Например, K_3 — треугольник, K_4 имеет 6 рёбер, K_5 имеет 10 рёбер. Регулярный, или однородный, граф степени r — граф, у которого степени всех вершин равны r :

$$\forall v \in V : \deg(v) = r.$$

Если граф r -регулярен и имеет n вершин, то

$$nr = 2m, \quad m = \frac{nr}{2}.$$

Следовательно, произведение nr должно быть чётным. Для простого графа выполняется $0 \leq r \leq n - 1$. Полный граф K_n является $(n - 1)$ -регулярным, цикл C_n при $n \geq 3$ — 2-регулярным, граф без рёбер — 0-регулярным. Кубический граф — 3-регулярный граф, то есть у каждой вершины степень 3. Для кубического графа на n вершинах

$$3n = 2m,$$

поэтому n должно быть чётным. Примеры кубических графов: K_4 , граф куба, некоторые регулярные сетевые структуры. Двудольный граф — граф, множество вершин которого можно разбить на две непересекающиеся доли X и Y :

$$V = X \cup Y, \quad X \cap Y = \emptyset,$$

так, что каждое ребро соединяет вершину из X с вершиной из Y , а рёбер внутри одной доли нет. Критерий двудольности: граф двудолен тогда и только тогда, когда он не содержит циклов нечётной длины. Пути и деревья всегда двудольны, чётные циклы двудольны, нечётные циклы не двудольны. Полный двудольный граф $K_{m,n}$ имеет две доли размеров m и n , причём каждая вершина первой доли соединена с каждой вершиной второй доли. Для него

$$\|V\| = m + n, \quad \|E\| = mn.$$

Степени вершин первой доли равны n , второй — m . Граф $K_{3,3}$ является важным примером непланарного графа. Цепь, или путь-граф P_n , — граф из n вершин, расположенных последовательно. Он имеет $n - 1$ ребро. У двух крайних вершин степень 1, у внутренних — степень 2. Цикл C_n получается из P_n добавлением ребра между крайними вершинами; он имеет n вершин, n рёбер и все степени равны 2. Дерево — связный граф без циклов. Для дерева на n вершинах выполняется

$$\|E\| = n - 1.$$

Между любыми двумя вершинами дерева существует ровно один простой путь. Удаление любого ребра дерева нарушает связность, а добавление любого нового ребра создаёт цикл. Дополнение простого графа $\bar{G}$ имеет то же множество вершин, что и G , а ребро между двумя различными вершинами есть тогда и только тогда, когда его нет в исходном графе:

$$E(\bar{G}) = \{\{u, v\} : u \neq v, \{u, v\} \notin E(G)\}.$$

Если G имеет n вершин и m рёбер, то

$$\|E(\bar{G})\| = \frac{n(n - 1)}{2} - m.$$

Изоморфизм графов — переименование вершин, сохраняющее структуру смежности. Графы G_1 и G_2 изоморфны, если существует биекция $f : V_1 \rightarrow V_2$, такая что

$$\{u, v\} \in E_1 \iff \{f(u), f(v)\} \in E_2.$$

Необходимые признаки изоморфизма: одинаковое число вершин, рёбер, компонент связности, одинаковая последовательность степеней, совпадение числа циклов определённых длин, одинаковые специальные свойства. Эти признаки не всегда достаточны, но позволяют быстро доказать неизоморфность. # Билет 4. Связность графов, разрезы и компоненты связности Связность описывает возможность перехода между вершинами по рёбрам или дугам. В неориентированном графе $G = (V, E)$ путь между вершинами u и v — последовательность

$$u = v_0, v_1, \dots, v_k = v,$$

такая, что каждая пара соседних вершин соединена ребром. Граф называется связным, если для любых $u, v \in V$ существует путь из u в v . Если это не выполняется, граф несвязен и распадается на компоненты связности. Компонента связности — максимальный по включению связный подграф. Множество вершин $C \subseteq V$ является компонентой, если любые две вершины из C соединены путём и нельзя добавить к C другую вершину, сохранив связность. Компоненты попарно не пересекаются и покрывают всё V . В ориентированном графе $D = (V, A)$ направление

дуг принципиально важно. Орграф называется сильно связным, если для любых вершин u и v существует ориентированный путь из u в v и из v в u . Сильно связная компонента — максимальное множество вершин, внутри которого любые две вершины взаимно достижимы. Орграф слабо связан, если после замены всех дуг неориентированными рёбрами получается связный граф. Разрез — множество рёбер или вершин, удаление которых нарушает связность или отделяет одну часть графа от другой. Рёберный разрез можно задать через разбиение вершин:

$$V = S \cup (V \setminus S), \quad S \neq \emptyset, \quad V \setminus S \neq \emptyset.$$

Тогда

$$\delta(S) = \{uv \in E : u \in S, v \in V \setminus S\}$$

— множество рёбер, пересекающих разбиение. Если граф взвешенный, вес разреза равен сумме весов рёбер, входящих в разрез. Рёберный $s - t$ -разрез — множество рёбер, после удаления которых нет пути из s в t . Вершинный разрез — множество вершин, удаление которых вместе с инцидентными рёбрами делает граф несвязным или отделяет заданные вершины. Вершинный $s - t$ -разрез отделяет s от t , если после удаления его вершин пути между s и t нет. Мост — ребро, удаление которого увеличивает число компонент связности. Ребро является мостом тогда и только тогда, когда оно не принадлежит ни одному циклу. Если ребро лежит на цикле, после его удаления связь между его концами сохраняется по оставшейся части цикла. Точка сочленения — вершина, удаление которой увеличивает число компонент связности. Мосты и точки сочленения показывают уязвимые места сети. Рёберная связность $\lambda(G)$ — минимальное число рёбер, удаление которых делает связный граф несвязным:

$$\lambda(G) = \min\{\|F\| : G - F \text{ несвязен}\}.$$

Вершинная связность $\kappa(G)$ — минимальное число вершин, удаление которых делает граф несвязным или тривиальным:

$$\kappa(G) = \min\{\|S\| : G - S \text{ несвязен}\}.$$

Для нетривиального связного графа выполняется важное неравенство:

$$\kappa(G) \leq \lambda(G) \leq \delta(G),$$

где $\delta(G)$ — минимальная степень вершины. Это означает: чтобы разрушить граф, обычно достаточно удалить все рёбра, инцидентные вершине минимальной степени, поэтому $\lambda(G) \leq \delta(G)$. Вершинная связность не превосходит рёберную. Компоненты связности неориентированного графа находят обходом DFS или BFS. Алгоритм: все вершины помечаются как непосещённые; выбирается непосещённая вершина; из неё запускается обход, который помечает всю компоненту; затем выбирается следующая непосещённая вершина. Число запусков обхода равно числу компонент. Сложность при списках смежности:

$$O(\|V\| + \|E\|).$$

Для орграфов слабые компоненты ищутся так же после игнорирования направлений. Сильно связные компоненты находят алгоритмами Косарайю, Тарьяна или Габова. Алгоритм Косарайю выполняет DFS в исходном графе, затем DFS в транспонированном графе в порядке убывания времени выхода. Алгоритм Тарьяна использует один DFS, стек и значения tin , low , также работает за

$$O(\|V\| + \|E\|).$$

Билет 5. Эйлеровы графы Эйлеровость описывает возможность пройти по всем рёбрам графа ровно один раз. Пусть $G = (V, E)$. Маршрут — последовательность

$$v_0, e_1, v_1, e_2, \dots, e_k, v_k,$$

где каждое ребро соединяет соседние вершины. Цепь — маршрут без повторения рёбер. Эйлерова цепь, или эйлеров путь, — цепь, проходящая по каждому ребру графа ровно один раз. Эйлеров цикл — замкнутая эйлерова цепь, которая начинается и заканчивается в одной вершине. Эйлеров граф — граф, содержащий эйлеров цикл. Полуэйлеров граф содержит эйлерову цепь, но не обязательно эйлеров цикл. Важно отличать эйлеровы и гамильтоновы задачи. В эйлеровой задаче нужно

пройти все рёбра ровно один раз, вершины могут повторяться. В гамильтоновой задаче нужно пройти все вершины ровно один раз, а рёбра выбираются не обязательно все. Эйлеровость имеет простой критерий через степени и связность, а гамильтоновость в общем случае такого критерия не имеет. Для неориентированного графа изолированные вершины не мешают эйлерову обходу, потому что в них нет рёбер. Поэтому связность проверяют только для вершин ненулевой степени: все такие вершины должны лежать в одной компоненте связности. Неориентированный граф имеет эйлеров цикл тогда и только тогда, когда выполнены два условия: 1. все вершины ненулевой степени лежат в одной компоненте связности; 2. степень каждой вершины чётна.

Объяснение: при прохождении через вершину каждый вход должен иметь соответствующий выход. В цикле начальная вершина совпадает с конечной, поэтому для неё действует то же правило. Следовательно, все инцидентные рёбра разбиваются на пары «вошли — вышли», а степени чётны. Неориентированный граф имеет эйлерову цепь тогда и только тогда, когда: 1. все вершины ненулевой степени лежат в одной компоненте связности; 2. число вершин нечётной степени равно 0 или 2.

Если нечётных вершин 0, существует эйлеров цикл, и обход можно начать в любой вершине ненулевой степени. Если нечётных вершин 2, существует незамкнутая эйлерова цепь: она начинается в одной нечётной вершине и заканчивается в другой. Если нечётных вершин больше двух, эйлеровой цепи нет. Для ориентированного графа используются полустепени:

$$\deg^+(v) = \text{число выходящих дуг}, \quad \deg^-(v) = \text{число входящих дуг}.$$

Орграф имеет эйлеров цикл, если все вершины с ненулевой степенью лежат в одной слабой компоненте, достижимость согласована с направлением обхода, и для каждой вершины

$$\deg^+(v) = \deg^-(v).$$

Орграф имеет эйлеров путь из s в t , если для всех промежуточных вершин вход равен выходу, для стартовой вершины

$$\deg^+(s) = \deg^-(s) + 1,$$

для конечной вершины

$$\deg^-(t) = \deg^+(t) + 1,$$

а все остальные вершины удовлетворяют $\deg^+(v) = \deg^-(v)$. Также все вершины с ненулевой степенью должны принадлежать одной слабой компоненте. Для построения эйлерова цикла используют алгоритм Флёри или алгоритм Хиерхольцера. Алгоритм Флёри строит цепь пошагово и старается не выбирать мост, если есть альтернатива. Он нагляден, но неэффективен. Алгоритм Хиерхольцера эффективнее: сначала строится произвольный цикл, затем пока есть вершина цикла с неиспользованными рёбрами, из неё строится дополнительный цикл и вставляется в основной. При списках смежности сложность может быть

$$O(\|E\|).$$

Типовой пример: если степени вершин графа равны 2, 2, 4, 4, и все вершины ненулевой степени связны, граф эйлеров. Если степени 1, 2, 2, 3, есть ровно две нечётные вершины, значит существует эйлерова цепь, но не цикл. Если нечётных степеней четыре, эйлерова цепь невозможна. # Билет 6. Гамильтоновы графы Пусть дан граф $G = (V, E)$. Гамильтонов путь — простой путь, проходящий через каждую вершину графа ровно один раз:

$$v_1, v_2, \dots, v_n,$$

где все v_i различны, и для каждого $i = 1, \dots, n - 1$ существует ребро $v_i v_{i+1}$. Гамильтонов цикл — простой цикл, проходящий через каждую вершину ровно один раз и возвращающийся в начало:

$$v_1, v_2, \dots, v_n, v_1.$$

Гамильтонов граф — граф, содержащий гамильтонов цикл. Если есть только гамильтонов путь, но нет цикла, граф обычно не называют гамильтоновым. Гамильтоновость отличается от эйлеровости. В эйлеровой задаче нужно пройти все рёбра ровно один раз, вершины могут повторяться. В гамильтоновой задаче нужно пройти все вершины ровно один раз, рёбра могут использоваться

не все. Для эйлера цикла в неориентированном графе есть точный критерий через чётность степеней и связность. Для гамильтонова цикла в произвольном графе простого полного критерия нет. Если граф гамильтонов, то он обязан удовлетворять нескольким необходимым условиям. Во-первых, он связан, потому что гамильтонов цикл проходит через все вершины. Во-вторых, минимальная степень не меньше 2:

$$\delta(G) \geq 2,$$

так как в цикле каждая вершина имеет двух соседей: предыдущую и следующую. В-третьих, в гамильтоновом графе не может быть моста. Если ребро является мостом, то для перехода из одной части графа в другую и возврата пришлось бы использовать его дважды, что невозможно в простом цикле. Эти условия необходимы, но не достаточны. Задача проверки существования гамильтонова цикла в общем случае NP-полна. Это означает, что известные точные алгоритмы в худшем случае имеют экспоненциальную сложность, а простого универсального критерия вроде эйлера не существует. Поэтому на практике используют достаточные условия, перебор, динамическое программирование и эвристики. Теорема Дирака: если простой граф G имеет $n \geq 3$ вершин и минимальная степень каждой вершины не меньше половины числа вершин,

$$\delta(G) \geq \frac{n}{2},$$

то граф гамильтонов. Это достаточное условие: если оно выполнено, цикл точно есть; если не выполнено, граф всё равно может быть гамильтоновым. Теорема Оре: если простой граф G имеет $n \geq 3$ вершин и для любой пары несмежных вершин u, v выполняется

$$\deg(u) + \deg(v) \geq n,$$

то граф гамильтонов. Теорема Дирака является частным следствием теоремы Оре: если каждая степень не меньше $n/2$, то сумма степеней любых двух вершин не меньше n . Полный граф K_n при $n \geq 3$ гамильтонов, потому что можно расположить вершины в любом порядке и замкнуть цикл. Цикл C_n гамильтонов сам по себе. Любой граф, содержащий C_n как остовный цикл, является гамильтоновым. Дерево с $n > 2$ не гамильтоново, потому что имеет листья степени 1, а гамильтонов цикл требует степень хотя бы 2. Для поиска гамильтонова цикла можно использовать перебор с возвратом. Строится последовательность вершин; на каждом шаге выбирается ещё не посещённая вершина, смежная с текущей. Если путь содержит все вершины и последняя смежна с первой, найден гамильтонов цикл. Если продолжение невозможно, алгоритм возвращается назад. В худшем случае перебирается порядка $n!$ вариантов. Для задачи коммивояжёра гамильтонов цикл связан с поиском цикла минимального веса, проходящего через все вершины ровно один раз. В полном взвешенном графе требуется минимизировать

$$\sum_{i=1}^n w(v_i, v_{i+1}), \quad v_{n+1} = v_1.$$

Это уже оптимизационная задача: гамильтонов цикл должен не только существовать, но и иметь минимальную стоимость. # Билет 7. Деревья, остовное дерево и фундаментальное множество циклов. Дерево — связный неориентированный граф без циклов. Пусть $T = (V, E)$, $\|V\| = n$. Для конечного графа следующие условия эквивалентны и могут использоваться как определения дерева: 1. T связан и не содержит циклов; 2. T ациклический и имеет $n - 1$ ребро; 3. T связан и имеет $n - 1$ ребро; 4. между любыми двумя вершинами существует ровно один простой путь; 5. T минимально связан: удаление любого ребра нарушает связность; 6. T максимально ациклический: добавление любого нового ребра создаёт цикл.

Главная формула дерева:

$$\|E(T)\| = \|V(T)\| - 1 = n - 1.$$

Она удобна для проверки: если связный граф на n вершинах имеет ровно $n - 1$ ребро, он является деревом; если ациклический граф имеет $n - 1$ ребро, он также является деревом. Лес — неориентированный граф без циклов. Каждая компонента связности леса является деревом. Если лес имеет n вершин, m рёбер и k компонент связности, то

$$m = n - k.$$

Доказательство: в каждой компоненте с n_i вершинами $n_i - 1$ ребро, значит

$$m = \sum_{i=1}^k (n_i - 1) = n - k.$$

Лист дерева — вершина степени 1. В дереве с $n \geq 2$ всегда есть хотя бы два листа. Это следует из рассмотрения самого длинного простого пути: его концы не могут иметь дополнительные рёбра, иначе путь можно было бы продолжить или возник бы цикл. Корневое дерево — дерево, в котором выделена вершина r , называемая корнем. Для любой вершины v существует единственный путь из r в v . Это позволяет определить уровень вершины как расстояние от корня, родителя как предыдущую вершину на пути к корню, детей как соседей следующего уровня, предков и потомков. Корневые деревья используются в алгоритмах поиска, структурах данных, синтаксических деревьях, иерархиях файлов и организационных схемах. Остовное дерево связного графа $G = (V, E)$ — подграф $T = (V, E_T)$, который содержит все вершины исходного графа и является деревом. Поэтому

$$E_T \subseteq E, \quad \|E_T\| = \|V\| - 1.$$

Остовное дерево существует тогда и только тогда, когда исходный граф связан. Если граф сам является деревом, его единственное остовное дерево — он сам. Если граф содержит циклы, обычно существует несколько остовных деревьев: каждое получается удалением лишних рёбер так, чтобы сохранить связность и убрать циклы. Хорда относительно остовного дерева — ребро исходного графа, которое не входит в выбранное остовное дерево:

$$e \in E \setminus E_T.$$

Если добавить хорду $e = uv$ к остовному дереву T , в дереве уже существует единственный путь из u в v . Добавленное ребро вместе с этим путём образует ровно один цикл. Этот цикл называется фундаментальным циклом относительно остова T . Число хорд равно

$$\|E\| - \|E_T\| = m - (n - 1) = m - n + 1$$

для связного графа. Поэтому число фундаментальных циклов относительно выбранного остова также равно

$$\mu(G) = m - n + 1.$$

Эта величина называется цикломатическим числом связного графа и показывает, сколько независимых циклов имеется в графе. Для несвязного графа с k компонентами формула принимает вид

$$\mu(G) = m - n + k.$$

Фундаментальное множество циклов зависит от выбранного остовного дерева, но его размер одинаков для всех остовов. На практике оно используется в анализе электрических цепей, транспортных сетей и алгоритмах, где циклы нужно описать через базис. # Билет 8. Остовные деревья и минимальные остовы Остовное дерево связного графа — это способ сохранить все вершины и связность, но удалить циклы. Пусть $G = (V, E)$ — конечный неориентированный граф, $\|V\| = n$. Остовный подграф содержит все вершины исходного графа. Остовное дерево $T = (V, E_T)$ — остовный подграф, который является деревом:

$$E_T \subseteq E, \quad T \text{ связан, } T \text{ ацикличен.}$$

Любое остовное дерево связного графа содержит ровно

$$n - 1$$

ребро. Если граф несвязен и имеет c компонент, вместо остовного дерева строится остовный лес, содержащий

$$n - c$$

рёбер. Если каждому ребру задан вес $w : E \rightarrow \mathbb{R}$, вес остова равен

$$w(T) = \sum_{e \in E(T)} w(e).$$

Минимальное остовное дерево, или MST, — остовное дерево минимального суммарного веса:

$$T^* = \arg \min_T \sum_{e \in E(T)} w(e).$$

Если граф несвязен, минимального остовного дерева всего графа нет, но можно искать минимальный остовный лес по компонентам. Основные свойства остовов: остов существует тогда и только тогда, когда граф связен; добавление к дереву любого ребра из $E \setminus E(T)$ создаёт ровно один цикл; удаление любого ребра из дерева разбивает его на две компоненты; если граф уже дерево, его единственный остов — он сам. Задача перечисления остовных деревьев состоит в получении всех различных остовов графа. Их число может быть очень большим. Для полного графа K_n число остовных деревьев задаётся формулой Кэли:

$$\tau(K_n) = n^{n-2}.$$

Например, K_4 имеет $4^2 = 16$ остовных деревьев. Для подсчёта числа остовов произвольного графа используется матричная теорема Кирхгофа. Строится матрица Лапласа

$$L = D - A,$$

где D — диагональная матрица степеней, A — матрица смежности. Число остовных деревьев $\tau(G)$ равно любому алгебраическому дополнению матрицы L , то есть определителю матрицы, полученной удалением одной строки и одного столбца:

$$\tau(G) = \det L^{(i)}.$$

Алгоритм Краскала строит MST, сортируя рёбра по неубыванию веса. Далее рёбра просматриваются по порядку и добавляются в остов, если не создают цикл. Для проверки циклов используется система непересекающихся множеств DSU. Алгоритм заканчивает работу, когда выбрано $n - 1$ ребро. Сложность определяется сортировкой:

$$O(m \log m).$$

Краскал удобен для разреженных графов и естественно следует свойству разреза: минимальное ребро, пересекающее некоторый разрез, безопасно для MST. Алгоритм Прима начинает с произвольной вершины и постепенно расширяет дерево. На каждом шаге выбирается ребро минимального веса, соединяющее уже построенную часть с новой вершиной. Если используется очередь с приоритетом и списки смежности, сложность:

$$O(m \log n).$$

При матрице смежности возможна реализация за $O(n^2)$, удобная для плотных графов. Алгоритм Борушки начинает с того, что каждая вершина считается отдельной компонентой. На каждой фазе для каждой компоненты выбирается минимальное исходящее ребро, после чего компоненты объединяются. Число компонент быстро уменьшается, а алгоритм хорошо параллелится. Корректность алгоритмов MST основана на двух свойствах. Свойство разреза: если разрез пересекает множество рёбер, то ребро минимального веса среди пересекающих этот разрез можно включить в некоторое MST. Свойство цикла: если в цикле есть ребро строго максимального веса, то оно не входит ни в одно MST. Если веса различны, минимальное остовное дерево единственно. Если равные веса есть, MST может быть несколько. # Билет 9. Планарность графа и теорема Понтрягина — Куратовского Планарность описывает возможность изобразить граф на плоскости без пересечений рёбер во внутренних точках. Вершины можно располагать произвольно, а рёбра проводить не только прямыми отрезками, но и непрерывными кривыми. Важно только, чтобы разные рёбра встречались лишь в общих концах. Рисунок графа на плоскости — изображение, при котором каждой вершине сопоставлена точка, каждому ребру — простая кривая между концами, и кривая ребра не проходит через чужие вершины. Рисунок может иметь пересечения. Если два ребра пересекаются не в общей вершине, это недопустимое пересечение для плоской укладки. Планарный граф — граф, который можно изобразить на плоскости без пересечений рёбер. Плоский граф, или плоская укладка, — конкретное изображение планарного графа без пересечений. Один и тот же планарный граф может иметь разные укладки. Грани — связные

области плоскости, на которые плоская укладка делит плоскость. Одна грань всегда внешняя, неограниченная; остальные грани внутренние. Для связного плоского графа с v вершинами, e рёбрами и f гранями выполняется формула Эйлера:

$$v - e + f = 2.$$

Для графа с c компонентами:

$$v - e + f = 1 + c.$$

Граница грани — замкнутый обход по рёбрам и вершинам, отделяющий грань от остальных частей плоскости. В 2-связных плоских графах границы граней являются циклами, но в общем случае граница может проходить по вершине или ребру несколько раз. В дереве, например, есть одна грань — внешняя, и каждое ребро входит в её границу дважды. Гомеоморфизм графов связан с разбиением рёбер. Разбить ребро uv означает заменить его путём $u-x-v$, добавив новую вершину степени 2. Графы называются гомеоморфными, если их можно получить из одного и того же графа последовательным разбиением рёбер. Иными словами, при гомеоморфизме важна «форма» графа, а вершины степени 2 на рёбрах могут появляться или удаляться. Два классических непланарных графа — K_5 и $K_{3,3}$. Граф K_5 имеет 5 вершин, каждая соединена с каждой, число рёбер равно

$$\|E(K_5)\| = \frac{5 \cdot 4}{2} = 10.$$

Если бы K_5 был планарным, для простого планарного графа при $n \geq 3$ выполнялось бы

$$m \leq 3n - 6.$$

Для $n = 5$ получаем $m \leq 9$, но у K_5 десять рёбер, значит он непланарен. Граф $K_{3,3}$ — полный двудольный граф с долями по 3 вершины. Он имеет 6 вершин и 9 рёбер. В двудольном графе нет циклов нечётной длины, значит в плоской укладке каждая грань имела бы длину не меньше 4. Тогда для простого двудольного планарного графа выполняется

$$m \leq 2n - 4.$$

Для $n = 6$ получаем $m \leq 8$, но у $K_{3,3}$ девять рёбер. Следовательно, $K_{3,3}$ непланарен. Теорема Понтрягина — Куратовского даёт полный критерий непланарности: конечный граф непланарен тогда и только тогда, когда он содержит подграф, гомеоморфный K_5 или $K_{3,3}$. То есть любая непланарность сводится к наличию внутри графа «скрытого» K_5 или «скрытого» $K_{3,3}$, где отдельные рёбра могли быть заменены путями. При проверке планарности важно понимать: один рисунок с пересечениями не доказывает непланарность. Нужно либо найти укладку без пересечений, либо доказать невозможность через теорему Куратовского, формулу Эйлера или оценки числа рёбер. Типовые необходимые условия планарности для простого связного графа при $n \geq 3$:

$$m \leq 3n - 6.$$

Если граф двудольный и $n \geq 3$, то

$$m \leq 2n - 4.$$

Эти условия не являются достаточными: граф может удовлетворять им и всё равно быть непланарным, но их нарушение сразу доказывает непланарность. # Билет 10. Теорема Эйлера, толщина графа и дуальные графы Планарный граф можно нарисовать на плоскости так, чтобы рёбра пересекались только в общих вершинах. Конкретная укладка такого графа называется плоским графом. Рёбра плоского графа разбивают плоскость на грани: внутренние ограниченные области и одну внешнюю неограниченную область. Для связного плоского графа с n вершинами, m рёбрами и f гранями выполняется теорема Эйлера:

$$n - m + f = 2.$$

Эквивалентные формы:

$$f = m - n + 2,$$

$$m = n + f - 2.$$

Идея доказательства: если граф содержит цикл, удалим ребро из цикла. Связность сохранится, а две грани сольются в одну, поэтому m и f уменьшатся на 1, а величина $n - m + f$ не изменится. Повторяя удаление, получим дерево. Для дерева $m = n - 1$, $f = 1$, значит

$$n - (n - 1) + 1 = 2.$$

Следовательно, формула верна и для исходного графа. Для несвязного плоского графа с c компонентами формула имеет вид

$$n - m + f = 1 + c.$$

Если граф состоит только из n изолированных вершин, то $m = 0$, $c = n$, $f = 1$, и действительно

$$n - 0 + 1 = 1 + n.$$

Сумма длин всех граней равна удвоенному числу рёбер:

$$\sum_F \ell(F) = 2m.$$

Каждое ребро имеет две стороны и при подсчёте границ граней учитывается дважды. Если ребро является мостом, оно может дважды входить в границу одной и той же грани. Из формулы Эйлера выводятся оценки числа рёбер. В простом планарном графе при $n \geq 3$ каждая грань имеет длину не меньше 3, поэтому

$$3f \leq 2m.$$

Совмещая с $f = 2 - n + m$, получаем

$$3(2 - n + m) \leq 2m,$$

$$m \leq 3n - 6.$$

Это необходимое условие планарности. Если простой граф имеет больше $3n - 6$ рёбер, он непланарен. Для простого двудольного планарного графа минимальная длина грани не меньше 4, так как нечётных циклов нет. Тогда

$$4f \leq 2m,$$

откуда

$$m \leq 2n - 4.$$

Эта оценка доказывает непланарность $K_{3,3}$: у него $n = 6$, $m = 9$, но $2n - 4 = 8$. Толщина графа $\theta(G)$ — минимальное число планарных подграфов, на которые можно разбить множество рёбер графа. Формально нужно представить

$$E(G) = E_1 \cup E_2 \cup \dots \cup E_k,$$

так, чтобы каждый граф $G_i = (V, E_i)$ был планарным. Минимальное такое k называется толщиной:

$$\theta(G) = \min k.$$

Если G планарен, то $\theta(G) = 1$. Если граф непланарен, его толщина больше 1. Толщина показывает, сколько «слоёв» нужно, чтобы нарисовать граф без пересечений в каждом слое. Это применимо в проектировании печатных плат, сетей и схем. Нижняя оценка толщины следует из планарной оценки числа рёбер. Если простой граф имеет n вершин и m рёбер, а каждый планарный слой содержит не более $3n - 6$ рёбер, то

$$\theta(G) \geq \left\lceil \frac{m}{3n - 6} \right\rceil.$$

Для двудольных слоёв можно использовать более сильную оценку $2n - 4$. Дуальный граф строится по плоской укладке планарного графа. Каждой грани исходного графа соответствует вершина дуального графа. Если две грани имеют общее ребро, между соответствующими вершинами дуального графа проводится ребро. Если одно и то же ребро граничит с одной гранью с двух сторон, в дуальном графе возникает петля. Если две грани имеют несколько общих рёбер, возникают кратные рёбра. Если G — связный плоский граф, то в его дуальном графе G^* :

$$\|V(G^*)\| = f(G), \quad \|E(G^*)\| = \|E(G)\|,$$

а грани G^* соответствуют вершинам G . Дуальность зависит от выбранной плоской укладки: один и тот же планарный граф может иметь разные дуальные графы. Дуальные графы важны для связи циклов и разрезов. Циклу в исходном графе часто соответствует разрез в дуальном графе, а разрезу в исходном — цикл в дуальном. Это используется в задачах потоков, электрических цепей, картографии и анализе планарных сетей. # Билет 11. Алгоритмы поиска кратчайшего пути в графе Пусть дан граф

$$G = (V, E),$$

где V — множество вершин, E — множество рёбер или дуг. Если граф взвешенный, каждому ребру (u, v) сопоставлен вес $w(u, v)$, например длина, время, стоимость или штраф. Задача кратчайшего пути состоит в поиске пути минимальной суммарной стоимости. Для пути P

$$\text{len}(P) = \sum_{(u,v) \in P} w(u, v).$$

В невзвешенном графе каждое ребро обычно имеет вес 1, поэтому кратчайший путь минимизирует число рёбер. Основные постановки: путь от s до t , пути от одного источника s до всех вершин, пути между всеми парами вершин. Если в графе есть отрицательный цикл, достижимый из источника, корректного конечного кратчайшего пути до некоторых вершин может не быть: проходя по циклу многократно, можно бесконечно уменьшать длину. Для восстановления пути почти все алгоритмы используют массив предков:

$$\text{parent}[v] = \text{вершина, из которой пришли в } v \text{ по лучшему пути.}$$

Инициализация:

$$d[s] = 0, \quad d[v] = +\infty \text{ для } v \neq s.$$

Если найдено улучшение через u , выполняется релаксация:

$$d[v] > d[u] + w(u, v) \Rightarrow d[v] = d[u] + w(u, v), \quad \text{parent}[v] = u.$$

BFS применяется в невзвешенных графах. Он просматривает вершины слоями: сначала вершины на расстоянии 0, затем на расстоянии 1, затем 2 и так далее. Поэтому первое посещение вершины даёт кратчайшее расстояние по числу рёбер. Сложность при списках смежности:

$$O(\|V\| + \|E\|).$$

Алгоритм Дейкстры применяется для графов с неотрицательными весами:

$$w(e) \geq 0.$$

Он поддерживает множество вершин с окончательно найденным расстоянием и каждый раз выбирает непосещённую вершину с минимальным текущим $d[v]$. После выбора вершины релаксируются все исходящие рёбра. Корректность основана на неотрицательности весов: если вершина выбрана как ближайшая, более короткий путь к ней уже не появится. С бинарной кучей сложность:

$$O((\|V\| + \|E\|) \log \|V\|).$$

Алгоритм Беллмана — Форда допускает отрицательные веса, но не допускает отрицательных циклов, достижимых из источника. Он выполняет $\|V\| - 1$ проходов по всем рёбрам, потому что простой кратчайший путь содержит не более $\|V\| - 1$ ребра. Затем делается дополнительный проход: если ещё возможна релаксация, существует отрицательный цикл. Сложность:

$$O(\|V\| \|E\|).$$

Для ориентированного ациклического графа DAG кратчайшие пути можно искать за линейное время после топологической сортировки. Вершины обрабатываются в топологическом порядке, и каждое ребро релаксируется один раз. Алгоритм допускает отрицательные веса, потому что циклов нет. Сложность:

$$O(\|V\| + \|E\|).$$

Алгоритм Флойда — Уоршелла находит кратчайшие пути между всеми парами вершин. Используется матрица расстояний $d[i][j]$. Переход:

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j]).$$

После обработки промежуточных вершин $1, \dots, k$ значение $d[i][j]$ равно длине кратчайшего пути из i в j , использующего только эти промежуточные вершины. Сложность:

$$O(\|V\|^3), \quad \text{память } O(\|V\|^2).$$

A^* используется, когда нужен путь от s к t и есть эвристика $h(v)$, оценивающая расстояние до цели. Алгоритм выбирает вершину по функции

$$f(v) = g(v) + h(v),$$

где $g(v)$ — известная стоимость пути от старта до v . Если эвристика допустима, то есть не переоценивает истинное расстояние до цели, A^* находит оптимальный путь. Алгоритм Джонсона решает задачу всех пар в разреженных графах с возможными отрицательными весами, но без отрицательных циклов. Сначала Беллман — Форд строит потенциалы, затем веса перенормируются так, чтобы стать неотрицательными, после чего из каждой вершины запускается Дейкстра. Для разреженных графов это может быть эффективнее Флойда — Уоршелла. Правило выбора алгоритма: невзвешенный граф — BFS; взвешенный граф без отрицательных весов — Дейкстра; есть отрицательные веса — Беллман — Форд или алгоритм для DAG; нужны все пары и граф небольшой — Флойд — Уоршелл; большой разреженный граф со всеми парами — Джонсон; есть хорошая эвристика и одна цель — A^* . # Билет 12. Раскраска вершин, рёбер и граней графов Раскраска графа — назначение цветов объектам графа так, чтобы конфликтующие объекты получали разные цвета. Различают вершинную раскраску, рёберную раскраску и раскраску граней плоского графа. Обычно рассматривают простые неориентированные графы: петля делает правильную вершинную раскраску невозможной, потому что вершина была бы смежна сама с собой. Правильная вершинная раскраска графа $G = (V, E)$ в k цветов — отображение

$$c : V \rightarrow \{1, 2, \dots, k\},$$

такое что для любого ребра $uv \in E$

$$c(u) \neq c(v).$$

Хроматическое число $\chi(G)$ — минимальное число цветов, достаточное для правильной вершинной раскраски:

$$\chi(G) = \min\{k : G \text{ имеет правильную раскраску в } k \text{ цветов}\}.$$

Для полного графа K_n :

$$\chi(K_n) = n,$$

потому что все вершины попарно смежны. Для дерева с хотя бы одним ребром $\chi(G) = 2$, так как дерево двудольно. Для чётного цикла C_{2m} хроматическое число равно 2, для нечётного цикла $C_{2m+1} = 3$. Нижние оценки хроматического числа. Если $\omega(G)$ — размер наибольшей клики, то

$$\chi(G) \geq \omega(G),$$

потому что все вершины клики должны иметь разные цвета. Если граф содержит нечётный цикл, то $\chi(G) \geq 3$. Если $\alpha(G)$ — размер наибольшего независимого множества, то

$$\chi(G) \geq \left\lceil \frac{\|V\|}{\alpha(G)} \right\rceil,$$

поскольку один цвет может покрывать только независимое множество. Верхняя оценка через максимальную степень:

$$\chi(G) \leq \Delta(G) + 1,$$

где

$$\Delta(G) = \max_{v \in V} \deg(v).$$

Она следует из жадной раскраски: у вершины не больше Δ соседей, значит среди $\Delta + 1$ цветов всегда найдётся свободный. Двудольные графы являются 2-раскрашиваемыми. Критерий: граф двудолен тогда и только тогда, когда не содержит нечётных циклов. Проверка выполняется BFS или DFS: стартовой вершине задаётся цвет 1, её соседям цвет 2, соседям соседей снова цвет 1.

Если обнаружено ребро между вершинами одного цвета, граф не двудолен. Жадный алгоритм раскраски обрабатывает вершины в некотором порядке и назначает каждой минимальный допустимый цвет, не использованный её уже раскрашенными соседями. Алгоритм прост и работает быстро, но результат зависит от порядка вершин и не всегда оптимален. Сложность при списках смежности обычно порядка

$$O(\|V\| + \|E\|)$$

при эффективной проверке цветов соседей. Рёберная раскраска назначает цвета рёбрам так, чтобы смежные рёбра, имеющие общую вершину, получали разные цвета. Правильная рёберная раскраска в k цветов — отображение

$$c : E \rightarrow \{1, 2, \dots, k\},$$

такое что если рёбра e_1 и e_2 имеют общую вершину, то

$$c(e_1) \neq c(e_2).$$

Хроматический индекс $\chi'(G)$ — минимальное число цветов для правильной рёберной раскраски. Нижняя оценка:

$$\chi'(G) \geq \Delta(G),$$

потому что все рёбра, инцидентные вершине максимальной степени, попарно конфликтуют. По теореме Визинга для простого графа

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1.$$

То есть любому простому графу достаточно либо Δ , либо $\Delta + 1$ цветов для рёбер. Раскраска граней применяется к плоским графам. Граням назначаются цвета так, чтобы соседние грани, имеющие общую границу-ребро, были разных цветов. Эта задача эквивалентна вершинной раскраске дуального графа: грани исходного графа становятся вершинами дуального, а соседство граней становится ребром. Теорема о четырёх красках утверждает, что всякий планарный граф можно раскрасить в 4 цвета:

$$\chi(G) \leq 4$$

для любого планарного графа. В терминах карт это означает: любую плоскую карту можно раскрасить четырьмя цветами так, чтобы соседние области имели разные цвета. Прикладной смысл раскраски: вершины могут обозначать экзамены, процессы или задачи, а ребро — конфликт; тогда цвет соответствует временному слоту или ресурсу. Рёберная раскраска моделирует распределение каналов, матчей или передач. Раскраска граней используется в картографии и планарных схемах. Главная цель во всех случаях — минимизировать число цветов, то есть число ресурсов.

Contents

Анализ данных и машинное обучение	1
Билет 1. Постановка задачи машинного обучения. Типы задач и их различия. Обучение с учителем и без учителя	2
Билет 2. Предварительная обработка данных. Методы обработки. Примеры	5
Билет 3. Визуализация данных. Примеры графиков. Библиотеки в Python	9
Билет 4. Корреляционный анализ. Коэффициент корреляции	13
Билет 5. Регрессионный анализ. Метод наименьших квадратов. Метрики качества регрессии	15
Билет 6. Постановка задачи классификации. Примеры. Метрики качества классификации. ROC-кривая	20
Билет 7. Обучающая и валидационная выборки. Этапы решения задачи классификации	24
Билет 8. Постановка задачи кластеризации. Примеры. Метрики качества кластеризации	28
Билет 9. Методы решения задачи кластеризации. Иерархические методы. Алгоритм k-means	32
Билет 10. Деревья решений. Обучение дерева решений	36
Билет 11. Алгоритм k-ближайших соседей	40
Билет 12. Переобучение и недообучение. Способы обнаружения и предотвращения	44
Билет 13. Нейронные сети. Принцип работы. Основные определения. Однослойные и многослойные нейронные сети	47
Билет 14. Алгоритм обратного распространения ошибки	50
Билет 15. Сферы применения нейронных сетей. Примеры. Виды нейронных сетей	54
Билет 16. Свёрточные нейронные сети	57
Билет 17. Постановка задачи анализа временных рядов. Основные методы решения	60
Билет 18. Ансамбли моделей. Основные понятия: бэггинг, бустинг, стэкинг. Примеры алгоритмов	64
Билет 19. Дисбаланс классов в задачах классификации. Причины возникновения проблем. Методы борьбы с дисбалансом	67
Билет 20. Обработка естественного языка (NLP). Основные задачи: классификация текстов и генерация	71

Анализ данных и машинное обучение

Файл собран из загруженных билетов. Версия не является агрессивной выжимкой: сохранены определения, формулы, этапы решения, метрики и практические примеры. Убраны только явные повторы между билетами и приведена единая структура оформления.

Содержание

1. Постановка задачи машинного обучения. Типы задач и их различия. Обучение с учителем и без учителя
2. Предварительная обработка данных. Методы обработки. Примеры
3. Визуализация данных. Примеры графиков. Библиотеки в Python
4. Корреляционный анализ. Коэффициент корреляции

5. Регрессионный анализ. Метод наименьших квадратов. Метрики качества регрессии
6. Постановка задачи классификации. Примеры. Метрики качества классификации. ROC-кривая
7. Обучающая и валидационная выборки. Этапы решения задачи классификации
8. Постановка задачи кластеризации. Примеры. Метрики качества кластеризации
9. Методы решения задачи кластеризации. Иерархические методы. Алгоритм k-means
10. Деревья решений. Обучение дерева решений
11. Алгоритм k-ближайших соседей
12. Переобучение и недообучение. Способы обнаружения и предотвращения
13. Нейронные сети. Принцип работы. Основные определения. Однослойные и многослойные нейронные сети
14. Алгоритм обратного распространения ошибки
15. Сферы применения нейронных сетей. Примеры. Виды нейронных сетей
16. Свёрточные нейронные сети
17. Постановка задачи анализа временных рядов. Основные методы решения
18. Ансамбли моделей. Основные понятия: бэггинг, бустинг, стэкинг. Примеры алгоритмов
19. Дисбаланс классов в задачах классификации. Причины возникновения проблем. Методы борьбы с дисбалансом
20. Обработка естественного языка (NLP). Основные задачи: классификация текстов и генерация

Билет 1. Постановка задачи машинного обучения. Типы задач и их различия. Обучение с учителем и без учителя

Основные понятия и определения

Машинное обучение - это область искусственного интеллекта, в которой алгоритм строит правило решения задачи по данным. В отличие от обычной программы, где разработчик вручную задает все правила, модель машинного обучения настраивает параметры на примерах и затем применяет найденную закономерность к новым объектам.

Базовая единица данных называется объектом. Объектом может быть письмо, клиент банка, фотография, медицинская запись, транзакция или строка в таблице. Каждый объект описывается признаками: числовыми, категориальными, текстовыми, графическими или другими измеримыми характеристиками. Признаковое описание объекта обычно обозначают x , а множество всех возможных описаний - X .

Если для объекта известен правильный ответ, его обозначают y , а множество возможных ответов - Y . Обучающая выборка в задаче с учителем записывается так:

$$D = \{(x_i, y_i)\}_{i=1}^n,$$

где n - число размеченных примеров. Цель состоит в построении алгоритма $a : X \rightarrow Y$, который хорошо предсказывает ответы не только на обучающих объектах, но и на новых данных. Это свойство называют обобщающей способностью.

Следует различать модель, алгоритм обучения и обученный алгоритм. Модель задает семейство возможных функций, например линейные модели, деревья решений или нейронные сети. Алгоритм обучения выбирает конкретные параметры модели по данным. Обученный алгоритм - это уже готовое правило, применяемое на практике.

Главное деление задач машинного обучения связано с наличием правильных ответов. В обучении с учителем каждому обучающему объекту соответствует метка или числовое значение. В обучении без учителя меток нет: алгоритм должен найти структуру в самих данных. Также существуют промежуточные постановки, например полубучение и обучение с подкреплением, но базовое различие проходит именно между размеченными и неразмеченными данными.

Теория, формулы и интуиция

Формальная постановка задачи включает пространство объектов X , пространство ответов Y , обучающую выборку, семейство моделей и критерий качества. В реальности существует неизвестная зависимость между объектами и ответами. Модель пытается приблизить эту зависимость по ограниченному набору примеров.

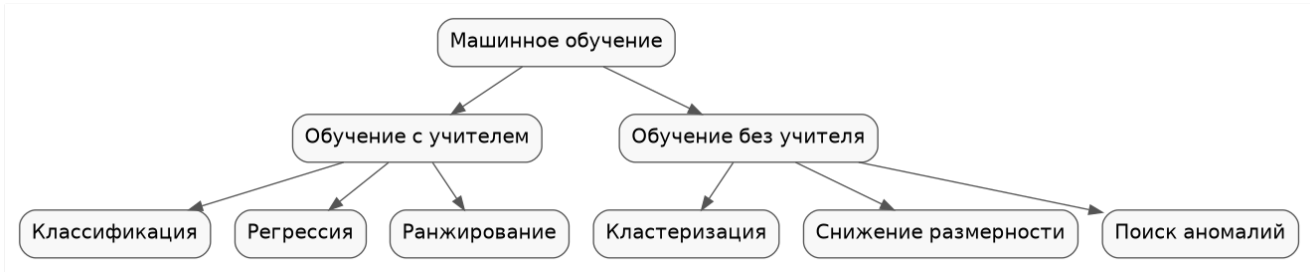


Figure 1: Схема 1

Качество модели в обучении с учителем задают через функцию потерь $L(a(x), y)$. Она показывает, насколько предсказание отличается от правильного ответа. Идеальная цель - минимизировать ожидаемый риск:

$$R(a) = \mathbb{E}_{(x,y) \sim P} L(a(x), y),$$

где P - неизвестное распределение данных. Так как распределение неизвестно, на практике минимизируют эмпирический риск на обучающей выборке:

$$\hat{R}(a) = \frac{1}{n} \sum_{i=1}^n L(a(x_i), y_i).$$

Интуитивно это означает: выбирается такая модель, которая меньше ошибается на известных примерах. Но минимальная ошибка на обучении не гарантирует хорошую работу в будущем. Если модель слишком сложная, она может запомнить шум и случайные особенности выборки. Это переобучение. Если модель слишком простая и не улавливает закономерность, возникает недообучение. Поэтому важна проверка на отложенных данных и контроль сложности модели. Часто к функции потерь добавляют регуляризацию:

$$\frac{1}{n} \sum_{i=1}^n L(a(x_i), y_i) + \lambda \Omega(a),$$

где $\Omega(a)$ - штраф за сложность, а λ - параметр силы штрафа. Регуляризация помогает модели не подстраиваться чрезмерно под отдельные примеры.

В классификации ответ принадлежит конечному числу классов. Например, письмо является спамом или не спамом, пациент относится к группе риска или не относится, изображение содержит один из нескольких объектов. Если классов два, задача бинарная; если больше двух - многоклассовая. Метриками могут быть accuracy, precision, recall, F1-мера, ROC-AUC и log loss. Выбор метрики зависит от цены ошибок: в медицинской диагностике пропустить болезнь и ошибочно направить здорового человека на проверку - разные ситуации.

В регрессии ответ является числом. Примеры: цена квартиры, спрос на товар, температура, время доставки. Типичные метрики:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad MAE = \frac{1}{n} \sum_{i=1}^n \|y_i - \hat{y}_i\|.$$

MSE сильнее штрафует большие ошибки, потому что отклонение возводится в квадрат. MAE устойчивее к выбросам, так как ошибка растет линейно.

В ранжировании важно упорядочить объекты. Поисковая система должна поставить более релевантные документы выше, а рекомендательная система - расположить товары или фильмы в полезном для пользователя порядке. Здесь оценивают не только правильность ответа, но и позицию объекта в списке, используя, например, NDCG, MAP или MRR.

В обучении без учителя выборка имеет вид

$$D = \{x_i\}_{i=1}^n.$$

Правильных ответов нет, поэтому нельзя напрямую посчитать ошибку предсказания. Задача состоит в обнаружении структуры: групп похожих объектов, скрытых факторов, компактного представления или необычных наблюдений. Поэтому оценка результата часто зависит от предметного смысла и дальнейшего применения.

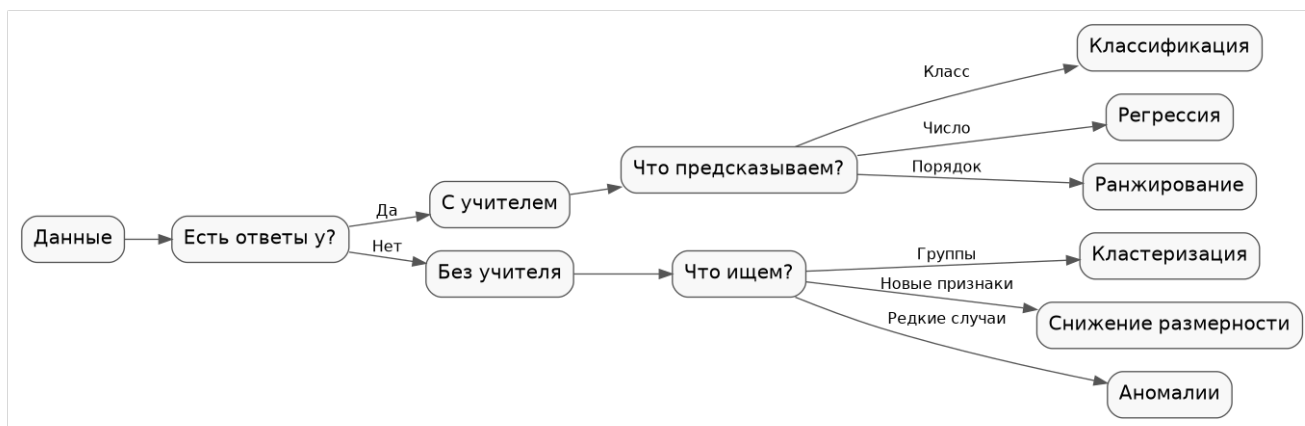


Figure 2: Схема 2

Методы, алгоритмы и этапы решения

Работа над задачей начинается с постановки цели. Нужно определить, какое решение будет приниматься по результату модели, какие данные доступны, что считается ошибкой и как измеряется успех. Формулировка “улучшить продажи” слишком общая; корректнее поставить задачу как прогноз вероятности покупки, выбор набора рекомендаций или сегментацию клиентов.

Затем собирают и готовят данные. Пропуски, дубликаты, ошибки ввода и разные единицы измерения могут испортить обучение. Категориальные признаки кодируют, числовые признаки иногда масштабируют, тексты переводят в векторное представление, изображения - в массивы пикселей или признаки нейронной сети. Важное требование - не допустить утечки данных, когда в признаки попадает информация, недоступная в момент реального предсказания.

В обучении с учителем данные обычно делят на обучающую, валидационную и тестовую части. На обучающей выборке модель настраивает параметры. На валидационной выбирают гиперпараметры и сравнивают варианты. Тестовая выборка нужна для финальной оценки. Если данных мало, применяют кросс-валидацию.

К основным алгоритмам классификации и регрессии относятся линейные модели, метод ближайших соседей, деревья решений, случайный лес, градиентный бустинг, метод опорных векторов и нейронные сети. Линейная регрессия использует зависимость

$$\hat{y} = w_0 + w_1 x_1 + \dots + w_m x_m.$$

Логистическая регрессия применяется в классификации и оценивает вероятность класса:

$$P(y = 1 | x) = \frac{1}{1 + e^{-w^T x}}.$$

Деревья решений строят цепочки правил по признакам. Они понятны и интерпретируемы, но отдельное дерево легко переобучается. Случайный лес усредняет много деревьев и уменьшает нестабильность. Градиентный бустинг строит ансамбль последовательно: каждая новая модель исправляет ошибки предыдущих. На табличных данных это один из самых сильных подходов.

Метод ближайших соседей использует идею, что похожие объекты имеют похожие ответы. Для нового объекта ищут k ближайших обучающих примеров и агрегируют их ответы. Метод прост, но зависит от выбора метрики расстояния и масштаба признаков.

Для задач без учителя применяют другие алгоритмы. К-средних разбивает объекты на k кластеров, поочередно назначая объекты ближайшему центру и пересчитывая центры. Иерархическая

кластеризация строит дерево вложенных групп. DBSCAN выделяет области высокой плотности и может находить шумовые объекты.

Снижение размерности часто выполняют методом главных компонент, PCA. Он ищет новые оси, вдоль которых разброс данных максимален, и проецирует объекты на меньшее число признаков. PCA полезен для визуализации, сжатия и удаления шума. Для нелинейной визуализации используют t-SNE и UMAP, но их двумерные картинки требуют осторожной интерпретации.

После обучения модель оценивают, интерпретируют и внедряют. На практике работа не заканчивается внедрением: распределение данных может измениться. Поведение пользователей, рыночные цены, мошеннические схемы и язык текстов со временем сдвигаются, поэтому качество модели нужно мониторить и периодически обновлять.

Практические примеры

Фильтрация спама - пример классификации. Объектом является письмо, признаками служат слова, отправитель, ссылки, вложения и технические заголовки. Ответ имеет два класса: спам или не спам. Здесь важно учитывать разные цены ошибок: пропущенный спам неприятен, но ошибочно скрытое важное письмо может быть серьезнее.

Прогноз цены квартиры - пример регрессии. Признаками являются площадь, район, этаж, год постройки, расстояние до метро и состояние дома. Ответ - цена. Простая линейная модель дает понятную зависимость, а градиентный бустинг лучше учитывает сложные нелинейные эффекты.

Поисковая выдача - пример ранжирования. Для пары “запрос - документ” модель оценивает релевантность, но главная цель - поставить лучшие документы выше. Ошибка на первой позиции обычно важнее ошибки внизу списка.

Сегментация клиентов интернет-магазина - пример кластеризации. У клиентов есть частота покупок, средний чек, любимые категории и реакция на скидки, но заранее неизвестно, какие группы существуют. Алгоритм может выделить постоянных клиентов, редких покупателей, любителей акций и покупателей дорогих товаров. Ценность результата определяется тем, можно ли использовать эти группы в маркетинге.

Поиск подозрительных банковских операций - пример обнаружения аномалий. Большинство транзакций похоже на обычное поведение клиента, но некоторые выделяются суммой, географией, временем или устройством. Так как размеченных мошеннических случаев мало, методы без учителя помогают найти необычные операции для последующей проверки.

Итак, постановка задачи машинного обучения требует определить объекты, признаки, ответы, метрику качества, ограничения и способ применения результата. Обучение с учителем решает задачи предсказания по размеченным примерам. Обучение без учителя применяется, когда правильных ответов нет и нужно обнаружить структуру данных. От этой разницы зависят выбор алгоритмов, метрик, процедура проверки и интерпретация результата.

Билет 2. Предварительная обработка данных. Методы обработки. Примеры

Основные понятия и определения

Предварительная обработка данных — это совокупность действий, которые переводят исходные данные в форму, пригодную для анализа, построения моделей машинного обучения, визуализации или принятия решений. В реальных задачах данные редко бывают сразу готовы к использованию: в них встречаются пропуски, дубликаты, выбросы, разные форматы записи, ошибки ввода, несогласованные единицы измерения, несбалансированные классы и признаки с разными масштабами. Поэтому качество модели часто определяется не только выбранным алгоритмом, но и подготовкой данных.

Исходные данные — это данные в том виде, в котором они получены из базы данных, файла, API, датчиков, анкет, журналов событий или другой системы. Такие данные могут содержать как полезный сигнал, так и шум. Шумом называют случайные или систематические искажения, которые затрудняют выделение закономерностей: например, опечатки в категориях, некорректные значения возраста, резкие скачки показаний датчика.

Пропущенное значение — это отсутствие значения признака у объекта. Пропуск может возникать случайно, например из-за технического сбоя, или неслучайно, например когда клиент не указывает доход именно в определенной группе заявок. Важно понимать причину пропуска: иногда его можно восстановить, иногда нужно удалить объект, а иногда сам факт отсутствия значения является информативным признаком.

Выброс — это наблюдение, заметно отличающееся от основной массы данных. Выбросы бывают ошибочными, например цена товара 999999 из-за сбоя ввода, и реальными, например очень крупная транзакция в банковских данных. Удалять выбросы автоматически нельзя: сначала нужно понять предметную область и последствия такого решения.

Признак — измеряемое свойство объекта: возраст пользователя, сумма покупки, температура, категория товара, текст отзыва. Целевая переменная — величина, которую нужно предсказать или объяснить: класс письма, цена квартиры, вероятность отказа клиента. Матрица признаков обычно обозначается X , а вектор ответов — y .

Кодирование признаков — преобразование категориальных или текстовых значений в числовой вид. Масштабирование — изменение числовых признаков так, чтобы они были сопоставимы по диапазону или распределению. Нормализация и стандартизация являются частными способами масштабирования.

Утечка данных — ситуация, когда при подготовке данных в обучающую выборку попадает информация, которая в реальном применении модели была бы неизвестна. Например, если среднее значение для заполнения пропусков посчитать по всей таблице до разделения на обучение и тест, то тестовая часть косвенно повлияет на обучение. Это приводит к завышенной оценке качества модели.

Теория, формулы и интуиция

Главная идея предварительной обработки состоит в том, чтобы сделать данные согласованными, численно устойчивыми и содержательно корректными. Алгоритмы анализа данных работают не с «смыслом» таблицы напрямую, а с ее числовым представлением. Если это представление искажено, модель будет находить закономерности в ошибках, технических особенностях сбора или случайном шуме.

Одно из базовых преобразований — стандартизация числового признака. Для значения x она задается формулой:

$$z = \frac{x - \mu}{\sigma},$$

где μ — среднее значение признака, а σ — стандартное отклонение. После стандартизации признак имеет среднее около нуля и стандартное отклонение около единицы. Это важно для методов, чувствительных к масштабу: линейной и логистической регрессии с регуляризацией, метода ближайших соседей, метода опорных векторов, нейронных сетей, кластеризации на основе расстояний.

Другой распространенный способ — min-max нормализация:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Она переводит значения в диапазон от 0 до 1. Такой подход удобен, когда нужно сохранить форму распределения, но привести признаки к единому диапазону. Недостаток состоит в чувствительности к выбросам: одно экстремальное значение может сильно сжать остальные наблюдения.

Для обнаружения выбросов часто используют межквартильный размах:

$$IQR = Q_3 - Q_1.$$

Значения ниже $Q_1 - 1.5 * IQR$ или выше $Q_3 + 1.5 * IQR$ считают потенциальными выбросами. Этот критерий устойчивее среднего и стандартного отклонения, потому что основан на квартилях и меньше зависит от экстремальных значений.

Заполнение пропусков можно рассматривать как оценивание неизвестных значений. Самый простой вариант — заменить пропуски средним, медианой или модой. Для числовых признаков медиана часто надежнее среднего, если распределение асимметрично или содержит выбросы. Для категориальных признаков используют наиболее частую категорию или отдельную категорию «неизвестно». Более сложные методы восстанавливают пропуски по соседним объектам, регрессионным моделям или итеративным алгоритмам.

Интуитивно предварительная обработка похожа на настройку измерительного прибора перед экспериментом. Если шкалы несогласованы, часть показаний пропала, а часть записана в разных единицах, то даже хороший метод анализа даст сомнительный результат. Обработка не должна искусственно подгонять данные под желаемый ответ. Ее задача — убрать технические и методические помехи, сохранив реальную структуру явления.

Важное теоретическое требование — разделять операции, которые можно обучать на данных, и операции, которые применяются к данным. Например, параметры стандартизации, словарь категорий, медианы для заполнения пропусков и границы выбросов нужно вычислять только на обучающей выборке. Затем эти же параметры применяются к валидационной, тестовой и будущей рабочей выборке. Такой подход сохраняет честность оценки качества.

Методы, алгоритмы и этапы решения

Первый этап — сбор и первичный осмотр данных. На этом шаге выясняют размер таблицы, типы признаков, диапазоны значений, долю пропусков, количество уникальных категорий, наличие дубликатов и очевидных ошибок. Для табличных данных полезны описательные статистики: среднее, медиана, минимум, максимум, стандартное отклонение, квантили. Для категориальных признаков смотрят частоты категорий. Для временных рядов проверяют равномерность временного шага и наличие пропущенных периодов.

Второй этап — очистка данных. Сюда относится удаление полных дубликатов, исправление некорректных форматов, приведение регистра строк, унификация названий категорий и единиц измерения. Например, значения Moscow, Москва и moscow могут обозначать один город, но для алгоритма это разные категории. Очистка особенно важна при объединении нескольких источников.

Третий этап — обработка пропусков. Возможны несколько стратегий. Если пропусков мало и они случайны, строки или столбцы иногда удаляют. Если признак важен, пропуски заполняют статистикой: средним, медианой, модой, константой или отдельной категорией. В задачах с временными рядами используют прямое заполнение предыдущим значением, интерполяцию или сглаживание. В машинном обучении также добавляют бинарный индикатор пропуска: он показывает, было ли исходное значение известно.

Четвертый этап — работа с выбросами. Сначала выбросы обнаруживают: с помощью диаграмм размаха, правил на основе квартилей, z-оценки, предметных ограничений или алгоритмов аномалий. Затем выбирают действие: исправить очевидную ошибку, удалить наблюдение, ограничить значение сверху или снизу, заменить на квантиль, логарифмировать признак или оставить как есть. Решение зависит от смысла признака. Например, отрицательная цена товара почти наверняка ошибка, а высокая сумма заказа может быть важным бизнес-сигналом.

Пятый этап — преобразование признаков. Числовые признаки масштабируют, логарифмируют, дискретизируют или строят на их основе новые признаки. Логарифмирование полезно для положительных величин с длинным правым хвостом: доход, цена, число просмотров. Дискретизация переводит непрерывную величину в интервалы, например возраст в группы. Генерация признаков может включать отношения величин, разности дат, агрегаты по пользователю, скользящие средние, частоты событий.

Шестой этап — кодирование категориальных признаков. При one-hot encoding для каждой категории создается отдельный бинарный столбец. Такой метод прост и хорошо работает при небольшом числе категорий. Для порядковых категорий применяют ordinal encoding, если порядок действительно имеет смысл: например низкий, средний, высокий. Для признаков с большим числом категорий используют частотное кодирование, target encoding или специальные методы, но target encoding требует особенно аккуратной валидации, чтобы не допустить утечки целевой переменной.

Седьмой этап — обработка текстов, изображений или временных данных, если они присутствуют. Для текста применяют токенизацию, приведение к нижнему регистру, удаление лишних символов, лемматизацию, стемминг, удаление стоп-слов, построение TF-IDF или эмбедингов. Для изображений используют изменение размера, нормализацию пикселей, выравнивание каналов, аугментации. Для временных рядов строят лаговые признаки, сезонные признаки, скользящие средние и признаки календаря.

Восьмой этап — балансировка выборки. В классификации классы часто представлены неравномерно: например мошеннических транзакций намного меньше обычных. Если обучать модель напрямую, она может почти всегда выбирать большинство. Методы решения включают взвешивание классов, under-sampling большинства, oversampling меньшинства и синтетическое порождение объектов, например SMOTE. При этом качество нужно оценивать подходящими метриками: precision, recall, F1, ROC-AUC или PR-AUC, а не только accuracy.

Девятый этап — разделение данных и построение конвейера обработки. Хорошая практика — описывать все преобразования как pipeline: сначала обучение параметров обработки на train, затем применение тех же шагов к validation, test и новым данным. Это делает эксперимент воспроизводимым и снижает риск случайной утечки. Для временных рядов нельзя случайно перемешивать прошлое и будущее: обучение должно происходить на более ранних периодах, а

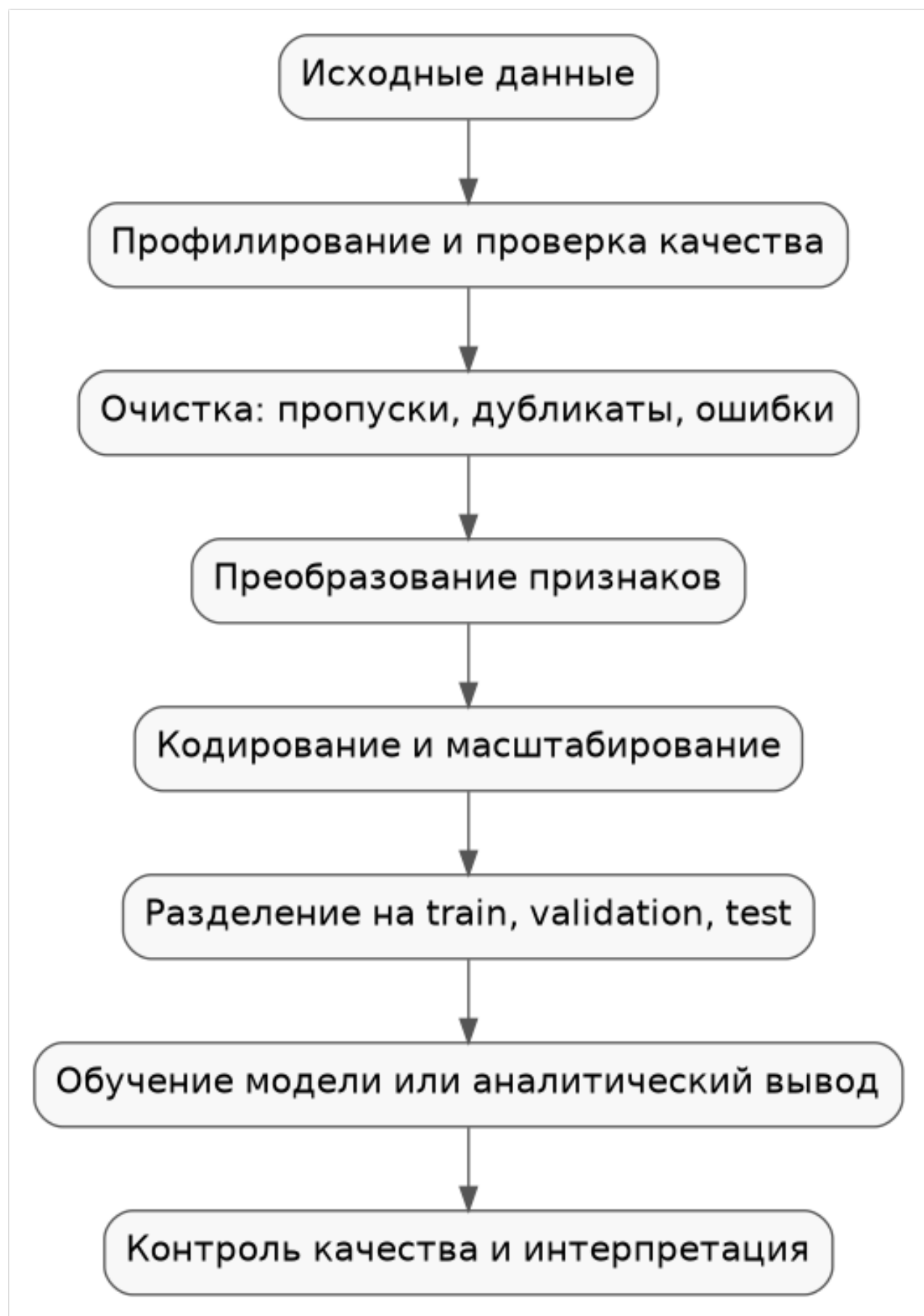


Figure 3: Схема 3

проверка — на более поздних.

Практические примеры

Рассмотрим задачу прогнозирования цены квартиры. В исходной таблице есть площадь, число комнат, этаж, район, год постройки и цена. На первом этапе проверяют, нет ли квартир с нулевой или отрицательной площадью, слишком большим числом комнат, пропусками в районе. Дубликаты объявлений удаляют, потому что они могут зависеть влияние повторяющихся объектов. Пропуски в годе постройки можно заполнить медианой по району или добавить категорию «год неизвестен», если отсутствие информации само по себе влияет на цену.

Числовые признаки цены и площади часто имеют асимметричные распределения, поэтому для них применяют логарифмирование. Район кодируют one-hot encoding, потому что это категориальный признак без естественного порядка. Этаж можно преобразовать в несколько признаков: первый этаж, последний этаж, относительный этаж в доме. После этого признаки стандартизируют, если используется линейная регрессия, метод ближайших соседей или нейронная сеть. Все параметры преобразований считают только на обучающей части данных.

Другой пример — классификация писем на спам и не спам. Исходные письма содержат HTML-разметку, разные кодировки, ссылки, повторяющиеся пробелы и служебные заголовки. Предобработка включает удаление или разбор HTML, приведение текста к единому регистру, токенизацию, нормализацию слов и построение признаков. Простая модель может использовать TF-IDF: каждому слову или n-грамме назначается вес, который растет при частом появлении в конкретном письме и уменьшается при частом появлении во всей коллекции. Если классы несбалансированы, например спама меньше, применяют веса классов или меняют порог принятия решения.

Третий пример — анализ данных интернет-магазина. В таблице заказов могут быть отрицательные количества, отмененные заказы, разные валюты, пропуски в идентификаторе клиента и товары с разными вариантами написания. Сначала приводят валюты к единой базе, отделяют отмены от успешных покупок, нормализуют названия товаров и проверяют даты. Затем строят агрегаты: число заказов на клиента, средний чек, давность последней покупки, частота покупок, доля возвратов. Эти признаки уже лучше отражают поведение клиента, чем отдельные строки заказов.

Для временного ряда продаж предобработка имеет свои особенности. Нельзя просто случайно удалить строки или перемешать данные. Нужно восстановить календарную сетку: если магазин работает ежедневно, каждый день должен присутствовать в ряду, даже если продаж не было. Пропущенные дни можно заполнить нулями или интерполировать, в зависимости от смысла отсутствия записи. Затем добавляют признаки дня недели, месяца, праздников, лаги продаж за предыдущие дни и скользящие средние. Проверка модели выполняется на будущих периодах, а не на случайной части данных.

В медицинских данных предварительная обработка требует особенно осторожной интерпретации. Например, пропуск анализа может означать не техническую ошибку, а то, что врач не назначил исследование из-за отсутствия симптомов. Удаление таких пропусков может исказить выборку. Поэтому часто добавляют индикаторы факта измерения и консультируются с предметными специалистами. Также важно сохранять воспроизводимость: фиксировать правила фильтрации, версии данных, параметры заполнения и причины исключения наблюдений.

Итог предварительной обработки — не просто «чистая таблица», а воспроизводимый процесс перехода от сырых данных к аналитическому набору признаков. Хорошая обработка делает модель устойчивее, метрики честнее, а выводы понятнее. Она не заменяет понимание предметной области, но помогает методам машинного обучения работать с реальной структурой данных, а не с ошибками их записи.

Билет 3. Визуализация данных. Примеры графиков. Библиотеки в Python

Основные понятия и определения

Визуализация данных — это представление числовой, категориальной, временной или пространственной информации в графической форме. Ее цель не просто «сделать красиво», а помочь увидеть структуру данных: распределения, зависимости, выбросы, тренды, группы, аномалии и сравнения. Хороший график сокращает время понимания данных и делает выводы проверяемыми: зритель может сопоставить подписи, шкалы, легенду и форму графика с исходным смыслом признаков. Данные обычно состоят из наблюдений и признаков. Наблюдение — отдельная строка таблицы, например студент, сделка, измерение температуры за час. Признак — переменная, описывающая наблюдение: возраст, цена, дата, категория товара, координаты. Признаки бывают количественными,

категориальными, порядковыми, временными и географическими. От типа признака зависит выбор графика: для количественного распределения подходит гистограмма, для сравнения категорий — столбчатая диаграмма, для динамики во времени — линейный график.

График строится из нескольких элементов. Оси задают шкалы и единицы измерения. Геометрические элементы отображают данные: точки, линии, столбцы, области, прямоугольники, цветовые ячейки. Визуальные каналы — это свойства, которыми кодируются значения: положение, длина, площадь, цвет, оттенок, форма, размер, прозрачность. Подписи, заголовок, легенда и сетка помогают читать изображение, но не должны перегружать его. Важный принцип: один график должен отвечать на понятный аналитический вопрос.

Различают разведочную и объясняющую визуализацию. Разведочная визуализация используется исследователем при анализе: строится много графиков, проверяются гипотезы, ищутся неожиданные зависимости. Объясняющая визуализация предназначена для передачи уже найденного вывода аудитории: она должна быть особенно аккуратной, с ясным акцентом, минимальным шумом и корректным контекстом. Например, при разведке можно построить десятки диаграмм рассеяния между признаками, а в отчете оставить одну, где явно видно влияние площади квартиры на цену.

Теория, формулы и интуиция

Основа визуализации — отображение данных в визуальные координаты. Если есть числовой признак x , то график переводит его значение в положение на оси. Обычно используется линейная шкала:

$$x_{\text{screen}} = a * x + b$$

Коэффициенты a и b подбираются так, чтобы минимальное и максимальное значения данных попали в границы области построения. Если данные сильно различаются по порядку величины, применяют логарифмическую шкалу:

$$x_{\text{screen}} = a * \log(x) + b$$

Логарифмическая шкала полезна, когда важны относительные изменения: рост с 10 до 100 воспринимается так же, как рост со 100 до 1000, потому что оба изменения равны увеличению в 10 раз. Но такую шкалу нужно явно подписывать, иначе график может ввести в заблуждение.

Для гистограммы важна группировка числовой переменной по интервалам. Пусть диапазон значений разбит на k корзин ширины h . Тогда высота столбца показывает число наблюдений в интервале или плотность. При подсчете частот:

$$\text{count}_j = \text{число } x_i, \text{ попавших в интервал } j$$

Если нужна плотность вероятности, высоты нормируют так, чтобы суммарная площадь столбцов была равна единице:

$$\text{density}_j = \text{count}_j / (n * h)$$

Интуитивно гистограмма показывает форму распределения: симметричность, скошенность, несколько пиков, редкие экстремальные значения. Однако вид гистограммы зависит от числа корзин. Слишком мало корзин скрывает детали, слишком много создает шум.

Диаграмма рассеяния показывает связь двух числовых признаков. Каждая точка имеет координаты (x_i, y_i) . Если точки вытянуты вдоль возрастающей прямой, связь положительная; если вдоль убывающей — отрицательная. Формально линейная связь часто оценивается коэффициентом корреляции Пирсона:

$$r = \text{cov}(X, Y) / (\text{std}(X) * \text{std}(Y))$$

Значение r близкое к 1 означает сильную положительную линейную связь, близкое к -1 — сильную отрицательную, около 0 — отсутствие именно линейной связи. Но график часто важнее одного числа: зависимость может быть нелинейной, содержать кластеры или выбросы, которые корреляция скрывает.

Временной ряд обычно отображается линией, потому что соседние точки упорядочены по времени и между ними есть естественная последовательность. Линия помогает увидеть тренд, сезонность и резкие изменения. При этом не следует соединять линией несвязанные категории: линия визуально обещает непрерывный переход, которого в данных может не быть.

Цвет в визуализации требует осторожности. Для категорий используют качественные палитры с различимыми цветами. Для числовых величин — последовательные палитры от светлого к темному или расходящиеся палитры, если есть важный ноль или среднее значение. Нельзя кодировать слишком много категорий цветом: легенда станет трудной, а график потеряет читаемость. Также важно учитывать дальтонизм и не полагаться только на различие красного и зеленого.

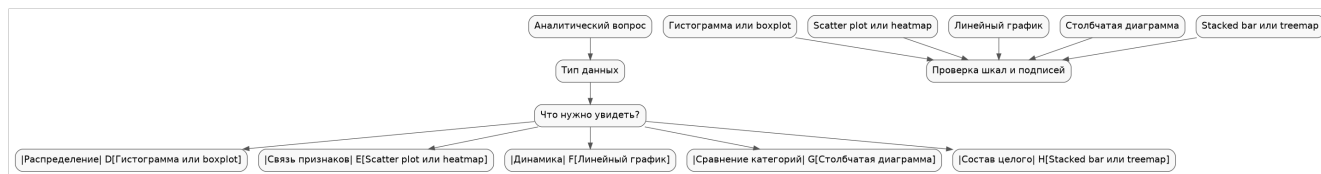


Figure 4: Схема 4

Методы, алгоритмы и этапы решения

Процесс построения визуализации начинается не с выбора библиотеки, а с постановки вопроса. Например: «Какие товары дают основную выручку?», «Есть ли зависимость между опытом и зарплатой?», «Когда происходят пики нагрузки?». Без вопроса легко получить набор декоративных картинок, которые не дают вывода.

Второй этап — подготовка данных. Нужно проверить типы столбцов, пропуски, дубликаты, выбросы, единицы измерения и формат дат. Часто данные агрегируют: считают сумму продаж по месяцам, средний чек по сегментам, количество обращений по дням. Для категорий с большим числом значений выбирают топ-группы, а остальные объединяют в «прочие», иначе график становится нечитаемым.

Третий этап — выбор типа графика. Для распределения одной числовой переменной используют гистограмму, график плотности, boxplot или violin plot. Boxplot показывает медиану, квартили и выбросы. Он компактен и удобен для сравнения распределений по группам. Для связи двух числовых признаков используют диаграмму рассеяния; если точек слишком много, применяют прозрачность, hexbin-график или двумерную тепловую карту. Для матрицы корреляций подходит heatmap. Для категориальных сравнений чаще всего лучше столбчатая диаграмма, причем категории полезно сортировать по значению.

Четвертый этап — настройка визуального кодирования. Наиболее точный канал восприятия — положение на общей шкале, поэтому линейные и точечные графики обычно читаются лучше, чем круговые диаграммы. Длина воспринимается достаточно хорошо, площадь и объем — хуже. Поэтому пузырьковые диаграммы требуют осторожности: зритель может неверно оценить различия по площади. Если нужно показать доли, часто лучше использовать нормированную столбчатую диаграмму, чем круговую, особенно при большом числе категорий.

Пятый этап — оформление. У графика должны быть понятный заголовок, подписи осей с единицами измерения, читаемые деления шкал и легенда, если используется цвет или форма. Нулевую линию следует показывать там, где она важна для интерпретации. Для столбчатых диаграмм ось значений обычно должна начинаться с нуля, иначе разница высот будет визуально преувеличена. Для линейных графиков допустимо сужать диапазон оси, если это явно оправдано и не скрывает контекст.

Шестой этап — проверка вывода. Нужно спросить: «Можно ли по этому графику сделать тот вывод, ради которого он построен? Не искажает ли масштаб смысл? Не скрыты ли важные группы?». Хорошая практика — строить несколько альтернативных графиков для одной задачи и выбирать наиболее честный и простой. В аналитической работе визуализация часто идет вместе со статистическими расчетами: график показывает структуру, а числа подтверждают силу эффекта.

В Python типичный конвейер выглядит так: загрузить данные в pandas, очистить и агрегировать таблицу, построить график средствами matplotlib, seaborn, plotly или другой библиотеки, затем сохранить изображение или встроить его в отчет. Для воспроизводимости код построения графиков хранят вместе с анализом, например в Jupyter Notebook или Python-скрипте.

Практические примеры

Базовая библиотека визуализации в Python — matplotlib. Она низкоуровневая, гибкая и лежит в основе многих других инструментов. С ее помощью можно управлять почти всеми элементами фигуры: осями, шрифтами, сеткой, подписями, цветами, несколькими областями построения. Пример линейного графика:

```
import matplotlib.pyplot as plt
```

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]
revenue = [120, 135, 128, 160, 190]
```

```
plt.figure(figsize=(7, 4))
plt.plot(months, revenue, marker="o")
plt.title("Выручка по месяцам")
```



```
plt.xlabel("Месяц")
plt.ylabel("Выручка, тыс. руб.")
plt.grid(True, alpha=0.3)
plt.show()
```

Такой график подходит для временной динамики: видно, что после небольшого снижения в марте показатель вырос в апреле и мае. Маркеры полезны, если точек немного; при длинном временном ряде их часто убирают.

seaborn — библиотека более высокого уровня, удобная для статистической визуализации. Она хорошо работает с таблицами pandas и позволяет быстро строить графики распределений, категориальные сравнения и связи между переменными. Например, диаграмма рассеяния с цветом по категории:

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")

sns.scatterplot(
    data=tips,
    x="total_bill",
    y="tip",
    hue="time",
    style="time"
)
plt.title("Связь суммы счета и чаевых")
plt.xlabel("Счет")
plt.ylabel("Чаевые")
plt.show()
```

Здесь каждая точка — один заказ, координаты показывают сумму счета и размер чаевых, а цвет разделяет обед и ужин. По такому графику можно увидеть общую положительную связь: большие счета часто сопровождаются большими чаевыми, но разброс остается значительным.

Для интерактивной визуализации часто применяют plotly. Графики Plotly можно увеличивать, наводить курсор для просмотра значений, фильтровать элементы легенды и встраивать в веб-страницы. Это удобно для дашбордов и исследовательских отчетов:

```
import plotly.express as px

df = px.data.gapminder().query("year == 2007")

fig = px.scatter(
    df,
    x="gdpPercap",
    y="lifeExp",
    size="pop",
    color="continent",
    hover_name="country",
    log_x=True,
    title="ВВП на душу населения и продолжительность жизни"
)
fig.show()
```

В этом примере логарифмическая ось x нужна из-за сильного разброса ВВП между странами. Размер точки кодирует население, цвет — континент, а всплывающая подсказка показывает страну. Такой график уже передает несколько измерений данных, но важно не перегрузить его дополнительными кодировками.

pandas имеет встроенные методы .plot(), которые удобны для быстрых проверок. Например, если есть таблица продаж по датам, можно написать df.groupby("date")["sales"].sum().plot(). Это не всегда лучший вариант для финального отчета, но очень полезный инструмент разведочного анализа. Для специализированных задач существуют и другие библиотеки. bokeh применяется для интерактивных веб-графиков, altair строит декларативные визуализации на основе грамматики Vega-Lite, georandas и folium используются для карт, missingno помогает визуально анализировать пропуски, yellowbrick строит диагностические графики для моделей машинного обучения. В научных расчетах часто встречаются matplotlib и seaborn, в продуктовой аналитике и BI-отчетах — plotly, Dash или Streamlit.

Пример выбора графика по задаче: если нужно сравнить среднюю оценку студентов по факультетам,

подойдет отсортированная столбчатая диаграмма с доверительными интервалами. Если нужно понять, различается ли распределение оценок между факультетами, лучше использовать boxplot или violin plot. Если нужно изучить связь времени подготовки и результата экзамена, разумен scatter plot с линией регрессии. Если нужно показать изменение среднего балла по годам, подойдет линейный график. Главное качество визуализации — соответствие данным и вопросу. Сильная визуализация не маскирует неопределенность, не преувеличивает эффект и не заставляет зрителя угадывать смысл кодировок. В Python есть широкий набор библиотек: от универсального matplotlib до интерактивного plotly, но выбор инструмента вторичен. Сначала нужно понять тип данных, аналитическую цель и аудиторию, а затем выбрать самый простой график, который честно показывает нужную закономерность.

Билет 4. Корреляционный анализ. Коэффициент корреляции

Основные понятия и определения

Корреляционный анализ - это совокупность статистических методов, с помощью которых изучают наличие, направление и тесноту связи между признаками. Он применяется тогда, когда исследователю важно понять, изменяются ли две величины согласованно: например, связаны ли время подготовки и результат экзамена, цена и спрос, возраст оборудования и число отказов, доход и расходы.

Корреляционная связь отличается от функциональной. При функциональной зависимости каждому значению x соответствует строго определенное значение y . При корреляционной зависимости такой жесткости нет: одному и тому же значению x могут соответствовать разные значения y , но в среднем наблюдается некоторая тенденция. Именно поэтому корреляционный анализ работает не с отдельным наблюдением, а с совокупностью парных данных.

Главная характеристика линейной связи двух количественных признаков - коэффициент корреляции. Чаще всего под ним понимают коэффициент корреляции Пирсона. Он показывает, насколько точки на диаграмме рассеяния близки к прямой линии, а также задает направление связи. Значение коэффициента лежит в интервале от -1 до 1.

Если коэффициент положителен, связь называется прямой: при увеличении одного признака другой в среднем также увеличивается. Если коэффициент отрицателен, связь обратная: рост одного признака сопровождается средним уменьшением другого. Если коэффициент близок к нулю, линейная связь отсутствует или выражена слабо. Однако это не означает полного отсутствия зависимости: связь может быть нелинейной, например параболической.

Различают генеральный и выборочный коэффициенты корреляции. Генеральный коэффициент, обычно обозначаемый ρ , относится ко всей изучаемой совокупности. Выборочный коэффициент r вычисляется по имеющейся выборке и является оценкой ρ . Так как выборка случайна, значение r может отличаться от истинной корреляции в генеральной совокупности.

Важное ограничение: корреляция не доказывает причинность. Если два признака связаны, это еще не значит, что один является причиной другого. Возможна связь через третий фактор, влияние времени, сезонность, особенности отбора объектов или случайное совпадение. Поэтому корреляционный анализ измеряет статистическую согласованность признаков, но причинное объяснение требует дополнительной содержательной проверки.

Теория, формулы и интуиция

Пусть есть n пар наблюдений:

$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Сначала вычисляют выборочные средние:

$\bar{x} = (1/n) * \sum(x_i)$

$\bar{y} = (1/n) * \sum(y_i)$

Затем анализируют отклонения от средних: $x_i - \bar{x}$ и $y_i - \bar{y}$. Если большие значения x часто соответствуют большим значениям y , произведения отклонений в основном положительны. Если большие значения x соответствуют малым значениям y , произведения отклонений в основном отрицательны. Если устойчивой согласованности нет, положительные и отрицательные произведения взаимно компенсируются.

Выборочная ковариация определяется так:

$s_{xy} = (1/(n - 1)) * \sum((x_i - \bar{x})(y_i - \bar{y}))$

Ковариация показывает совместную изменчивость признаков, но зависит от единиц измерения. Если рост измерять не в сантиметрах, а в метрах, численное значение ковариации изменится. Поэтому

ее нормируют на стандартные отклонения признаков. Так получается коэффициент корреляции Пирсона:

$$r = \frac{\sum((x_i - \bar{x})(y_i - \bar{y}))}{\sqrt{\sum((x_i - \bar{x})^2) * \sum((y_i - \bar{y})^2)}}$$

или короче:

$$r = s_{xy} / (s_x * s_y)$$

где s_x и s_y - выборочные стандартные отклонения. Нормировка делает коэффициент безразмерным и ограничивает его пределами $[-1; 1]$. При $r = 1$ все точки лежат на возрастающей прямой, при $r = -1$ - на убывающей прямой. При $r = 0$ линейная согласованность отклонений отсутствует.

Интуитивно корреляцию можно понимать как среднее согласование стандартизированных отклонений. Если оба признака одновременно выше своих средних или одновременно ниже своих средних, вклад в корреляцию положителен. Если один признак выше среднего, а другой ниже, вклад отрицателен. Чем стабильнее такие совпадения по всем наблюдениям, тем ближе коэффициент к 1 или -1.

Для приближенной интерпретации часто используют ориентиры: $|r| < 0,3$ - слабая связь, $0,3 \leq |r| < 0,7$ - умеренная, $|r| \geq 0,7$ - сильная. Эти границы условны. В разных областях одна и та же величина корреляции может иметь разную практическую значимость.

Для проверки статистической значимости часто проверяют гипотезу $H_0: \rho = 0$. При нормальности данных и независимости наблюдений используют статистику:

$$t = r * \sqrt{(n - 2) / (1 - r^2)}$$

Она имеет распределение Стьюдента с $n - 2$ степенями свободы при верной нулевой гипотезе. Если p -значение мало, гипотезу об отсутствии линейной корреляции отвергают. Но статистическая значимость не равна практической важности: на большой выборке даже очень слабая связь может стать формально значимой.

Коэффициент Пирсона хорошо описывает линейные связи, но чувствителен к выбросам. Если связь монотонная, но нелинейная, или данные представлены рангами, применяют ранговую корреляцию Спирмена. Она вычисляется как корреляция Пирсона между рангами значений. Для порядковых данных и небольших выборок также используют коэффициент Кендалла, основанный на сравнении согласованных и несогласованных пар.

Методы, алгоритмы и этапы решения

Корреляционный анализ обычно выполняют по последовательному плану. Сначала формулируют задачу: какие признаки сравниваются, почему между ними ожидается связь и для чего нужен результат. Затем собирают парные наблюдения, то есть для каждого объекта должны быть известны оба признака. Следующий этап - проверка качества данных. Нужно обнаружить пропуски, ошибки ввода, дубли, невозможные значения и выбросы. Пропуски можно удалить или заменить расчетными значениями, но выбранный способ должен быть обоснован. Выбросы также нельзя механически исключать: они могут быть ошибками измерения, но могут отражать важные реальные случаи.

После подготовки данных строят диаграмму рассеяния. Это обязательный шаг, потому что сам коэффициент не показывает форму зависимости. На графике можно увидеть линейную тенденцию, криволинейность, отдельные группы объектов, резкие выбросы или отсутствие видимой структуры.

Затем выбирают коэффициент. Для количественных данных и примерно линейной связи используют Пирсона. Для монотонной зависимости, ранговых шкал или данных с сильными выбросами выбирают Спирмена или Кендалла. После этого вычисляют коэффициент, проверяют значимость, при необходимости строят доверительный интервал и дают содержательную интерпретацию.

При ручном расчете удобно составить таблицу. В ней для каждой пары наблюдений находят отклонения $x_i - \bar{x}$ и $y_i - \bar{y}$, затем считают три столбца: квадрат первого отклонения, квадрат второго отклонения и произведение отклонений. Суммы этих столбцов подставляют в формулу Пирсона. Если признаков много, рассчитывают корреляционную матрицу, где каждая ячейка содержит коэффициент корреляции между двумя переменными.

Практические примеры

Первый пример - связь между подготовкой и результатом экзамена. Пусть по пяти студентам известны часы подготовки и баллы:

Студент	Часы x	Балл y	$(x - \bar{x})^2$	$(y - \bar{y})^2$	$(x - \bar{x})(y - \bar{y})$
1	1	55	4	16	-8
2	2	60	1	9	-3
3	3	68	0	64	0
4	4	74	1	100	12
5	5	82	4	144	24

Здесь при росте числа часов балл также растет. Диаграмма рассеяния будет похожа на возрастающую прямую, а коэффициент Пирсона окажется близким к 1. Это означает сильную положительную



Figure 5: Схема 5

линейную связь в данной выборке. Корректный вывод: большее время подготовки связано с более высоким баллом. Некорректный вывод: время подготовки является единственной причиной результата. На оценку также могут влиять способности, качество подготовки, сложность билета и состояние студента.

Второй пример - цена и спрос. Если магазин анализирует цену товара и число проданных единиц, часто наблюдается отрицательная корреляция: при росте цены спрос уменьшается. Значение $r = -0,75$ можно интерпретировать как сильную обратную линейную связь. Но для управленческого решения этого мало: нужно учесть сезон, рекламу, акции конкурентов, доход покупателей и наличие заменителей.

Третий пример показывает ограничение Пирсона. Пусть урожайность сначала растет при увеличении дозы удобрения, затем достигает максимума и начинает снижаться из-за избытка удобрения. Такая зависимость может быть сильной, но нелинейной. Коэффициент Пирсона при этом способен оказаться близким к нулю, потому что возрастающая и убывающая части компенсируют друг друга. Поэтому перед интерпретацией корреляции важно смотреть график.

На практике корреляционный анализ используют как предварительный этап перед регрессией, прогнозированием и построением моделей. Он помогает найти связанные признаки, выявить избыточные переменные и сформулировать гипотезы. Надежный вывод строится не только по числу r , а по сочетанию графика, формулы, проверки значимости и понимания предметной области.

Билет 5. Регрессионный анализ. Метод наименьших квадратов. Метрики качества регрессии

Основные понятия и определения

Регрессионный анализ - это раздел статистики и машинного обучения, изучающий зависимость числовой целевой переменной от одного или нескольких факторов. В задачах регрессии требуется не отнести объект к классу, а предсказать количественное значение: цену квартиры, спрос на товар, время выполнения операции, урожайность, температуру, прибыль предприятия. Формально имеется выборка наблюдений: для каждого объекта известен вектор признаков $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})$ и числовой отклик y_i . Нужно построить функцию $\hat{y} = f(x)$, которая по новым признакам дает разумное приближение истинного значения.

Целевая переменная, или зависимая переменная, обозначается y . Признаки, или независимые переменные, обозначаются $x_1, x_2, \dots, x_p$. Слово "независимые" здесь не всегда означает статистическую независимость, а лишь указывает, что эти переменные используются как

объясняющие факторы. Предсказанное моделью значение обозначается $\hat{y}$. Разность между реальным и предсказанным значением называется ошибкой или остатком: $e_i = y_i - \hat{y}_i$. Самый важный частный случай - линейная регрессия. В ней предполагается, что условное среднее отклика можно описать линейной комбинацией признаков:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}.$$

Коэффициент β_0 называется свободным членом или интерсептом. Коэффициент β_j показывает, как в среднем меняется предсказание при увеличении признака x_j на единицу при фиксированных остальных признаках. Если модель описывает цену квартиры, коэффициент при площади может интерпретироваться как средняя прибавка к цене при увеличении площади на один квадратный метр, если район, этаж и другие признаки не меняются.

Регрессия бывает парной и множественной. В парной регрессии используется один фактор, например зависимость зарплаты от стажа. Во множественной регрессии факторов несколько: зарплата может зависеть от стажа, образования, региона, отрасли и должности. Также различают линейные и нелинейные модели. Важно, что линейная регрессия линейна по параметрам, а не обязательно по исходным признакам. Например, модель

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

является линейной по коэффициентам β , хотя зависимость от x имеет параболический вид.

Теория, формулы и интуиция

Классическая статистическая запись линейной регрессии выглядит так:

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \varepsilon_i,$$

где ε_i - случайная ошибка, отражающая влияние неучтенных факторов, шум измерений и естественную изменчивость процесса. В матричной форме для всех наблюдений:

$$y = X\beta + \varepsilon.$$

Здесь y - вектор ответов размера n , X - матрица признаков размера $n \times (p+1)$, первый столбец которой обычно состоит из единиц для учета свободного члена, β - вектор коэффициентов, ε - вектор ошибок. Метод наименьших квадратов (МНК) выбирает коэффициенты так, чтобы сумма квадратов остатков была минимальной:

$$RSS(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - x_i^T \beta)^2.$$

В матричной форме целевая функция записывается компактно:

$$RSS(\beta) = |y - X\beta|^2 = (y - X\beta)^T (y - X\beta).$$

Если матрица $X^T X$ обратима, решение МНК имеет аналитический вид:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

Интуитивно МНК подбирает такую гиперплоскость в пространстве признаков, которая проходит "как можно ближе" к точкам выборки. Квадрат ошибки используется по нескольким причинам. Во-первых, положительные и отрицательные ошибки не компенсируют друг друга. Во-вторых, большие промахи штрафуются сильнее малых, что полезно, если крупные ошибки действительно нежелательны. В-третьих, квадратичная функция удобна математически: она дифференцируема и приводит к явному решению при стандартных условиях.

Для парной линейной регрессии можно представить задачу геометрически: на плоскости есть облако точек, и нужно провести прямую, минимизирующую сумму квадратов вертикальных расстояний от

точек до прямой. Эти вертикальные расстояния и есть остатки. МНК не минимизирует расстояния до прямой по перпендикуляру; он минимизирует именно ошибки по оси отклика y , потому что задача состоит в предсказании y по x .

У классической линейной регрессии есть предпосылки, важные для статистической интерпретации коэффициентов и доверительных интервалов. Обычно предполагают линейность связи по параметрам, отсутствие точной мультиколлинеарности, нулевое условное математическое ожидание ошибки $E(\varepsilon | X) = 0$, постоянную дисперсию ошибок (гомоскедастичность), независимость наблюдений и, для точных маловыборочных выводов, нормальность ошибок. Для построения предсказательной модели часть предпосылок может быть смягчена, но их нарушение влияет на надежность выводов. Например, сильная мультиколлинеарность делает коэффициенты нестабильными: качество предсказаний может оставаться приемлемым, но интерпретировать отдельные коэффициенты становится трудно. Схематично процесс регрессионного анализа можно представить так:

Методы, алгоритмы и этапы решения

Типовой алгоритм применения регрессионного анализа начинается с постановки задачи. Нужно определить, какая переменная является откликом, какие факторы доступны до момента предсказания и как будет измеряться качество. Например, если прогнозируется стоимость объекта недвижимости, нельзя использовать признак “финальная сумма сделки”, если он становится известен только после продажи.

Затем выполняют первичный анализ данных. Проверяют пропуски, выбросы, шкалы признаков, распределение целевой переменной, связи между признаками и откликом. Для линейной регрессии особенно полезны диаграммы рассеяния, корреляционная матрица и анализ выбросов. Категориальные признаки обычно кодируют численно, например one-hot-кодированием. Числовые признаки при обычном МНК не обязаны масштабироваться для корректности формул, но масштабирование полезно при регуляризации, градиентных методах и сравнении влияния признаков. Далее данные часто делят на обучающую и тестовую выборки. На обучающей выборке оцениваются коэффициенты, на тестовой проверяется способность модели работать на новых данных. Если данных мало или требуется более надежная оценка качества, используют кросс-валидацию: выборку несколько раз делят на обучающие и проверочные части, после чего усредняют метрики.

После подготовки строится матрица признаков X , добавляется столбец единиц для свободного члена, и коэффициенты оцениваются методом наименьших квадратов. На практике аналитическая формула через явное обращение матрицы не всегда является лучшим вычислительным способом, потому что обращение матрицы может быть численно неустойчивым. В программных библиотеках часто используют QR-разложение, сингулярное разложение или специализированные численные методы. Если признаков очень много, а данных тоже много, могут применяться итерационные методы оптимизации, например градиентный спуск.

После обучения анализируют остатки. Если остатки имеют выраженную структуру, например растут вместе с предсказанием или зависят от времени, модель не уловила важную закономерность. При гетероскедастичности стандартные ошибки коэффициентов могут быть некорректными; при автокорреляции в временных рядах нарушается независимость ошибок; при нелинейной зависимости нужна другая форма признаков или модель.

Для борьбы с переобучением и нестабильностью часто используют регуляризованные варианты линейной регрессии. Ridge-регрессия добавляет штраф за сумму квадратов коэффициентов:

$$\sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2.$$

Она уменьшает разброс оценок и особенно полезна при мультиколлинеарности. Lasso-регрессия добавляет штраф за сумму модулей коэффициентов:

$$\sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^p \|\beta_j\|.$$

Lasso может занулять часть коэффициентов и тем самым выполнять отбор признаков. Параметр λ обычно выбирают по кросс-валидации.

Качество регрессии оценивают с помощью нескольких метрик, потому что каждая показывает свой



Figure 6: Схема 6

аспект ошибки. Средняя абсолютная ошибка:

$$MAE = \frac{1}{n} \sum_{i=1}^n \|y_i - \hat{y}_i\|.$$

MAE измеряется в тех же единицах, что и целевая переменная, и хорошо интерпретируется: если MAE цены равна 50000 рублей, то средний модуль ошибки составляет 50000 рублей. Она менее чувствительна к выбросам, чем квадратичные метрики.

Среднеквадратичная ошибка:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

MSE сильнее штрафует крупные ошибки, но измеряется в квадратных единицах, что неудобно для объяснения. Поэтому часто используют корень из MSE:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

RMSE находится в исходных единицах отклика и особенно полезна, когда большие промахи нужно считать существенно хуже нескольких малых ошибок.

Коэффициент детерминации показывает, какую долю разброса целевой переменной модель объясняет по сравнению с простым прогнозом средним значением:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

Если $R^2 = 0.8$, это означает, что модель объясняет около 80% вариации отклика относительно среднего. Значение R^2 может быть отрицательным на тестовой выборке, если модель хуже, чем прогноз константой $\bar{y}$. Для сравнения моделей с разным числом признаков используют скорректированный коэффициент детерминации:

$$R_{adj}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}.$$

Он штрафует добавление лишних признаков и поэтому лучше подходит для интерпретации в классической статистической регрессии.

Иногда используют MAPE:

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left\| \frac{y_i - \hat{y}_i}{y_i} \right\|.$$

Эта метрика выражает ошибку в процентах, но плохо работает, если истинные значения близки к нулю или равны нулю. Поэтому выбор метрики должен соответствовать предметной области. Для прогноза выручки процентная ошибка может быть удобна, а для температуры или остатков на складе она может вести себя неадекватно.

Практические примеры

Рассмотрим задачу прогнозирования цены квартиры. Пусть признаки: площадь, расстояние до метро, этаж, возраст дома и район. Линейная модель может иметь вид:

$$\widehat{price} = \beta_0 + \beta_1 area + \beta_2 metro + \beta_3 floor + \beta_4 age + \beta_5 district.$$

Если после обучения коэффициент при площади равен 180000, то при прочих равных дополнительный квадратный метр увеличивает прогноз цены примерно на 180000 рублей. Если коэффициент при расстоянии до метро отрицателен, это означает снижение цены при удалении от метро. Однако такая

интерпретация корректна только при разумном наборе признаков и отсутствии сильной зависимости между ними. Например, площадь, число комнат и тип квартиры могут быть тесно связаны, из-за чего отдельные коэффициенты становятся менее устойчивыми.

Другой пример - прогнозирование продаж магазина. В качестве факторов можно использовать цену, день недели, наличие акции, расходы на рекламу, сезон, остаток товара и данные о прошлых продажах. Если модель строится для планирования закупок, важно оценивать не только среднюю ошибку, но и цену ошибки. Недопрогноз может привести к дефициту товара, а перепрогноз - к излишкам на складе. Поэтому иногда выбирают не симметричную MSE, а бизнес-метрику или взвешенную ошибку, где разные направления ошибки имеют разную стоимость.

В учебном числовом примере пусть фактические значения спроса равны $y = [100, 120, 130]$, а прогнозы модели $\hat{y} = [90, 125, 135]$. Остатки равны $[10, -5, -5]$. Тогда

$$MAE = \frac{\|10\| + \|-5\| + \|-5\|}{3} \approx 6.67,$$

$$MSE = \frac{10^2 + (-5)^2 + (-5)^2}{3} = 50,$$

$$RMSE = \sqrt{50} \approx 7.07.$$

Эти значения показывают, что средняя ошибка прогноза составляет примерно 6-7 единиц спроса. Если в другом варианте модели MAE чуть ниже, но RMSE заметно выше, это значит, что вторая модель иногда допускает крупные ошибки. Выбор между ними зависит от того, насколько опасны редкие большие промахи.

На практике регрессионный анализ полезен не только как инструмент прогноза, но и как инструмент объяснения. В экономике с его помощью оценивают влияние цены на спрос, рекламы на продажи, образования на доход. В инженерии строят зависимости ресурса детали от нагрузок и температуры. В медицине анализируют связь показателей пациента с количественным исходом, например уровнем давления или временем восстановления. При этом всегда нужно помнить: регрессия обнаруживает статистическую связь, но сама по себе не доказывает причинность. Для причинных выводов требуются дизайн исследования, контроль смешивающих факторов, эксперименты или специальные квазиэкспериментальные методы.

Итак, регрессионный анализ решает задачу предсказания и объяснения числовой переменной по набору факторов. Метод наименьших квадратов является базовым способом оценки параметров линейной модели: он минимизирует сумму квадратов остатков и при стандартных условиях имеет явное матричное решение. Качество модели оценивают через MAE, MSE, RMSE, R^2 , скорректированный R^2 , MAPE и другие метрики, выбирая их с учетом смысла ошибки в конкретной прикладной задаче.

Билет 6. Постановка задачи классификации. Примеры. Метрики качества классификации. ROC-кривая

Основные понятия и определения

Классификация в машинном обучении — это задача обучения с учителем, в которой по описанию объекта требуется предсказать дискретную метку класса. Объект обычно задается вектором признаков $x = (x_1, x_2, \dots, x_d)$, а правильный ответ — меткой y из конечного множества классов Y . Обучающая выборка имеет вид $\{(x_i, y_i)\}_{i=1}^n$, где для каждого объекта известен правильный класс. Цель состоит в том, чтобы построить алгоритм $a(x)$, который хорошо предсказывает класс новых, ранее не виденных объектов.

Если классов два, говорят о бинарной классификации. Обычно один класс называют положительным, например «болен», «спам», «мошенническая операция», а другой — отрицательным. Если классов больше двух, задача называется многоклассовой: например, распознавание цифр от 0 до 9, определение темы текста или классификация изображения как «кошка», «собака», «автомобиль». Есть также многометочная классификация, когда объект может относиться сразу к нескольким классам, например

текст может одновременно иметь темы «экономика» и «политика».

Важное отличие классификации от регрессии состоит в типе целевой переменной. В регрессии предсказывается число, например цена квартиры или температура, а в классификации — категория. При этом многие классификаторы внутри себя могут оценивать не только сам класс, но и вероятность или скор принадлежности к классу. Например, модель может выдать $P(y = 1 \mid x) = 0.83$, а затем по выбранному порогу решить, относить объект к положительному классу или нет.

Качество классификации нельзя оценивать только по факту совпадения ответов, если классы несбалансированы или ошибки имеют разную цену. Например, в медицинской диагностике пропустить болезнь обычно хуже, чем направить здорового пациента на дополнительное обследование. Поэтому используются разные метрики качества: accuracy, precision, recall, F1-мера, ROC-кривая, AUC и другие.

Теория, формулы и интуиция

Для бинарной классификации удобно описывать результаты модели через матрицу ошибок. Она сравнивает истинный класс и предсказание алгоритма. Возможны четыре исхода:

- TP , true positive: объект положительный и модель предсказала положительный класс;
- TN , true negative: объект отрицательный и модель предсказала отрицательный класс;
- FP , false positive: объект отрицательный, но модель ошибочно предсказала положительный класс;
- FN , false negative: объект положительный, но модель ошибочно предсказала отрицательный класс.

На основе этих величин строятся основные метрики. Доля правильных ответов, или accuracy, равна

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Эта метрика понятна, но может быть обманчивой при дисбалансе классов. Если в выборке 99% писем не являются спамом, классификатор, всегда отвечающий «не спам», получит accuracy 99%, хотя он совершенно бесполезен для поиска спама.

Точность положительных предсказаний, или precision, показывает, какая доля объектов, предсказанных как положительные, действительно положительна:

$$Precision = \frac{TP}{TP + FP}.$$

Эта метрика важна, когда ложные срабатывания дороги. Например, если система блокирует банковскую карту при подозрении на мошенничество, высокий precision означает, что блокировки чаще обоснованы.

Полнота, или recall, показывает, какую долю настоящих положительных объектов модель нашла:

$$Recall = \frac{TP}{TP + FN}.$$

Recall важен, когда опасно пропускать положительные случаи. В диагностике тяжелой болезни высокий recall означает, что модель находит большую часть больных пациентов.

Precision и recall часто конфликтуют. Если понизить порог классификации и чаще относить объекты к положительному классу, recall обычно растет, потому что модель пропускает меньше положительных объектов. Но precision может падать, потому что вместе с ними растет число ложных срабатываний. Для компромиссной оценки используют F1-меру:

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}.$$

Это гармоническое среднее precision и recall. Оно мало, если хотя бы одна из двух метрик мала, поэтому F1 полезна, когда важны обе стороны качества.

Для многоклассовой классификации метрики часто обобщают через усреднение. При macro-averaging метрика считается отдельно для каждого класса, а затем усредняется с одинаковым весом для всех классов. Это хорошо показывает качество на редких классах. При micro-averaging сначала суммируются TP , FP , FN по всем классам, а затем считается метрика; такой подход сильнее учитывает частые классы. Weighted-averaging усредняет метрики по классам с весами, пропорциональными числу объектов каждого класса.

ROC-кривая используется для анализа бинарного классификатора, который выдает скор или

вероятность положительного класса. Для разных порогов решения вычисляются две величины: true positive rate и false positive rate.

$$TPR = Recall = \frac{TP}{TP + FN},$$

$$FPR = \frac{FP}{FP + TN}.$$

ROC-кривая строится в координатах (FPR, TPR) . Каждая точка соответствует некоторому порогу. Если порог очень высокий, модель почти никого не относит к положительному классу: и TPR , и FPR малы. Если порог очень низкий, почти все объекты становятся положительными: TPR стремится к 1, но FPR тоже растет. Хорошая модель поднимает ROC-кривую ближе к левому верхнему углу: высокий TPR при низком FPR .

Площадь под ROC-кривой называется AUC ROC. Значение AUC равно 1 для идеального ранжирования и примерно 0.5 для случайного классификатора. Интуитивно AUC можно понимать как вероятность того, что случайно выбранный положительный объект получит скор выше, чем случайно выбранный отрицательный объект. Поэтому AUC оценивает качество ранжирования, а не качество при одном конкретном пороге.



Figure 7: Схема 7

Важно понимать, что ROC-кривая не выбирает порог автоматически. Она показывает, как меняется поведение модели при разных порогах. Финальный порог выбирают с учетом прикладной цены ошибок. В одних задачах лучше терпеть больше ложных срабатываний ради высокого recall, в других — наоборот.

Методы, алгоритмы и этапы решения

Решение задачи классификации начинается с формализации классов и сбора размеченных данных. Нужно определить, что именно считается объектом, какие классы допустимы и как присваивается правильная метка. Например, в задаче антиспама объектом является письмо, признаки могут включать слова, адрес отправителя, наличие ссылок и вложений, а метка показывает, является ли письмо спамом.

Следующий этап — подготовка признаков. Числовые признаки часто масштабируют, категориальные кодируют, тексты превращают в векторы с помощью мешка слов, TF-IDF или эмбедингов, изображения подают в сверточные нейронные сети либо описывают заранее извлеченными признаками. Качество признаков существенно влияет на результат: даже сильный алгоритм плохо работает, если данные не отражают различия между классами.

После этого выборку делят на обучающую, валидационную и тестовую части. На обучающей части модель настраивает параметры, на валидационной подбираются гиперпараметры и порог классификации, на тестовой оценивается итоговое качество. Такое разделение нужно, чтобы не переоценить качество модели из-за подгонки под уже известные данные.

Для классификации применяются разные алгоритмы. Логистическая регрессия строит линейную границу решения и оценивает вероятность положительного класса через сигмоиду. Она интерпретируема, быстро обучается и часто служит сильной базовой моделью. Метод k ближайших соседей относит объект к классу, наиболее часто встречающемуся среди ближайших объектов обучающей выборки. Деревья решений последовательно разбивают пространство признаков по условиям вида $x_j < t$; они понятны, но склонны к переобучению. Случайный лес и градиентный бустинг строят ансамбли деревьев и обычно дают высокое качество на табличных данных. Метод опорных векторов ищет разделяющую поверхность с максимальным зазором между классами. Нейронные сети особенно эффективны для изображений, речи, текстов и других сложных структурированных сигналов.

Отдельный этап — выбор функции потерь. Для бинарной классификации часто используется логистическая потеря, или binary cross-entropy:

$$L = -\frac{1}{n} \sum_{i=1}^n (y_i \log p_i + (1 - y_i) \log(1 - p_i)),$$

где p_i — предсказанная вероятность положительного класса. Для многоклассовой классификации применяют categorical cross-entropy с softmax-выходом. Функция потерь оптимизируется при обучении, а метрики качества используются для оценки результата с точки зрения задачи.

После обучения важно проанализировать матрицу ошибок и метрики. Если классы сбалансированы и ошибки примерно одинаково важны, ассигасу может быть достаточной. Если положительный класс редкий, полезнее смотреть precision, recall, F1, PR-кривую и ROC AUC. Если модель используется для ранжирования заявок, AUC может быть особенно информативной. Если модель принимает автоматические решения, необходимо выбрать рабочий порог и проверить качество именно при нем. На практике также используют кросс-валидацию, регуляризацию, балансировку классов и настройку порога. Дисбаланс можно частично учитывать через веса классов, oversampling редкого класса, under-sampling частого класса или специальные методы вроде SMOTE. Однако балансировку нужно делать только на обучающей части, иначе возникает утечка информации из теста.

Практические примеры

Первый пример — медицинский скрининг. Пусть модель должна определить, есть ли у пациента заболевание. Положительный класс — «болен». Если модель ошибочно относит больного пациента к здоровым, возникает FN , и болезнь может остаться без лечения. Поэтому в такой задаче часто стремятся к высокому recall. Это может привести к увеличению FP : часть здоровых пациентов получит подозрение на болезнь и пройдет дополнительные обследования. Такой компромисс может быть приемлемым, если цена пропуска болезни выше цены лишней проверки.

Второй пример — фильтрация спама. Положительный класс — «спам». Если обычное письмо попало в спам, это FP , и пользователь может пропустить важное сообщение. Если спам прошел во входящие, это FN . В корпоративной почте может быть важно снижать FP , чтобы не блокировать рабочие письма, то есть повышать precision. В личной почте пользователь может предпочесть более агрессивную фильтрацию, если спам сильно мешает.

Третий пример — кредитный скоринг. Банк оценивает вероятность дефолта заемщика. Модель может выдавать скор риска, а решение принимается по порогу: если риск выше порога, заявка отклоняется или отправляется на ручную проверку. ROC-кривая помогает понять, насколько хорошо модель отделяет надежных клиентов от рискованных при разных порогах. Но сам порог должен выбираться с учетом прибыли, потерь, требований регуляторов и бизнес-стратегии.

Четвертый пример — распознавание рукописных цифр. Это многоклассовая классификация: каждый объект относится к одному из десяти классов. Здесь можно считать ассигасу, но полезно также смотреть confusion matrix: например, модель может часто путать 4 и 9 или 3 и 8. Такая диагностика показывает, какие классы требуют дополнительных данных, улучшения признаков или изменения архитектуры.

Пятый пример — модерация комментариев. Комментарий может быть «нормальным», «оскорбительным», «спамом», «угрозой» или одновременно иметь несколько нежелательных признаков. Если система автоматически скрывает сообщения, ложные срабатывания могут ухудшить пользовательский опыт. Если система ничего не скрывает, вредный контент останется видимым. Поэтому часто используют не один порог, а несколько режимов: уверенно вредный контент удаляется автоматически, сомнительный отправляется модератору, безопасный пропускается.

Таким образом, задача классификации состоит не только в построении модели, которая предсказывает метки классов. Важны корректная постановка классов, качественная разметка, выбор признаков, адекватная схема проверки, подбор метрик и интерпретация ошибок. ROC-кривая и AUC полезны для оценки способности модели ранжировать объекты по вероятности положительного класса, а precision, recall и F1 помогают понять практические последствия выбранного порога. Хороший ответ на задачу классификации всегда связывает математические метрики с реальной ценой ошибок в конкретной предметной области.

Билет 7. Обучающая и валидационная выборки. Этапы решения задачи классификации

Основные понятия и определения

В задаче классификации требуется по описанию объекта предсказать его класс: например, определить, является ли письмо спамом, болен ли пациент, относится ли изображение к классу “кошка” или “собака”. Формально объект описывается вектором признаков $x = (x_1, x_2, \dots, x_d)$, а правильный ответ задается меткой класса y . В бинарной классификации y принимает одно из двух значений, например 0 и 1. В многоклассовой классификации классов больше двух, например цифры от 0 до 9. Данные в машинном обучении обычно представлены выборкой:

$$D = \{(x_i, y_i)\}_{i=1}^n,$$

где x_i - признаки i -го объекта, y_i - его истинная метка, n - число объектов. Главная цель обучения состоит не в том, чтобы идеально запомнить ответы на уже известных объектах, а в том, чтобы построить алгоритм $a(x)$, который хорошо работает на новых данных из той же прикладной области. Обучающая выборка, или train set, - это часть данных, на которой модель подбирает свои параметры. Например, в логистической регрессии на обучающей выборке подбираются веса признаков, в дереве решений выбираются условия разбиения, в нейронной сети настраиваются веса связей. Именно обучающая выборка “видна” алгоритму во время обучения.

Валидационная выборка, или validation set, - это отдельная часть данных, которую используют для промежуточной оценки качества модели и выбора настроек. На ней обычно подбирают гиперпараметры: глубину дерева, коэффициент регуляризации, число соседей в kNN, порог классификации, архитектуру модели, способ обработки признаков. Валидационная выборка не должна участвовать в прямом обучении параметров модели, иначе оценка качества станет слишком оптимистичной.

Тестовая выборка, или test set, - это независимая часть данных для финальной проверки уже выбранной модели. Ее смысл в том, чтобы имитировать будущие реальные данные. Если много раз подбирать модель по тестовой выборке, она фактически превращается в валидационную, и финальная оценка перестает быть честной.

Разбиение данных на train, validation и test нужно для контроля обобщающей способности. Обобщающая способность - это способность модели делать правильные предсказания не только на известных примерах, но и на ранее не встречавшихся объектах. Если модель хорошо работает на train, но плохо на validation, говорят о переобучении. Если она плохо работает и на train, и на validation, возможно недообучение: модель слишком простая, признаки слабые или обучение настроено неудачно.

Теория, формулы и интуиция

При обучении классификатора обычно минимизируется эмпирический риск, то есть средняя ошибка на обучающей выборке:

$$Q_{train}(a) = \frac{1}{n_{train}} \sum_{i=1}^{n_{train}} L(a(x_i), y_i),$$

где L - функция потерь. Для простой классификации можно использовать индикатор ошибки:

$$L(a(x_i), y_i) = [a(x_i) \neq y_i],$$

который равен 1 при ошибке и 0 при правильном ответе. На практике часто используют гладкие функции потерь, например логистическую потерю для бинарной классификации:

$$L(p_i, y_i) = -y_i \log p_i - (1 - y_i) \log(1 - p_i),$$

где $p_i = P(y = 1 | x_i)$ - оцененная моделью вероятность положительного класса.

Однако минимальная ошибка на обучающей выборке не гарантирует хорошего качества на новых данных. Слишком сложная модель может выучить случайные особенности конкретной выборки: шум в метках, редкие совпадения, выбросы, технические артефакты сбора данных. Поэтому качество

оценивают отдельно:

$$Q_{val}(a) = \frac{1}{n_{val}} \sum_{i=1}^{n_{val}} L(a(x_i), y_i).$$

Если Q_{train} низкий, а Q_{val} заметно выше, модель, скорее всего, переобучилась. Если обе ошибки велики, модель недостаточно выразительна или данные плохо подготовлены. Интуитивно обучающая выборка нужна для “настройки”, а валидационная - для проверки, не стала ли настройка слишком узкой и случайной.

В классификации качество часто описывают не только функцией потерь, но и метриками. Ассурасу показывает долю правильных ответов:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Для несбалансированных классов важнее могут быть precision, recall и F1-мера:

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN},$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}.$$

Выбор метрики должен соответствовать смыслу задачи. В медицинском скрининге часто важна полнота, потому что опасно пропустить больного пациента. В антиспам-филт্রে может быть важна точность, потому что плохо ошибочно отправлять важные письма в спам. На валидационной выборке можно подобрать не только модель, но и порог принятия решения. Например, если классификатор выдает вероятность $P(y = 1 | x)$, стандартный порог 0.5 не всегда оптимален. Его можно изменить так, чтобы достичь нужного баланса между precision и recall.

При разбиении данных важно соблюдать независимость выборок. Все операции, которые “учатся” по данным, нужно настраивать только на train. Например, среднее и стандартное отклонение для стандартизации признаков вычисляют на обучающей выборке, а затем применяют к validation и test. То же относится к отбору признаков, заполнению пропусков, кодированию категорий, балансировке классов. Если использовать информацию из validation или test при подготовке признаков, возникает утечка данных: модель получает косвенное знание о будущих объектах, и оценка качества становится завышенной.

Обычное случайное разбиение может быть неудачным при дисбалансе классов. Если положительный класс редкий, он может случайно почти исчезнуть из validation. Тогда метрики будут нестабильными. Поэтому для классификации часто используют стратифицированное разбиение: доли классов сохраняются примерно одинаковыми во всех частях данных. Например, если мошеннических операций 3%, то и в train, и в validation, и в test желательно иметь примерно ту же долю.

Если данных мало, вместо одного разбиения применяют кросс-валидацию. При k -fold cross-validation данные делят на k частей. Модель обучается k раз: каждый раз одна часть используется как validation, остальные - как train. Итоговое качество усредняется. Это дает более устойчивую оценку, хотя требует больше вычислений. Для классификации также желательно использовать stratified k-fold, чтобы в каждом fold сохранялось распределение классов.

Методы, алгоритмы и этапы решения

Решение задачи классификации удобно рассматривать как последовательный pipeline. Он начинается с постановки задачи: нужно понять, что является объектом, какая целевая переменная предсказывается, какие ошибки особенно нежелательны и по какой метрике будет оцениваться результат.

Первый этап - постановка задачи. Нужно определить классы, единицу наблюдения и целевую метку. Например, в задаче кредитного скоринга объектом является заявка клиента, признаки включают доход, возраст, кредитную историю, а метка показывает, был ли дефолт. Также важно определить цену ошибок: отказ хорошему клиенту и одобрение рискованного кредита имеют разную стоимость. Второй этап - сбор и разметка данных. Качество классификатора ограничено качеством данных. Если метки противоречивы, признаки собраны после наступления события или данные не отражают будущую эксплуатацию модели, алгоритм может показать хорошую валидационную метрику, но

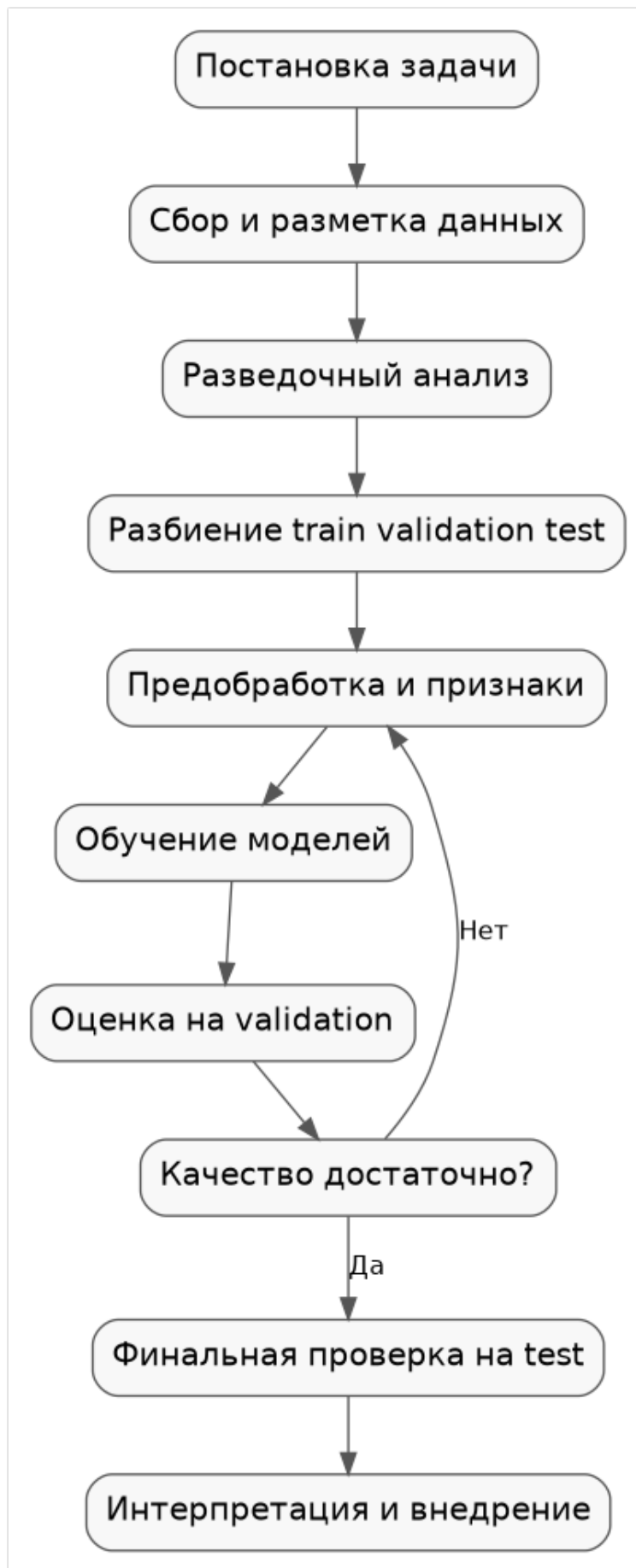


Figure 8: Схема 8

провалиться в реальности.

Третий этап - разведочный анализ данных. На нем смотрят распределение классов, пропуски, выбросы, типы признаков, корреляции, возможные источники утечки. Для классификации особенно важно проверить баланс классов. Если один класс встречается намного чаще другого, ассигурацию может быть бесполезной: модель, всегда предсказывающая частый класс, получит высокую долю правильных ответов, но не решит задачу.

Четвертый этап - разбиение данных. Часто используют пропорции 60/20/20, 70/15/15 или 80/10/10 для train, validation и test. Конкретное соотношение зависит от объема данных. При большом датасете можно выделить достаточно большую тестовую часть без ущерба для обучения. При малом датасете лучше использовать кросс-валидацию и аккуратно оставить небольшой финальный test. Если данные имеют временную природу, например заявки идут по датам, случайное перемешивание может быть неправильным. Тогда обучают на более раннем периоде, валидируют и тестируют на более поздних периодах.

Пятый этап - предобработка и построение признаков. Числовые признаки могут масштабироваться, категориальные - кодироваться, текстовые - преобразовываться в векторы, пропуски - заполняться. Важно, чтобы весь preprocessing был частью единого pipeline: сначала он настраивается на train, затем без перенастройки применяется к validation и test. Это обеспечивает честную проверку.

Шестой этап - обучение базовой модели. Обычно начинают с простого и интерпретируемого алгоритма: логистической регрессии, дерева решений, k ближайших соседей, наивного байесовского классификатора. Базовая модель дает ориентир. Затем можно пробовать более сильные методы: случайный лес, градиентный бустинг, SVM, нейронные сети. Сравнение моделей проводится по одинаковому разбиению и одной заранее выбранной метрике.

Седьмой этап - настройка гиперпараметров. Для этого используют validation или кросс-валидацию. Примеры гиперпараметров: максимальная глубина дерева, число деревьев в ансамбле, коэффициент регуляризации, скорость обучения, число соседей в kNN. Простой перебор называется grid search, случайный перебор - random search. При большом числе параметров применяют более продвинутые методы оптимизации, но принцип остается тем же: параметры модели обучаются на train, а настройки выбираются по validation.

Восьмой этап - анализ метрик и ошибок. Недостаточно увидеть одно число. Нужно посмотреть матрицу ошибок, precision, recall, F1, ROC-AUC или PR-AUC, качество по отдельным группам объектов. Если модель часто ошибается на важной подгруппе, это может быть критично даже при хорошем среднем качестве. На этом этапе можно принять решение изменить признаки, собрать больше данных, сбалансировать классы, поменять метрику или выбрать другой порог.

Девятый этап - финальное обучение и тестирование. После выбора подхода модель часто переобучают на объединении train и validation, чтобы использовать больше данных, а затем один раз оценивают на test. Результат на test считается финальной независимой оценкой. После этого модель можно готовить к внедрению: сохранять preprocessing, фиксировать версии данных и кода, описывать метрики и ограничения.

Практические примеры

Рассмотрим задачу фильтрации спама. Есть письма с признаками: частоты слов, наличие ссылок, домен отправителя, длина письма, число вложений. Метка $y = 1$ означает спам, $y = 0$ - нормальное письмо. Данные делят стратифицированно, чтобы доля спама была одинаковой в train, validation и test. На train обучают логистическую регрессию или наивный байесовский классификатор. На validation подбирают порог: если слишком много важных писем попадает в спам, повышают precision; если спам часто проходит во входящие, повышают recall. На test проверяют итоговое качество выбранного решения.

Другой пример - медицинская диагностика. Объектом является пациент, признаки включают результаты анализов и анкетные данные, метка показывает наличие болезни. Здесь опасна ошибка FN , когда больной пациент признан здоровым. Поэтому на validation выбирают модель и порог с высоким recall. При этом precision тоже нельзя игнорировать: слишком много ложных тревог перегрузит врачей и приведет к лишним обследованиям. В такой задаче полезно посмотреть не только ассигурацию, но и матрицу ошибок, recall, precision, ROC-AUC и калибровку вероятностей.

В задаче кредитного скоринга объектом является заявка на кредит. Модель выдает вероятность дефолта. Обучающая выборка содержит прошлые заявки, валидационная используется для выбора алгоритма и порога риска, тестовая - для финальной проверки. Здесь особенно важна временная валидация: если обучаться и проверяться на случайно перемешанных старых и новых заявках, можно получить завышенную оценку. Более честно обучаться на старых данных, а проверять качество на более позднем периоде, потому что именно так модель будет работать после внедрения.

Наконец, рассмотрим классификацию изображений рукописных цифр. Объекты - изображения, классы

- цифры от 0 до 9. Обучающая выборка нужна для настройки модели, validation - для выбора архитектуры, числа эпох и регуляризации, test - для финального сравнения. Для многоклассовой задачи важно смотреть качество по каждому классу: модель может хорошо распознавать цифру 1, но путать 3 и 8. Поэтому полезны матрица ошибок и макро-усредненные метрики.

Во всех примерах роль обучающей и валидационной выборок различна. Train отвечает на вопрос “как настроить модель”, validation - “какая модель и какие настройки лучше”, test - “какого качества ожидать на новых данных”. Если эти роли смешиваются, результат становится ненадежным. Поэтому грамотный pipeline классификации строится вокруг честного разделения данных, выбора подходящей метрики, аккуратной предобработки и проверки модели на независимых объектах.

Билет 8. Постановка задачи кластеризации. Примеры. Метрики качества кластеризации

Основные понятия и определения

Кластеризация - это задача машинного обучения без учителя, в которой требуется разбить множество объектов на группы, называемые кластерами, так, чтобы объекты внутри одного кластера были похожи друг на друга, а объекты из разных кластеров существенно различались. В отличие от классификации, в кластеризации заранее не заданы правильные метки классов: алгоритм сам выявляет структуру данных по признакам объектов.

Формально имеется выборка объектов $X = \{x_1, x_2, \dots, x_n\}$, где каждый объект x_i описан набором признаков. Если признаки числовые, объект можно рассматривать как точку в пространстве $\mathbb{R}^d$. Требуется построить отображение

$$a : X \rightarrow \{1, 2, \dots, K\},$$

где $a(x_i)$ - номер кластера, к которому отнесен объект, а K - число кластеров. Иногда K известно заранее, но часто его нужно выбирать по данным или по смыслу прикладной задачи.

Ключевое понятие в кластеризации - мера близости или расстояния между объектами. Для числовых признаков часто используют евклидово расстояние:

$$\rho(x_i, x_j) = \sqrt{\sum_{m=1}^d (x_{im} - x_{jm})^2}.$$

Также применяются манхэттенское расстояние, косинусная близость, расстояние Жаккара для множеств, расстояние Хэмминга для бинарных векторов, специальные метрики для временных рядов и графов. Выбор расстояния влияет на результат не меньше, чем выбор алгоритма: два объекта могут быть близкими в одном смысле и далекими в другом.

Кластер - это не обязательно шарообразная группа точек. В зависимости от алгоритма кластеры могут пониматься как плотные области пространства, ветви иерархического дерева, компоненты графа близости или группы объектов, хорошо описываемые одним прототипом. Поэтому постановка задачи всегда включает не только данные, но и предположение о том, что считать естественной группировкой.

Теория, формулы и интуиция

Интуитивно хорошая кластеризация должна обладать двумя свойствами: компактностью и разделимостью. Компактность означает, что объекты внутри кластера расположены близко друг к другу. Разделимость означает, что разные кластеры находятся достаточно далеко или имеют разные области плотности. Эти свойства могут конфликтовать: если взять слишком много кластеров, компактность улучшится, но смысловая интерпретация ухудшится; если взять слишком мало, разные группы будут искусственно объединены.

Одна из классических формализаций используется в методе k -средних. Пусть задано число кластеров K , а у каждого кластера есть центр μ_k . Нужно минимизировать сумму квадратов расстояний от объектов до центров своих кластеров:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} |x_i - \mu_k|^2.$$

Здесь C_k - множество объектов k -го кластера. Чем меньше значение J , тем компактнее кластеры в евклидовом смысле. Однако эта функция не оценивает смысловую правильность разбиения и склонна предпочитать сферические кластеры примерно одинакового размера. Для иерархической кластеризации важна идея расстояния между кластерами. Если A и B - два кластера, то можно определить:

$$d_{\min}(A, B) = \min_{x \in A, y \in B} \rho(x, y),$$

$$d_{\max}(A, B) = \max_{x \in A, y \in B} \rho(x, y),$$

$$d_{\text{avg}}(A, B) = \frac{1}{\|A\| \|B\|} \sum_{x \in A} \sum_{y \in B} \rho(x, y).$$

Эти варианты называются single linkage, complete linkage и average linkage. Они дают разные деревья кластеров: первый хорошо находит вытянутые цепочки, но чувствителен к мостам из точек; второй строит более компактные группы; третий является компромиссом.

Качество кластеризации оценивают внутренними, внешними и относительными метриками. Внутренние метрики используют только признаки и результат кластеризации. Они полезны, когда истинных меток нет. Внешние метрики сравнивают кластеры с известной разметкой, если она есть, например в эксперименте или на контрольном наборе данных. Относительные метрики помогают сравнивать разные числа кластеров или разные алгоритмы.

Коэффициент силуэта показывает, насколько объект похож на свой кластер по сравнению с ближайшим чужим кластером. Для объекта i обозначим $a(i)$ как среднее расстояние до объектов своего кластера, а $b(i)$ как минимальное среднее расстояние до объектов другого кластера. Тогда

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}.$$

Значение $s(i)$ лежит примерно от -1 до 1 . Чем ближе к 1 , тем лучше объект вписывается в свой кластер. Значение около 0 означает, что объект находится на границе кластеров. Отрицательное значение указывает, что объект, возможно, отнесен не туда.

Индекс Дэвиса-Болдина измеряет среднюю похожесть каждого кластера на наиболее близкий к нему другой кластер:

$$DB = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} \frac{S_i + S_j}{M_{ij}},$$

где S_i - среднее расстояние объектов кластера i до его центра, а M_{ij} - расстояние между центрами кластеров i и j . Чем меньше DB, тем лучше: внутри кластеров должно быть мало разброса, а между центрами - большое расстояние.

Индекс Калински-Харабаша основан на отношении межкластерной и внутрикластерной дисперсии:

$$CH = \frac{B/(K-1)}{W/(n-K)}.$$

Здесь B - межкластерная дисперсия, W - внутрикластерная дисперсия, n - число объектов. Большое значение индекса обычно говорит о более выраженной кластерной структуре.

Если известны истинные классы, применяют внешние метрики. Adjusted Rand Index сравнивает пары объектов: оказались ли они вместе или отдельно в истинной и найденной разметке. Коррекция делает значение устойчивым к случайным совпадениям. Normalized Mutual Information измеряет, сколько информации найденные кластеры дают об истинных классах. Эти метрики важны в научных экспериментах, но в реальных задачах настоящая разметка часто отсутствует.

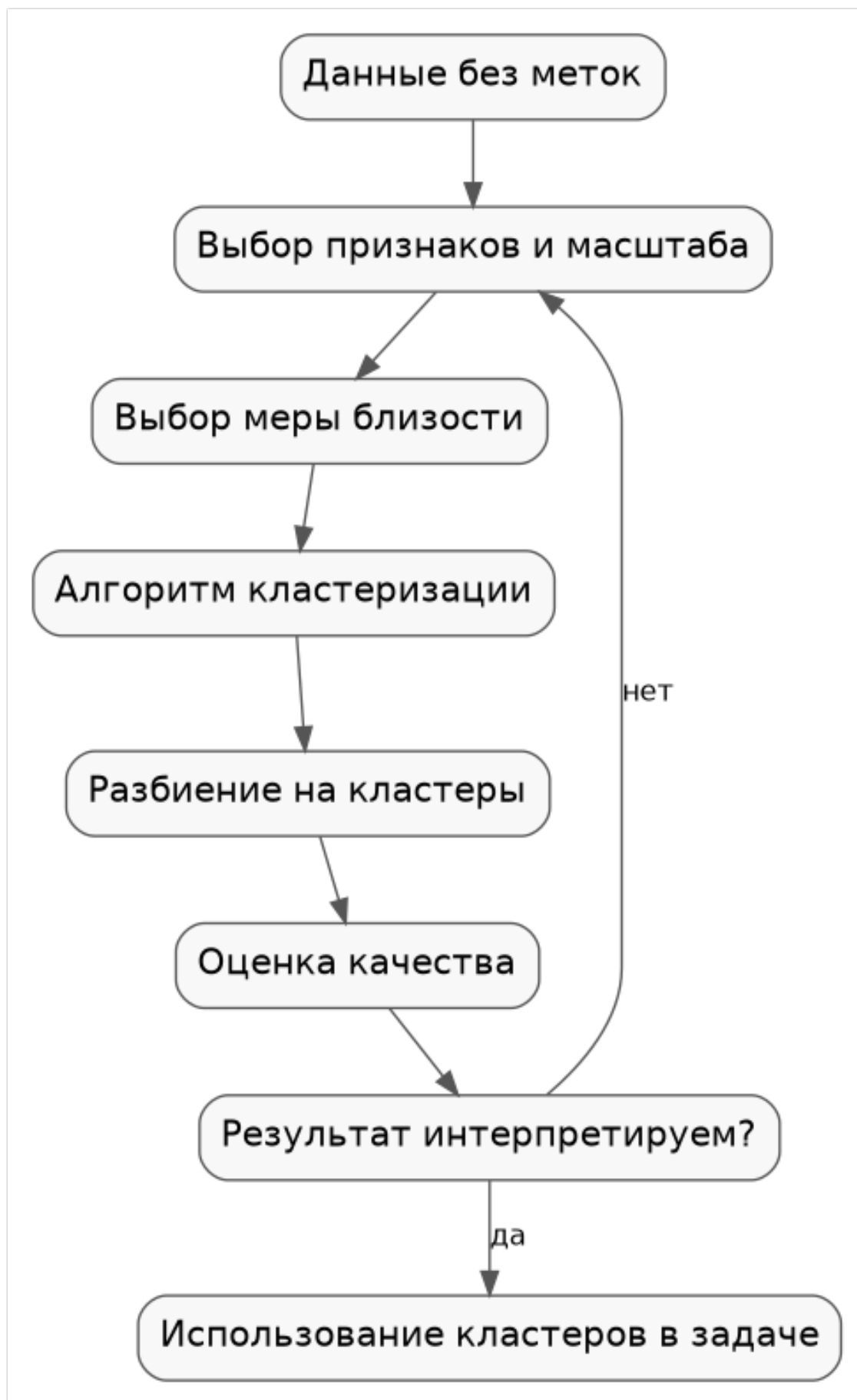


Figure 9: Схема 9

Методы, алгоритмы и этапы решения

Обычно решение задачи кластеризации начинается с подготовки данных. Числовые признаки масштабируют, потому что признаки с большими значениями могут доминировать в расстоянии. Например, если один признак измеряется в рублях, а другой в процентах, евклидово расстояние без нормировки будет почти полностью определяться рублевым признаком. Часто применяют стандартизацию, min-max нормировку, обработку выбросов, заполнение пропусков и отбор информативных признаков.

Затем выбирают меру близости. Для обычных табличных числовых данных часто подходит евклидово расстояние. Для текстов после векторизации удобно использовать косинусную близость, потому что важнее направление вектора частот или эмбединга, а не его длина. Для категориальных признаков нужны другие подходы: кодирование, расстояние Гауэра или алгоритмы, специально рассчитанные на смешанные данные.

Метод k -средних относится к прототипным методам. Он работает итеративно: сначала выбираются K центров, затем каждый объект относится к ближайшему центру, после чего центры пересчитываются как средние значения объектов в кластерах. Процесс повторяется до стабилизации. Метод прост, быстр и хорошо работает на больших данных, но требует заранее выбрать K , чувствителен к выбросам и предполагает кластеры примерно выпуклой формы.

Метод k -медоидов похож на k -средних, но центром кластера является реальный объект, а не среднее значение. Это делает алгоритм более устойчивым к выбросам и позволяет использовать произвольные матрицы расстояний. Недостаток - большая вычислительная сложность.

Иерархическая кластеризация строит дерево вложенных кластеров - дендрограмму. В агломеративном варианте каждый объект сначала является отдельным кластером, затем ближайшие кластеры последовательно объединяются. Пользователь может выбрать уровень разреза дерева и получить нужное число групп. Такой подход хорошо интерпретируется и не требует сразу фиксировать число кластеров, но плохо масштабируется на очень большие выборки.

DBSCAN относится к плотностным методам. Он формирует кластеры как области высокой плотности, разделенные областями низкой плотности. У алгоритма есть параметры ε , задающий радиус окрестности, и min_samples , задающий минимальное число соседей для плотной точки. DBSCAN умеет находить кластеры произвольной формы и выделять шум, но плохо работает при сильно различающихся плотностях кластеров.

Gaussian Mixture Model рассматривает данные как смесь нескольких нормальных распределений. Каждый объект получает не только номер кластера, но и вероятности принадлежности к компонентам. Это полезно, когда границы между группами размыты. Модель обучается обычно ЕМ-алгоритмом: на Е-шаге оцениваются вероятности принадлежности, на М-шаге обновляются параметры распределений.

Спектральная кластеризация строит граф близости объектов и использует собственные векторы матрицы графа для перехода в новое пространство, где структура кластеров становится проще. Она может хорошо работать для сложных нелинейных форм, но требует аккуратного выбора графа и может быть дорогой по памяти.

Выбор числа кластеров - отдельная задача. Для k -средних часто используют метод локтя: строят граф зависимости внутрикластерной суммы квадратов от K и ищут точку, после которой улучшение становится слабым. Также применяют силуэт, индекс Калински-Харабаша, информационные критерии AIC и BIC для вероятностных моделей. Но окончательный выбор должен учитывать смысл задачи: математически удобное число кластеров не всегда является полезным для принятия решений.

Практические примеры

В маркетинге кластеризация используется для сегментации клиентов. Объекты - клиенты, признаки - частота покупок, средний чек, давность последней покупки, категории товаров, реакция на акции. После кластеризации могут обнаружиться группы: постоянные покупатели с высоким чеком, редкие покупатели, чувствительные к скидкам, новые клиенты без устойчивого поведения. Для такой задачи важно не только получить высокий силуэт, но и дать каждому сегменту понятную бизнес-интерпретацию.

В обработке текстов кластеризация помогает группировать документы по темам. Документы переводят в векторы с помощью TF-IDF или эмбедингов, затем применяют k -средних, иерархическую кластеризацию или HDBSCAN. Например, новости могут автоматически разделиться на экономику, спорт, политику и технологии. Для текстов часто используется косинусная близость, поскольку абсолютная длина документа не должна быть главным фактором.

В медицине кластеризация может применяться для поиска подтипов заболеваний. Пациенты описываются лабораторными показателями, симптомами, возрастом, историей лечения. Найденные кластеры могут помочь сформулировать гипотезы о разных механизмах болезни. Однако такая

интерпретация требует особой осторожности: кластер - это статистическая группа, а не автоматически доказанная клиническая категория.

В анализе изображений кластеризация используется для сегментации. Например, пиксели изображения можно кластеризовать по цвету и координатам, чтобы выделить области: фон, объект, тень, текстуру. Метод k -средних часто применяют для сжатия цветовой палитры: все цвета изображения заменяются ближайшими центрами кластеров, и изображение сохраняет общий вид при меньшем числе цветов.

В информационной безопасности кластеризация помогает искать аномалии. Нормальные сетевые соединения образуют плотные группы по длительности, объему трафика, числу пакетов, портам и протоколам. Точки, не попадающие ни в один плотный кластер, могут быть подозрительными. Здесь особенно полезны плотностные методы, потому что они явно выделяют шум и не требуют относить каждый объект к какой-либо группе.

На практике качество кластеризации проверяют не одной метрикой, а сочетанием критериев. Например, можно сравнить несколько алгоритмов по silhouette score, Davies-Bouldin и Calinski-Harabasz, затем визуализировать данные через PCA, t-SNE или UMAP, после чего проверить, имеют ли кластеры прикладной смысл. Хороший результат - это не просто красивые численные значения, а устойчивое, интерпретируемое и полезное разбиение данных.

Билет 9. Методы решения задачи кластеризации. Иерархические методы. Алгоритм k-means

Основные понятия и определения

Кластеризация - это задача обучения без учителя, в которой требуется разбить множество объектов на группы, называемые кластерами, так, чтобы объекты внутри одной группы были похожи друг на друга, а объекты из разных групп заметно различались. В отличие от классификации, здесь заранее нет правильных меток классов: алгоритм сам ищет скрытую структуру данных по признаковому описанию объектов.

Пусть задана выборка $X = \{x_1, x_2, \dots, x_n\}$, где каждый объект x_i описан вектором признаков из пространства $\mathbb{R}^d$. Результатом кластеризации является разбиение выборки на K непересекающихся подмножеств $C_1, C_2, \dots, C_K$, таких что каждый объект относится хотя бы к одному кластеру. В классической жесткой кластеризации объект принадлежит ровно одному кластеру. В мягкой кластеризации, например в вероятностных моделях смесей, объект может иметь степени принадлежности к нескольким кластерам.

Понятие похожести обычно задается через меру расстояния или сходства. Для числовых признаков часто используют евклидово расстояние:

$$\rho(x_i, x_j) = \sqrt{\sum_{m=1}^d (x_{im} - x_{jm})^2}.$$

Также применяются манхэттенское расстояние, косинусная мера, расстояние Чебышева, расстояния для категориальных признаков и специальные доменные метрики. Выбор метрики принципиален: алгоритм кластеризации не знает смысл данных, он видит только геометрию, заданную признаками и расстояниями. Поэтому перед кластеризацией обычно проводят нормализацию или стандартизацию признаков, иначе признаки с большими числовыми масштабами начинают доминировать.

Кластер можно понимать по-разному. В центроидных методах кластер - это группа объектов вокруг некоторого центра. В иерархических методах кластер - это узел дерева объединений или разбиений. В плотностных методах кластер - это область повышенной плотности точек. В графовых подходах кластер соответствует плотно связанной компоненте графа похожести. Поэтому не существует универсального алгоритма, который одинаково хорошо решает все задачи: форма кластеров, наличие шума, размерность данных и требование интерпретируемости влияют на выбор метода.

Теория, формулы и интуиция

Общая идея кластеризации состоит в оптимизации некоторого критерия качества. Такой критерий может явно задаваться, как в k-means, или быть неявным, как в агломеративной иерархической кластеризации, где на каждом шаге выбирается пара наиболее близких кластеров. Хорошая кластеризация обычно имеет малую внутрикластерную вариативность и большую межкластерную

отделимость.

Для кластера C_k можно определить центроид:

$$\mu_k = \frac{1}{\|C_k\|} \sum_{x_i \in C_k} x_i.$$

Центроид - это средний вектор объектов кластера. Внутрикластерная сумма квадратов расстояний до центроидов записывается так:

$$WCSS = \sum_{k=1}^K \sum_{x_i \in C_k} |x_i - \mu_k|^2.$$

Именно эту величину минимизирует алгоритм k-means. Интуитивно он ищет такие центры кластеров, чтобы каждая точка была как можно ближе к центру своего кластера. Чем меньше $WCSS$, тем компактнее кластеры. Однако само по себе уменьшение $WCSS$ не означает, что найдено содержательно правильное разбиение: при увеличении K значение критерия почти всегда уменьшается, потому что большее число центров лучше подгоняет данные.

Для выбора числа кластеров часто используют метод локтя. Строят график зависимости $WCSS(K)$ от числа кластеров K . Сначала при росте K ошибка резко падает, затем улучшение становится слабым. Точка перегиба воспринимается как разумный компромисс между простотой модели и качеством описания данных. Другой подход - силуэт:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))},$$

где $a(i)$ - среднее расстояние от объекта i до объектов своего кластера, а $b(i)$ - минимальное среднее расстояние от объекта i до объектов другого ближайшего кластера. Значение силуэта близко к 1 означает удачное размещение объекта, около 0 - объект находится на границе кластеров, отрицательное значение - объект, вероятно, попал не в свой кластер.

Иерархическая кластеризация строит не одно разбиение, а целую вложенную систему кластеров. Ее результат удобно изображать дендрограммой - деревом, где листья соответствуют отдельным объектам, а внутренние узлы показывают объединение кластеров. Высота объединения отражает расстояние или меру неоднородности между объединяемыми кластерами. Чтобы получить конкретное число кластеров, дендрограмму "разрезают" на выбранной высоте.

Главный теоретический вопрос в агломеративной иерархической кластеризации - как определить расстояние между кластерами A и B , если каждый из них содержит много точек. Используются разные правила связи:

- single linkage: $D(A, B) = \min_{x \in A, y \in B} \rho(x, y)$, расстояние между ближайшими точками кластеров;
- complete linkage: $D(A, B) = \max_{x \in A, y \in B} \rho(x, y)$, расстояние между самыми удаленными точками;
- average linkage: среднее расстояние между всеми парами точек из разных кластеров;
- centroid linkage: расстояние между центроидами кластеров;
- метод Уорда: объединяются кластеры, дающие минимальный прирост внутрикластерной суммы квадратов.

Single linkage хорошо находит вытянутые цепочечные структуры, но чувствителен к "мостикам" из близких точек и может объединять разные группы через шум. Complete linkage стремится строить компактные кластеры, но может дробить вытянутые формы. Average linkage является более сбалансированным вариантом. Метод Уорда часто хорошо работает с евклидовыми числовыми данными и дает кластеры, похожие по духу на результат k-means, но при этом сохраняет иерархическую структуру.

Методы, алгоритмы и этапы решения

Методы кластеризации можно разделить на несколько больших групп. Центроидные методы представляют кластеры через центры; главный пример - k-means. Иерархические методы строят дерево объединений или разбиений. Плотностные методы, например DBSCAN, ищут области высокой плотности и позволяют выделять шум. Модельные методы предполагают, что данные порождены вероятностной моделью, например смесью гауссовских распределений. Спектральные методы

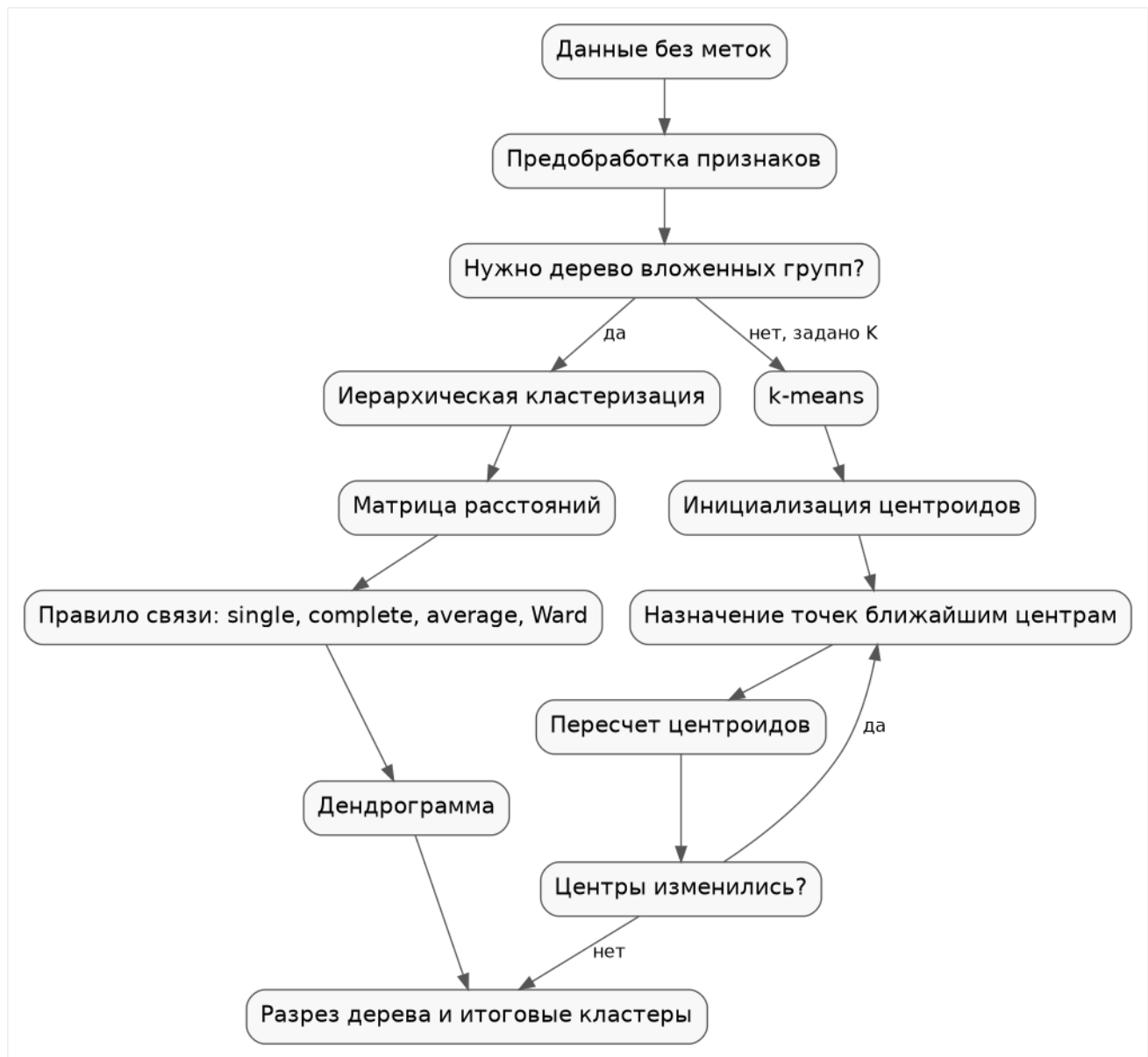


Figure 10: Схема 10

используют собственные векторы матриц графа похожести и полезны для сложной нелинейной структуры данных.

Иерархические методы бывают агломеративными и дивизивными. Агломеративный подход работает снизу вверх: сначала каждый объект считается отдельным кластером, затем на каждом шаге объединяются два наиболее близких кластера. Процесс продолжается, пока не останется один общий кластер или пока не будет достигнуто заданное число групп. Дивизивный подход работает сверху вниз: сначала все объекты находятся в одном кластере, затем он последовательно разбивается на части. На практике агломеративные методы используются чаще, потому что они проще и имеют более стандартную реализацию.

Алгоритм агломеративной иерархической кластеризации можно описать так:

1. Выбрать признаки, метрику расстояния и способ связи между кластерами.
2. Поместить каждый объект в отдельный кластер.
3. Вычислить расстояния между всеми парами кластеров.
4. Найти пару кластеров с минимальным расстоянием и объединить ее.
5. Обновить расстояния от нового кластера до остальных кластеров.
6. Повторять объединения до получения полной дендрограммы.
7. Выбрать уровень разреза дендрограммы и получить итоговое разбиение.

Сильная сторона иерархических методов - интерпретируемость. Исследователь видит не только итоговые группы, но и то, как они появляются при разных уровнях детализации. Это полезно, когда заранее неизвестно число кластеров или нужно объяснить структуру данных. Недостатки тоже существенны: вычисление матрицы расстояний требует порядка $O(n^2)$ памяти, а полный алгоритм плохо масштабируется на очень большие выборки. Кроме того, ранние ошибочные объединения обычно нельзя исправить на следующих шагах.

Алгоритм k-means решает другую задачу: он сразу ищет K компактных групп вокруг центроидов. Число кластеров K задается заранее. Классическая версия работает с евклидовым расстоянием и числовыми признаками. Этапы алгоритма следующие:

1. Задать число кластеров K .
2. Инициализировать K центроидов, случайно или методом k-means++.
3. Для каждой точки найти ближайший центроид и назначить точку соответствующему кластеру.
4. Для каждого кластера пересчитать центроид как среднее всех его точек.
5. Повторять шаги назначения и пересчета, пока центроиды или назначения почти не перестанут изменяться.

Формально на шаге назначения для каждого объекта выбирается номер кластера:

$$c_i = \arg \min_{k \in \{1, \dots, K\}} |x_i - \mu_k|^2.$$

Затем центры обновляются по формуле среднего:

$$\mu_k = \frac{1}{\|C_k\|} \sum_{x_i \in C_k} x_i.$$

Каждая итерация k-means не увеличивает $WCSS$, поэтому алгоритм сходится за конечное число шагов к локальному минимуму. Но локальный минимум может зависеть от начальной инициализации. Поэтому на практике алгоритм запускают несколько раз с разными начальными центрами и выбирают решение с минимальным $WCSS$. Инициализация k-means++ уменьшает риск плохого старта: первый центр выбирается случайно, а следующие выбираются с большей вероятностью среди точек, удаленных от уже выбранных центров.

Преимущества k-means - простота, скорость и хорошая масштабируемость. При фиксированном K , числе итераций T , числе объектов n и размерности d сложность примерно равна $O(nKdT)$, что позволяет применять метод к большим данным. Но у него есть ограничения. Он предполагает компактные примерно сферические кластеры сходного размера, чувствителен к выбросам, требует заранее выбрать K , плохо работает с категориальными признаками без специального кодирования и может давать неустойчивый результат при сильной корреляции или разном масштабе признаков.

Перед применением обоих подходов важно подготовить данные. Обычно удаляют или обрабатывают пропуски, кодируют категориальные признаки, стандартизуют числовые признаки, анализируют выбросы и при необходимости уменьшают размерность методом PCA или другими техниками. После получения кластеров результат интерпретируют: сравнивают распределения признаков по группам,

строят профили кластеров, проверяют устойчивость при изменении параметров и оценивают качество через силуэт, индекс Калински-Харабаса, индекс Дэвиса-Болдина или внешние метрики, если есть экспертные метки.

Практические примеры

Первый пример - сегментация клиентов интернет-магазина. Для каждого клиента можно взять признаки: частота покупок, средний чек, давность последней покупки, доля возвратов, интерес к категориям товаров. Если бизнес заранее хочет получить, например, пять рабочих сегментов для маркетинговых кампаний, подойдет k-means. После стандартизации признаков алгоритм может выделить группы “частые покупатели с высоким чеком”, “редкие покупатели”, “новые клиенты”, “клиенты с риском ухода” и “охотники за скидками”. Названия кластеров задает не алгоритм, а аналитик после изучения средних значений признаков в каждой группе.

Второй пример - анализ учебных результатов студентов. Если неизвестно, сколько типов траекторий существует, полезна иерархическая кластеризация. Объекты описываются баллами по темам, посещаемостью, активностью в системе и сроками сдачи работ. Дендрограмма может показать, что на верхнем уровне студенты делятся на сильную и проблемную группы, а на более низком уровне внутри проблемной группы есть разные причины: слабая база, нерегулярная работа или трудности только с отдельной темой. Такой результат удобен для преподавателя, потому что дерево дает несколько уровней детализации.

Третий пример - группировка документов. Тексты переводят в векторы признаков, например через TF-IDF или эмбединги. Для коротких новостей k-means может быстро сгруппировать документы по темам, если заранее задано число рубрик. Но если нужно исследовать коллекцию без заранее известной структуры, иерархический метод с косинусным расстоянием позволит увидеть близкие подтемы: экономика может делиться на банки, рынки и налоги, а политика - на выборы, международные отношения и законодательство.

Четвертый пример - обработка изображений. В задаче сжатия цвета k-means можно применить к пикселям, где каждый пиксель описан тройкой RGB. Если выбрать $K = 16$, алгоритм найдет 16 типичных цветов изображения, а каждый пиксель заменяется ближайшим центроидом. Так изображение хранится с меньшим числом цветов. Это показывает, что кластеризация полезна не только для аналитики таблиц, но и как технический инструмент преобразования данных.

Иерархические методы и k-means часто дополняют друг друга. Иерархическую кластеризацию можно использовать на подвыборке, чтобы понять примерное число групп и выбрать K , а затем применить k-means ко всей большой выборке. Обратная стратегия тоже возможна: сначала быстро получить много мелких кластеров с помощью k-means, а затем кластеризовать их центроиды иерархическим методом. В реальных задачах качество решения определяется не только алгоритмом, но и тем, насколько осмысленно выбраны признаки, метрика расстояния, масштабирование и способ интерпретации результата.

Билет 10. Деревья решений. Обучение дерева решений

Основные понятия и определения

Дерево решений — это модель машинного обучения, которая делает прогноз как последовательность простых проверок над признаками объекта. Внутренние вершины дерева содержат условия вида $x_j \leq t$, $x_j \in A$ или «значение признака равно категории», ребра соответствуют исходам проверки, а листья содержат итоговый ответ модели. В задаче классификации лист обычно хранит класс или распределение вероятностей по классам, в задаче регрессии — численное значение, например среднее целевой переменной среди объектов, попавших в этот лист.

Главная идея дерева решений состоит в разбиении пространства признаков на области, внутри которых ответ можно считать достаточно однородным. Если модель решает задачу выдачи кредита, дерево может сначала проверить доход клиента, затем возраст, затем наличие просрочек. Если объект проходит по некоторому пути от корня к листу, лист определяет прогноз: «одобрить» или «отказать». Поэтому дерево удобно интерпретировать как набор правил вида «если выполнены такие-то условия, то ответ такой-то».

Обучающая выборка задается набором объектов и ответов:

$$D = \{(x_i, y_i)\}_{i=1}^n,$$

где $x_i = (x_{i1}, \dots, x_{id})$ — вектор признаков, y_i — целевая переменная. Для классификации y_i принимает

значения из конечного множества классов, для регрессии является числом. Дерево строится по обучающим данным так, чтобы в листьях оказывались группы объектов с как можно более похожими ответами.

Важные элементы дерева:

- корень — начальная вершина, куда попадают все объекты;
- внутренний узел — вершина с условием разбиения;
- лист — конечная вершина, в которой выдается прогноз;
- глубина дерева — максимальное число проверок на пути от корня до листа;
- разбиение, или *split*, — выбор признака и условия, по которому множество объектов делится на подмножества;
- критерий качества разбиения — мера того, насколько после разделения данные стали более однородными.

Деревья относятся к непараметрическим моделям: они не задают заранее фиксированную формулу зависимости y от x , а адаптируют форму решения к данным. Это делает их гибкими: они умеют находить нелинейные зависимости, учитывать взаимодействия признаков и работать как с числовыми, так и с категориальными признаками. При этом у дерева есть важный недостаток: если не ограничивать рост, оно легко переобучается, то есть хорошо запоминает обучающую выборку, но хуже работает на новых объектах.

Теория, формулы и интуиция

Обучение дерева основано на жадном рекурсивном выборе разбиений. На каждом шаге алгоритм рассматривает текущую группу объектов и пытается найти такой вопрос к данным, который лучше всего уменьшит неоднородность целевой переменной. Для классификации неоднородность означает смешение классов в узле: если в узле есть объекты разных классов, прогноз менее уверен; если почти все объекты одного класса, узел чистый. Для регрессии неоднородность обычно означает большой разброс числовых ответов.

Пусть в узел S попало m объектов. Для классификации доля класса k в этом узле равна

$$p_k = \frac{1}{m} \sum_{(x_i, y_i) \in S} [y_i = k].$$

Один из популярных критериев неоднородности — энтропия:

$$H(S) = - \sum_{k=1}^K p_k \log_2 p_k.$$

Если все объекты в узле принадлежат одному классу, энтропия равна нулю. Если классы представлены примерно одинаково, энтропия максимальна. Интуитивно энтропия показывает неопределенность ответа в узле.

Другой распространенный критерий — индекс Джини:

$$Gini(S) = 1 - \sum_{k=1}^K p_k^2.$$

Он также равен нулю для полностью чистого узла и растет, когда классы перемешаны. Индекс Джини часто используется в алгоритме CART, потому что он прост в вычислении и хорошо работает на практике.

Чтобы оценить конкретное разбиение узла S на подмножества S_L и S_R , сравнивают неоднородность до и после *split*. Например, информационный выигрыш через энтропию:

$$IG(S, split) = H(S) - \frac{\|S_L\|}{\|S\|} H(S_L) - \frac{\|S_R\|}{\|S\|} H(S_R).$$

Чем больше IG , тем лучше разбиение: оно сильнее уменьшает неопределенность. Для критерия Джини аналогично минимизируют взвешенную сумму неоднородностей дочерних узлов:

$$Q(split) = \frac{\|S_L\|}{\|S\|} Gini(S_L) + \frac{\|S_R\|}{\|S\|} Gini(S_R).$$

Лучшим считается разбиение с минимальным Q или максимальным уменьшением Джини. Для регрессии вместо энтропии и Джини используют меры разброса. Чаще всего в листе предсказывается среднее значение:

$$\hat{y} * S = \frac{1}{\|S\|} \sum_{(x_i, y_i) \in S} y_i,$$

а качество узла измеряется суммой квадратов отклонений:

$$MSE(S) = \frac{1}{\|S\|} \sum_{(x_i, y_i) \in S} (y_i - \hat{y}_S)^2.$$

Разбиение выбирают так, чтобы уменьшить среднеквадратичную ошибку в дочерних узлах. Интуитивно дерево регрессии пытается разделить данные на области, где целевая переменная почти постоянна.

Пример структуры дерева можно представить так:

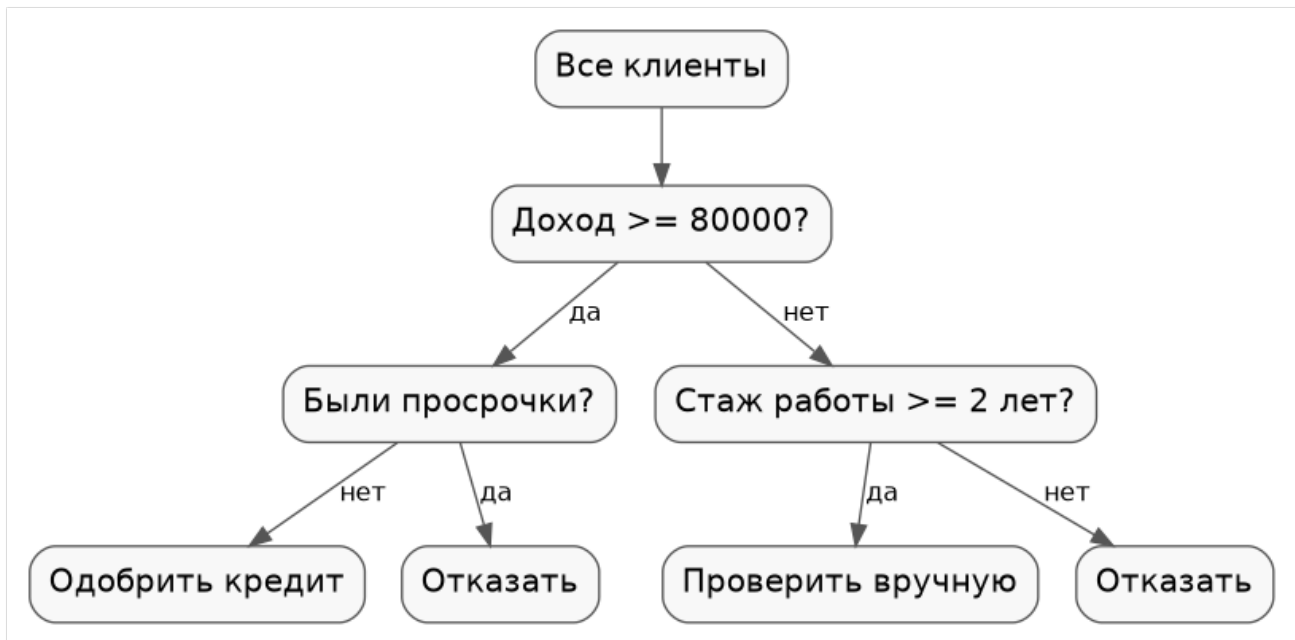


Figure 11: Схема 11

В этой схеме каждый путь от корня к листу соответствует правилу. Например: если доход не меньше 80000 и просрочек нет, дерево выдает «одобрить кредит». Такие правила легко объяснить человеку, что отличает деревья от многих менее интерпретируемых моделей.

Однако высокая интерпретируемость сохраняется только у небольших деревьев. Глубокое дерево может содержать сотни условий, и тогда оно становится сложным для анализа. Кроме того, глубокие деревья имеют высокую дисперсию: малое изменение обучающей выборки может заметно изменить структуру дерева. Поэтому на практике важны ограничения роста и методы обрезки.

Методы, алгоритмы и этапы решения

Классическое обучение дерева решений выполняется рекурсивно. В корневой узел помещают всю обучающую выборку. Затем алгоритм перебирает возможные разбиения и выбирает лучшее по выбранному критерию. После этого он повторяет процедуру отдельно для каждого дочернего узла. Процесс продолжается до выполнения критерия остановки.

Общий алгоритм построения бинарного дерева можно описать так:

1. Поместить текущую выборку S в узел.
2. Проверить условия остановки: узел уже достаточно чистый, достигнута максимальная глубина, объектов слишком мало или улучшение качества слишком мало.

3. Если остановка выполнена, сделать узел листом и записать в него прогноз.
4. Иначе перебрать кандидаты на разбиение: признаки и пороги для числовых признаков, варианты разделения категорий для категориальных.
5. Для каждого кандидата вычислить качество разбиения.
6. Выбрать лучший split и разделить S на дочерние подмножества.
7. Рекурсивно повторить процедуру для дочерних узлов.

Для числового признака x_j обычно рассматриваются пороги между соседними отсортированными уникальными значениями. Условие имеет вид $x_j \leq t$. Например, если признак «возраст» принимает значения 20, 25, 30 и 40, возможные пороги могут быть 22.5, 27.5 и 35. Для категориального признака можно проверять принадлежность некоторому множеству категорий, но полный перебор всех множеств дорогой, поэтому реализации часто используют упрощения или предварительное кодирование категорий.

Существуют разные исторические алгоритмы деревьев решений. ID3 использует информационный выигрыш на основе энтропии и ориентирован прежде всего на категориальные признаки. C4.5 развивает ID3: поддерживает числовые признаки, пропущенные значения и использует gain ratio, который уменьшает склонность выбирать признаки с большим числом уникальных значений. CART строит бинарные деревья и применим как к классификации, так и к регрессии; для классификации часто использует индекс Джини, для регрессии — MSE.

Критерии остановки нужны, чтобы дерево не запоминало шум. Типичные параметры:

- max_depth — максимальная глубина дерева;
- min_samples_split — минимальное число объектов, при котором узел можно делить;
- min_samples_leaf — минимальное число объектов в листе;
- min_impurity_decrease — минимальное уменьшение неоднородности для выполнения split;
- max_leaf_nodes — максимальное число листьев.

Еще один важный подход — обрезка дерева, или pruning. Предварительная обрезка ограничивает рост дерева во время построения с помощью перечисленных параметров. Пост-обрезка сначала строит большое дерево, а затем удаляет части, которые плохо улучшают качество на валидации или слишком сильно усложняют модель. В CART используется cost-complexity pruning: минимизируется функционал, где ошибка дерева дополняется штрафом за число листьев:

$$R_\alpha(T) = R(T) + \alpha \|T\|,$$

где $R(T)$ — ошибка дерева, $\|T\|$ — число листьев, α — коэффициент сложности. Чем больше α , тем сильнее штраф за размер дерева и тем проще итоговая модель.

При работе с деревьями важно корректно оценивать качество. Данные делят на обучающую, валидационную и тестовую части или используют кросс-валидацию. Гиперпараметры дерева подбирают по валидационной выборке, а итоговую оценку получают на тестовой, которую не использовали при настройке. Для классификации применяют accuracy, precision, recall, F1, ROC-AUC; для регрессии — MAE, MSE, RMSE, R^2 .

Деревья умеют оценивать важность признаков. В простом варианте важность признака измеряют суммарным уменьшением неоднородности во всех узлах, где этот признак использовался. Если разбиения по признаку часто дают большое улучшение, признак считается важным. Но такую важность нужно трактовать осторожно: признаки с большим числом возможных разбиений могут получать завышенную оценку, а коррелирующие признаки делят важность между собой.

Практические примеры

Рассмотрим задачу классификации заявок на кредит. Есть признаки: доход, возраст, стаж работы, наличие прошлых просрочек, размер кредита. Целевая переменная — «кредит возвращен» или «кредит не возвращен». Дерево может сначала выбрать признак «были просрочки», потому что он сильно разделяет клиентов по риску. Затем для клиентов без просрочек оно может проверить отношение суммы кредита к доходу, а для клиентов с просрочками — стаж работы. В итоге получаются понятные правила: «если просрочек нет и долговая нагрузка мала, заявка надежна».

Для небольшой иллюстрации пусть в узле 10 клиентов: 6 надежных и 4 рискованных. Доли классов равны $p_1 = 0.6$ и $p_2 = 0.4$. Индекс Джини:

$$Gini = 1 - 0.6^2 - 0.4^2 = 0.48.$$

Если разбиение по признаку «просрочки» дает левый узел из 5 надежных и 1 рискованного клиента, а

правый узел из 1 надежного и 3 рискованных, то неоднородность заметно уменьшается. Для левого узла

$$Gini_L = 1 - (5/6)^2 - (1/6)^2 \approx 0.278,$$

для правого:

$$Gini_R = 1 - (1/4)^2 - (3/4)^2 = 0.375.$$

Взвешенная неоднородность после разбиения:

$$Q = \frac{6}{10} \cdot 0.278 + \frac{4}{10} \cdot 0.375 \approx 0.317.$$

Так как $0.317 < 0.48$, разбиение улучшает чистоту узлов. Алгоритм сравнит это улучшение с другими возможными разбиениями и выберет лучшее.

В задаче регрессии дерево можно использовать для прогнозирования цены квартиры. Признаки: площадь, район, этаж, год постройки, расстояние до метро. Первый split может быть по площади, например «площадь ≤ 45 ». Затем внутри малых квартир дерево может разделить объекты по району, а внутри больших — по расстоянию до метро. В каждом листе прогнозом будет средняя цена квартир, попавших в этот лист. Такое дерево фактически строит кусочно-постоянную аппроксимацию: для каждой области признакового пространства назначается своя средняя цена.

На практике дерево решений часто используют как базовую интерпретируемую модель. Его удобно применять, когда нужно объяснить прогноз в терминах простых условий. Например, в медицине дерево может показать, какие признаки пациента привели к предварительному решению о риске заболевания. В бизнес-аналитике дерево помогает сегментировать клиентов: одни листья соответствуют клиентам с высокой вероятностью оттока, другие — с низкой.

При этом одиночное дерево редко является самой точной моделью на сложных данных. Из-за склонности к переобучению его часто заменяют ансамблями деревьев: случайным лесом, градиентным бустингом, XGBoost, LightGBM или CatBoost. Эти методы используют деревья как строительные блоки и обычно дают более высокое качество, но теряют часть простой интерпретируемости. Поэтому выбор зависит от задачи: если важна прозрачность, подходит небольшое ограниченное дерево; если важнее максимальная точность, чаще выбирают ансамбли.

Итоговый рабочий процесс обычно выглядит так: подготовить данные, обработать пропуски и категориальные признаки, разделить выборку, обучить дерево с разумными ограничениями глубины, подобрать гиперпараметры по валидации, оценить качество на тесте и проанализировать получившиеся правила. Такой подход позволяет использовать сильные стороны деревьев — наглядность, гибкость и способность находить нелинейные зависимости — и одновременно контролировать их главный риск, то есть переобучение.

Билет 11. Алгоритм k-ближайших соседей

Основные понятия и определения

Алгоритм k-ближайших соседей, или k-Nearest Neighbors (kNN), - это метод машинного обучения с учителем, применяемый для классификации и регрессии. Его основная идея проста: похожие объекты обычно имеют похожие ответы. Если новый объект находится рядом с объектами одного класса, то его разумно отнести к этому классу. Если нужно предсказать числовое значение, то его можно оценить по значениям у ближайших похожих объектов.

kNN называют ленивым алгоритмом обучения. На этапе обучения он не строит явную модель, не подбирает коэффициенты и не оценивает параметры распределения. Он фактически запоминает обучающую выборку. Основная работа выполняется во время предсказания: для каждого нового объекта алгоритм ищет ближайшие объекты из обучающего набора и агрегирует их ответы.

Пусть дана обучающая выборка:

$$X = \{(x_i, y_i)\}_{i=1}^n,$$

где x_i - вектор признаков i-го объекта, y_i - известный ответ, n - число объектов. В задаче классификации

ответ принадлежит конечному набору классов, например “спам” или “не спам”. В задаче регрессии ответ является числом: ценой, температурой, рейтингом, вероятной выручкой.

Параметр k задает количество соседей, участвующих в принятии решения. При $k = 1$ новый объект получает ответ самого близкого объекта. При большем k решение становится устойчивее к отдельным ошибочным меткам и выбросам, но слишком большое значение может сгладить локальную структуру данных. Поэтому k обычно подбирают по валидационной выборке или с помощью кросс-валидации.

Важнейшая часть kNN - метрика расстояния. Она определяет, какие объекты считать похожими. Для числовых признаков часто используют евклидово расстояние. Для признаков, где важна сумма абсолютных отличий, применяют манхэттенское расстояние. Для текстовых векторов и эмбедингов часто используют косинусную близость. Если метрика не соответствует смыслу задачи, алгоритм будет находить формально близкие, но практически непохожие объекты.

Теория, формулы и интуиция

Каждый объект можно представить точкой в пространстве признаков. Например, студент может описываться двумя признаками: количеством часов подготовки и баллом за пробный тест. Тогда похожие студенты окажутся рядом на плоскости. В реальных задачах признаков может быть десятки или тысячи, но логика сохраняется: ответ для нового объекта оценивается по его локальному окружению.

Евклидово расстояние между объектами x и z задается формулой:

$$\rho(x, z) = \sqrt{\sum_{j=1}^m (x_j - z_j)^2},$$

где m - число признаков. Манхэттенское расстояние имеет вид:

$$\rho(x, z) = \sum_{j=1}^m \|x_j - z_j\|.$$

Обобщением является расстояние Минковского:

$$\rho_p(x, z) = \left(\sum_{j=1}^m \|x_j - z_j\|^p \right)^{1/p}.$$

При $p = 1$ получается манхэттенская метрика, при $p = 2$ - евклидова.

Для классификации после поиска k ближайших соседей используется голосование. Пусть $N_k(x)$ - множество индексов k ближайших объектов к новому объекту x . Тогда:

$$\hat{y}(x) = \arg \max_c \sum_{i \in N_k(x)} I(y_i = c),$$

где $I(y_i = c)$ равно 1, если сосед принадлежит классу c , и 0 иначе. Предсказывается класс, который чаще всего встречается среди соседей.

Часто применяют взвешенное голосование: более близкий сосед влияет сильнее, чем более дальний. Например:

$$w_i = \frac{1}{\rho(x, x_i) + \varepsilon},$$

где ε - малое положительное число, защищающее от деления на ноль. Тогда класс выбирается по максимальной сумме весов соседей этого класса:

$$\hat{y}(x) = \arg \max_c \sum_{i \in N_k(x)} w_i I(y_i = c).$$

Для регрессии базовое предсказание равно среднему значению целевой переменной среди соседей:

$$\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i.$$

Во взвешенном варианте:

$$\hat{y}(x) = \frac{\sum_{i \in N_k(x)} w_i y_i}{\sum_{i \in N_k(x)} w_i}.$$

Интуитивно kNN строит локальное приближение зависимости. Он не ищет одну глобальную формулу для всех данных, как линейная регрессия, а предполагает, что в небольшой окрестности объекта ответы меняются относительно плавно. Поэтому метод может хорошо работать при сложных границах классов: граница решения возникает из расположения обучающих объектов.

При этом kNN чувствителен к масштабу признаков. Если один признак измеряется в тысячах, а другой в долях единицы, первый почти полностью определит расстояние. Поэтому перед применением алгоритма признаки обычно стандартизируют:

$$x' = \frac{x - \mu}{\sigma},$$

или нормализуют в диапазон от 0 до 1:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Еще одна проблема - проклятие размерности. В пространствах большой размерности расстояния становятся менее информативными: многие объекты оказываются примерно одинаково далекими. Для борьбы с этим используют отбор признаков, снижение размерности, например PCA, или более подходящие метрики.

Сильная сторона kNN состоит в том, что результат можно объяснить через конкретные наблюдения: модель не просто выдает класс или число, а показывает, какие объекты оказались ближайшими. Это удобно в учебных, аналитических и прикладных задачах, где важно не только получить прогноз, но и понять его основание. В то же время качество такого объяснения зависит от качества данных: если в выборке много дублей, ошибок, нерелевантных признаков или устаревших наблюдений, ближайшие соседи будут вводить в заблуждение.

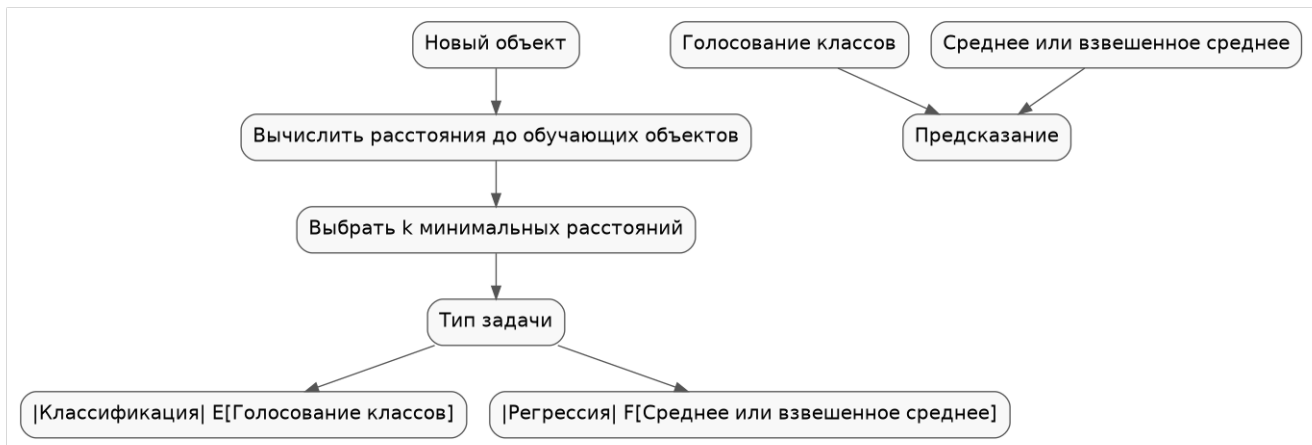


Figure 12: Схема 12

Методы, алгоритмы и этапы решения

Практическое применение kNN обычно состоит из следующих этапов.

1. Подготовить данные. Нужно выбрать признаки, обработать пропуски, удалить явные ошибки, закодировать категориальные переменные. Например, категориальный признак “район” можно представить с помощью one-hot encoding.
2. Масштабировать признаки. Для метрического алгоритма это почти всегда обязательный шаг. Без

масштабирования расстояние будет отражать не реальную похожесть объектов, а единицы измерения признаков.

3. Выбрать метрику. Евклидова метрика подходит как базовый вариант для числовых признаков. Манхэттенская может быть устойчивее, если признаки имеют разные направления изменения. Косинусная мера полезна для текстов и векторных представлений, где важен угол между векторами.

4. Подобрать k . Малое k делает модель гибкой, но чувствительной к шуму. Большое k уменьшает влияние случайностей, но может привести к чрезмерному усреднению. На практике проверяют несколько значений, например 1, 3, 5, 7, 11, 15, и выбирают лучшее по валидационной метрике.

5. Для нового объекта вычислить расстояния до всех объектов обучающей выборки. В наивной реализации это полный перебор.

6. Отсортировать объекты по расстоянию и выбрать k ближайших.

7. Получить ответ: для классификации выполнить голосование, для регрессии посчитать среднее или взвешенное среднее.

8. Оценить качество на тестовой выборке. Для классификации применяют accuracy, precision, recall, F1-score, ROC-AUC; для регрессии - MAE, MSE, RMSE, R^2 .

Наивный поиск соседей для одного нового объекта требует порядка $O(nm)$ операций, где n - число обучающих объектов, m - число признаков. Если данных много, предсказание становится дорогим. Для ускорения используют KD-деревья, ball tree и методы приближенного поиска ближайших соседей. Такие структуры особенно полезны при умеренной размерности пространства.

Выбор k связан с компромиссом между смещением и разбросом. При $k = 1$ модель почти идеально следует обучающим данным, но легко переобучается. При большом k модель устойчивее, но может игнорировать локальные закономерности. Хорошее значение обычно находится между этими крайностями.

Также нужно учитывать дисбаланс классов. Если один класс встречается намного чаще, обычное голосование может систематически выбирать его. В этом случае помогают взвешивание соседей, балансировка данных или выбор метрик качества, учитывающих редкий класс.

Практические примеры

Первый пример - классификация клиентов банка. Пусть для каждого клиента известны возраст, доход, сумма кредита и факт просрочки. Нужно определить, склонен ли новый клиент к просрочке. После масштабирования признаков алгоритм ищет, например, $k = 5$ наиболее похожих клиентов. Если среди них четыре клиента имели просрочку, а один не имел, kNN предскажет класс "просрочка". Если используется взвешенное голосование, то самый похожий клиент будет иметь больший вес, чем сосед, расположенный дальше.

Второй пример - оценка стоимости квартиры. Признаками могут быть площадь, этаж, расстояние до метро, район и год постройки. Для новой квартиры алгоритм находит похожие проданные квартиры и усредняет их цены. Если ближайшие объекты действительно похожи, результат легко объяснить: прогноз основан на конкретных аналогах. Это делает kNN интерпретируемым методом.

Рассмотрим короткий числовой пример классификации. Новый объект имеет координаты $x = (4, 4)$. В обучающей выборке есть объекты $A = (3, 4)$ класса 0, $B = (5, 4)$ класса 0, $C = (4, 6)$ класса 1 и $D = (8, 8)$ класса 1. Евклидовы расстояния до A и B равны 1, до C равно 2, до D равно $\sqrt{32}$. При $k = 3$ ближайшие соседи - A, B, C . Два соседа имеют класс 0, один - класс 1, значит предсказание: класс 0.

Для регрессии пример аналогичен. Пусть нужно предсказать цену товара по весу и рейтингу. Три ближайших товара стоят 900, 1000 и 1100 рублей. При $k = 3$ прогноз равен:

$$\hat{y} = \frac{900 + 1000 + 1100}{3} = 1000.$$

Если первый товар расположен значительно ближе остальных, лучше использовать взвешенное среднее, чтобы его цена сильнее повлияла на результат.

kNN полезен как базовая модель и как понятный пример метрического подхода. Он хорошо подходит для задач, где похожесть объектов действительно связана с целевым ответом, данных не слишком много, а интерпретируемость важна. Однако для больших выборок, шумных данных и пространств высокой размерности часто нужны ускорение поиска, тщательная предобработка или другие модели.

Билет 12. Переобучение и недообучение. Способы обнаружения и предотвращения

Основные понятия и определения

В машинном обучении модель строится не ради хорошего запоминания обучающей выборки, а ради способности правильно работать на новых объектах. Это свойство называют обобщающей способностью. Если алгоритм показывает низкую ошибку на обучающих данных, но заметно ошибается на отложенной выборке или в реальном применении, говорят о переобучении. Если же он плохо работает даже на обучающей выборке, говорят о недообучении. Оба состояния связаны с несоответствием сложности модели, объема данных, качества признаков и истинной сложности зависимости между признаками и целевой переменной.

Пусть есть обучающая выборка $\{(x_i, y_i)\}_{i=1}^n$, где x_i - вектор признаков объекта, а y_i - правильный ответ. Модель $a(x)$ подбирается так, чтобы минимизировать некоторую функцию потерь $L(y, a(x))$, например квадрат ошибки в регрессии или логистическую потерю в классификации. Эмпирический риск, то есть средняя ошибка на обучающей выборке, равен

$$R_{train}(a) = \frac{1}{n} \sum_{i=1}^n L(y_i, a(x_i)).$$

Однако на практике важнее не R_{train} , а ожидаемый риск на новых данных:

$$R(a) = \mathbb{E}_{(x,y)} L(y, a(x)).$$

Так как истинное распределение данных неизвестно, ожидаемый риск оценивают на валидационной или тестовой выборке. Именно разрыв между обучающей и проверочной ошибкой чаще всего показывает, насколько модель обобщает закономерность, а не просто воспроизводит частные особенности обучающего набора.

Переобучение возникает, когда модель слишком сложна относительно данных. Она улавливает не только полезный сигнал, но и шум: случайные выбросы, ошибки разметки, неустойчивые корреляции, особенности конкретной выборки. Типичные признаки: очень высокая точность на train, существенно худшее качество на validation/test, большая чувствительность к небольшим изменениям данных, нестабильные коэффициенты или слишком сложные правила. Например, глубокое дерево решений без ограничений может построить отдельные ветви под единичные наблюдения и идеально классифицировать train, но плохо работать на новых объектах.

Недообучение возникает, когда модель слишком проста или плохо обучена. Она не способна уловить даже основные закономерности. Типичные признаки: высокая ошибка на train и примерно такая же высокая ошибка на validation/test. Например, попытка описать явно нелинейную зависимость простой линейной регрессией без новых признаков может дать систематические ошибки и на обучающих, и на новых данных.

Важное различие состоит в источнике проблемы. При переобучении модель имеет слишком низкое смещение относительно обучающей выборки, но высокую вариативность: она слишком сильно зависит от конкретного набора данных. При недообучении модель имеет высокое смещение: выбранный класс моделей не содержит достаточно хорошего приближения истинной зависимости или обучение остановлено слишком рано.

Теория, формулы и интуиция

Интуитивно процесс обучения можно представить как поиск баланса между простотой и гибкостью. Слишком простая модель сглаживает данные чрезмерно грубо. Слишком сложная модель изгибается под каждую точку. На графике зависимости ошибки от сложности модели обычно видны три области: сначала и обучающая, и проверочная ошибки велики; затем проверочная ошибка уменьшается; после некоторой точки обучающая ошибка продолжает падать, а проверочная начинает расти. Эта последняя область соответствует переобучению.

Классическая теория описывает ошибку через компромисс смещения и разброса. Для регрессии с квадратичной потерей математическое ожидание ошибки в точке x можно разложить так:

$$\mathbb{E}[(y - \hat{f}(x))^2] = Bias^2(\hat{f}(x)) + Var(\hat{f}(x)) + \sigma^2.$$

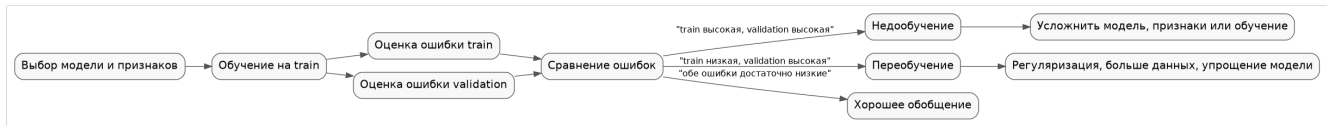


Figure 13: Схема 13

Здесь $Bias^2$ показывает систематическую ошибку модели: насколько среднее предсказание отличается от истинной зависимости. Var показывает изменчивость модели при обучении на разных выборках из того же распределения. σ^2 - неустраняемый шум данных: случайность, которую нельзя предсказать по имеющимся признакам. Недообучение обычно связано с большим $Bias^2$, а переобучение - с большим Var .

Пример с полиномиальной регрессией хорошо иллюстрирует эту идею. Полином первой степени может не описать кривую зависимость и будет недообучен. Полином средней степени может приблизить общий тренд. Полином очень высокой степени может пройти почти через все обучающие точки, но между ними вести себя неустойчиво. Формально все три модели минимизируют похожую функцию потерь, но отличаются размером пространства гипотез: чем больше степень полинома, тем больше вариантов формы доступно модели.

Еще один важный инструмент - разделение данных. Обычно выделяют train для подбора параметров модели, validation для выбора гиперпараметров и test для финальной честной оценки. Если тестовую выборку использовать много раз при подборе модели, она фактически превращается в валидационную, и итоговая оценка становится завышенной. Это особая форма переобучения к процедуре оценки.

Для диагностики полезны кривые обучения. На них по оси x откладывают размер обучающей выборки, а по оси y - ошибку или качество на train и validation. Если при увеличении данных train-ошибка низкая, а validation-ошибка остается высокой, модель склонна к переобучению и дополнительные данные могут помочь. Если обе ошибки высоки и близки, модель недообучена: простое добавление данных часто не решит проблему, нужна более выразительная модель, лучшие признаки или более длительное обучение.

Также применяют кривые валидации: фиксируют все настройки, кроме одного гиперпараметра, и смотрят, как меняются train- и validation-метрики. Например, у дерева решений можно менять максимальную глубину. При малой глубине обе ошибки высоки. При умеренной глубине качество на validation улучшается. При слишком большой глубине train-ошибка падает, но validation-ошибка растет. Это помогает выбрать значение гиперпараметра, соответствующее лучшему обобщению.

Методы, алгоритмы и этапы решения

Обнаружение переобучения и недообучения обычно начинают с корректной схемы оценки. Первый этап - разделить данные на обучающую, валидационную и тестовую части или использовать кросс-валидацию. При временных рядах нельзя перемешивать наблюдения случайно: нужно сохранять временной порядок, чтобы не допустить утечки будущей информации. При дисбалансе классов полезна стратификация, чтобы доли классов в разбиениях были близки.

Второй этап - выбрать метрику, соответствующую задаче. В регрессии это может быть MAE, RMSE или R^2 , в классификации - accuracy, precision, recall, F1, ROC AUC, PR AUC. Неверная метрика способна скрыть проблему: например, при редком положительном классе высокая accuracy может сочетаться с полной неспособностью находить этот класс. После выбора метрики сравнивают качество на train и validation. Низкое качество на обеих выборках указывает на недообучение. Высокое качество на train и низкое на validation указывает на переобучение.

Третий этап - построить диагностические зависимости: кривые обучения, кривые валидации, графики остатков в регрессии, матрицу ошибок в классификации. Важно смотреть не только одно число, но и характер ошибок. Если модель хорошо описывает центральную часть данных, но систематически ошибается на отдельных диапазонах признаков, возможно, ей не хватает признаков или нелинейности. Если ошибки на validation хаотичны, а train почти идеален, вероятно, модель выучила шум.

Для предотвращения недообучения применяют усложнение модели или улучшение представления данных. Можно добавить информативные признаки, преобразования признаков, взаимодействия между признаками, полиномиальные члены, нелинейные модели вместо линейных. В деревьях решений можно увеличить максимальную глубину, уменьшить минимальное число объектов в листе, разрешить больше разбиений. В нейронных сетях можно увеличить число слоев или нейронов, обучать больше эпох, подобрать скорость обучения, улучшить архитектуру. Иногда проблема не в

модели, а в данных: признаки плохо измеряют целевое явление, разметка ошибочна или важные факторы отсутствуют.

Для предотвращения переобучения используют методы ограничения сложности. Самый общий подход - регуляризация, то есть добавление штрафа за сложность модели к функции потерь:

$$Q(w) = \frac{1}{n} \sum_{i=1}^n L(y_i, a(x_i; w)) + \lambda \Omega(w).$$

Параметр λ управляет силой штрафа. При L2-регуляризации $\Omega(w) =$

$\|w\|_2^2$, поэтому большие веса становятся невыгодными, а модель получается более гладкой. При L1-регуляризации $\Omega(w) =$

$\|w\|_1$, часть коэффициентов может стать нулевой, что одновременно упрощает модель и выполняет отбор признаков. В линейных моделях эти варианты известны как ridge-регрессия и lasso-регрессия.

В деревьях решений переобучение предотвращают ограничением глубины, минимального числа объектов в листе, минимального выигрыша от разбиения, а также обрезкой дерева. В ансамблях используют усреднение и случайность. Random Forest снижает разброс за счет обучения многих деревьев на бутстрэп-выборках и случайных подмножествах признаков. Градиентный бустинг контролируют числом деревьев, глубиной базовых деревьев, скоростью обучения, subsampling и регуляризацией листьев.

В нейронных сетях распространены early stopping, dropout, weight decay, batch normalization, аугментация данных и уменьшение архитектуры. Early stopping останавливает обучение, когда validation-ошибка перестает улучшаться, даже если train-ошибка продолжает падать. Dropout случайно отключает часть нейронов во время обучения, вынуждая сеть не полагаться на отдельные активации. Аугментация создает дополнительные варианты объектов: повороты и сдвиги изображений, шум, замены слов в текстах, небольшие искажения сигналов. Это расширяет обучающую выборку и делает модель устойчивее.

Отдельно нужно контролировать утечки данных. Утечка возникает, когда в признаки попадает информация, недоступная в момент реального предсказания, или когда нормализация, отбор признаков, кодирование категорий выполняются на всей выборке до разбиения. Тогда validation/test качество выглядит высоким, но в эксплуатации падает. Правильный способ - все преобразования обучать только на train и затем применять к validation/test через единый pipeline.

Практические примеры

В задаче предсказания цены квартиры линейная регрессия только по площади может недообучиться: цена зависит также от района, этажа, состояния дома, расстояния до метро, года постройки. На train и validation RMSE будет высоким, остатки покажут систематические ошибки: дорогие районы недооцениваются, старые дома переоцениваются. Решение - добавить признаки, учесть категориальные переменные, попробовать регуляризованную линейную модель, дерево решений или градиентный бустинг. Если же взять очень сложную модель с большим числом признаков и без регуляризации, она может запомнить редкие сочетания адресов и случайные выбросы цен. Тогда train-ошибка будет малой, а validation-ошибка высокой. Здесь помогут кросс-валидация, регуляризация, ограничение глубины деревьев, удаление явных выбросов только по обоснованным правилам и проверка качества на отложенном районе или периоде.

В классификации писем на спам простая модель, использующая только длину письма и число ссылок, может недообучиться: она не учитывает слова, домены отправителей, шаблоны заголовков и другие признаки. Качество будет низким на всех выборках. Добавление текстовых признаков, TF-IDF, n-грамм и признаков отправителя повысит выразительность. Но если использовать слишком редкие токены без ограничения словаря, модель может выучить конкретные адреса и фразы из train. На новых письмах такие совпадения исчезнут, и качество упадет. Для борьбы применяют ограничение словаря, регуляризацию логистической регрессии, кросс-валидацию и оценку на письмах из более позднего временного периода.

В медицинской диагностике переобучение особенно опасно, потому что выборки часто малы, а число признаков велико. Модель может найти случайные корреляции между лабораторными показателями и диагнозом, которые возникли только в конкретной больнице или партии измерений. Поэтому важно иметь внешнюю тестовую выборку, проверять стабильность признаков, использовать регуляризацию и не подбирать гиперпараметры по финальному тесту. Недообучение здесь тоже вредно: слишком грубая модель может пропускать важные случаи болезни. Баланс достигается не максимальной сложностью,

а проверенной обобщающей способностью.

В задачах с нейронными сетями типичная картина переобучения выглядит так: loss на train монотонно падает, а loss на validation сначала падает, затем начинает расти. В этот момент модель продолжает лучше запоминать обучающие примеры, но хуже обобщает. Практическое решение - сохранить веса с лучшим validation-качеством, включить early stopping, добавить аугментацию и weight decay, уменьшить сеть или собрать больше данных. Если же и train loss долго остается высоким, нужно проверить скорость обучения, архитектуру, масштабирование признаков, корректность разметки и достаточность числа эпох.

Билет 13. Нейронные сети. Принцип работы. Основные определения. Однослойные и многослойные нейронные сети

Основные понятия и определения

Нейронная сеть - это параметрическая модель машинного обучения, составленная из связанных между собой простых вычислительных элементов, которые по аналогии называют нейронами. Ее задача состоит в том, чтобы по входному вектору признаков x построить ответ $\hat{y}$: класс объекта, вероятность события, численное значение, последовательность токенов и так далее. В отличие от явно заданного алгоритма, нейронная сеть не получает готовые правила принятия решения. Эти правила формируются в процессе обучения через настройку весов и смещений.

Искусственный нейрон принимает несколько входных чисел $x_1, x_2, \dots, x_d$, умножает их на веса $w_1, w_2, \dots, w_d$, добавляет смещение b и применяет функцию активации. Формально:

$$z = \sum_{j=1}^d w_j x_j + b, \quad a = \varphi(z).$$

Здесь z называют линейной комбинацией, или входом активации, a - выходом нейрона, φ - функцией активации. Вес показывает, насколько сильно соответствующий признак влияет на ответ. Смещение позволяет сдвигать границу решения и не привязывать ее к началу координат.

Функция активации нужна, чтобы внести нелинейность. Если убрать активации или заменить их тождественной функцией во всех слоях, даже глубокая сеть сведется к одной линейной модели, потому что композиция линейных преобразований остается линейным преобразованием. Часто используют sigmoid $\sigma(z) = 1/(1 + e^{-z})$, гиперболический тангенс $\tanh(z)$, ReLU $\max(0, z)$, а в выходном слое многоклассовой классификации - softmax:

$$\text{softmax}(z) * k = \frac{e^{z_k}}{\sum_r e^{z_r}}.$$

Эпоха - это один полный проход алгоритма обучения по всей обучающей выборке. Если выборка большая, ее обычно делят на мини-пакеты, или batches. Тогда внутри одной эпохи сеть несколько раз делает прямой проход, считает ошибку, вычисляет градиенты и обновляет параметры. Число эпох влияет на степень обученности: слишком мало эпох может дать недообучение, слишком много без контроля качества может привести к переобучению.

Обучение нейронной сети - это процесс подбора параметров W, b , при котором уменьшается функция потерь. Функция потерь измеряет, насколько прогноз $\hat{y}$ отличается от правильного ответа y . Для регрессии часто используют среднеквадратичную ошибку:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Для классификации часто используют кросс-энтропию, например в многоклассовом случае:

$$L = - \sum_{k=1}^K y_k \log \hat{p}_k,$$

где y_k - индикатор правильного класса, а $\hat{p}_k$ - предсказанная вероятность класса k .

Однослойная нейронная сеть обычно означает сеть без скрытых слоев: входные признаки напрямую связаны с выходным слоем. Примеры - перцептрон, логистическая регрессия в нейронной записи, линейный классификатор с softmax. Такая модель может хорошо работать, если данные разделимы линейной границей или если признаки уже содержат нужные нелинейные преобразования.

Многослойная нейронная сеть содержит один или несколько скрытых слоев между входом и выходом. Скрытый слой строит новое представление данных: каждый следующий слой комбинирует выходы предыдущего. Благодаря этому сеть может приближать сложные нелинейные зависимости. Классическая полносвязная многослойная сеть называется многослойным перцептроном, или MLP.

Теория, формулы и интуиция

Работу нейронной сети удобно понимать как последовательное преобразование пространства признаков. На входе объект описан исходными признаками: возрастом клиента, доходом, количеством покупок, пикселями изображения или частотами слов. Первый слой строит из них простые комбинации. Следующие слои строят комбинации комбинаций. В итоге выходной слой получает представление, в котором проще отделить классы или вычислить нужное число.

Для одного слоя с m нейронами вычисление можно записать в матричной форме:

$$a = \varphi(Wx + b),$$

где W - матрица весов размера $m \times d$, b - вектор смещений, φ применяется поэлементно. Для многослойной сети:

$$a^{(0)} = x,$$

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \varphi^{(l)}(z^{(l)}), \quad l = 1, \dots, L.$$

Последний слой дает $\hat{y} = a^{(L)}$ или параметры распределения ответа. В регрессии выход может быть одним числом без нелинейной активации. В бинарной классификации часто ставят sigmoid и получают вероятность положительного класса. В многоклассовой классификации softmax превращает набор чисел в вероятности классов, сумма которых равна единице.

Интуиция однослойной сети связана с гиперплоскостью. Бинарный перцептрон фактически проверяет, по какую сторону от границы $w^T x + b = 0$ лежит объект. Если точки разных классов можно разделить прямой в двумерном случае, плоскостью в трехмерном или гиперплоскостью в многомерном пространстве, однослойная модель может справиться. Но если классы устроены как задача XOR, где правильный ответ зависит от нелинейного сочетания признаков, одной линейной границы недостаточно.

Многослойная сеть решает эту проблему через скрытые признаки. Например, первый скрытый слой может выделить несколько простых областей, а следующий слой объединит их в сложную форму границы. Поэтому MLP с нелинейными активациями способен аппроксимировать широкий класс функций. Чем больше слоев и нейронов, тем выше выразительность модели, но тем больше риск переобучения, вычислительная стоимость и требования к данным.

Обучение основано на минимизации эмпирического риска:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i; \theta)),$$

где θ обозначает все веса и смещения сети. Параметры обновляют градиентным методом:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta),$$

где η - скорость обучения. Если η слишком мала, обучение идет медленно. Если слишком велика, оптимизация может стать неустойчивой: ошибка будет колебаться или расти.

Главный технический механизм обучения многослойных сетей - обратное распространение ошибки, или backpropagation. Сначала выполняется прямой проход: сеть получает вход, последовательно вычисляет все слои и дает прогноз. Затем считается функция потерь. После этого по правилу цепочки вычисляются производные ошибки по параметрам каждого слоя, начиная с выходного слоя и двигаясь

назад к входу. Эти производные показывают, какие веса нужно увеличить или уменьшить, чтобы снизить ошибку.



Figure 14: Схема 14

Важную роль играет нормализация входов. Если один признак измеряется в рублях и принимает значения до миллионов, а другой лежит в диапазоне от 0 до 1, градиентная оптимизация может идти хуже. Поэтому числовые признаки часто стандартизуют: вычитают среднее и делят на стандартное отклонение. Для изображений пиксели приводят к диапазону $[0, 1]$ или нормируют по среднему и дисперсии.

Методы, алгоритмы и этапы решения

Типичный процесс построения нейронной сети начинается с постановки задачи. Нужно определить, что является входом, что является целевой переменной и какая метрика качества важна. Для кредитного скоринга это может быть вероятность дефолта и ROC AUC, для прогноза цены - MAE или RMSE, для распознавания цифр - accuracy или F1.

Далее готовят данные. Удаляют или обрабатывают пропуски, кодируют категориальные признаки, нормализуют числовые признаки, разбивают данные на обучающую, валидационную и тестовую выборки. Тестовую выборку нельзя использовать для подбора архитектуры и гиперпараметров, иначе итоговая оценка будет завышенной.

Затем выбирают архитектуру. Для простой табличной задачи можно начать с небольшой полносвязной сети: входной слой по числу признаков, один-два скрытых слоя с ReLU, выходной слой под тип задачи. Для бинарной классификации выходной нейрон обычно один, с sigmoid. Для многоклассовой классификации выходов столько, сколько классов, и применяется softmax. Для регрессии выход часто линейный.

После этого параметры инициализируют малыми случайными значениями. Если все веса сделать одинаковыми, нейроны одного слоя будут получать одинаковые градиенты и обучаться одинаково, поэтому сеть не сможет использовать их как разные признаки. Современные варианты инициализации, например Xavier или He, подбирают масштаб весов с учетом числа входов и типа активации.

Один шаг обучения на мини-пакете включает несколько действий:

1. Подать мини-пакет объектов на вход сети.
2. Выполнить прямой проход и получить прогнозы.
3. Посчитать функцию потерь.
4. Выполнить обратное распространение ошибки.
5. Обновить веса оптимизатором.

Простейший оптимизатор - стохастический градиентный спуск. На практике часто используют Adam, RMSProp или SGD с momentum. Они учитывают историю градиентов и могут быстрее сходиться на сложных задачах. При этом оптимизатор не заменяет корректную постановку задачи: плохие данные, неверная разметка и утечки признаков не исправляются выбором более сложного алгоритма. В конце каждой эпохи качество обычно проверяют на валидационной выборке. Если ошибка на обучении падает, а на валидации начинает расти, сеть переобучается. Тогда применяют регуляризацию: L2-штраф на веса, dropout, раннюю остановку, уменьшение размера сети, добавление данных или аугментации. Ранняя остановка означает, что обучение прекращают, когда валидационное качество перестало улучшаться в течение заданного числа эпох.

Отдельно подбирают гиперпараметры: число слоев, число нейронов, скорость обучения, размер batch, число эпох, тип активации, коэффициент регуляризации. Их нельзя получить прямым обучением как обычные веса, поэтому используют валидацию, поиск по сетке, случайный поиск или более умные методы оптимизации гиперпараметров.

Практические примеры

Первый пример - бинарная классификация заявок на кредит. Пусть входные признаки включают доход, возраст, кредитную историю, текущую долговую нагрузку и стаж работы. Однослойная сеть с sigmoid фактически строит логистическую регрессию: она оценивает вероятность дефолта как нелинейное преобразование линейной суммы признаков. Такая модель проста, быстро обучается и хорошо интерпретируется: знак и величина весов показывают направление влияния признаков. Но если риск зависит от сложных сочетаний, например высокий доход снижает риск только при определенном

стаже и отсутствии просрочек, многослойная сеть может уловить такие взаимодействия лучше. Второй пример - распознавание рукописных цифр. Если использовать только однослойную сеть по пикселям, она будет искать линейные шаблоны: одни пиксели повышают вероятность цифры, другие понижают. Это может дать приемлемый результат на простой задаче, но модель плохо учитывает локальные формы: петли, наклоны, сдвиги. Многослойная сеть сначала может выделять простые элементы изображения, затем их комбинации, а потом целые контуры цифр. Для изображений особенно эффективны сверточные сети, но базовый принцип слоев, активаций, функции потерь и обратного распространения остается тем же.

Третий пример - прогноз цены квартиры. Входами могут быть площадь, этаж, район, расстояние до метро, год постройки и состояние дома. Регрессионная нейронная сеть с линейным выходом обучается минимизировать MSE или MAE. Скрытые слои позволяют учитывать нелинейные зависимости: дополнительный квадратный метр может иметь разную ценность в разных районах, а влияние этажа может зависеть от типа дома. Однако для табличных данных нейронная сеть не всегда лучше градиентного бустинга или линейных моделей с хорошими признаками, поэтому качество нужно сравнивать на валидации.

Четвертый пример - простая задача XOR. Даны два бинарных признака x_1 и x_2 , ответ равен 1, если признаки различны, и 0, если одинаковы. Однослойная сеть не может провести одну прямую, которая отделит положительные точки от отрицательных. Многослойная сеть с двумя нейронами в скрытом слое может построить промежуточные признаки, соответствующие разным областям, а выходной нейрон объединит их. Этот пример показывает, зачем нужны скрытые слои: они не просто добавляют параметры, а меняют представление задачи.

На практике нейронные сети особенно полезны там, где данные сложные и плохо описываются вручную заданными признаками: изображения, звук, текст, последовательности, большие многомерные наборы сигналов. В таких задачах сеть одновременно учит признаки и решающее правило. Но базовые элементы остаются одинаковыми: нейрон вычисляет взвешенную сумму и активацию, слой преобразует вектор, функция потерь измеряет ошибку, обучение изменяет веса, эпохи организуют многократный проход по данным, а различие между однослойными и многослойными сетями определяет, насколько сложные зависимости модель способна выразить.

Билет 14. Алгоритм обратного распространения ошибки

Основные понятия и определения

Алгоритм обратного распространения ошибки, или backpropagation, - это основной способ вычисления градиентов в многослойных нейронных сетях. Он отвечает на практический вопрос: как понять, насколько каждый вес и каждое смещение сети повлияли на итоговую ошибку? После такого вычисления параметры можно обновить градиентным методом и постепенно уменьшать значение функции потерь.

Нейронная сеть состоит из слоев. Каждый слой получает входной вектор, выполняет аффинное преобразование и применяет нелинейную функцию активации. Для слоя с номером l обычно записывают:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \varphi^{(l)}(z^{(l)}),$$

где $a^{(l-1)}$ - вход слоя, $W^{(l)}$ - матрица весов, $b^{(l)}$ - вектор смещений, $z^{(l)}$ - линейная комбинация до активации, $a^{(l)}$ - выход слоя, φ - функция активации. Для первого слоя $a^{(0)} = x$, то есть это исходные признаки объекта.

Функция потерь $L(\hat{y}, y)$ измеряет расхождение между предсказанием сети $\hat{y}$ и правильным ответом y . В регрессии часто используется среднеквадратичная ошибка, в классификации - кросс-энтропия. Цель обучения - найти такие параметры $\theta = \{W^{(l)}, b^{(l)}\}$, при которых средняя ошибка на обучающей выборке становится малой:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(f(x_i; \theta), y_i).$$

Градиент - это вектор частных производных функции потерь по параметрам. Например, $\frac{\partial L}{\partial W^{(l)}}$ показывает, как изменится ошибка при малом изменении весов слоя l . Если производная

положительна, увеличение соответствующего веса увеличивает ошибку; если отрицательна, увеличение веса уменьшает ошибку. Поэтому в градиентном спуске параметры двигают в противоположную сторону градиента:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}},$$

где η - скорость обучения. Backpropagation не является отдельным методом оптимизации: он только эффективно вычисляет градиенты. А уже SGD, mini-batch SGD, Momentum, RMSProp, Adam и другие оптимизаторы используют эти градиенты для обновления параметров.

Теория, формулы и интуиция

Главная идея обратного распространения основана на правиле цепочки для производных сложной функции. Нейронная сеть - это композиция многих преобразований: вход проходит через первый слой, затем через второй, затем через следующие слои, пока не получится предсказание. Ошибка зависит от последнего выхода, последний выход зависит от предыдущего слоя, предыдущий слой зависит от более раннего, и так далее до входа. Чтобы найти влияние раннего веса на итоговую ошибку, нужно перемножить локальные производные вдоль пути от этого веса к функции потерь.

Если сеть имеет L слоев, прямой проход можно представить так:

$$x = a^{(0)} \rightarrow z^{(1)} \rightarrow a^{(1)} \rightarrow \dots \rightarrow z^{(L)} \rightarrow a^{(L)} = \hat{y} \rightarrow L(\hat{y}, y).$$

Наивное вычисление производных для каждого веса отдельно было бы очень дорогим. Backpropagation использует динамическое программирование: он один раз считает прямой проход, сохраняет промежуточные значения $z^{(l)}$ и $a^{(l)}$, а затем идет назад от функции потерь к первым слоям, переиспользуя уже найденные производные. Поэтому стоимость вычисления всех градиентов оказывается сравнимой со стоимостью одного прямого прохода, а не умножается на число параметров. Введем величину ошибки слоя:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}.$$

Это не просто ошибка предсказания в бытовом смысле, а производная функции потерь по линейному входу слоя до активации. Именно $\delta^{(l)}$ удобно распространять назад. Для последнего слоя:

$$\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} \odot \varphi'^{(L)}(z^{(L)}),$$

где $\odot$ означает поэлементное умножение. Для скрытого слоя:

$$\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \varphi'^{(l)}(z^{(l)}).$$

Эта формула хорошо показывает интуицию алгоритма. Ошибка текущего слоя получается из ошибки следующего слоя, умноженной на веса, которые связывают слои, и на производную локальной активации. То есть слой получает “сигнал ответственности”: насколько его выходы повлияли на ошибку дальше по сети и насколько сама активация была чувствительна к изменению входа.

После вычисления $\delta^{(l)}$ градиенты по параметрам слоя записываются просто:

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

Для mini-batch из m объектов обычно усредняют градиенты:

$$\frac{\partial J}{\partial W^{(l)}} = \frac{1}{m} \Delta^{(l)} (A^{(l-1)})^T, \quad \frac{\partial J}{\partial b^{(l)}} = \frac{1}{m} \sum_{i=1}^m \delta_i^{(l)}.$$

Здесь матрицы A и Δ содержат активации и ошибки сразу для нескольких объектов. На практике библиотеки глубокого обучения используют векторизованные операции, потому что они намного

быстрее циклов по отдельным примерам.

Для выходного слоя формулы часто упрощаются. Например, если в задаче многоклассовой классификации используется softmax и кросс-энтропия, то:

$$\delta^{(L)} = \hat{y} - y.$$

Это одна из причин популярности такой комбинации: градиент получается численно удобным и легко интерпретируемым. Если предсказанная вероятность класса выше правильной, соответствующий сигнал ошибки положителен и весам нужно уменьшить поддержку этого класса; если ниже - сигнал отрицателен и сеть будет усиливать этот класс.

Важная роль принадлежит функциям активации. Без нелинейностей несколько слоев свелись бы к одному линейному преобразованию, и сеть не могла бы моделировать сложные зависимости. Но производные активаций влияют на качество обратного распространения. У sigmoid и tanh производные малы на насыщенных участках, поэтому в глубоких сетях может возникать затухание градиента: при движении назад производные многократно перемножаются, сигнал становится почти нулевым, ранние слои обучаются медленно. У ReLU производная равна 1 для положительных входов и 0 для отрицательных, поэтому она часто лучше сохраняет градиент, хотя может возникать проблема “мертвых” нейронов.

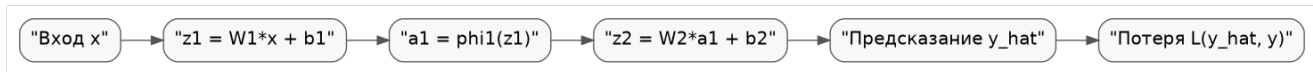


Figure 15: Схема 15

Диаграмма показывает два направления. Сплошные стрелки - прямой проход, где сеть получает предсказание. Пунктирные стрелки - обратный проход, где ошибка превращается в градиенты для каждого слоя.

Методы, алгоритмы и этапы решения

Классический цикл обучения нейронной сети с обратным распространением состоит из нескольких этапов.

1. Инициализация параметров. Веса задаются малыми случайными числами, смещения часто инициализируются нулями или малыми константами. Случайность нужна, чтобы разные нейроны не начали обучаться одинаково. Для глубоких сетей используют схемы Xavier/Glorot или He, которые учитывают размер входа и выхода слоя и помогают стабилизировать дисперсию активаций и градиентов.

2. Прямое распространение. Для входного объекта или mini-batch последовательно вычисляются все $z^{(l)}$ и $a^{(l)}$, затем получается предсказание $\hat{y}$. Промежуточные значения сохраняются, потому что они понадобятся при вычислении производных. Например, чтобы найти градиент по весам слоя, нужен вход этого слоя $a^{(l-1)}$, а чтобы найти ошибку слоя, нужна производная активации в точке $z^{(l)}$.

3. Вычисление функции потерь. Модель сравнивает предсказание с правильным ответом. Для одного объекта это $L(\hat{y}, y)$, для mini-batch - среднее значение по объектам. Иногда к потере добавляют регуляризацию, например L_2 -штраф λ

$\|W\|_2^2$, чтобы веса не становились слишком большими и модель меньше переобучалась.

4. Обратное распространение. Сначала вычисляется δ для выходного слоя. Затем для каждого предыдущего слоя, двигаясь от последнего к первому, считается $\delta^{(l)}$ через ошибку следующего слоя. После этого находятся $\frac{\partial L}{\partial W^{(l)}}$ и $\frac{\partial L}{\partial b^{(l)}}$. На этом этапе не изменяются сами параметры; алгоритм только собирает информацию о направлении изменения ошибки.

5. Обновление параметров. Оптимизатор применяет найденные градиенты. В простейшем случае используется градиентный спуск. В mini-batch SGD обновления выполняются после каждой небольшой группы объектов, что ускоряет обучение и добавляет полезный шум. Adam дополнительно хранит скользящие средние градиентов и квадратов градиентов, поэтому часто сходится быстрее и требует меньше ручной настройки.

6. Повторение по эпохам. Один проход по всей обучающей выборке называется эпохой. После каждой эпохи обычно считают качество на валидационной выборке. Это помогает понять, улучшается ли обобщающая способность модели, или сеть уже начала переобучаться.

В псевдокоде алгоритм можно записать так:

```
инициализировать W, b
для каждой эпохи:
    разбить обучающие данные на mini-batch
    для каждого mini-batch:
        выполнить прямой проход и сохранить z, a
        вычислить значение функции потерь
        вычислить delta для выходного слоя
        для слоев от L-1 до 1:
            распространить delta назад
        вычислить градиенты по W и b
        обновить W и b выбранным оптимизатором
```

В современных фреймворках, таких как PyTorch, TensorFlow и JAX, пользователь обычно не реализует эти формулы вручную. Он задает вычислительный граф: операции прямого прохода и функцию потерь. Система автоматического дифференцирования запоминает граф операций и при вызове backward автоматически применяет правило цепочки. Тем не менее понимание backpropagation важно: оно помогает выбирать архитектуру, функцию активации, скорость обучения, размер batch, регуляризацию и диагностировать проблемы обучения.

Практические примеры

Рассмотрим простую сеть для бинарной классификации: два входных признака, один скрытый слой с тремя нейронами ReLU и выходной нейрон sigmoid. Пусть объект описывает клиента: x_1 - нормированный доход, x_2 - нормированный возраст. Нужно предсказать, совершит ли клиент целевое действие. Прямой проход:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(1)} = \text{ReLU}(z^{(1)}),$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \quad \hat{y} = \sigma(z^{(2)}).$$

Для бинарной классификации удобно использовать бинарную кросс-энтропию:

$$L(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}).$$

Если $y = 1$, а сеть предсказала $\hat{y} = 0.3$, ошибка велика: модель слишком слабо уверена в правильном классе. Для связки sigmoid и бинарной кросс-энтропии градиент выходного слоя упрощается:

$$\delta^{(2)} = \hat{y} - y = 0.3 - 1 = -0.7.$$

Отрицательное значение означает, что выход $z^{(2)}$ нужно увеличить, чтобы sigmoid дала большую вероятность класса 1. Градиент по весам второго слоя:

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)}(a^{(1)})^T.$$

Если какой-то нейрон скрытого слоя имел большую положительную активацию, соответствующий вес сильнее получит корректировку. Если активация была нулевой из-за ReLU, вклад этого нейрона в градиент выходного веса для данного объекта будет нулевым.

Затем ошибка передается в скрытый слой:

$$\delta^{(1)} = (W^{(2)})^T \delta^{(2)} \odot \text{ReLU}'(z^{(1)}).$$

Если у скрытого нейрона $z^{(1)} \leq 0$, производная ReLU равна нулю, и для этого объекта нейрон не получает обновления через данный путь. Если $z^{(1)} > 0$, градиент проходит дальше к весам первого слоя. Так сеть определяет, какие входные признаки и какие скрытые нейроны ответственны за неправильное предсказание.

В программной практике один шаг обучения в PyTorch выглядит почти так же, как математическое описание, только ручные формулы скрыты внутри автоматического дифференцирования:

```
optimizer.zero_grad()
pred = model(x_batch)
loss = criterion(pred, y_batch)
loss.backward()
optimizer.step()
```

Команда `zero_grad()` очищает старые градиенты, потому что во многих библиотеках они накапливаются. Прямой вызов `model` выполняет forward pass. `loss.backward()` запускает обратное распространение по вычислительному графу и заполняет поля градиентов у параметров. `optimizer.step()` обновляет веса.

На практике эффективность обратного распространения зависит не только от формул, но и от подготовки данных и настройки обучения. Нормализация признаков делает масштабы входов сопоставимыми и улучшает численную устойчивость. Подходящий learning rate критически важен: слишком большой приводит к скачкам и расходимости, слишком маленький делает обучение медленным. Регуляризация, dropout, batch normalization и ранняя остановка помогают стабилизировать обучение и уменьшить переобучение.

Таким образом, обратное распространение ошибки - это вычислительный механизм, который делает обучение глубоких нейронных сетей практически возможным. Он превращает единственное число ошибки на выходе в набор точных градиентов для всех параметров сети. Благодаря правилу цепочки, сохранению промежуточных результатов и движению от последнего слоя к первому алгоритм эффективно распределяет ответственность за ошибку между весами, смещениями и слоями модели.

Билет 15. Сферы применения нейронных сетей. Примеры. Виды нейронных сетей

Основные понятия и определения

Нейронная сеть - это модель машинного обучения, состоящая из связанных искусственных нейронов. Она получает входные данные, преобразует их в слоях и выдает ответ: класс, число, текст, изображение, рекомендацию или управляющее действие. В отличие от алгоритма с явно заданными правилами, нейронная сеть подбирает правила поведения по примерам.

Искусственный нейрон имеет обучаемые параметры: веса и смещение. Слои между входом и выходом называют скрытыми. Если скрытых слоев много, сеть называют глубокой, а соответствующий подход - глубоким обучением. Обучение сети означает настройку параметров так, чтобы предсказания были ближе к правильным ответам.

Данные делят на обучающую, валидационную и тестовую выборки. Обучающая часть нужна для изменения весов, валидационная - для подбора настроек и контроля переобучения, тестовая - для итоговой проверки. Переобучение возникает, когда модель запоминает обучающие примеры, но плохо работает на новых данных. Регуляризация - набор методов, уменьшающих этот риск.

Нейронные сети применяют там, где зависимость между входом и ответом сложна для ручного описания, но есть данные: в компьютерном зрении, обработке речи и текста, медицине, финансах, промышленности, рекомендациях, робототехнике и генерации контента.

Теория, формулы и интуиция

Один нейрон вычисляется по формуле:

$$z = \sum_{i=1}^n w_i x_i + b, \quad y = \varphi(z),$$

где x_i - признаки, w_i - веса, b - смещение, φ - функция активации. Активация нужна для нелинейности: без нее вся многослойная сеть была бы эквивалентна одному линейному преобразованию. Часто используют ReLU, sigmoid, tanh, GELU и softmax.

Многослойная сеть является композицией функций:

$$\hat{y} = f(x; \theta) = f_L(f_{L-1}(\dots f_1(x))),$$

где θ - все параметры. Интуитивно каждый слой строит более удобное представление данных. В изображениях ранние слои находят края и текстуры, поздние - части объектов и целые объекты. В текстах первые представления связаны с токенами, а глубокие слои учитывают контекст.

Для обучения задают функцию потерь. В регрессии часто используют среднеквадратичную ошибку:

$$L = \frac{1}{m} \sum_{j=1}^m (\hat{y}_j - y_j)^2.$$

В классификации применяют кросс-энтропию:

$$L = - \sum_{k=1}^K y_k \log \hat{p}_k.$$

Параметры обновляются градиентным спуском или его вариантами:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t),$$

где η - скорость обучения. Градиенты вычисляются обратным распространением ошибки: сначала сеть делает прямой проход, затем ошибка проходит назад по слоям, и для каждого веса определяется полезное направление изменения.

Основные виды нейронных сетей:

- MLP, или полносвязные сети, применяются для табличных данных, регрессии и простой классификации.
- CNN, или сверточные сети, подходят для изображений, видео и медицинских снимков.
- RNN, LSTM и GRU обрабатывают последовательности и временные ряды.
- Трансформеры используют механизм внимания и применяются для текста, кода, перевода, поиска и мультимодальных задач.
- Автоэнкодеры сжимают и восстанавливают данные, поэтому полезны для снижения размерности, шумоподавления и поиска аномалий.
- Генеративные модели, включая GAN и диффузионные модели, создают новые изображения, тексты, звуки или синтетические данные.
- GNN, или графовые нейронные сети, анализируют вершины и ребра: социальные связи, молекулы, маршруты, рекомендации.

Также нейронные сети различают по режиму обучения. В обучении с учителем каждому объекту соответствует правильный ответ, например метка класса или численное значение. В обучении без учителя модель ищет структуру данных сама: группы похожих объектов, скрытые признаки, аномалии. В обучении с подкреплением агент выбирает действия в среде и получает награду или штраф; такой подход применяют в играх, робототехнике и управлении.

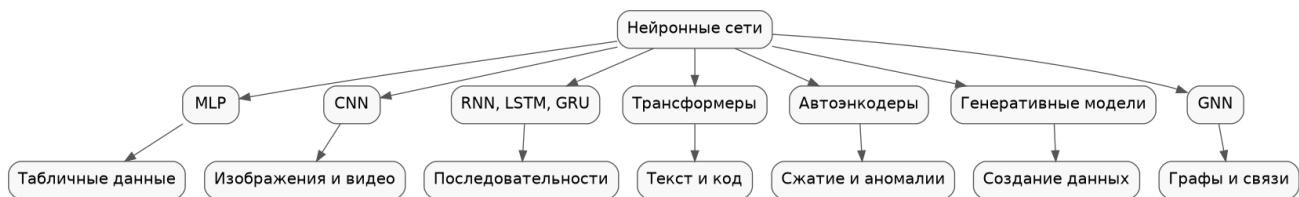


Figure 16: Схема 16

Методы, алгоритмы и этапы решения

Сначала формулируют задачу: какие данные подаются на вход, какой ответ нужен и какой метрикой измеряется качество. Для классификации используют accuracy, precision, recall, F1-меру, ROC-AUC; для регрессии - MAE, MSE, RMSE. Метрика выбирается по смыслу задачи: в диагностике важна полнота, а в рекомендациях - релевантность выдачи.

Затем готовят данные. Числовые признаки нормализуют, категориальные кодируют, пропуски обрабатывают. Изображения приводят к одному размеру, нормализуют и иногда дополняют аугментациями: поворотами, отражениями, обрезками. Тексты токенизируют. Для временных рядов формируют окна наблюдений и не допускают попадания будущей информации в признаки.

После этого выбирают архитектуру. Для таблиц используют MLP или классические модели. Для изображений - CNN или vision transformer. Для текста и кода - трансформеры. Для временных рядов - RNN, LSTM, GRU, temporal convolution или трансформеры рядов. Для объектов со связями - GNN.

Далее задают функцию потерь, оптимизатор и режим обучения. В бинарной классификации типична связка sigmoid и binary cross-entropy, в многоклассовой - softmax и кросс-энтропия, в регрессии - линейный выход и MSE или MAE. Обучение идет по батчам несколько эпох, качество проверяется на валидационной выборке.

Если сеть переобучается, используют dropout, L2-регуляризацию, early stopping, аугментацию данных, batch normalization или layer normalization. Часто применяют перенос обучения: берут заранее обученную модель и дообучают ее на своей задаче. После обучения модель тестируют, анализируют ошибки и внедряют. В эксплуатации важны скорость ответа, стоимость вычислений, интерпретируемость, защита данных и мониторинг качества.

Перед внедрением также проверяют устойчивость модели к шуму, редким случаям и изменению распределения данных. Это важно, потому что хорошая оценка на тестовой выборке не всегда гарантирует надежную работу в реальной среде.

Отдельно оценивают ошибки модели. Важно смотреть не только среднюю метрику, но и примеры неправильных ответов: какие классы путаются, какие входы слишком шумные, где не хватает данных. Такой анализ помогает улучшить разметку, добавить признаки, изменить архитектуру или собрать новые примеры.

Практические примеры

В компьютерном зрении CNN и трансформеры распознают объекты, лица, дефекты и области на изображении. На производстве модель может находить трещины, сколы и царапины деталей. В медицине нейронные сети анализируют рентген, КТ, МРТ и снимки глазного дна, помогая врачу быстрее заметить подозрительные зоны.

В обработке естественного языка трансформеры применяются для перевода, суммаризации, поиска по смыслу, классификации обращений и генерации текста. Например, служба поддержки может определить тему заявки: оплата, доставка, возврат или техническая ошибка, а затем предложить оператору подходящую статью из базы знаний.

В распознавании речи сеть преобразует аудио в текст. Это используется в голосовых ассистентах, субтитрах, диктовке и расшифровке звонков. В синтезе речи модель решает обратную задачу: строит звучание по тексту с учетом пауз, ударений и интонации.

В рекомендательных системах сети прогнозируют, какие товары, фильмы, статьи или треки будут интересны пользователю. Они учитывают историю действий, свойства объектов, похожесть пользователей и контекст. Графовые сети дополнительно используют связи между пользователями, товарами и событиями.

В финансах модели оценивают кредитный риск, выявляют мошенничество и анализируют поведение клиентов. Подозрительная транзакция может быть найдена по необычному сочетанию суммы, места, времени и прошлых операций. В промышленности нейронные сети прогнозируют отказ оборудования по данным датчиков температуры, вибрации, давления и тока.

В транспорте и робототехнике нейронные сети помогают распознавать дорожные знаки, пешеходов, препятствия, положение объектов и команды оператора. В беспилотных системах они обрабатывают данные камер, радаров и лидаров, а в складской робототехнике помогают выбрать объект и спланировать захват.

В генеративных задачах модели создают изображения, музыку, тексты, программный код и синтетические данные. Диффузионные модели превращают шум в изображение по описанию, а языковые модели помогают работать с документами и программами. Такие системы полезны в дизайне, образовании, аналитике и разработке, но требуют проверки фактов, качества, авторских прав и безопасности результата.

Итак, нейронные сети особенно полезны там, где есть данные и трудно вручную описать правила. Вид сети выбирают по типу входа: изображения требуют пространственной обработки, текст и речь - учета контекста, временные ряды - анализа динамики, графы - работы со связями, а генеративные задачи - моделирования распределения данных.

Билет 16. Свёрточные нейронные сети

Основные понятия и определения

Свёрточная нейронная сеть (Convolutional Neural Network, CNN) - это класс нейронных сетей, особенно хорошо подходящий для данных с пространственной или временной структурой: изображений, видеопотоков, спектрограмм, медицинских снимков, временных рядов. Главная идея CNN состоит в том, что признаки в таких данных имеют локальный характер. Например, на изображении сначала полезно находить простые элементы: границы, углы, перепады яркости, цветовые пятна. Затем из них можно собирать более сложные признаки: текстуры, части объектов, целые объекты.

Обычная полносвязная сеть рассматривает вход как длинный вектор и связывает каждый входной признак с каждым нейроном следующего слоя. Для изображения это неэффективно: теряется явная геометрия пикселей, а число параметров быстро становится огромным. CNN использует свёрточные слои, которые применяют небольшие обучаемые фильтры к локальным областям входа. Один и тот же фильтр проходит по всему изображению, поэтому сеть может распознавать один и тот же признак в разных местах.

Основные элементы CNN:

- Входной тензор - обычно изображение размера $H \times W \times C$, где H - высота, W - ширина, C - число каналов. Для RGB-изображения $C = 3$.
- Ядро, или фильтр, - небольшая матрица весов, например 3×3 или 5×5 , которая скользит по входу и вычисляет локальные отклики.
- Карта признаков - результат применения одного фильтра ко всем допустимым позициям входа. Если фильтров много, получается несколько карт признаков.
- Шаг (stride) - величина сдвига фильтра при движении по изображению. Шаг 1 сохраняет больше пространственной детализации, шаг 2 уменьшает размер выходной карты.
- Дополнение (padding) - добавление рамки, часто из нулей, вокруг входа. Оно позволяет контролировать размер выхода и учитывать пиксели на границах.
- Пулинг (pooling) - операция уменьшения пространственного размера карты признаков, например max pooling или average pooling.
- Функция активации - нелинейное преобразование, чаще всего ReLU: $\text{ReLU}(x) = \max(0, x)$.
- Полносвязная часть - завершающие слои, которые используют извлечённые признаки для классификации, регрессии или другой целевой задачи.

CNN можно рассматривать как систему автоматического извлечения признаков. В классическом компьютерном зрении признаки часто проектировались вручную: контуры, дескрипторы углов, гистограммы градиентов. В CNN фильтры обучаются по данным вместе с остальными параметрами модели.

Теория, формулы и интуиция

Дискретная двумерная свёртка для одного канала может быть записана так:

$$Y_{i,j} = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X_{i+u,j+v} K_{u,v} + b,$$

где X - входная матрица, K - ядро размера $k_h \times k_w$, b - смещение, $Y_{i,j}$ - значение выходной карты признаков в позиции (i, j) . На практике в библиотеках глубокого обучения часто используется не математическая свёртка с переворотом ядра, а кросс-корреляция, но в контексте CNN её обычно тоже называют свёрткой.

Для изображения с несколькими каналами фильтр имеет размер $k_h \times k_w \times C$. Тогда отклик вычисляется суммированием по высоте, ширине и каналам:

$$Y_{i,j,m} = \sum_{c=1}^C \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X_{i+u,j+v,c} K_{u,v,c,m} + b_m,$$

где m - номер фильтра. Если в слое M фильтров, то выход будет иметь M каналов.

Размер выходной карты признаков по одной пространственной координате вычисляется формулой:

$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1,$$

где I - размер входа, K - размер ядра, P - padding, S - stride. Например, при входе 32×32 , ядре 3×3 , padding 1 и stride 1 выход остаётся 32×32 . Это удобно, когда нужно сохранить пространственное разрешение.

Ключевые свойства CNN:

- Локальная связность. Нейрон свёрточного слоя смотрит не на весь вход, а на небольшое рецептивное поле. Это соответствует природе изображений: соседние пиксели обычно связаны сильнее, чем далёкие.
- Разделение весов. Один фильтр используется во всех позициях изображения. Благодаря этому число параметров резко уменьшается, а сеть получает свойство эквивариантности к сдвигу: если объект немного сдвинулся, карта признаков тоже сдвинется.
- Иерархия признаков. Ранние слои находят простые паттерны, средние слои комбинируют их в части объектов, глубокие слои кодируют более абстрактные признаки.

Интуитивно фильтр можно понимать как маленький шаблон. Если участок изображения похож на шаблон, отклик фильтра будет большим. Например, один фильтр может реагировать на вертикальные границы, другой - на горизонтальные, третий - на цветовой переход. В процессе обучения сеть сама подбирает такие фильтры, чтобы минимизировать функцию потерь.

После свёртки обычно применяют нелинейность. Без нелинейных функций несколько свёрточных слоёв свелись бы к одному линейному преобразованию, и сеть не смогла бы описывать сложные зависимости. ReLU проста, вычислительно дешева и помогает уменьшать проблему затухающих градиентов.

Пулинг уменьшает размер карт признаков и делает представление менее чувствительным к малым сдвигам. Max pooling выбирает максимум в локальном окне, например 2×2 , и тем самым сохраняет наиболее сильный отклик признака. Average pooling берёт среднее значение и работает более сглаживающе. В современных архитектурах иногда вместо пулинга используют свёртки со stride больше 1, потому что они тоже уменьшают разрешение, но остаются обучаемыми.

Обучение CNN выполняется методом обратного распространения ошибки. Сначала выполняется прямой проход: вход проходит через свёртки, активации, нормализацию, пулинг и классификатор. Затем считается функция потерь, например кросс-энтропия для классификации:

$$L = - \sum_{r=1}^R y_r \log \hat{y}_r,$$

где R - число классов, y_r - истинная метка в one-hot представлении, $\hat{y}_r$ - предсказанная вероятность класса. После этого градиенты по весам фильтров вычисляются автоматически и параметры обновляются оптимизатором, например SGD, Adam или AdamW.

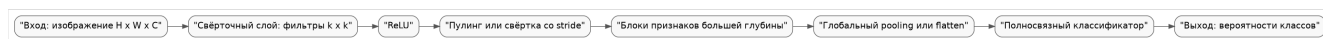


Figure 17: Схема 17

Методы, алгоритмы и этапы решения

Типовая работа с CNN начинается с постановки задачи. Нужно определить входные данные, целевую переменную и метрику качества. Для классификации изображений это могут быть accuracy, precision, recall, F1-мера или ROC-AUC. Для сегментации используют IoU и Dice coefficient. Для детекции объектов важна mAP.

Следующий этап - подготовка данных. Изображения приводят к единому размеру, нормализуют значения пикселей, разбивают выборку на обучающую, валидационную и тестовую части. Часто применяют аугментации: случайные повороты, отражения, обрезки, изменение яркости и контраста. Аугментации расширяют эффективное разнообразие обучающих данных и помогают модели не переобучаться на конкретные положения и условия съёмки.

Затем выбирают архитектуру. Для простой задачи может хватить небольшой сети вида:

1. Свёртка 3×3 , ReLU.
2. Свёртка 3×3 , ReLU.
3. Max pooling.
4. Несколько аналогичных блоков с увеличением числа каналов.

5. Global average pooling или flatten.
6. Полносвязный слой и softmax.

Для более сложных задач часто используют готовые архитектуры: LeNet, AlexNet, VGG, Inception, ResNet, DenseNet, EfficientNet. Важная идея ResNet - остаточные связи. Вместо того чтобы обучать преобразование $H(x)$, блок обучает добавку $F(x)$, а выход имеет вид:

$$y = F(x) + x.$$

Такие связи облегчают обучение глубоких сетей, потому что градиент может проходить через короткие пути. Это снижает риск деградации качества при увеличении глубины.

После выбора архитектуры задают функцию потерь, оптимизатор, learning rate, batch size и число эпох. На каждой эпохе модель получает мини-батчи изображений, делает предсказания, считает ошибку, распространяет градиенты назад и обновляет веса. Валидационная выборка используется для контроля качества на данных, не участвующих в обновлении параметров.

Для борьбы с переобучением применяют регуляризацию. Распространённые методы: увеличение данных через аугментации, dropout в полносвязной части, weight decay, batch normalization, ранняя остановка по валидационной метрике. Batch normalization нормализует распределение активаций внутри мини-батча и обычно ускоряет обучение, делая его стабильнее.

Отдельный практический метод - transfer learning. Если данных мало, не всегда разумно обучать CNN с нуля. Можно взять модель, предварительно обученную на большом наборе изображений, например ImageNet, заменить последнюю классификационную голову под свои классы и дообучить модель. Часто сначала замораживают ранние слои, потому что они кодируют универсальные признаки вроде границ и текстур, а затем при необходимости размораживают часть сети для тонкой настройки.

При анализе обученной CNN полезно смотреть не только на итоговую метрику, но и на ошибки. Матрица ошибок показывает, какие классы сеть путает. Визуализации активаций и методы вроде Grad-CAM помогают понять, на какие области изображения модель опирается при решении. Это особенно важно в медицине, промышленной диагностике и других областях, где требуется интерпретируемость.

Практические примеры

Классический пример - распознавание рукописных цифр MNIST. Входом является серое изображение $28 \times 28 \times 1$, а выходом - один из десяти классов. Даже небольшая CNN с двумя свёрточными слоями обычно показывает высокое качество, потому что форма цифр хорошо описывается локальными признаками: дугами, пересечениями, наклонами и замкнутыми контурами.

В задаче классификации фотографий товаров интернет-магазина CNN может определять категорию: обувь, сумка, куртка, часы. Ранние слои находят края и текстуры, средние - элементы вроде подошвы, ремешка или рукава, глубокие - общий образ товара. На практике здесь полезны аугментации, потому что товар может быть снят под разными углами и при разном освещении.

В медицине CNN применяют для анализа рентгеновских снимков, МРТ и КТ. Например, сеть может классифицировать снимок как нормальный или содержащий патологию, а в задаче сегментации - выделять область опухоли или повреждения. В таких задачах важно тщательно валидировать модель, контролировать смещение данных и проверять, не учится ли сеть на посторонних признаках, например на метках оборудования или особенностях конкретной клиники.

В промышленности CNN используют для визуального контроля качества. Камера снимает детали на конвейере, а модель определяет наличие трещин, царапин, деформаций или неправильной сборки. Преимущество CNN здесь в том, что дефекты часто локальны и могут иметь разные положения на изображениях. Свёрточные фильтры позволяют искать один и тот же визуальный паттерн по всей поверхности детали.

CNN применяются не только к изображениям. В обработке речи спектрограмму можно рассматривать как двумерное изображение, где одна ось соответствует времени, другая - частоте. Свёрточная сеть выделяет локальные паттерны в частотно-временной области. В анализе временных рядов одномерные свёртки помогают находить короткие характерные фрагменты сигнала, например всплески нагрузки, аномалии датчиков или повторяющиеся паттерны поведения пользователя.

Итак, свёрточные нейронные сети эффективны там, где данные имеют локальную структуру и полезно искать одинаковые признаки в разных позициях. Их сила основана на локальных рецептивных полях, разделении весов, иерархическом извлечении признаков и обучении фильтров напрямую по целевой задаче. На практике качество CNN определяется не только архитектурой, но и подготовкой данных, регуляризацией, корректной валидацией и анализом ошибок.

Билет 17. Постановка задачи анализа временных рядов. Основные методы решения

Основные понятия и определения

Временной ряд – это последовательность наблюдений некоторой величины, упорядоченная по времени: значения температуры по дням, курс валюты по минутам, объем продаж по месяцам, нагрузка на сервер по секундам. Обычно временной ряд записывают как набор значений $y_1, y_2, \dots, y_T$, где индекс t соответствует моменту времени. Главное отличие временного ряда от обычной выборки состоит в том, что наблюдения часто зависимы: значение сегодня связано со значениями вчера, неделю назад или год назад.

Задача анализа временных рядов состоит в том, чтобы описать структуру этих зависимостей, выделить закономерности, объяснить поведение процесса и, если нужно, построить прогноз будущих значений. В зависимости от цели анализ может быть описательным, прогностическим, диагностическим или управляющим. Например, экономисту важно понять тренд инфляции, инженеру – обнаружить аномалию в датчике, менеджеру – спрогнозировать спрос, а системе автоматического управления – принять решение на основе текущего состояния процесса.

Ключевыми компонентами временного ряда считаются тренд, сезонность, цикличность и случайная составляющая. Тренд отражает долгосрочное направление изменения уровня ряда: рост, спад или плавную стабилизацию. Сезонность описывает регулярные повторения с известным периодом, например недельные колебания посещаемости сайта или годовую сезонность продаж. Цикличность похожа на сезонность, но ее период обычно менее строгий и может зависеть от экономических, социальных или технологических факторов. Остаток, или шум, содержит нерегулярные колебания, которые не объяснены выбранной моделью.

Важное понятие – стационарность. Ряд называют стационарным в широком смысле, если его математическое ожидание постоянно, дисперсия конечна и постоянна, а ковариация между y_t и y_{t-k} зависит только от лага k , но не от самого времени t . Многие классические методы, особенно ARMA/ARIMA-подходы, удобнее применять к стационарным рядам. Если ряд нестационарен из-за тренда или меняющейся дисперсии, его часто преобразуют: берут разности, логарифмы, сезонные разности или удаляют тренд.

Еще одно базовое понятие – лаг. Лаг k означает сдвиг ряда на k моментов назад. Автокорреляция показывает, насколько текущее значение связано с прошлыми значениями. Частная автокорреляция измеряет связь между y_t и y_{t-k} после учета промежуточных лагов. Эти характеристики помогают понять память процесса и выбрать порядок модели.

Теория, формулы и интуиция

Один из простейших способов описания временного ряда – разложение на компоненты. В аддитивной модели предполагается

$$y_t = T_t + S_t + E_t,$$

где T_t – тренд, S_t – сезонная компонента, E_t – случайная ошибка. Такая запись удобна, когда амплитуда сезонных колебаний примерно постоянна. Если сезонные колебания растут вместе с уровнем ряда, используют мультипликативную модель

$$y_t = T_t \cdot S_t \cdot E_t.$$

После логарифмирования мультипликативная модель часто превращается в аддитивную, потому что произведение становится суммой.

Интуитивно анализ временного ряда можно представить как попытку отделить закономерность от случайности. Если ряд растет, нужно понять, является ли рост устойчивым трендом или случайным всплеском. Если значения повторяются каждую неделю, нужно учесть сезонный эффект, иначе прогноз будет систематически ошибаться. Если остатки после модели все еще автокоррелированы, значит модель не извлекла всю временную структуру.

В классической вероятностной постановке временной ряд рассматривается как реализация случайного процесса. Для модели авторегрессии порядка p , обозначаемой $AR(p)$, текущее значение зависит от p предыдущих значений:

$$y_t = c + \varphi_1 y_{t-1} + \varphi_2 y_{t-2} + \dots + \varphi_p y_{t-p} + \varepsilon_t.$$

Здесь c – константа, φ_i – параметры, ε_t – белый шум, то есть случайная ошибка с нулевым средним, постоянной дисперсией и отсутствием автокорреляции.

В модели скользящего среднего $MA(q)$ текущее значение зависит не от прошлых значений самого ряда, а от прошлых ошибок:

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}.$$

Комбинация этих идей дает $ARMA(p, q)$. Если исходный ряд нестационарен, но становится стационарным после взятия d разностей, используют $ARIMA(p, d, q)$. Разность первого порядка задается формулой

$$\Delta y_t = y_t - y_{t-1}.$$

Для сезонных рядов применяют $SARIMA$, где добавляются сезонные авторегрессионные, разностные и скользящие компоненты.

Качество прогноза обычно оценивают на отложенном временном участке, потому что перемешивать наблюдения при проверке временных рядов нельзя: модель должна обучаться на прошлом и проверяться на будущем. Часто используют метрики MAE , MSE , $RMSE$ и $MAPE$:

$$MAE = \frac{1}{n} \sum_{t=1}^n \|y_t - \hat{y}_t\|, \quad RMSE = \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}.$$

MAE легче интерпретировать в исходных единицах, $RMSE$ сильнее штрафует крупные ошибки, а $MAPE$ показывает относительную ошибку в процентах, но плохо работает при нулевых или очень малых фактических значениях.

Общая логика анализа временных рядов может быть представлена так:

Методы, алгоритмы и этапы решения

Первый этап анализа – постановка задачи. Нужно определить, что именно требуется: прогноз на несколько шагов вперед, обнаружение аномалий, классификация рядов, оценка влияния внешних факторов или сглаживание шума. Важно уточнить горизонт прогноза, допустимую ошибку, частоту наблюдений, наличие пропусков и возможность использовать дополнительные признаки. Например, прогноз продаж на один день и прогноз на год требуют разных моделей и разных допущений.

Второй этап – предварительный анализ данных. Строят график ряда, проверяют пропуски, выбросы, смену режима, резкие разрывы, изменение частоты наблюдений. Пропуски можно заполнять интерполяцией, средними значениями по сезону, модельными прогнозами или оставить специальными признаками, если выбранный алгоритм умеет с ними работать. Выбросы не всегда являются ошибками: для финансового рынка скачок цены может быть важным событием, а для датчика температуры одиночное невозможное значение может быть техническим сбоем.

Третий этап – анализ структуры. Здесь оценивают тренд, сезонность, автокорреляционную функцию ACF и частную автокорреляционную функцию $PACF$. Для проверки стационарности применяют тесты, например Дики-Фуллера или KPSS. Если ряд нестационарен, используют преобразования. Логарифм стабилизирует относительные колебания, разности удаляют тренд, сезонные разности помогают убрать регулярный сезонный уровень.

К простым базовым методам относятся наивный прогноз, среднее, скользящее среднее и экспоненциальное сглаживание. Наивный прогноз полагает, что будущее значение равно последнему наблюдению: $\hat{y}_{t+1} = y_t$. Несмотря на простоту, это сильная базовая линия для многих финансовых и слабо предсказуемых рядов. Скользящее среднее сглаживает локальные колебания, но запаздывает при резкой смене тренда. Экспоненциальное сглаживание присваивает недавним наблюдениям больший вес. В простой форме оно задается как

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t,$$

где $0 < \alpha < 1$. Чем больше α , тем быстрее модель реагирует на новые данные. Метод Хольта добавляет тренд, а метод Хольта-Уинтерса учитывает и тренд, и сезонность.

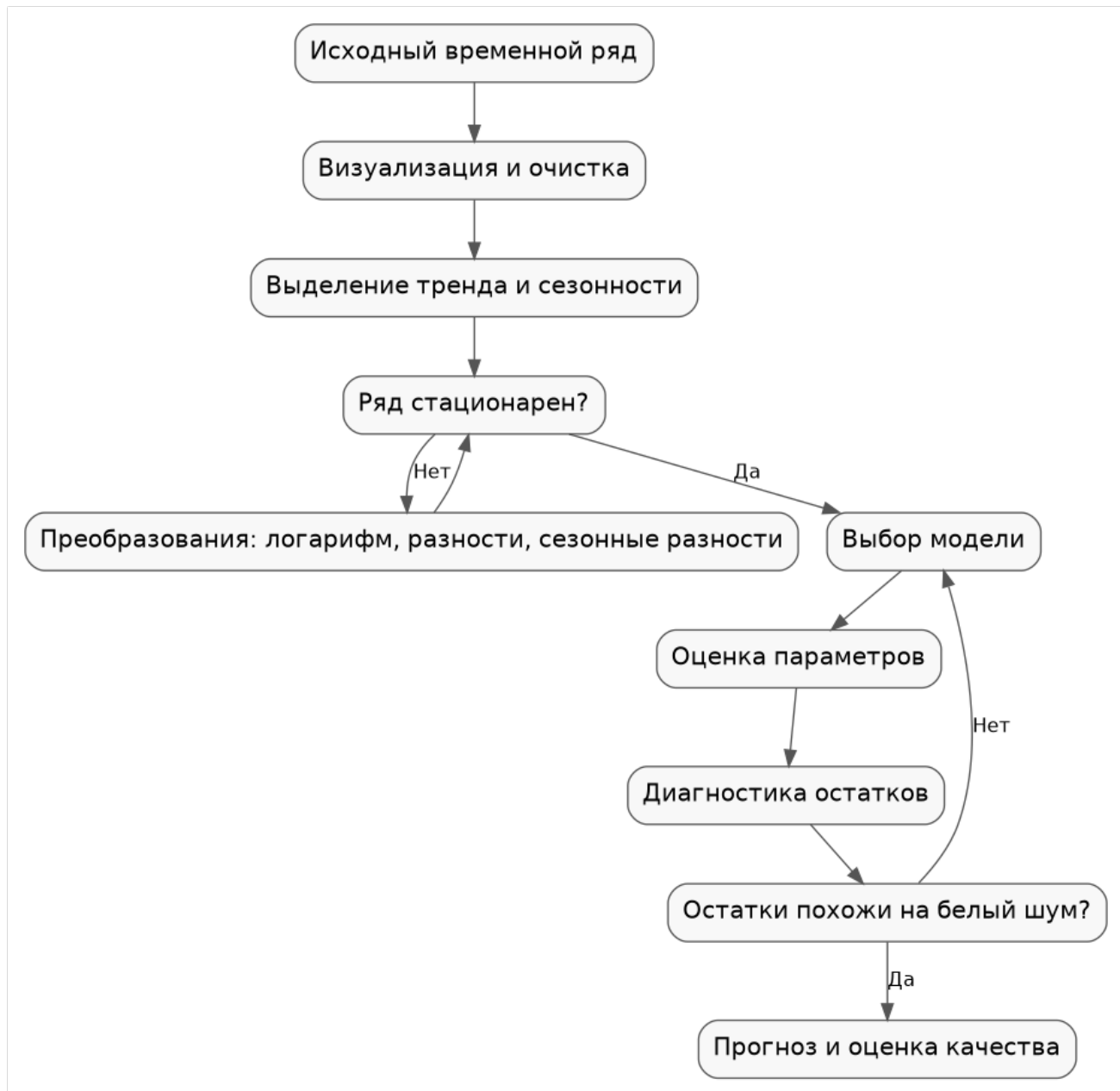


Figure 18: Схема 18

Классические статистические методы включают AR, MA, ARMA, ARIMA и SARIMA. Их преимущество – интерпретируемость и строгая теория. Они хорошо работают на сравнительно коротких рядах с понятной автокорреляционной структурой. Подбор параметров можно выполнять по графикам ACF/PACF, информационным критериям AIC/BIC или перебором на валидационном участке. После обучения обязательно проверяют остатки: если в них сохраняется автокорреляция, модель недоописала структуру ряда.

Если на ряд влияют внешние переменные, используют регрессионные модели с временными признаками или ARIMAX/SARIMAX. Например, спрос на товар может зависеть не только от прошлых продаж, но и от цены, праздников, рекламных кампаний, погоды и дня недели. В машинном обучении временной ряд часто превращают в табличную задачу: создают лаговые признаки y_{t-1} , y_{t-2} , агрегаты по окнам, календарные признаки, индикаторы праздников и затем обучают линейную регрессию, деревья решений, случайный лес, градиентный бустинг или нейронную сеть. Важно соблюдать временную схему проверки, иначе возникает утечка информации из будущего.

Для сложных и длинных рядов применяют нейросетевые методы: рекуррентные сети, LSTM, GRU, Temporal Convolutional Networks и трансформеры для временных рядов. Они способны учитывать нелинейные зависимости и множество входных признаков, но требуют больше данных, аккуратной регуляризации и серьезной настройки. В практических задачах они не всегда превосходят хорошо построенные статистические модели или градиентный бустинг с качественными признаками.

Отдельная группа методов связана с обнаружением аномалий. Подходы бывают статистическими, например контрольные карты и анализ остатков, и алгоритмическими, например Isolation Forest, One-Class SVM или автоэнкодеры. Часто аномалией считают не само большое значение, а большое отклонение от ожидаемого поведения с учетом тренда и сезонности. Так, высокий объем заказов в декабре может быть нормой, а тот же объем в обычный день – сигналом необычного события.

Итоговый алгоритм решения обычно выглядит так: сформулировать цель, собрать и упорядочить данные, очистить ряд, построить графики, выделить сезонность и тренд, выбрать базовую модель, затем более сложные модели, проверить качество на будущем интервале, проанализировать остатки, выбрать модель по метрикам и устойчивости, после чего внедрить прогноз или выводы в прикладной процесс. Для реальной эксплуатации также нужны мониторинг качества и регулярное переобучение, потому что структура временного ряда может меняться.

Практические примеры

Первый пример – прогноз месячных продаж магазина. Значение y_t – выручка за месяц. На графике видно, что продажи растут и резко увеличиваются в декабре. Здесь есть тренд и годовая сезонность, поэтому простое среднее будет давать плохой прогноз. Подходящими базовыми методами будут Хольт-Уинтерс или SARIMA с сезонным периодом 12. Если известны скидки, рекламные акции и праздники, можно добавить внешние признаки и использовать SARIMAX или градиентный бустинг. Проверять модель нужно на последних месяцах, например обучиться на данных за несколько лет и оценить прогноз на последнем году.

Второй пример – мониторинг нагрузки на сервер. Данные приходят каждую минуту: число запросов, время ответа, загрузка процессора. Здесь часто есть суточная и недельная сезонность: днем нагрузка выше, ночью ниже, в рабочие дни поведение отличается от выходных. Для прогноза можно использовать сезонное сглаживание или модель с календарными признаками. Для аномалий удобно строить ожидаемый диапазон значений: если фактическая нагрузка сильно выше прогноза и остаток выходит за допустимые границы, система подает сигнал.

Третий пример – анализ курса валюты. Такой ряд часто не имеет устойчивого прогнозируемого тренда на коротком горизонте, а его изменения ближе к случайному процессу. Наивная модель, где завтрашний курс равен сегодняшнему, может оказаться сильной базовой линией. Вместо прогнозирования самого курса иногда анализируют доходности $r_t = \log(y_t/y_{t-1})$, потому что они ближе к стационарным. Для оценки риска используют модели волатильности, например ARCH/GARCH, которые описывают не среднее значение ряда, а изменчивость.

Четвертый пример – временной ряд температуры воздуха. В нем выражена годовая сезонность, но также есть шум и локальные погодные колебания. Если нужно восстановить пропущенные значения, можно использовать интерполяцию и сезонные средние. Если нужно прогнозировать температуру на несколько дней, одного исторического ряда мало: потребуются метеорологические признаки. Этот пример показывает, что временной ряд не всегда сам по себе содержит всю информацию, необходимую для качественного прогноза.

Таким образом, анализ временных рядов объединяет статистическое описание, вероятностное моделирование, машинное обучение и предметную интерпретацию. Главная идея состоит в том, чтобы учитывать порядок наблюдений во времени, отделять регулярные компоненты от случайных колебаний и проверять модель так, как она будет использоваться на практике: прошлое объясняет

будущее, а не наоборот.

Билет 18. Ансамбли моделей. Основные понятия: бэггинг, бустинг, стэкинг. Примеры алгоритмов

Основные понятия и определения

Ансамбль моделей в машинном обучении - это метод построения прогноза, при котором итоговое решение получается не одной моделью, а комбинацией нескольких базовых моделей. Идея ансамблирования основана на том, что разные модели могут ошибаться по-разному: одна модель хорошо описывает одни области пространства признаков, другая - другие, третья может быть устойчива к шуму. Если правильно объединить их ответы, можно получить более точный и устойчивый алгоритм, чем любой отдельный участник ансамбля.

Базовую модель ансамбля часто называют слабым или базовым учеником. Слово “слабый” не всегда означает плохую модель: в бустинге слабым учеником обычно называют алгоритм, который лишь немного лучше случайного угадывания, например неглубокое дерево решений. В бэггинге и стэкинге базовые модели могут быть достаточно сильными: деревья, линейные модели, нейронные сети, SVM, kNN и другие методы.

Главные способы объединения прогнозов - усреднение, голосование и обучение метамодели. Для задачи регрессии часто используют среднее:

$$\hat{y}(x) = \frac{1}{M} \sum_{m=1}^M f_m(x),$$

где M - число моделей в ансамбле, $f_m(x)$ - прогноз m -й модели. Для классификации применяется голосование по классам:

$$\hat{y}(x) = \arg \max_k \sum_{m=1}^M I(f_m(x) = k),$$

или усреднение вероятностей классов:

$$\hat{p}(y = k | x) = \frac{1}{M} \sum_{m=1}^M p_m(y = k | x).$$

Три классические группы ансамблевых методов - бэггинг, бустинг и стэкинг. Бэггинг обучает базовые модели независимо на разных выборках и затем усредняет их ответы. Бустинг строит модели последовательно: каждая следующая модель уделяет больше внимания ошибкам предыдущих. Стэкинг обучает несколько моделей первого уровня, а затем отдельную модель второго уровня, которая учится объединять их прогнозы.

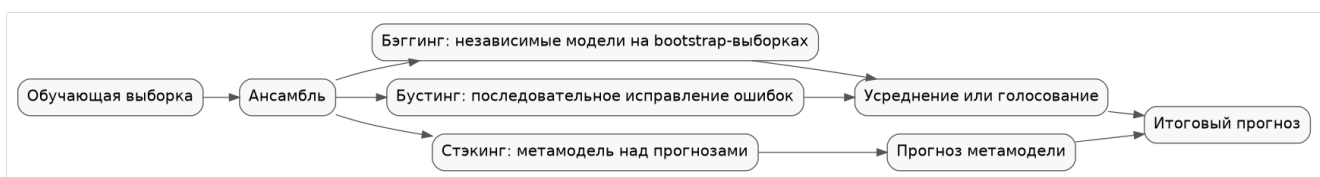


Figure 19: Схема 19

Теория, формулы и интуиция

Качество модели удобно рассматривать через разложение ошибки на смещение, разброс и шум. В регрессии для квадратичной функции потерь математически это записывают как:

$$E[(y - \hat{f}(x))^2] = Bias^2[\hat{f}(x)] + Var[\hat{f}(x)] + \sigma^2,$$

где смещение показывает, насколько в среднем модель далека от истинной зависимости, разброс показывает чувствительность модели к обучающей выборке, а σ^2 - неустраняемый шум данных. Ансамбли полезны потому, что позволяют управлять смещением и разбросом.

Бэггинг прежде всего снижает разброс. Если обучить несколько нестабильных моделей, например глубоких деревьев решений, на немного разных выборках и усреднить их ответы, случайные колебания отдельных деревьев частично компенсируются. Для независимых моделей с одинаковой дисперсией σ^2 дисперсия среднего равна σ^2/M . На практике модели не независимы, поэтому более точная формула учитывает корреляцию ρ :

$$Var\left(\frac{1}{M} \sum_{m=1}^M f_m(x)\right) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2.$$

Из этой формулы видно, что простое увеличение числа моделей не решает проблему полностью, если они слишком похожи. Поэтому в случайном лесе используют не только bootstrap-выборки объектов, но и случайный выбор признаков при построении разбиений в деревьях: это уменьшает корреляцию между деревьями.

Бустинг работает иначе. Его цель - постепенно построить сильную модель как сумму более простых моделей:

$$F_M(x) = \sum_{m=1}^M \alpha_m h_m(x),$$

где $h_m(x)$ - базовый алгоритм, а α_m - вес его вклада. Интуитивно бустинг похож на последовательное уточнение ответа: первая модель строит грубое приближение, вторая исправляет ее ошибки, третья исправляет ошибки уже составной модели и так далее. В градиентном бустинге это формализуется как оптимизация функции потерь. На шаге m новая модель обучается приближать антиградиент функции потерь по текущим предсказаниям:

$$r_i^{(m)} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right] * F = F * m - 1.$$

После этого добавляется новая компонента:

$$F_m(x) = F_{m-1}(x) + \eta \gamma_m h_m(x),$$

где η - скорость обучения, а γ_m - коэффициент, найденный минимизацией потерь. Малое значение η делает обучение более осторожным: обычно нужно больше деревьев, но модель лучше обобщает.

Стэкинг не ограничивается усреднением или последовательным исправлением. Он строит новую обучающую выборку, где признаками являются прогнозы базовых моделей. Например, если есть логистическая регрессия, дерево решений и градиентный бустинг, то для каждого объекта можно получить три прогноза или три набора вероятностей классов. Метамоделю, например линейная регрессия или логистическая регрессия, учится определять, каким базовым прогнозам больше доверять в разных ситуациях. Важное условие стэкинга - метамоделю должна обучаться на прогнозах, полученных без утечки целевой переменной. Поэтому обычно используют out-of-fold прогнозы: базовые модели обучаются на части фолдов, а предсказывают на отложенном фолде.

Сильная сторона ансамблей связана с разнообразием ошибок. Если все модели делают одинаковые ошибки, их объединение почти ничего не дает. Поэтому полезны разные источники разнообразия: разные подвыборки объектов, разные подмножества признаков, разные гиперпараметры, разные семейства моделей, разные функции потерь. Однако ансамбль не является гарантией качества. Он может переобучиться, стать слишком сложным, потерять интерпретируемость и потребовать значительно больше вычислительных ресурсов.

Методы, алгоритмы и этапы решения

Бэггинг, или bootstrap aggregating, строится по простой схеме. Сначала из исходной обучающей выборки

много раз формируют bootstrap-выборки: каждая такая выборка имеет тот же размер, что и исходная, но объекты берутся с возвращением. Поэтому одни объекты могут попасть несколько раз, а другие не попасть вовсе. Затем на каждой bootstrap-выборке независимо обучается базовая модель. На этапе предсказания ответы моделей объединяются голосованием или усреднением.

Классический пример бэггинга - BaggingClassifier или BaggingRegressor с деревьями решений. Более известный частный случай - случайный лес. В случайном лесе каждое дерево обучается на bootstrap-выборке, а при поиске лучшего разбиения в узле рассматривает не все признаки, а случайное подмножество. Это делает деревья менее похожими и повышает качество усреднения. Случайный лес хорошо работает на табличных данных, устойчив к выбросам и нелинейностям, сравнительно мало требует масштабирования признаков, но может быть тяжелым по памяти и хуже экстраполировать в регрессии.

Для оценки качества в бэггинге можно использовать out-of-bag оценку. Так как примерно треть объектов не попадает в конкретную bootstrap-выборку, для каждого объекта находятся модели, которые не видели его при обучении. Их прогнозы можно использовать как внутреннюю валидацию без отдельного отложенного набора. Это особенно удобно для случайного леса.

Бустинг имеет несколько важных разновидностей. AdaBoost - один из ранних алгоритмов бустинга. В классификации он обучает слабые классификаторы последовательно и изменяет веса объектов: неверно классифицированные объекты получают больший вес, поэтому следующая модель фокусируется на них. Вес самой модели зависит от ее ошибки:

$$\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m},$$

где ε_m - взвешенная ошибка слабого классификатора. Итоговый ответ получается взвешенным голосованием.

Градиентный бустинг обобщает идею AdaBoost на разные функции потерь. Вместо прямого изменения весов он обучает новую модель на остатках или псевдоостатках. Для квадратичной ошибки в регрессии псевдоостатки совпадают с обычными остатками:

$$r_i = y_i - F_{m-1}(x_i).$$

Популярные реализации градиентного бустинга - Gradient Boosting Machines, XGBoost, LightGBM и CatBoost. XGBoost известен регуляризацией, эффективной работой с разреженными признаками и контролем сложности деревьев. LightGBM использует эффективное построение деревьев и часто очень быстр на больших табличных данных. CatBoost хорошо работает с категориальными признаками, потому что имеет специальные методы кодирования категорий, уменьшающие утечку и переобучение. Основные гиперпараметры бустинга - число деревьев, глубина деревьев, скорость обучения, минимальное число объектов в листьях, доля объектов и признаков для стохастического обучения, параметры регуляризации. Бустинг часто дает высокое качество, но более чувствителен к настройкам, шуму в данных и утечкам. При слишком большом числе деревьев и слабой регуляризации он может переобучиться, поэтому важны валидационная выборка и early stopping.

Стэкинг обычно строится в несколько этапов. Сначала выбирают несколько достаточно разных моделей первого уровня. Затем обучают их по кросс-валидационной схеме и получают out-of-fold прогнозы для обучающей выборки. Эти прогнозы становятся признаками для метамодел. После обучения метамодел базовые алгоритмы переобучают на всей обучающей выборке. Для нового объекта сначала получают прогнозы базовых моделей, затем передают их метамодел, которая выдает окончательный ответ.

Для классификации в качестве входа метамодел часто используют вероятности классов, а не жесткие метки. Это сохраняет больше информации: прогноз “класс 1 с вероятностью 0.51” и “класс 1 с вероятностью 0.99” имеют разную уверенность, хотя метка одна и та же. Для регрессии метамодел получает численные прогнозы базовых регрессоров. Часто хорошей метамоделю является простая линейная или логистическая регрессия: она снижает риск переобучения и интерпретируемо взвешивает базовые прогнозы.

Отдельно можно упомянуть blending - упрощенный вариант стэкинга. В нем часть обучающей выборки выделяется как holdout-набор. Базовые модели обучаются на основной части, предсказывают на holdout, а метамодел обучается на этих предсказаниях. Blending проще, но менее эффективно использует данные, чем кросс-валидационный стэкинг.

Практические примеры

В задаче кредитного скоринга есть табличные данные: возраст клиента, доход, история платежей,

наличие просрочек, тип занятости. Одиночное дерево решений может легко переобучиться: оно построит слишком специфические правила для отдельных групп клиентов. Случайный лес уменьшит этот риск, потому что много деревьев будут обучены на разных подвыборках и разных признаках. Итоговое голосование даст более стабильную оценку вероятности дефолта. Такой подход полезен, когда важны надежность и устойчивость к шуму.

В задаче предсказания цены квартиры градиентный бустинг часто оказывается сильнее случайного леса. Цена зависит от нелинейного сочетания площади, района, этажа, года постройки, расстояния до метро, состояния дома и рыночных факторов. Бустинг последовательно исправляет ошибки: первые деревья могут уловить общий вклад площади и района, следующие - поправки для этажности, типа дома и редких комбинаций признаков. CatBoost особенно удобен, если много категориальных признаков: район, серия дома, тип ремонта, застройщик.

В задаче классификации текстов можно применить стэкинг. Пусть первая модель - логистическая регрессия на TF-IDF признаках, вторая - градиентный бустинг на статистических признаках текста, третья - нейронная модель на эмбедингах. Логистическая регрессия хорошо ловит ключевые слова, бустинг использует длину текста, частоты символов и другие агрегаты, нейронная модель учитывает семантическое сходство. Метамоделю на вероятностях этих трех моделей можно научиться, что для коротких текстов надежнее TF-IDF, а для перефразированных сообщений - эмбединги.

В медицинской диагностике ансамбли применяют осторожно, потому что важны интерпретируемость и контроль ошибок. Например, случайный лес может помочь при предварительной оценке риска заболевания по анализам и анамнезу, но врачам нужны объяснения: важность признаков, калибровка вероятностей, проверка на независимой выборке. Бустинг может дать высокую точность, но при плохой валидации легко выучит особенности конкретной клиники или прибора. Поэтому практическое применение ансамблей всегда требует строгого разделения train/test, анализа утечек и проверки стабильности.

В соревнованиях по машинному обучению ансамбли часто используются как финальный способ повысить качество. Участники обучают несколько сильных моделей с разными признаками, seed-значениями и архитектурами, затем усредняют вероятности или применяют стэкинг. Такой ансамбль может улучшить метрику на несколько десятых или сотых процента, что важно в соревновании. Но в промышленной системе цена усложнения может оказаться выше выигрыша: ансамбль труднее поддерживать, дольше обучать, сложнее обновлять и объяснять пользователям.

Практический выбор метода зависит от данных и требований. Если нужна устойчивая модель с минимальной настройкой, часто начинают со случайного леса или простого бэггинга. Если нужен максимум качества на табличных данных и есть возможность тщательно валидировать модель, обычно пробуют градиентный бустинг: XGBoost, LightGBM или CatBoost. Если уже есть несколько сильных и разнообразных моделей, имеет смысл применить стэкинг, но обязательно строить метапризнаки без утечки через кросс-валидацию. Общая идея остается одной: ансамбль выигрывает тогда, когда отдельные модели достаточно точны и при этом достаточно различны в своих ошибках.

Билет 19. Дисбаланс классов в задачах классификации. Причины возникновения проблем. Методы борьбы с дисбалансом

Основные понятия и определения

Дисбаланс классов в задаче классификации - это ситуация, при которой объекты разных классов представлены в обучающей выборке существенно неравномерно. Например, в задаче выявления мошеннических транзакций нормальных операций может быть 99,9%, а мошеннических - 0,1%; в медицинской диагностике здоровых пациентов может быть намного больше, чем пациентов с редким заболеванием. Класс объектов которого много, называют мажоритарным, а класс с малым числом объектов - миноритарным.

Важно отличать дисбаланс классов от простой малой выборки. Если оба класса представлены примерно одинаково, но данных мало, проблема в основном связана с высокой дисперсией оценки и переобучением. При дисбалансе же модель может получить много информации о типичном поведении мажоритарного класса и слишком мало - о границе, признаках и разнообразии миноритарного класса. В результате алгоритм может формально показывать высокую точность, но плохо решать прикладную задачу.

Дисбаланс бывает бинарным и многоклассовым. В бинарном случае положительный класс часто соответствует редкому, но важному событию: болезни, отказу оборудования или атаке. В

многоклассовой классификации распределение может иметь длинный хвост: несколько популярных классов содержат большую долю данных, а многие классы представлены десятками или единицами примеров.

Степень дисбаланса часто описывают отношением размеров классов:

$$IR = \frac{N_{majority}}{N_{minority}},$$

где IR - imbalance ratio, $N_{\{majority\}}$ - число объектов мажоритарного класса, $N_{\{minority\}}$ - число объектов миноритарного класса. Если $IR = 10$, это умеренный дисбаланс; если $IR = 1000$, проблема обычно требует специальной обработки. Однако важна не только пропорция, но и цена ошибок. Иногда соотношение 1:20 критично, если пропуск редкого события очень дорог, а иногда 1:100 приемлемо, если редкий класс хорошо отделим признаками.

Теория, формулы и интуиция

Главная причина, почему дисбаланс создает трудности, связана с функцией потерь и критерием качества. Многие стандартные алгоритмы минимизируют среднюю ошибку по всем объектам. Если редкий класс занимает 1% выборки, то ошибки на нем дают малый вклад в общую среднюю потерю. Модель может снизить глобальную ошибку, почти всегда предсказывая мажоритарный класс.

Простейший пример: в выборке 990 объектов класса 0 и 10 объектов класса 1. Классификатор, который всегда предсказывает 0, имеет ассигасу:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{990}{1000} = 0,99.$$

На первый взгляд качество 99% выглядит отличным, но модель не находит ни одного объекта редкого класса. Поэтому ассигасу при дисбалансе часто является вводящей в заблуждение метрикой. В таких задачах нужно смотреть на матрицу ошибок:

	Предсказан положительный класс		Предсказан отрицательный класс		
Истинно положительный	TP	FN	FP	TN	

Для миноритарного положительного класса особенно важны precision и recall:

$$Precision = \frac{TP}{TP + FP},$$

$$Recall = \frac{TP}{TP + FN}.$$

Precision показывает, какая доля объектов, помеченных моделью как положительные, действительно положительна. Recall показывает, какую долю настоящих положительных объектов модель смогла найти. Их гармоническое среднее называется F1-мерой:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}.$$

Если важнее не пропустить редкий класс, обычно оптимизируют recall или используют F-beta:

$$F_{\beta} = (1 + \beta^2) \cdot \frac{Precision \cdot Recall}{\beta^2 \cdot Precision + Recall},$$

где при $\beta > 1$ больше весит полнота, а при $\beta < 1$ - точность.

Также применяют ROC-AUC и PR-AUC. ROC-кривая строится по true positive rate и false positive rate:

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}.$$

ROC-AUC полезна, но при сильном дисбалансе может быть чрезмерно оптимистичной, потому что FPR нормируется на большое число отрицательных объектов. PR-AUC, то есть площадь под precision-recall

кривой, часто информативнее для редкого положительного класса, поскольку напрямую показывает компромисс между точностью обнаружений и полнотой.

Еще одна важная идея - априорные вероятности классов. Байесовский классификатор выбирает класс с максимальной апостериорной вероятностью:

$$\hat{y} = \arg \max_k P(y = k | x).$$

Если один класс встречается намного чаще, его априорная вероятность $P(y=k)$ велика. При слабых или шумных признаках модель естественно склоняется к более частому классу. Это не обязательно ошибка алгоритма: проблема возникает, когда целевая функция не соответствует реальной цене решений.

У дисбаланса есть и геометрическая сторона. Миноритарный класс может занимать небольшие, разрозненные области пространства признаков. Если объектов мало, трудно надежно оценить границу между классами. При этом мажоритарные объекты могут окружать миноритарные, образуя перекрытие классов. Тогда простое увеличение веса редкого класса не всегда помогает: модель начинает ловить шум и растет число ложных срабатываний.

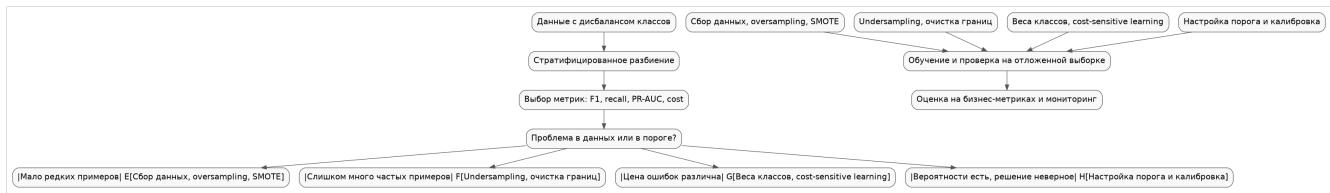


Figure 20: Схема 20

Методы, алгоритмы и этапы решения

Первый этап работы с дисбалансом - корректная постановка задачи. Нужно определить, какой класс является редким, какие ошибки дороже и какую метрику следует оптимизировать. Например, в антифроде ложноположительное срабатывание может раздражать клиента, но ложноотрицательное может привести к финансовым потерям. В медицине цена пропуска болезни часто выше цены дополнительного обследования. Без этого выбора невозможно понять, что значит “хорошая модель”.

Второй этап - правильная валидация. Разбиение на train, validation и test должно быть стратифицированным, то есть сохранять доли классов. Иначе в тестовой выборке может случайно оказаться слишком мало редких объектов, и оценка качества станет нестабильной. При малом числе миноритарных объектов полезна стратифицированная кросс-валидация. Все операции ресемплинга нужно выполнять только на обучающей части внутри каждого фолда, чтобы не допустить утечки данных.

Один из базовых подходов - изменение весов классов. Взвешенная функция потерь штрафует ошибки на редком классе сильнее:

$$L = \sum_{i=1}^n w_{y_i} \cdot \ell(f(x_i), y_i),$$

где $w_{\{y_i\}}$ - вес класса объекта i . Часто веса выбирают обратно пропорционально частотам классов:

$$w_k = \frac{n}{K \cdot n_k},$$

где n - общее число объектов, K - число классов, n_k - число объектов класса k . Такой подход есть во многих алгоритмах: логистической регрессии, SVM, деревьях, случайном лесе, градиентном бустинге. Преимущество метода в том, что он не меняет сами данные. Недостаток - при очень шумном редком классе модель может переобучаться на ошибочные или нетипичные примеры.

Второй класс методов - oversampling, то есть увеличение числа объектов миноритарного класса. Простое дублирование редких объектов может помочь алгоритму “увидеть” этот класс, но не добавляет новой информации и повышает риск переобучения. Более развитый метод - SMOTE. Он создает

синтетические объекты как интерполяции между близкими миноритарными примерами:

$$x_{new} = x_i + \lambda(x_{nn} - x_i), \quad \lambda \in [0, 1],$$

где x_i - редкий объект, x_{nn} - один из его ближайших соседей того же класса. Идея SMOTE состоит в расширении области миноритарного класса, а не в копировании точек. Есть модификации вроде Borderline-SMOTE и ADASYN, которые уделяют больше внимания объектам около границы. Однако синтетические примеры могут быть вредны, если классы сильно перекрываются или признаки категориальные без корректной обработки.

Третий класс методов - undersampling, то есть уменьшение числа объектов мажоритарного класса. Случайное удаление частых объектов ускоряет обучение, но может выбросить полезную информацию. Более аккуратные методы очищают область границы: Tomek links удаляют пограничные пары ближайших соседей разных классов, а Edited Nearest Neighbors удаляет объекты, чьи метки не согласуются с соседями. Undersampling особенно уместен, когда мажоритарных объектов очень много и они избыточны.

Четвертый подход - настройка порога классификации. Многие модели возвращают вероятность или score, а итоговое решение принимается по порогу:

$$\hat{y} = \begin{cases} 1, & p(y = 1 | x) \geq t, \\ 0, & p(y = 1 | x) < t. \end{cases}$$

Стандартный порог $t = 0.5$ не обязан быть оптимальным. При дисбалансе и разной цене ошибок порог часто сдвигают: чтобы повысить recall редкого класса, его уменьшают; чтобы снизить число ложных тревог, увеличивают. Порог выбирают на validation-выборке по целевой метрике, например максимальному F1, требуемому recall при ограничении на precision или минимальной ожидаемой стоимости ошибок:

$$Cost = C_{FP} \cdot FP + C_{FN} \cdot FN.$$

Пятый подход - cost-sensitive learning, обучение с учетом стоимости ошибок. В отличие от простого балансирования классов, здесь веса задаются не только частотами, но и прикладными потерями. Например, если пропуск дефекта стоит в 100 раз дороже ложной проверки, модель должна учитывать это в функции потерь или в выборе порога. Такой подход особенно важен, когда реальное распределение классов менять нельзя, но решение должно быть экономически обоснованным.

Шестой подход - ансамбли и специальные алгоритмы. Balanced Random Forest обучает деревья на сбалансированных подвыборках. EasyEnsemble строит несколько моделей на разных undersampling-выборках мажоритарного класса и объединяет их. Для нейронных сетей применяют focal loss:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t),$$

которая уменьшает вклад легких примеров и усиливает вклад трудных. Это полезно, когда огромное число простых объектов мажоритарного класса доминирует в обучении.

Седьмой путь - сбор и улучшение данных. Иногда никакой алгоритмический прием не заменяет дополнительные размеченные примеры редкого класса. Полезны активное обучение, целевой сбор редких случаев, уточнение разметки или переход к anomaly detection, если положительных примеров почти нет. При аномалиях модель часто обучают на нормальном классе и ищут отклонения.

Практические примеры

Рассмотрим задачу выявления мошеннических транзакций. Пусть мошенничество составляет 0,2% операций. Если обучить модель и оценивать ее по ассигасу, почти бесполезный классификатор может получить качество выше 99%. Поэтому сначала выбирают метрики: например, recall мошенничества не ниже 80% при precision не ниже 10%, потому что каждое срабатывание проверяет антифрод-команда. Далее данные разбивают стратифицированно, обучают модель с весами классов или градиентный бустинг, затем подбирают порог по PR-кривой. Если ложных тревог слишком много, повышают порог или добавляют признаки: историю клиента, географию, частоту операций, поведение устройства.

Другой пример - медицинская диагностика редкого заболевания. Здесь пропуск больного пациента может быть критичен, поэтому целевой метрикой может быть высокая полнота. Но нельзя просто заставить модель предсказывать болезнь всем пациентам: precision станет низкой, система перегрузит врачей и вызовет ненужные обследования. На практике используют взвешенное обучение, калибруют

вероятности, выбирают несколько порогов риска и разделяют решения: низкий риск - обычное наблюдение, средний - дополнительный анализ, высокий - срочное обследование. Так классификатор становится частью процесса принятия решений, а не единственным источником диагноза.

В промышленном контроле качества брак может встречаться редко, но данные часто имеют визуальную или сенсорную природу. Если дефектов мало, применяют аугментации изображений, oversampling дефектных примеров, focal loss или методы обнаружения аномалий. При этом важно проверять модель на реальных новых партиях продукции: синтетическое увеличение данных может улучшить валидацию, но не покрыть новые типы дефектов. После внедрения нужен мониторинг долей классов, качества разметки, числа ложных тревог и изменений в производстве.

Итак, дисбаланс классов - не просто статистическая неприятность, а несоответствие между распределением данных, функцией потерь и реальной целью задачи. Борьба с ним требует не одного универсального приема, а последовательного процесса: понять цену ошибок, выбрать корректные метрики, обеспечить честную валидацию, применить веса или ресемплинг, настроить порог и проверить результат на независимых данных. Лучшие решения обычно сочетают алгоритмические методы с улучшением данных и ясной прикладной постановкой.

Билет 20. Обработка естественного языка (NLP). Основные задачи: классификация текстов и генерация

Основные понятия и определения

Обработка естественного языка, или NLP (Natural Language Processing), — область машинного обучения и искусственного интеллекта, которая занимается автоматическим анализом, представлением, пониманием и порождением текстов на человеческом языке. В отличие от табличных данных или изображений, язык дискретен, неоднозначен и сильно зависит от контекста: одно и то же слово может иметь разные значения, смысл фразы может меняться из-за порядка слов, отрицаний, стиля, жанра и фоновых знаний.

Основная единица текстовой обработки — токен. Токеном может быть слово, часть слова, знак препинания или специальный символ. Современные модели часто используют subword-токенизацию: редкие слова разбиваются на более частые фрагменты, поэтому модель может работать с новыми словами, опечатками и морфологически богатыми языками. Например, слово может быть представлено не целиком, а как набор подсловных единиц.

Корпус — набор текстов, на котором обучается или оценивается модель. Корпус может быть размеченным, когда каждому примеру сопоставлена целевая метка, или неразмеченным, когда модель учится извлекать закономерности из самого текста. Разметка зависит от задачи: для классификации это класс документа, для анализа тональности — положительная или отрицательная оценка, для извлечения сущностей — границы и типы именованных объектов, для машинного перевода — пары предложений на разных языках.

Векторное представление текста — способ заменить слова, предложения или документы числовыми векторами. Машинная модель не работает с буквами и словами напрямую, поэтому текст нужно перевести в признаки. Ранние подходы использовали мешок слов и TF-IDF, где документ описывается частотами терминов. Более современные подходы используют эмбединги: плотные векторы, в которых похожие по смыслу слова или тексты оказываются близко друг к другу в векторном пространстве.

Классификация текстов — задача присвоить тексту одну или несколько категорий. Примеры: определение темы новости, фильтрация спама, анализ тональности отзыва, маршрутизация обращений в поддержку, определение токсичности комментария. Классификация может быть бинарной, многоклассовой или многометочной. В бинарной классификации выбирают один из двух классов, например «спам» или «не спам». В многоклассовой задаче выбирают один класс из нескольких. В многометочной классификации один текст может иметь несколько меток одновременно.

Генерация текста — задача построить новую последовательность токенов по некоторому условию или без него. Условием может быть начало фразы, вопрос пользователя, инструкция, набор фактов, исходный текст для пересказа или документ для перевода. К генеративным задачам относятся автодополнение, диалоговые системы, суммаризация, перевод, перефразирование, генерация ответов на вопросы и написание структурированных документов.

Теория, формулы и интуиция

С точки зрения вероятностного моделирования язык можно рассматривать как последовательность

токенов. Пусть текст задан последовательностью $x_1, x_2, \dots, x_n$. Языковая модель оценивает вероятность всей последовательности:

$$P(x_1, x_2, \dots, x_n) = \prod_{t=1}^n P(x_t | x_1, \dots, x_{t-1}).$$

Эта формула выражает простую идею: вероятность текста можно разложить на произведение вероятностей каждого следующего токена при условии всех предыдущих. Генерация работает именно так: модель получает контекст, оценивает распределение вероятностей по словарю и выбирает следующий токен. Затем выбранный токен добавляется к контексту, и процесс повторяется.

Для классификации текст обычно преобразуется в вектор признаков h , после чего классификатор вычисляет вероятности классов. В многоклассовом случае часто используется softmax:

$$P(y = k | x) = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)},$$

где z_k — логит класса k , а K — число классов. Модель выбирает класс с максимальной вероятностью или использует порог, если задача многометочная. Обучение обычно сводится к минимизации функции потерь, например кросс-энтропии:

$$L = - \sum_{k=1}^K y_k \log \hat{y}_k,$$

где y_k — истинная метка в one-hot-представлении, а $\hat{y}_k$ — предсказанная вероятность.

Ключевая интуиция NLP состоит в том, что значение слова определяется контекстом. В простых частотных моделях слово имеет почти фиксированный смысл: «банк» будет одним признаком и в тексте про финансы, и в тексте про берег реки. Контекстные модели, такие как BERT или GPT-подобные трансформеры, строят представление токена с учетом соседних слов и всей доступной последовательности. Поэтому один и тот же токен может получать разные векторы в разных предложениях.

В основе современных NLP-моделей лежит механизм внимания. Он позволяет каждому токenu учитывать другие токены последовательности с разными весами. Упрощенно, для матриц запросов Q , ключей K и значений V self-attention вычисляется так:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

Матрица QK^T показывает, насколько токены «смотрят» друг на друга, деление на $\sqrt{d_k}$ стабилизирует масштаб, softmax превращает оценки в веса, а умножение на V дает новое контекстное представление. Благодаря этому модель может связывать удаленные части предложения: подлежащее и сказуемое, местоимение и объект, начало инструкции и требуемый формат ответа.

Для генерации важны не только вероятности, но и стратегия выбора следующего токена. Жадный декодинг всегда берет самый вероятный токен, что может давать однообразные тексты. Beam search хранит несколько наиболее вероятных гипотез и часто используется в переводе или суммаризации. Sampling выбирает токен случайно с учетом распределения вероятностей, поэтому результат становится разнообразнее. Параметры temperature, top-k и top-p управляют степенью случайности: низкая температура делает текст более предсказуемым, высокая — более вариативным, но иногда менее точным.

Методы, алгоритмы и этапы решения

Классический конвейер NLP начинается с предобработки. В зависимости от задачи выполняют приведение к нижнему регистру, удаление лишних символов, нормализацию пробелов, лемматизацию, стемминг, обработку чисел и ссылок. Однако степень предобработки зависит от модели. Для TF-IDF и линейных классификаторов нормализация часто полезна. Для трансформеров агрессивная очистка может навредить, потому что пунктуация, регистр, эмодзи и специальные символы тоже несут смысл.



Figure 21: Схема 21

Следующий этап — токенизация. В русскоязычных задачах важно учитывать морфологию: окончания, приставки и формы слов. Поэтому subword-подходы удобны: они уменьшают проблему неизвестных слов и позволяют модели обрабатывать формы, которых не было в обучающей выборке. После токенизации текст переводится в числовые идентификаторы, дополняется служебными токенами и масками внимания, а затем подается в модель.

Для классификации один из простых и сильных базовых методов — TF-IDF плюс логистическая регрессия, линейный SVM или наивный байесовский классификатор. TF-IDF повышает вес слов, которые часто встречаются в конкретном документе, но редко встречаются во всем корпусе:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \cdot \log \frac{N}{\text{df}(t)}.$$

Здесь $\text{tf}(t, d)$ — частота термина t в документе d , N — число документов, $\text{df}(t)$ — число документов, где встречается термин. Такой подход хорошо работает для тематической классификации, спама и задач, где важны ключевые слова. Его преимущества — скорость, интерпретируемость и небольшие требования к данным. Недостатки — слабое понимание порядка слов, контекста и сложной семантики. Более мощный подход — использовать нейронные эмбединги. Сначала применялись word2vec, GloVe и fastText, где каждому слову или подсловной единице сопоставлялся плотный вектор. Затем поверх этих векторов строились CNN, RNN, LSTM или GRU. Рекуррентные сети обрабатывали текст последовательно и могли учитывать порядок слов, но плохо масштабировались на длинные контексты и хуже распараллеливались.

Современный стандарт — трансформеры. Для классификации часто используют encoder-модели, например BERT-подобную архитектуру. Вся последовательность обрабатывается сразу, а специальный агрегирующий вектор или усреднение токенов передается в классификационную голову. Такая модель дообучается на размеченных примерах: общие языковые знания берутся из предварительного обучения на больших корпусах, а конкретная граница классов уточняется на задаче пользователя.

Этапы решения задачи классификации обычно выглядят так: собрать и очистить датасет, определить схему меток, разделить данные на train, validation и test, выбрать baseline, обучить модель, подобрать гиперпараметры, оценить качество, проанализировать ошибки и внедрить модель. Для оценки используют accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix. Если классы несбалансированы, accuracy может вводить в заблуждение: модель, которая всегда выбирает частый класс, покажет высокую точность, но будет бесполезна для редких важных случаев. Поэтому в задачах токсичности, мошенничества или критических обращений часто особенно важны recall и F1.

Для генерации применяются decoder-модели или encoder-decoder-модели. Decoder-модель предсказывает следующий токен по предыдущим и хорошо подходит для диалога, автодополнения и ответов по инструкции. Encoder-decoder архитектура удобна, когда есть входная последовательность и нужно получить выходную: перевод, суммаризация, исправление ошибок, переформулирование. Encoder строит представление исходного текста, а decoder по нему порождает целевой текст.

Практический конвейер генерации включает подготовку промпта или входных данных, токенизацию, получение распределения следующего токена, декодирование, остановку по специальному токenu или лимиту длины и постобработку результата. Для прикладных систем важны ограничения: модель может генерировать правдоподобный, но фактически неверный текст; может повторяться; может нарушать формат; может быть чувствительной к формулировке запроса. Поэтому генерацию часто дополняют retrieval-augmented generation, проверкой фактов, шаблонами вывода, правилами валидации и человеческой модерацией.

Отдельно важна настройка моделей под задачу. Fine-tuning изменяет веса модели на специализированном датасете. Prompt engineering управляет поведением через инструкцию и примеры без изменения весов. Retrieval-подход сначала ищет релевантные документы во внешней базе знаний, затем передает их модели как контекст. Это полезно, когда нужно отвечать по актуальным или внутренним данным.

Практические примеры

Пример классификации — фильтр обращений в техническую поддержку. На вход поступает текст: «После обновления приложение не запускается, появляется ошибка авторизации». Система должна отнести обращение к категориям «ошибка входа», «проблема после обновления» и, возможно, «высокий приоритет». Если используется TF-IDF, важными признаками станут слова «обновления», «не запускается», «ошибка», «авторизации». Если используется трансформер, модель дополнительно учтет общий смысл: проблема возникла после обновления и связана со входом в систему. Результат можно использовать для автоматической маршрутизации заявки нужной команде.

Другой пример — анализ тональности отзывов. Отзыв «Доставка быстрая, но товар пришел с

дефектом» нельзя надежно классифицировать только по отдельным положительным и отрицательным словам. В нем есть смешанная оценка: положительная характеристика доставки и отрицательная характеристика товара. Для бизнеса может быть полезна не только общая тональность, но и aspect-based sentiment analysis, где модель отдельно оценивает аспекты: доставка, качество товара, цена, поддержка.

Пример генерации — автоматическая суммаризация длинной новости. На вход подается статья, а модель должна выдать краткое содержание из нескольких предложений. Экстрактивная суммаризация выбирает важные фрагменты из исходного текста. Абстрактивная суммаризация формулирует новый текст своими словами. Абстрактивный подход гибче, но требует контроля фактической точности: модель может добавить деталь, которой не было в источнике.

Еще один пример — генерация ответа в чат-боте поддержки. Пользователь пишет: «Как восстановить доступ, если я потерял телефон с двухфакторной аутентификацией?» Система может найти в базе знаний инструкцию, передать ее генеративной модели и получить связный ответ. Хорошая архитектура не должна полагаться только на память модели: она должна извлечь актуальную инструкцию, процитировать или пересказать ее, проверить наличие обязательных шагов и не придумывать несуществующие способы восстановления.

В образовательных системах NLP применяют для проверки открытых ответов. Классификационная часть может определить, раскрыта ли тема, а генеративная — сформировать пояснение: какие элементы ответа присутствуют и чего не хватает. При этом оценивание должно быть осторожным: для экзаменационных или юридически значимых решений нужны прозрачные критерии, тестирование на реальных данных и участие эксперта.

Таким образом, классификация и генерация — две центральные группы задач NLP. Классификация превращает текст в решение: метку, вероятность, приоритет или категорию. Генерация превращает условие в новый текст: ответ, перевод, резюме, объяснение или продолжение. Обе группы задач опираются на общую идею: язык нужно представить в числовой форме так, чтобы модель могла учитывать частоты, порядок, контекст и смысловые связи. Разница в том, что классификатор сжимает текст до решения, а генеративная модель разворачивает внутреннее представление обратно в последовательность токенов.